WNDRVR
SUPPORT NETWORK

VXWORKS 7 BSP AND DRIVER GUIDE, SR0650 EDITION 2

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 BSP and Driver Guide, SR0650 Edition 2

18 August 2020

1. OVERVIEW

1. BSP and Driver Model Overview

The VxWorks® 7 BSP and Driver Guide describes the VxWorks 7 board support package (BSP) and VxBus driver model, which allows you to get the VxWorks operating system running on your custom hardware with minimal effort.

The VxWorks 7 BSP and driver model is designed to allow for extensive reuse of drivers and architecture code within a specific processor family. This approach minimizes the work required to port VxWorks to your custom hardware. The VxBus driver framework allows for a consistent interface for drivers and easier reuse of driver code within VxWorks.

BSP Architecture

The BSP architecture is designed to optimize reuse and usability. BSPs that support processors within the same silicon vendor product family can be supported by a single BSP. Because BSPs and device drivers are configured dynamically by probing a device tree, you do not need to hard code the board configuration. This model simplifies porting for new boards (based on existing supported processors) because it allows you to reuse device drivers across families of peripherals and boards and decreases the overall code base.

The VxWorks 7 BSP architecture includes the following elements:

Board Subsystem

Provides a common interface between the OS and a BSP.

BSP

Provides the board-specific descriptors and functions.

Processor Support Library

Provides code and drivers that are common to processors within a specific processor family. This is the code that allows for reuse among hardware platforms that use a common family of processors.

VxBus

The term *VxBus* generally refers to one of two things: the device driver infrastructure or the format of the drivers themselves. Most often, the term refers to a specific infrastructure for support of device drivers in VxWorks, with minimal BSP support.

VxWorks device drivers are a means of communication between a hardware device and the VxWorks operating system. This release of VxWorks requires all device drivers to be VxBus-enabled. One version of VxBus driver is supported, VxBus GEN2 drivers. VxBus GEN1 drivers are no longer supported by VxWorks. Refer to VxBus GEN1 Driver Migration on page 233 for guidance on porting VxBus GEN1 source code drivers to VxBus GEN2.

The VxBus infrastructure supports device drivers by defining standard interfaces for the driver to interact with the operating system and device hardware. VxBus drivers share a common interface to the operating system or hardware.

2. GENERAL ARCHITECTURE

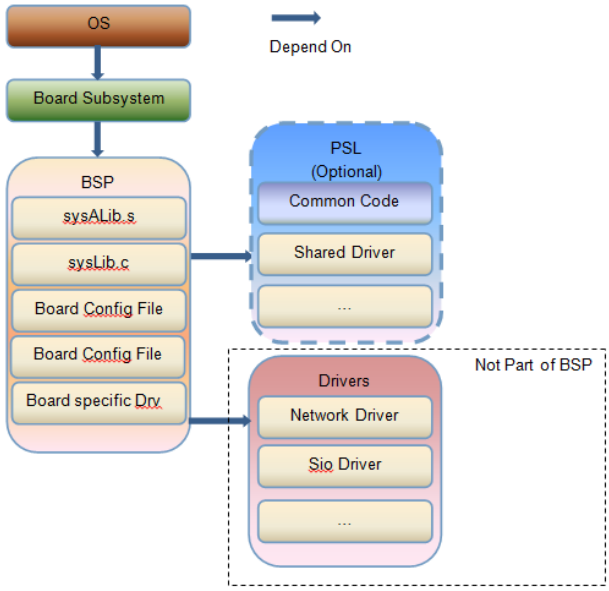
1. VxWorks 7 BSP Infrastructure Overview

A VxWorks 7 board support package (BSP) provides code and OS support for a specific target hardware platform. The BSP makes use of a *processor support library* (PSL) and the VxBus driver infrastructure to provide complete support for target hardware.

A BSP is composed of hardware configuration data, the code to parse the configuration, and the interface to the boot loader and OS. The BSP must define the hardware; supply access information, board initialization, and control methods; and prepare the OS run-time environment.

The BSP infrastructure includes a processor support library (PSL) that allows you to share common code and drivers among BSPs that support a similar hardware platform. It also includes a *board subsystem* interface to facilitate communications between the BSP and the OS, which are isolated.

The following figure illustrates the BSP framework:



BSP

The core BSP code includes only the board-specified descriptors and functions. This limits the number of files in any given BSP. A data set model describes the board instances. For example, one board might include one or more board instances, but each BSP determines if one image supports several boards. The BSP is the entry into the VxWorks image, and the BSP decides how to make use of the board subsystem and the PSL.

PSL

The PSL consists of the code and drivers that are shared among boards that belong to a particular processor family. Use of the PSL helps to limit the size of the BSP and allows for easier porting for processors in a common family.

Board Subsystem

The board subsystem retrieves all of the board data sets defined by the BSP. The subsystem generally makes use of a probe function to automatically match hardware on the board with driver support code. After that, the OS uses the API supplied by the board subsystem (for example, sysToMonitor( ) and sysUsDelay( )) to interface with the hardware.

Note: The board subsystem can work without a board probe utility.

About VxWorks 7 Board Support Packages on page 3 The board support package (BSP) code is the board-specific part of the overall hardware support infrastructure.
Customizing a BSP Component on page 5 This example shows how to add UART support to an ARM based BSP.
About the VxWorks 7 Processor Support Library on page 6 The processor support library (PSL) includes the common code and drivers shared among BSPs related to the same hardware product family. The PSL reduces code redundancy and enhances the maintainability and usability of BSPs.
About the VxWorks 7 Board Subsystem on page 6 The board subsystem provides the interface between the operating system and the board support package (BSP). The OS can only access board functionality through the board subsystem.

1.1. About VxWorks 7 Board Support Packages

The board support package (BSP) code is the board-specific part of the overall hardware support infrastructure.

The term BSP is often not well defined, and can refer to a collection of device drivers as well as the interface between the kernel and the hardware. In VxWorks 7, the BSP includes only the bare minimum amount of code specific to a single board or narrowly-defined family of common boards. Code that is specific to a given higher level architecture or broad processor family is contained in the processor support library (PSL) and shared with other BSPs. Common device drivers are also abstracted out for use with multiple BSPs. Only drivers specific to a particular target board are contained in the BSP code itself.

All BSPs reside under the following directory:

installDir **/vxworks-7/pkgs_v2/os/board/***vendor*

A typical BSP directory includes different content, depending on your target hardware architecture.

ARM, PowerPC, and RISC-V BSP Files

BSPs for ARM, PowerPC, and RISC-V targets typically include the following files:

sysALib.s

Identifies the assembly language entry point (the first code executed after the boot loader loads VxWorks). It typically saves arguments, initializes a few registers, does minimal MMU initialization, cache initialization, and sets up the stack for the C language.

sysLib.c

Implements board-specific functions. (This file implements the traditional sysHwInit( ), sysHwInit1( ), and sysHwInit2( ) functions.)

bsp.cdf and **20bsp.cdf**

These files contain board-specific definition and configuration parameters, such as the kernel entry address, or configuration parameters for certain components.

sysLib.c

This file contains the board definition structure (**BOARD_DESC**) and may contain some board-specific functions.

board_name.spec

An RPM spec file for installation information.

board_name.dts

A baseline device tree source (DTS) file for the board.

Typically, you need only customize the **board_name.dts** and the **20bsp.cdf** files. Rarely, you may be required to add some board-specific functionality (which is not part of the PSL) to the **sysLib.c** file.

Note: When the BSP only represents a single board, the **board_name.dts** file resides in the BSP directory. When the BSP represents a family of boards, the BSP directory contains the various DTS files, one for each single board variant.

See Customizing a BSP Component on page 5 for a sample ARM based BSP customization.

Intel Architecture BSP Files

The itl_generic BSP for Intel Architecture includes the following files:

sysALib.s

Identifies the assembly language entry point (the first code executed after the boot loader loads VxWorks). It typically saves arguments, initializes a few registers, does minimal MMU initialization, cache initialization, and sets up the stack for the C language.

sysLib.c

This file contains the board definition structure (**BOARD_DESC**) and may contain some board-specific functions.

bsp.cdf and **20bsp.cdf**

These files contain board-specific definition and configuration parameters, such as the kernel entry address, or configuration parameters for certain components.

board_name.spec

An RPM spec file for installation information.

board_name.dts

A baseline device tree source (DTS) file for the board.

romInit.s

This file is the first assembly code executed by a boot loader when booting from ROM.

20IaDrvCfg.cdf

Defines device components and parameters that the kernel uses.

usrMpApicData.c

Defines static configuration tables for the BSP mpApicData structure.

Typically, you need only customize the 20bsp.cdf( ) file and, if you are adding device drivers, the 20IaDrvCfg.cdf( ) file.

Parent topic: VxWorks 7 BSP Infrastructure Overview on page 2

1.2. Customizing a BSP Component

This example shows how to add UART support to an ARM based BSP.

About This Task

The Zynq-7000 ZC702 BSP is used to illustrate the general steps involved.

Before You Begin

You will need to obtain correct values for the following steps using the Xilinx design tool documentation.

Procedure

1. Copy the existing hardware node to the bsp's **.dts** file as follows and substitute correct values obtained from your Xilinx design documentation.

```
uart2: uart@ some_address
            {
            compatible = xlnx,xuartps;
            reg = value ;
            clock-frequency = value ;
            interrupts = value ;
            interrupt-parent = reference ;
            };
```

2. Add the following components to the VIP project to enable testing of the newly added device at run-time.

```
INCLUDE_VXBUS_SHOW
INCLUDE_VXBUS_IOCTL_SHOW
```

3. Build the VSB project.
4. Build the VIP project based on the above VSB project and boot the kernel .
5. Enter the following in the target shell.

```
-> vxbDevShow
zynq7ksio-0 (refs: 0) on simpleBus0 (refs: 19)
zynq7ksio-0: <zynq7k serial> (0x20019b4c)
zynq7ksio-1 (refs: 0) on simpleBus0 (refs: 19)
zynq7ksio-1: <zynq7k serial> (0x2002a580)
```

   vxbDevShow displays the newly added serial device **zynq7ksio-1** along with the first serial device **zynq7ksio-0**,

```
-> vxbIoctlShow
```

   The vxbIoctlShow command displays both the VxBus device nodes and the resource properties.

6. Confirm that the register base address and interrupt are mapped correctly.

Parent topic: VxWorks 7 BSP Infrastructure Overview on page 2

1.3. About the VxWorks 7 Processor Support Library

The processor support library (PSL) includes the common code and drivers shared among BSPs related to the same hardware product family. The PSL reduces code redundancy and enhances the maintainability and usability of BSPs.

Note: An example of a hardware product family is the **fsl_qoriq** family from Freescale, the **Omap** family from TI, or the **QSP** family from Simics.

PSL as a Common API

The PSL provides common APIs for the supported BSPs. You can upgrade the PSL separately from the BSPs that it supports. The PSL includes a single board function table with default functions such as *targetHw* Probe( ) or *targetHw* EarlyInit( ).

The following example hows the board function table for the am33xx ARM platform:

```
BOARD_FUNC_TBL am33xxFuncTbl = {
    /* .earlyInit  = */ am33xxEarlyInit,
    /* .init       = */ am33xxInit,
    /* .reset      = */ am33xxReset,
    /* .model      = */ am33xxModel,
    /* .usDelay    = */ am33xxUsDelay
};
```

Note: In older BSPs for the Intel Architecture, the board function table resides in the file **boardSys.c** in the BSP directory rather than in the PSL. For the itl_generic BSP, the table is in the PSL as **usrIaBoardSys.c**.

Content of a PSL

A PSL includes the following content:

- The *targetHw* **.c** file where *targetHw* is the name of the hardware platform supported by the PSL. This file includes the common APIs for the BSP (to reduce code redundancy).
- Drivers that are specific to the supported target hardware platform.
- Other shared code specific to the supported target hardware such as a clock module or a power management module.

Parent topic: VxWorks 7 BSP Infrastructure Overview on page 2

1.4. About the VxWorks 7 Board Subsystem

The board subsystem provides the interface between the operating system and the board support package (BSP). The OS can only access board functionality through the board subsystem.

The BSP or the processor support library (PSL) implements the functionality defined by the board subsystem. The board subsystem provides an API through a board function table. For more information, see About the VxWorks 7 Processor Support Library on page 6.

The following code snippet shows a typical board function table, **board_func_tbl**.

```
typedef struct board_func_tbl {
    UINT32 uVer;
    void    (*earlyInit) (void);
    void    (*init) (void); /* optional */
```


WNDRVR

```
    void    (*reset) (void);
    char *  (*model) (void);
    void    (*usDelay) (int); /* optional */

#ifdef _WRS_CONFIG_SMP
    STATUS  (*cpuEn) (unsigned int, WIND_CPU_STATE *);
#endif  /* _WRS_CONFIG_SMP */
} BOARD_FUNC_TBL;
```

The content of the example **board_func_tbl** is as follows:

> Note: Use *uVer* only if your board library does not use the board probe mode. Use the same string used by the board descriptor structure.

- The function earlyInit( ) is the first C entry for one board. The function usrInit( ) calls earlyInit( ) immediately after it calls sysStart( ) and usrFdtInit( ) (for architectures that use device trees).
- The function init( ) is optional. This function is invoked during the usrRoot( ) phase and does not need to call the VxBus library initialization function. The boardInit( ) function (within usrRoot( )) calls init( ) after initializing the MMU, cache, exception, kernel core, memory pool, I/O, and other subsystems, but before the VxBus library is initialized. The init( ) function can perform some board-specific initialization tasks required by the VxBus drivers (such as pin mux, initializing the clock, and so forth).

  > Note: The logMsg( ) and kprintf( ) functions do not work between usrMmuInit( ) and boardInit( ).

- The reset( ) function resets the entire board. The **sysToMonitor()** function calls reset( ) after it shuts down the CPUs.

  > Note: The reset( ) function performs a cold reset, and cannot jump to the boot loader directly because the boot loader is not part of the BSP.

- The model( ) function (called by sysModel( )) returns the model of the board.
- The usDelay( ) function delays the caller by the length of time specified (in microseconds), at a minimum. The sysUsDelay( ) function calls usDelay( ).
- The cpuEn( ) function (called by sysCpuEnable( )) enables the secondary cores of an SMP image.

Board Subsystem Functions

The board subsystem provides the following functions:

- The boardLibInit( ) function, which initializes the board subsystem. Synopsis:

  ```
  void boardLibInit()
  ```

- The sysToMonitor( ) function, which transfers control of the to the ROM monitor. Synopsis:

  ```
  void sysToMonitor( int   startType)
  ```

- The sysModel( ) function, which returns the model name of the CPU board. Synopsis:

  ```
  char * sysModel (void)
  ```

- The sysBspRev ( ) function, which returns the BSP package version retrieved from the file **bsp.vsbl**. Synopsis:

  ```
  char * sysBspRev (void)
  ```

- The sysUsDelay( ) function, which delays the caller by the specified amount of time (in microseconds), at a minimum. Synopsis:

```
void sysUsDelay( int  delay )
```

- The sysMsDelay( ) function, which delays the caller in one microsecond increments. Synopsis:

```
void sysMsDelay( int delay )
```

- The sysDelay( ) function, which delays the caller by a fixed one microsecond delay. Synopsis:

```
void sysDelay (void)
```

- The sysProcNumGet( ) function, which gets the processor number on which it is being called. Synopsis:

```
int sysProcNumGet (void)
```

- The sysCpuEnable( ) function, which enables a secondary CPU. Synopsis:

```
STATUS sysCpuEnable
            (
            unsigned int        cpuNum,
            Void                (*startFunc) (void),
            char *              stackPtr
            )
```

The boardLibInit() Function

The boardLibInit( ) function retrieves all of the board data sets defined in the BSP. It then uses the attached board descriptor to call the probe routine. Next, it connects the function table pointer to the global function table pointer defined in the board subsystem. If the tables match and all of the essential functions of the function table for the board descriptor are present, boardLibInit( ) calls earlyInit( ) to set up the board information and initialize the debug console.

```
STATUS boardLibInit ()
    {
#ifdef _WRS_CONFIG_BOARDLIB_NO_PROBE
    *gpBoardFuncTbl = _pBspBoardFuncTbl;
#else /* !_WRS_CONFIG_BOARDLIB_NO_PROBE */
    datasetIterate ("board", 0, 1, boardProbe, NULL);
#endif /* _WRS_CONFIG_BOARDLIB_NO_PROBE */

    if (*gpBoardFuncTbl != NULL)
        {
        if ((*gpBoardFuncTbl)->earlyInit != NULL)
            (*gpBoardFuncTbl)->earlyInit ();
        return (OK);
        }
    else
        return (ERROR);
    }
```

The following code snippet describes the boardProbe( ) function.

```
LOCAL STATUS boardProbe
    (
    void * pArg,
    void * pData
    )
```

```
    {
    BOARD_DESC * pBoardDesc = (BOARD_DESC * ) pData;
    if ((pBoardDesc->uVer != BOARD_DESC_VER_1_0) ||
        (pBoardDesc->probe == NULL) ||
        (pBoardDesc->pFuncTbl== NULL))
        return (ERROR);

    if (pBoardDesc->probe (pBoardDesc->pCompat) == TRUE)
        {
        gpBoardFuncTbl  = &(pBoardDesc->pFuncTbl);

        /* install one default delay routine */

        if ((*gpBoardFuncTbl)->usDelay == NULL)
            ((*gpBoardFuncTbl)->usDelay) = boardSwUsDelay;

        return (OK);
        }

    return (ERROR);
    }
```

The board descriptor, **pBoardDesc**, is part of the BSP and resides in the file **sysLib.c**. **pBoardDesc** defines the board type, version, and compatibility information. It also points to the main probing function and the main function table. The following code snippet provides an example from the **fsl_p1p2** BSP.

```
LOCAL BOARD_DESC fsl_p2020_rdb = {
    BOARD_DESC_VER_DEF,     /* uVer       */
    "fsl,p2020rdb",         /* pCompat    */
    P1P2_DESC_FLAG_DEF,     /* uFlag      */
    qoriqPorbe,             /* probe      */
    &qoriqFuncTbl           /* pFuncTbl   */
};

BOARD_DEF(fsl_p2020_rdb)

LOCAL BOARD_DESC fsl_p1020_rdb = {
    BOARD_DESC_VER_DEF,     /* uVer       */
    "fsl,p1020rdb",         /* pCompat    */
    P1P2_DESC_FLAG_DEF,     /* uFlag      */
    qoriqPorbe,             /* probe      */
    &qoriqFuncTbl           /* pFuncTbl   */
};

BOARD_DEF(fsl_p1020_rdb)

LOCAL BOARD_DESC fsl_p2020_ds = {
    BOARD_DESC_VER_DEF,     /* uVer       */
    "fsl,p2020ds",          /* pCompat    */
    P1P2_DESC_FLAG_DEF,     /* uFlag      */
    qoriqPorbe,             /* probe      */
    &qoriqFuncTbl           /* pFuncTbl   */
};

BOARD_DEF(fsl_p2020_ds)
```

Note: This single BSP supports three boards (**fsl_p2020_rdb**, **fsl_p1020_rdb**, and **fsl_p2020_ds**).

Parent topic: VxWorks 7 BSP Infrastructure Overview on page 2

2. BSP Directory Structure

To develop your own BSPs (or modify Wind River BSPs to suit your needs), you must understand the directory structure of the BSP and the location of relevant files and directories.

Note: All directory paths in this section are presented in relation to your Wind River VxWorks 7 installation directory (for example, *installDir* **/vxworks-7**).

All BSPs reside under the following directory:

installDir **/vxworks-7/pkgs_v2/os/board/**

Under each architecture-specific directory (**freescale**, **intel**, **nxp**, **renesas**, **sifive**, **ti**, **wrs**, **xilinx** or **other**), there are a variety of BSPs; one for each board family.

The content of the individual BSP directories is minimal and only includes the board-specified descriptors and functions.

PSL and Board Subsystem Components

The processor support library (PSL), which includes common code and drivers that are shared among BSPs that belong to the same product family (for example the **fsl_qoriq** family from Freescale), is located under **pkgs_v2/os/psl**.

The board subsystem library, which implements functionality that is not specific to a board or to an architecture (such as sysUsDelay( ), sysToMonitor( ), or boardProbe( )), is located under **pkgs_v2/os/utils/boardLib**.

Drivers for **gpio**, **dma**, **clock**, **pin-mux**, and so forth are included under **pkgs_v2/os/drv/vxbus/drv** and can used by the subsystem under **pkgs_v2/os/drv/vxbus/subsystem**.

The architecture-specific library, which implements functionality that is specific to an architecture such as cache and MMU control, exception handling, or math software support, is located under **pkgs_v2/os/arch**.

3. BSP CONFIGURATION

1. BSP Configuration Overview

If you need to configure your BSP for different architectures, you must be aware of the files that relate to hardware and component configuration.

VxWorks 7 BSP Configuration

For ARM, PowerPC, and RISC-V systems and for Intel systems supported by the itl_generic BSP, file configuration is based on a flat device tree (FDT) model. Configuration information is provided using device tree source (DTS) files.

Additional configuration information is provided by the BIOS, derived from either MultiProcessor (MP) tables or Advanced Configuration and Power Interface (ACPI), depending on your hardware.

Configuration information is also provided as part of the *xx* **bsp.cdf** files in the BSP.

BSP Configuration for Existing VxWorks Users

Traditionally in VxWorks, BSP configuration for components and hardware was done in the following files under the BSP directory:

hwconf.c

Hardware-related configuration.

20bsp.cdf

Component configuration.

config.h

Hardware-related configuration and component information for certain components.

In this release, the hardware configuration functionality of **hwconf.c** and **config.h** is moved to the DTS files (ARM, current Intel Architecture, and PowerPC) or the IADT (older Intel Architecture support).

On ARM and PowerPC, most of the remaining configuration information that resided in **config.h** has moved to **20bsp.cdf**.

For Intel Architecture systems supported by the itl_generic BSP, remaining configuration information that resided in **config.h** has moved to **00bsp.cdf**, **20bsp.cdf**, and **20IaDrvCfg.cdf**. On older Intel Architecture systems, the information moved to **00bsp.cdf** and **20bsp.cdf**.

2. Warm Boot Support

VxWorks warm boot support allows for preserving the persistent memory region through reboots, regardless of which boot loader is used.

Warm boot support provides a faster reboot process by skipping the boot loader phase of the boot operation. The warm boot process re-enters at the start of the kernel entry, preserving the persistent memory region through the reboot.

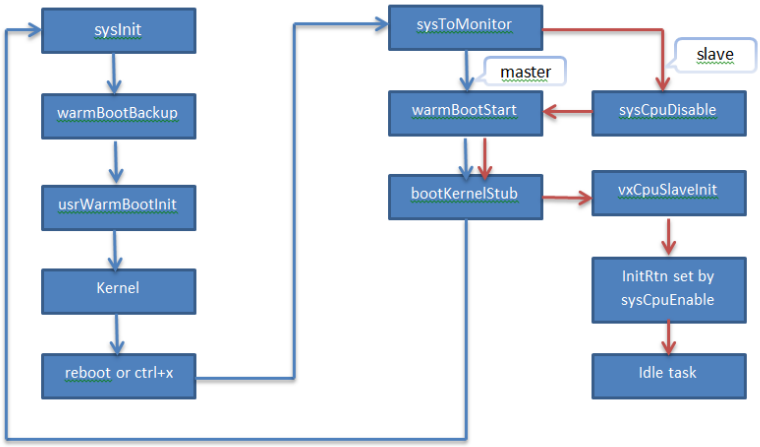
The persistent memory region is used by the kernel core dump facility, the error detection and reporting facility, and (optionally) by System Viewer.

The content of the persistent memory region is preserved by restarting the VxWorks image without using the boot loader. This is accomplished by having sysToMonitor( ) "jump back" to the loaded VxWorks image's entry point rather than resetting the board (which typically results in the boot loader coming into play). With a warm reboot support, the BSP backs up and restores the global variable, and restores the kernel environment to the same state as it was initially set by the boot loader. For SMP systems, the BSP must handle slave cores. For device drivers, vxbDevShutdown( ) is invoked at reboot to disable the devices.

To enable the fast warm boot, the INCLUDE_WARM_BOOT component must be included in the VIP project (except for VxSim, for which it is not required as the contents of persistent memory are always preserved). The optional INCLUDE_WARM_BOOT_PROTECT component write protects the data during a warm reboot (the data section and the header, which records the CRC for the text and data section). The INCLUDE_EDR_PM component is required for providing persistent memory support.

Warm Boot Process

The basic sequence of the warm boot process that is illustrated below is implemented somewhat differently for each architecture. For example, for IA it is handled by a slave core itself rather than a warm boot library. For ARM, the data is restored in the usrInit( ) call with a warm boot, because it does not use any global variables before that point, which means that the BSP does not need to call warmBootBackup( ).



VxWorks Warm Boot Library

The VxWorks warm boot library provides support for all architectures. The following functions can be used in the BSP or PSL.

warmBootBackup( )

Backs up the data segment and calculates the checksum for the data and text segment with the first boot.

The warmBootBackup( ) function should be called as soon as possible, before the global data has been updated.

For both PPC and IA, call warmBootBackup( ) in the sysInit( ) function. This function needs to use real-mode addresses (or a simulation thereof, as required for the processor, by doing a 1:1 mapping of physical and virtual addresses). The adjust parameter is equal to the offset between the load address and the link address (generally, the PC is equal to the real mode load address). The parameter is used to calculate the load addresses of the functions and variables that are backed up.

For ARM, call warmBootBackup( ) in usrInit( ). In usrInit( ), ARM enables the MMU first, then calls warmBootBackup( ), so the adjust parameter is 0.

warmBootStart( )

Starts the warm boot, checks the checksum for data and text segment to ensure the backed up data is correct, then restores the data segment and jumps to the bootstrap entry. To use this API, each architecture must support a bootKernelStub( ) function, which restores the kernel environment prior to re-entering the kernel.

PPC and IA use warmBootStart( ) to restore the backed up data, and call bootKernelStub( ) to restore the environment. ARM uses warmBootRestore( ) to restore the backed up data, and then usrInit( ).

Use the **sysWarmBootFunc** function pointer to call warmBootRestore( ).

warmBootRestore( )

Restores the saved data segment. This is only required by ARM BSPs. Instead of restoring data before the reboot, ARM BSPs do so in the early phase of kernel re-entry, in usrInit( ).

warmBootSlaveDisable( )

Disables the slave core. Call this function from the BSP or PSL. Only PPC BSPs use this function. Use the function pointer **sysWarmBootSlaveDisable** to call this function.

warmBootSlaveEnable( )

Enables a slave core that is polling spin state. Use the function pointer **sysWarmBootSlaveEnable** to call this function.

Adding Warm Boot Support

Warm boot support is only supported for the VxWorks 7 BSP model (in **installDir/vxworks-7/boardlib**).

To add warm boot support to a BSP, Wind River recommends using a similar BSP that already has warm boot support as a model for implementation. Refer to the existing BSP or PSL code. In general the architecture already provides code that can be used for preparing the kernel environment, and your BSP or PSL only needs call existing APIs.

Device drivers must use vxbDevShutdown( ) which should put the device in disable/reset state before rebooting.

3. Timer Support

The SYS_CLK_RATE_MIN, SYS_CLK_RATE_MAX, AUX_CLK_RATE_MIN, and AUX_CLK_RATE_MAX parameters defined in the BSP component description file (CDF) are used to qualify the system and auxiliary clocks in your VxWorks system. They are also used by the VxBus timer subsystem to filter for a suitable driver.

The following parameters are defined in the BSP CDF file and have the default values listed:

Parameter	Default Value (in Ticks)
SYS_CLK_RATE_MIN	60
SYS_CLK_RATE_MAX	300
AUX_CLK_RATE_MIN	60
AUX_CLK_RATE_MAX	10000

These parameters are used to qualify a potential timer device as the system clock timer; a timer device that is able to provide an interrupt rate higher than SYS_CLK_RATE_MAX or lower than SYS_CLK_RATE_MIN qualifies as a suitable device as the system clock timer. In addition, the system clock rate can only be set to a value between SYS_CLK_RATE_MIN and SYS_CLK_RATE_MAX. If the clock rate is set outside of that scope, it returns **ERROR**. This also applies to the auxiliary clock.

4. MAC Addresses and VxWorks Deployment

If you are going to deploy multiple instances of VxWorks on the same network, you should provide each instance with unique MAC Address.

By default, VxWorks uses the MAC address defined in the DTB file to program the network interface. This is not problem if only there is only one device using it, or for testing purpose. For formal product development and deployment, however, you must implement a method for changing it for multiple instances. These approaches are recommended:

- Use a pre-stage boot loader (for example, U-Boot) to change the MAC address stored in the DTB to a new value. You can do so by using an algorithm to calculate the MAC address based on hardware information, or to retrieve the MAC address from on board non-volatile media.
- Implement code for the function stub endMacGet( ), which is defined in the BSP's **board_func_table{}** structure. If you do so, the network subsystem will use the MAC address that it retrieves instead of the one defined in the DTB.

For further details, refer to the BSP Supplement. To find this, navigate to the General BSP category located on the Wind River product documentation site.

4. BSP SUPPORT FOR BOOT LOADERS

1. BSP Support for the VxBL Boot Loader

To add BSP support for the VxBL boot loader you create a sub_directory under the BSP's bootloader directory that contains the required code and makefile.

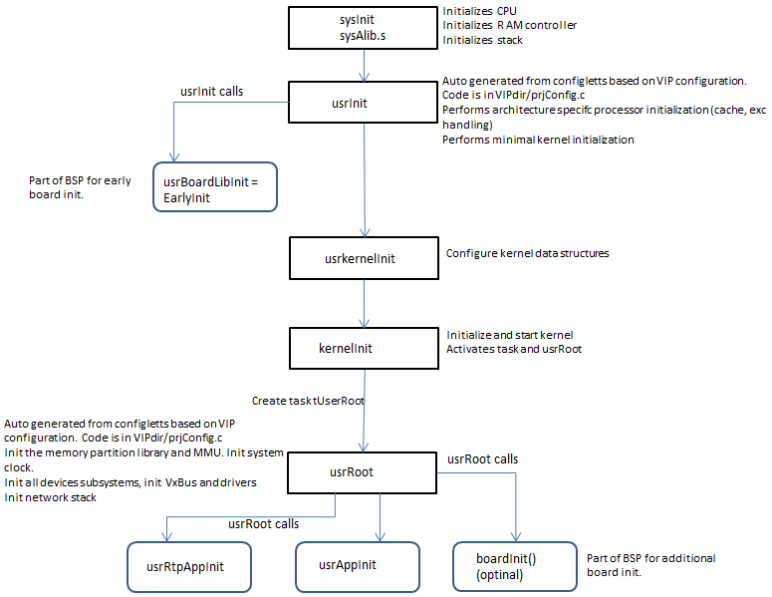
The sub-directory under the BSPs **_bootloader** directory must be named **vxbl**, and must hold the interface code and makefile. For an example, see *installDir* **/vxworks-7/pkgs_v2/os/board/freescale/fsl_imx6-x.x.x.x/_bootloader/vxbl**.

5. INITIALIZATION PROCESS

1. Initialization Process for PowerPC and ARM

The initialization process for a BSP is driven by a combination of board-specific, architecture-specific, and general OS initialization functions. The initialization processes for PowerPC and ARM architectures are the same.

Initialization Sequence



There are three initialization functions that are part of the BSP. One function is optional. The sequence is as follows:

1. sysInit( ) (entry point of the VxWorks kernel image)
2. *cpu* EarlyInit( )
3. *board* Init( ) (optional)

sysInit( )

sysInit( ) is the entry point of the VxWorks kernel image. It is the interface to the boot loader and is responsible for controlling the early initialization stage before usrInit( ). The following is a code example from the Freescale P2020RDB BSP:

```
FUNC_LABEL(_sysInit)
FUNC_BEGIN(sysInit)
#ifdef INCLUDE_STANDALONE_DTB
    /* save IMA size */

    lis     r31, HI(IMA_SIZE)
    ori     r31, r31, LO(IMA_SIZE)
#else /* !INCLUDE_STANDALONE_DTB */
    /* save DTB address and IMA size */
```

```
    mr      r30, r3
    mr      r31, r7
#endif /* INCLUDE_STANDALONE_DTB */
    /* initialize CPU */
    bl      vxCpuInit
    /* obtain runtime address */
    bl      rtAddr
rtAddr:
    mflr    r3      /* runtime address */
   ....
   ....
    /* initialize MMU early */
    mr      r5, r31
    bl      vxMmuEarlyInit
   ....
   ....
    /* jump to C routine */
    li      r3, 0   /* BOOT_NORMAL */
    b       usrInit
FUNC_END(sysInit)
```

sysInit( ) calls the architecture-specific functions for CPU-specified initialization, performs board-specified initialization, sets up the initialization stack, and finally jumps to usrInit( ).

For the architecture library, (**libraryArchitecture**), vxCpuInit( ) is provided to initialize the primary CPU. vxCpuMmuEarlyInit( ) is provided to map the kernel link address space to the run-time address space where the VxWorks kernel image is loaded and running.

usrInit( ) is not part of the BSP and is provided here as a reference:

```
void usrInit (int startType)
{
  sysStart (startType);               /* clear BSS and set up the vector table base address. */

  usrFdtInit ();                      /* Flat Device Tree library */
  usrBoardLibInit();                  /* initialize board library */
  usrCpuInit ();                      /* include CPU support */
  cacheLibInit (USER_I_CACHE_MODE, USER_D_CACHE_MODE); /* include cache support */
  excShowInit ();                     /* exception show routines */
  excVecInit ();                      /* exception handling */
  ...
}
```

cpuEarlyInit( )

The *cpu* EarlyInit( ) function is called by usrBoardLibInit( ) in usrInit( ). At this time, the only VxWorks features initialized are sysStart( ) and the flattened device tree (FDT) library (if applicable).

boardInit( )

The optional *board* Init( ) function is called before VxBus library initialization. This function can be called by boardInit( ) which is called by usrRoot( ). At that point in the initialization process, the architecture and the kernel are already initialized.

usrRoot( ) and *board* Init( ) are shown below:

```
void usrRoot (char *pMemPoolStart, unsigned memPoolSize)
{
  usrKernelCoreInit(); /* core kernel facilities */
  ..
  usrMmuInit((VIRT_ADDR) pMemPoolStart, memPoolSize); /* MMU global map support */
  usrKernelCreateInit ();  /* object creation routines */
  ...
  usrIosCoreInit();   /* core I/O system */
  vxbTimerStubInit(); /* vxBus Timer Stub Support for delay functions */
  boardInit(); /* call the boardInit() routine during system startup */
  vxbLibInit();    /* vxBus subsystem */
  excIntNestLogInit();
  ...
}

void boardInit()
{
  if ((gpBoardFuncTbl != NULL) &&
      (gpBoardFuncTbl->init != NULL))
         gpBoardFuncTbl->init();
}
```

After *board* Init( ), usrRoot( ) calls vxbLibInit( ) which initializes VxBus including all VxBus device drivers.

Generating the usrInit( ) and usrRoot( ) Functions

The usrInit( ) and usrRoot( ) functions are part of the auto-generated file, *yourVIPdir* **/prjConfig.c**.

This file is auto-generated from code and information that is provided as part of the project CDF and configlette files.

2. Initialization Process for Intel Architecture

The initialization process of the BSP is driven by a combination of board-specific, architecture-specific, and general OS initialization functions.

Initialization Sequence of the VxWorks Boot Loader

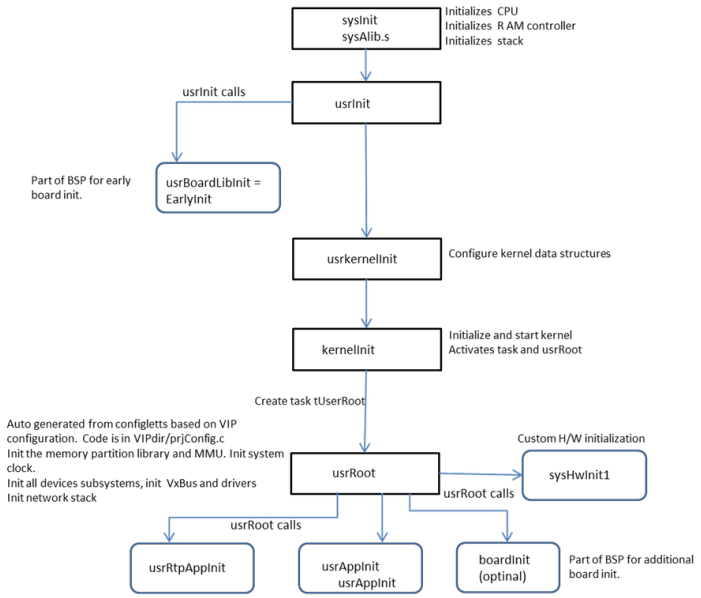
If you build a VxWorks boot loader (also referred to as **bootApp**, **bootapp.sys**, or **vxWorks_romCompress**), the loader that loads it (which can be UEFI, GRUB, SYSLINUX, or the VxWorks boot loader (VxBL) using **bootrom.sys**) loads the **bootApp** such that its **.text** address is at address 0x00408000. The first function, romInit( ) (in **romInit.s**) starts from that address and uncompresses the rest of the **bootApp**. It places the image from address 0x10008000 and up. It then jumps to the sysInit( ) function (in **sysALib.s**) which is at that address, 0x10008000. From that point, the execution is similar to the execution from usrInit( ).

The romInit( ) function clears and sets up a few registers, loads the global descriptor table (GDT), and initializes a static MMU table. It also brings the processor from the initial 16-bit mode all the way up to a 64-bit mode.

> Note: The work flow of the romInit( ) function somewhat varies depending on the boot type (cold boot, warm boot, or fast boot).

The sysInit( ) function disables interrupts, sets up the stack, and jumps to the usrInit( ) C function (in **usrConfig.c**).

Initialization Sequence of the VxWorks Kernel Image



A VxWorks image that is loaded by a boot loader such as UEFI, GRUB, SYSLINUX, or **bootApp** starts at the sysInit( ) function (in **sysALib.s**) at address 0x00408000. sysInit( ) then calls the usrInit( ) C function (which is auto generated in *yourVIPdir* **/prjConfig.c**).

Initialization Sequence Starting at usrInit()

The usrInit( ) function is not part of the BSP and is provided here as a reference:

```
void usrInit (int startType)
{
  sysStart (startType);                /* clear BSS and set up the vector table base address. */
  cacheLibInit (USER_I_CACHE_MODE, USER_D_CACHE_MODE); /* include cache support */
  usrBoardLibInit();                   /* initialize board library */
  usrAimCpuInit ();                    /* include CPU library support */
  excShowInit ();                      /* exception show routines */
  excVecInit ();                       /* exception handling */
  ...
  usrKernelInit (VX_GLOBAL_NO_STACK_FILL); /* context switch and interrupt handling (DO NOT REMOVE
). */
}
```

The *cpu* EarlyInit( ) function is called by usrBoardLibInit( ) in usrInit( ). At this time, the only VxWorks features initialized are sysStart( ).

usrInit( ) calls usrKernelInit( ) that configures internal data structures for context switching and interrupt handling. usrKernelInit( ) calls usrRoot( ) which is auto-generated in *yourVIPdir* **/prjConfig.c**

usrRoot( ) is the entry point for kernel initialization. It initializes page pools and the MMU, DMA, and hardware buses. It then calls sysHwInit1( ) (in *yourVIPdir* **/***BSPdir*/ **sysLib.c**).

sysHwInit1( ) connects system interrupts and does any additional configuration that is needed.

Next, usrRoot( ) calls *board* Init( )which is an optional function that is called before the VxBus library is initialized. The Init( ) function is where you can provide additional initialization. You must implement it in the BSP directory. At this stage, the architecture and the kernel are already initialized.

The usrRoot( ) and *board* Init( ) functions are as follows:

```
void usrRoot (char *pMemPoolStart, unsigned memPoolSize)
{
  usrKernelCoreInit(); /* core kernel facilities */
  ..
  usrMmuInit((VIRT_ADDR) pMemPoolStart, memPoolSize); /* MMU global map support */
  usrKernelCreateInit ();  /* object creation routinse */
  ...
  sysHwInit1();    /* call the BSPs sysHwInit1 routine during system startup */
  ...
  usrIosCoreInit();   /* core I/O system */
  vxbTimerStubInit(); /* vxBus Timer Stub Support for delay functions */
  .......
  boardInit(); /* call the boardInit() routine during system startup */
  vxbLibInit();    /* vxBus subsystem */
  ...
  ...
}

void boardInit()
{
  if ((gpBoardFuncTbl != NULL) &&
      (gpBoardFuncTbl->init != NULL))
         gpBoardFuncTbl->init();
}
```

After *board* Init( ), usrRoot( ) calls vxbLibInit( ) which initializes VxBus including all VxBus device drivers.

Generating the usrInit( ) and usrRoot( ) Functions

The functions usrInit( ) and usrRoot( ) are part of the auto-generated file, *yourVIPdir* **/prjConfig.c**. This file is auto-generated from code and information provided as part of the project CDF and configlette files.

WNDRVR

6. DETECTING AND CONFIGURING DEVICES

1. Detecting and Configuring Devices with a Device Tree

For PowerPC, ARM, and RISC-V systems, use a device tree to detect a device and provide configuration information.

> Note: Intel architecture systems that are supported by the itl_generic BSP use a combination of flattened device tree (FDT) and ACPI to configure devices which is slightly different from what is used for ARM, PowerPC, and RISC-V. For more information, see itl_generic BSP Supplement.

With ARM, PowerPC, and RISC-V architectures, you must use a flattened device tree (FDT) to detect (and sometimes to configure) a device.

A device tree is a tree data structure with nodes that describe the system hardware. Each node has property/value pairs that describe the characteristics for a device that is not dynamically detected. For example, your FDT does not have to contain code for PCI devices. However, it must contain code for the PCI host bridge. Generally, the device information is parsed either by the individual device drivers that call the FDT library when the device initializes, or by the VxBus core that also uses the that same FDT library.

For ARM, PowerPC, and RISC-V processors, the FDT format is used as the board configuration data format. The default format is FDT format version 17 which is accepted by the Embedded Power Architecture™ Platform Requirements (ePAPR) standard.

You can load the flattened device tree either using the boot loader (as a standalone file for a VxWorks kernel image), or by embedding it into a VxWorks kernel image. When loading a device tree blob (DTB) file, the boot loader must pass the DTB address to the FDT library. With an embedded DTB, the kernel uses the address of the **dt_blob_start** symbol instead.

The FDT typically provides the following information:

- the device memory address and range
- interrupt controllers and IRQ line connections
- the number and type of CPUs
- base addresses and size of RAM
- buses and bridges
- peripheral device connections
- boot parameters

> Note: When the VxWorks boot parameters are set in the FDT, these parameters cannot be modified from the running target's VxWorks kernel shell using bootChange( ).

FDT Source Code

The source code is in a device tree source (DTS) file, in text format, with the extension **.dts**. (The DTS file is compiled into a DTB file which is a binary representation of the information in the DTS file). The FDT standard supplies rules for every type of device. When you develop your BSP, you must supply a DTS file and ship it with your BSP.

> Note: Technically, the devices are not part of the BSP, However, the DTS file that describes the devices resides in the BSP directory.

Generally, device information is parsed by the device driver or VxBus. However, for some simple devices such as the Freescale Qoirq Global Utility Model (where a driver is unnecessary), BSP code is used for parsing instead.

The following example shows a typical device node. A device node provides information such as register mapping (base address and register file size), interrupt vectors, default initialization values, and computability information.

```
pictimer0: pic-timer@410f0 {
            compatible = "fsl,mpic-global-timer";
            reg = <0x410f0 0x400>;
            clk-divisor = <16>;
            clock-frequency = <600000000>;
            interrupts = <
                 0 0 3 0
                 1 0 3 0
                 2 0 3 0
                 3 0 3 0>;
        };
```

This node consists of standard data entries such as **reg** and **interrupts** as well as user defined entries such as **clk-divisor**. As long as the driver probe( ) and attach( ) methods include the necessary code to retrieve the values for the entries, there is no limitation on the type of device information that can be provided by the device tree.

The following code snippet demonstrates how the attach( ) method of this PIC timer device driver retrieves the clock divisor value and writes it into the corresponding register:

```
if((pVal = (void *)vxFdtPropGet( pFdtDev->offset, "clk-divisor", &len)) != NULL)

        pTI->clkDivisor  = vxFdt32ToCpu(*(UINT32 *)pVal);

else
        pTI->clkDivisor  = DEFAULT_CLK_DIVISOR;
```

A device tree also includes the following special nodes that the BSP must parse and handle:

root node

All device trees require a **root** node. Each device tree can have only one root node. The following code snippet describes the properties required by the ePAPR standard. The property **compatible** defines the board code (in this example, for a Freescale P2020 RDB board).

```
    #address-cells = <1>;
           #size-cells = <1>;
           model = "Freescale P2020 RDB";
           compatible = "fsl,p2020rdb";
```

memory node

The **memory** node, which is a child of the root node, describes the physical memory layout. All device trees require a **memory** node. The following example code snippet describes a **memory** node with two memory chunks:

```
    memory {
                    device_type = "memory";
                    reg = <0x00000000 0x10000000 0x30000000 0x10000000>;
                  }
```

chosen node

The **chosen** node, which is a child of the root node, describes parameters chosen or specified by the system firmware at run time. In the following example, the property **stdout-path** specifies the debug console. If **stdout-path** is not present, the board uses the first serial device.

```
  chosen {
                  bootargs = "motetsec(2,0) f=0x0 console=0,115200";
                  stdout-path = "/socp2020@ffe00000/serial@4500";
                }
```

Note: The boot loader must fix the property **bootargs** by overwriting the proper node in the DTB if the boot loader specifies the boot line.

cpus node

The **cpus** node, which is a child of the root node, contains child CPU nodes. These CPU nodes are hardware execution blocks that are sufficiently independent that they can run an operating system that does not interfere with other CPUs or other operating systems. The following example shows one CPU node. The FDT requires the properties **device_type**, **reg**, **clock-frequency**, and **timebase-frequency**. The remaining properties are specific to your implementation.

```
   cpus{
                #size-cells = <0>;
                #address-cells = <1>;
         cpu@0{
                device_type =  "cpu";
                reg = <0>;
                clock-frequency = <100000000>;
                timebase-frequency = <12500000>;
                i-cache-size = <32768>;
                d-cache-size = <32768>;
                i-cache-line-size = <32>;
                d-cache-line-size = <32>;
                };
              };
```

simple bus node

The **simple bus** node defines a bus for a system-on-a-chip processor with an internal I/O bus that cannot be probed for devices. You can access devices on the bus directly without any additional configuration in the FDT.

interrupt definition table

The **interrupt** definition table is part of the device's node. Note the following example:

```
enet0: ethernet@24000 {
  .....
  interrupts = <
     29 2 0 0
     30 2 0 0
     34 2 0 0>;
  interrupt-parent = <&mpic>;
```

In the above example:

- The first column (**29, 30, 34**) is the interrupt IRQ number.
- The second column (**2, 2, 2**) is the interrupt *group*. The group is determined by the hardware implementation of the interrupt controller. For example, when the interrupt controller is the MPIC, the interrupt vectors are divided into several groups: internal, external, pic timer, and so forth. Therefore, an interrupt is identified by a combination of pin number (or IRQ number) and group number. If the pin number is sufficient for positively identifying the interrupt on your hardware, the group is 0.
- The third column (**0, 0, 0**) is for the trigger type (edge, level, and so forth).
- The last (fourth) column (**0, 0, 0**) is for the level (active high, low, and so forth).

- The **interrupt-parent** is the interrupt controller to which the device is attached (in this example, the MPIC device).

When a device is created, the interrupt resource is created for the device and assigned to it. When the device driver calls vxbResourceAlloc( ), vxbIntConnect( ), or vxbIntEnable( ) in the attach( ) method, this information is used to configure the MPIC controller.

> Note: The device driver code only calls vxbResourceAlloc( ), vxbIntConnect( ), or vxbIntEnable( ). All of the interrupt controller configuration work is performed for the driver in the background by the VxBus FDT library and by the interrupt subsystem.

FDT Library

The VxWorks kernel for ARM, PowerPC, and RISC-V processors links against an FDT library that interprets the DTB at run time. You can download the FDT library from devicetree.org. The FDT library ships with VxWorks and resides in the following directory:

installDir **/vxworks-7/pkgs_v2/os/firmware/fdt**

Customizing a DTS File

Baseline DTS files for each ARM, PowerPC, or RISC-V board are available in the BSP directory. Occasionally, you need to customize a DTS file for additional devices on your board. A full description of the device tree data format is beyond the scope of this document. (For detailed information about device trees, refer to the ePAPR specification or go to devicetree.org). In general, start with the baseline DTS file provided for your board. Edit the file in an editor to add a device node (providing device information such as register addresses, interrupt numbers, and initialization parameters for the device).

Converting DTS Files to DTB Format

When using a separate .dtb file:

When you build or rebuild a VxWorks image project, the wrtool tool automatically compiles any DTS file in the *installDir* **/vxworks-7/pkgs_v2/os/board/***boardName* directory into a DTB file. You can load the DTB file separately with the boot loader.

When using an embedded DTB:

The DTS file will be compiled into an assembly file and included in **sysALib.s**, which is part of the kernel.

You can find the DTC tool in the following directories of your VxWorks installation:

- For a Linux host:
  installDir **/vxworks-7/host/x86-linux2/bin/dtc**
- For a Windows host:
  installDir **/vxworks-7/host/x86-win32/bin/dtc.exe**

The DTC tool is also available from devicetree.org.

VxWorks FDT API

The FDT API does not require a calling function to provide the DTB address. Some basic functions can use the standard API to make a direct call (such as to get the name of a device or a specific property). There is also an additional API you can use to perform general work (such as to get the device register address). The API can translate the register mapping for the upper level. There is also a VxBus-specific API to support operations such as creating the device register or interrupt resource for a child device. For information about the VxWorks FDT API, see VxWorks FDT API on page 28.

FDT Example Code

The following code snippets provide examples of how to implement an FDT for some typical use cases:

- Use a code snippet like the following to find a node based on a given path (in this example, to find the root node with the path **"/"**):

```
  offset = vxFdtPathOffset("/");
```

- Use a code snippet like the following to get information about a node (in this example, to get the interrupt device node for a given device node that VxBus requires). Use pHandle( ) to obtain the node.

```
offset = vxFdtNodeOffsetByPhandle(pHandleValue);
```

- Use a code snippet like the following to get a node register base for a specific node (in this example, to get information that VxBus requires to create a register resource for a new device):

```
vxFdtDefRegGet (offset, 0, &address, &size); # the second parameter specify the index
```

- Use a code snippet like the following to iterate an FDT device in the first sub-level (in this example, to scan an FDT bus driver for its child devices):

```
  for (offset = VX_FDT_CHILD(offset); offset > 0;offset = VX_FDT_PEER(offset)){  /* VxBus crea
te child device , for more
          detail, refer to the following use case, which creates the VxBus device node */
          }
```

- Use a code snippet like the following to create a VxBus child device (in this example, to create a child device for an FDT bus driver):

```
 /* Get the device basic information  */
        vxbFdtDevSetup(offset, pFdtDev);
        vxbDevNameSet(pCur, pFdtDev->name, FALSE);
        vxbResourceInit(&pFdtBusDevInfo->vxbResList);
        if (vxbFdtRegGet(&pFdtBusDevInfo->vxbResList, pFdtDev) != OK)
        {
          vxbFdtResFree(&pFdtBusDevInfo->vxbResList);
          vxbMemFree(pFdtBusDevInfo);
          vxbDevDestroy(pCur);
          continue;
        }
        if (vxbFdtIntGet(&pFdtBusDevInfo->vxbResList, pFdtDev) != OK)
        {
          vxbFdtResFree(&pFdtBusDevInfo->vxbResList);
          vxbMemFree(pFdtBusDevInfo);
          vxbDevDestroy(pCur);
          continue;
        }
        vxbDevIvarsSet(pCur, (void*)pFdtBusDevInfo);
        vxbDevClassSet(pCur, VXB_BUSID_FDT);
        vxbDevAdd(pDev, pCur);
```

- An FDT can match a device driver to a device using either of the two following scenarios:
  - A single platform driver supports multiple boards that have only minor differences (for example, the **ns16550** FDT driver supports both ARM, PowerPC, and RISC-V boards, except that the width of register are not the same on the two boards). In this case, the platform driver must supply a **VXB_FDT_DEV_MATCH_ENTRY** array that lists the supported boards. The driver then calls the score = vxbFdtDevMatch (pDev, matchTable, &pMatch) function to get a score (dependent on the matched compatible order) that allows the VxBus infrastructure to select best driver.

- A driver only supports one device (for example, the **e500 l2** cache driver, which only supports the E500 platform). In this case, you do not need a list array. Call the vxFdtNodeCheckCompatible( ) function with a compatible string to match the device.

- Use a code snippet like the following to get the value of a specific property (in this example, to get a driver-specific configuration parameter that the driver needs before it can enable the hardware). Note that not every property is standard in DTS; a user can define a private property.

```
pFdtDev = (VXB_FDT_DEV *)(vxbFdtDevGet(pDev));
       if (pFdtDev == NULL)
         return ERROR;
       pValue = vxFdtPropGet(pFdtDev->offset, "propertyName", &len);
```

2. Incrementally Adding to the Device Tree

You can add additional device specific parameters to the device tree from BSP, driver or kernel application code.

In some circumstances, it is useful to be able to add device parameters to the device tree without modifying the original DTS file. If you are not permitted to change BSP source files, but need to set specific parameters for an existing device on an application or board specific basis, you can incrementally add new device parameters to the representation of the device tree that is constructed in target memory.

Note: Even if your BSP does not use device tree, you can use this method to incrementally add devices to the VxWorks VxBus database at run time.

Add the additional device parameters to the target in three stages:

1. Declare new parameters in an array of type **VSB_PARAMS**.
2. Identify the VxBus device name and unit number that relate to the particular device declared in the DTS file.
3. Append the new device parameter values to the target by declaring VXB_DEV_PARAM_DEF ( ) in your code. This declaration uses the VxBus device name and unit number, not the DTS file device name and unit number. While VXB_DEV_PARAM_DEF ( ) looks like a function call, it is actually a declaration.

You can declare VXB_DEV_PARAM_DEF ( ) from any C source file. A good place to add this would be in the file **usrAppInit.c**, prior to the initializaton of the related VxBus driver.

Note: If you declare VXB_DEV_PARAM_DEF( ) after the VxBus driver has been initialized, then the initialized driver will not be aware of the additional device parameters you have added.

Declare the New Parameter Array

Each row in the array defines a new device parameter for a particular device and must contain the following:

- the parameter name string
- the parameter type
- the parameter value

Available parameter types are defined as macros in the header file **hwif/util/vxbParamLib.h**. You can define device parameters of the following types:

Table 1. Possible Device Tree Parameter Types

Required Additional Device Parameter Type	Parameter Type Macro
32-bit Integer	**VXB_PARAM_INT32**
64-bit Integer	**VXB_PARAM_INT64**
Character String	**VXB_PARAM_STRING**
Pointer	**VXB_PARAM_POINTER**

The final row in the table must define a **NULL** parameter of type **VXB_PARAM_END_OF_LIST**.

For example, lets assume you want to define a new parameter for two devices that already exist in the device tree called **serial0** and **serial1**. They are defined in the BSP DTS file. Each devices must have its own separate instance of the new parameter. Assuming the parameter is a single 32-bit integer device parameter called **example_parameter** with a value of 1, you would begin by declaring two arrays, one for each device:

```
#include <hwif/util/vxbParamLib.h>

    LOCAL VXB_PARAMS serial0_additionalParams[] =
        {
        { "example_parameter", VXB_PARAM_INT32, { (void *)1 } },
        { NULL, VXB_PARAM_END_OF_LIST, { NULL } }
        };

    LOCAL VXB_PARAMS serial1_additionalParams[] =
        {
        { "example_parameter", VXB_PARAM_INT32, { (void *)1 } },
        { NULL, VXB_PARAM_END_OF_LIST, { NULL } }
        };
```

Determine the VxBus Device Name and Unit Number

In order to append device parameters to a device declared in the DTS file, you must first identify the VxBus database representation that matches the original DTS file declaration of that device.

Assume that the BSP DTS file declares the target serial devices as follows:

```
    serial0: serial@11c500 {
        compatible = "fsl,ns16550", "ns16550";
        ...
    };

    serial1: serial@11c600 {
        compatible = "fsl,ns16550", "ns16550";
        ...
    };
```

In this example, the device names are **serial0** and **serial1**.

One way to establish the VxBus representation of the device details is by calling vxbDevShow( ) from the kernel shell on the running target.

```
-> vxbDevShow()
mainbus-0 at root nexus level(0)
mainbus-0: <Freescale P5020DS>
...
simpleBus-3: <simple bus controller>(0x20062ff0) level(3)
...
ns16550-0 (refs: 0) on simpleBus3 (refs:31)
ns16550-0: <ns16550 serial OF driver>(0x204f2170) level(3)
ns16550-1 (refs: 0) on simpleBus3 (refs:31)
ns16550-1: <ns16550 serial OF driver>(0x204f71c0) level(3)
```

This output shows that the following mapping exists:

Table 2. Mapping Between DTS and VxBus

Device Tree (DTS) Details	VxBus Details
serial0	**ns16550**, unit number 0
serial1	**ns16550**, unit number 1

Another way to determine the VxBus device name is to refer to the VxBus driver source code that is used by the BSP's serial driver. For example, *installDir* **/vxworks-7/pkgs_v2/os/drv/vxbus/drv/src/sio/vxbFdtNs16550Sio.c** declares the serial devices as follows:

```
VXB_DRV vxbFdtNs16550Drv =
  {
  {NULL},
  "ns16550",
  "ns16550 serial OF driver",
  VXB_BUSID_FDT,
```

Declare VSB_DEV_PARAM_DEF() to Apply Additional Device Parameters to the Target

The purpose of the VSB_DEV_PARAM_DEF( ) declaration is to append the new parameters you have defined to the serial devices referred to as **serial0** and **serial1** in the DTS file, and **ns16550** unit 1 and 2 on the target.

Make sure this declaration occurs in your code before the VxBus driver initialization happens. That way, the driver initialization code will benefit from the updated device tree.

Assuming that the device names are both **ns16550**, and the unit numbers are **0** and **1**, you will make the declarations as follows:

```
    VXB_DEV_PARAM_DEF ( ns16550, 0, serial0_additionalParams);

    VXB_DEV_PARAM_DEF ( ns16550, 1, serial1_additionalParams);
```

Note: If the original BSP DTS file also defines **example_parameter** statically for these serial devices, then the static value in the DTS file will always take precedence over the declaration VXB_DEV_PARAM_DEF ( ). You cannot use the incremental method to update the value of device parameters declared previously in the DTS file.

3. VxWorks FDT API

For ARM, PowerPC, and RISC-V developers and Intel Architecture developers using the itl_generic BSP, there is an API available that allows you to access FDT functionality when needed for your BSP and driver development.

WNDRVR

The FDT API provides a wide set of FDT functionality.

The following API functions are available to the BSP and architecture-specific drivers, and to VxBus device drivers:

vxFdt16ToCpu( )

Converts the FDT byte order (for a 16-bit value) to CPU byte order.

```
UINT16 vxFdt16ToCpu
(
  UINT16 val
)
```

vxFdt32ToCpu( )

Converts the FDT byte order (for a 32-bit value) to CPU byte order.

```
UINT32 vxFdt32ToCpu
(
  UINT32 val
)
```

vxFdt64ToCpu( )

Converts the FDT byte order (for a 64-bit value) to CPU byte order.

```
UINT64 vxFdt32ToCpu
(
  UINT64 val
)
```

vxCpuToFdt16( )

Converts the CPU byte order to FDT byte order (for a 16-bit value).

```
UINT16 vxCpuToFdt16
(
  UINT16 val
)
```

vxCpuToFdt32( )

Converts the CPU byte order to FDT byte order (for a 32-bit value).

```
UINT32 vxCpuToFdt32
(
  UINT32 val
)
```

vxCpuToFdt64( )

Converts the CPU byte order to FDT byte order (for a 64-bit value).

```
UINT64 vxCpuToFdt64
(
  UINT64 val
)
```

vxFdtPathOffset( )

Finds a tree node by its full path.

```
int vxFdtPathOffset
(
  char * pPath
)
```

vxFdtNextNode( )

Gets the next node in the FDT.

```
int vxFdtNextNode
(
 int offset,
 int *depth
)
```

vxFdtParentOffset( )

Finds the parent of a given node.

```
int vxFdtParentOffset
(
  int offset
)
```

vxFdtGetName( )

Retrieves the name of a given node. This function retrieves the name (including unit address) of the device tree node at the structure block offset node offset. If pLen is non-null, the length of this name is also returned in the integer pointed to by pLen.

```
const void *vxFdtGetName
(
  int offset,
  int * pLen
)
```

vxFdtPropGet( )

Retrieves the value of a given property. This function retrieves a pointer to the value of the property named pPropName of the node at the node offset.

```
const void *vxFdtPropGet
(
   int offset,
   char * pPropName,
   int * pLen
)
```

vxFdtIsEnabled( )

Checks that the FDT node status is set to enabled.

```
BOOL vxFdtIsEnabled
(
   int  offset
)
```

vxFdtNodeOffsetByPhandle( )

Finds the node with a given phandle. This function gets the offset of the node that has the given phandle value. If there is more than one node in the tree with the given phandle (an invalid tree), the results are undefined.

```
int vxFdtNodeOffsetByPhandle
(
 UINT32  phandle
)
```

vxFdtNodeOffsetByCompatible( )

Find nodes with a given **compatible** value. This function finds the offset of the first node from startoffset which has a **compatible** property that lists the given compatible string. If startoffset is **-1**, it returns the first node in the tree.

```
int vxFdtNodeOffsetByCompatible
(
   int startoffset,
   char *compatible
)
```

vxFdtNodeCheckCompatible( )

Checks the **compatible** property of a node against a given string.

```
int vxFdtNodeCheckCompatible
(
  int offset,
  char *compatible
)
```

vxFdtCellGet( )

Returns the cell size of a register.

```
void vxFdtCellGet
(
  int offset,     /* node offset */
  int *pAdrsCell, /* register address cell */
  int *pSzieCell  /* register size cell */
)
```

vxFdtDefRegGet( )

Gets the register address and size from the DTB.

```
STATUS vxFdtDefRegGet
(
  int          offset,      /*start node offset */
  UINT32      barIndx,     /* bar index */
  PHYS_ADDR * pRegBarAddr, /* register address */
  size_t * pRegBarSize  /* register size */
)
```

vxFdtStdoutGet( )

Obtains the **stdout** node ID.

```
int vxFdtStdoutGet(void)
```

vxFdtBootargsGet( )

Obtains the boot arguments from the DTB.

```
STATUS vxFdtBootargsGet
(
```

```
  char * pString,    /* where to copy bootargs */
  int    strLen      /* maximum number of bytes to copy  */
)
```

vxFdtStdDtb( )

When the flag of bootline is 0x1, the standard DTB is used. Otherwise, the VxWorks legacy DTB is used. vxFdtStdDtb( ) is presented to reflect this flag: `TRUE` means standard DTB and `FALSE` means legacy DTB. Drivers or other code that use DTBs should determine how to handle the DTB.

```
STATUS vxbIntCtrlMap
    (
     VXB_DEV_ID          pDev,
     UINT32            * hVec,
     VXB_INTR_ENTRY *    pVxbIntrEntry
     )
    {
    if (vxFdtStdDtb())
        return ERROR;

    /* assign hVec with right hardware interrupt vector by analyzing interrupts property in vx
bIntrEntry */

    return OK
    }
```

The following API functions are available to VxBus drivers:

vxbFdtDevMatch( )

Matches the FDT type device with a driver. The FDT describes the device using the **compatible** property. The driver also has a supported devices table that lists all supported devices. This function compares **compatible** strings and returns the minimum order number of a matched compatible string.

```
STATUS vxbFdtDevMatch
(
  VXB_DEV_ID                  pDev,          /* device to do match */
  VXB_FDT_DEV_MATCH_ENTRY *   pMatchTbl,     /* pointer to match table */
  VXB_FDT_DEV_MATCH_ENTRY **  pMatchedEntry  /* best matched entry */
)
```

vxbFdtDevGet( )

Returns the FDT bus device information.

```
VXB_FDT_DEV *vxbFdtDevGet
(
  VXB_DEV_ID pDev
)
```

vxbFdtDevSetup( )

Gets the FDT device information and saves the name and offset of the FDT device information structure.

```
void vxbFdtDevSetup
(
   int offset,
   VXB_FDT_DEV *pFdtDev
)
```

vxbFdtIntGet( )

Gets the device register information and creates the memory or I/O type resource for the device.

```
STATUS vxbFdtIntGet
(
    VXB_RESOURCE_LIST  * pResList,   /* resource list */
    VXB_FDT_DEV * pFdtDev            /* FDT device */
)
```

vxbFdtResFreeAll( )

Frees all resources from the resource list.

```
void vxbFdtResFree
(
  VXB_RESOURCE_LIST * pResList
)
```

7. VXBUS RTP-BASED DRIVERS

1. Introduction

Device drivers typically function only in the kernel context, since accessing hardware is a privileged operation. In some cases, however, it is desirable to access hardware directly from an RTP context.

Doing so places the driver code inside of a container in which it has access to a limited address space, which can mitigate the effects of bugs in the driver code. In contrast, a driver in kernel context has access to the entire kernel address space, and a bug in its code could potentially crash the entire OS. When running the driver from an RTP, its execution is restricted to a non-kernel MMU context, and bugs such as invalid memory accesses can only crash the RTP.

> Note: The MMU can't guard against a buggy driver program's hardware behaving erratically, and it is still possible for system crashes to occur.

A special RTP-based version of VxBus works in conjunction with VxBus in the kernel. This allows each RTP to create its own internal VxBus device tree and borrow devices from the kernel device tree for use with RTP-resident VxBus device drivers. The same basic VxBus facilities such as resource handling and interrupts that are available in the kernel are also available in the RTP context, allowing kernel driver code to be ported to an RTP with minimal changes.

2. Method of Operation

The kernel implementation of VxBus includes a **vxbRtpIoctlLib** module which provides RTPs with a way to borrow a VxBus device node from the kernel.

When a node is borrowed, it stays visible in the kernel-resident device tree, but is flagged so that it remains frozen in the orphaned state: even if a driver is present in the kernel which could support the device, the borrowed node remains unbound. This prevents the kernel and the borrowing RTP from trying to use the device at the same time. If the node has a kernel driver bound to it, then that driver is detached.

> Note: If detaching the driver fails, attempting to borrow the device will fail too.

When a node is borrowed, its resources are mapped to the borrowing RTP, allowing it to do the following:

- Map the device's registers into the RTP's address space
- Perform address translation operations (for setting up DMA transfers)
- Process interrupts

While there are APIs for performing the first two operations in RTPs, no other facility allows RTPs to handle interrupts. Additionally, VxBus integrates these operations into a single mechanism and handles most of the housekeeping tasks so that you do not need to develop your own solution.

The **vxbRtpIoctlLib** module implements a set of ioctl( )s that an RTP can use to borrow a device. This creates a duplicate of the VxBus object for that node within the RTP context and attaches it to the RTP's private device tree. The original node in the kernel is retained. ioctl( )s are provided to set up interrupts, proxy register accesses for certain cases where direct access might not be possible, and handle special address translation operations. Note that these ioctl( )s should not be used directly by application code: they are used internally by the RTP-resident version of **libvxbus.a**, which in turn provides an RTP-resident version of the same VxBus driver framework kernel-resident APIs and new ioctl( )s to handle device borrowing.

Once a device is borrowed, it is added to the RTP's internal VxBus device tree and is treated like any other VxBus device node. A driver added by the RTP can be bound to the node in the same way as in the kernel. In the process of borrowing the device, all of its

VXB_RESOURCE entries are duplicated and added to the copy of the node in the RTP. Mappings for memory-mapped register resources are also created so that the register ranges can be accessed from the RTP context.

When a device node is no longer needed, an RTP can return it to the kernel. Borrowed devices may also be returned automatically if an RTP terminates unexpectedly.

A device can only be borrowed by one RTP at a time. Attempting to borrow a device which has already been borrowed will result in an error since having two contexts compete for access to the same hardware would lead to erratic behavior.

It is possible to borrow a device without having to also borrow its parent. For example, if an RTP needs to claim a PCI ethernet controller device, it does not also need to claim its parent PCI bus controller. It is theoretically possible for an RTP to borrow a parent PCI bus controller device, however it would be necessary for the RTP to implement full PCI bus support and act as the parent of all child devices on that bus. This is not supported since the full VxBus PCI bus library is not compiled for user mode.

Interrupt handling is performed by proxying the interrupt events from the kernel using a task that runs within the RTP context. When a device hardware interrupt occurs, this task will be woken up to dispatch the RTP-resident driver's service routine. Note that the ISR runs in task context rather than interrupt context.

3. Typical Usage

Several steps are required in order to prepare the system for using the RTP-resident VxBus facilities.

1. The **INCLUDE_RTP** component must be included
2. The **INCLUDE_RTP_HOOKS** component must be included so that devices can be automatically returned to the kernel when an RTP terminates
3. If the project is for a 64-bit build and support for 32-bit PCI devices is required, the **INCLUDE_CACHE_DMA32_LIB** component should also be included
4. The VxWorks Source Build project (VSB) must have the **_WRS_CONFIG_VXBUS_USER** flag set, otherwise **vxbRtpIoctlLib** will not be not built.
5. The RTP project must be linked with the **libvxbus** library by adding the following to the Makefile or linker command.

```
ADDED_LIBS=-lvxbus
```

The following optional steps may also be performed before the RTP application code loads and the device is borrowed.

- If borrowing a PCI device from the kernel, a stub PCI driver is available to act as a proxy for certain PCI bus methods, notably those used to read and write PCI configuration registers. This driver should be loaded as follows:

```
[...]
#include <hwif/vxbus/vxbLib.h>
IMPORT VXB_DRV vxbPciDrv;
[...]
   vxbDrvAdd (&vxbPciDrv);
[...]
```

- If borrowing an FDT device from the kernel, a stub FDT driver is available as well, which can be enabled as follows:

```
[...]
#include <hwif/vxbus/vxbLib.h>
IMPORT VXB_DRV vxbFdtDrv;
[...]
   vxbDrvAdd (&vxbFdtDrv);
[...]
```

Once the RTP image has been compiled and linked, it can be loaded just like any other RTP. No special initialization of **libvxbus** is required as this is handled automatically using constructors. An internal VxBus device tree will be created with three nodes:

- a mainbus node (the root of the tree)
- a pciBus node.
- a fdtBus node.

The latter two nodes can be used as proxies for the PCI and FDT parent buses and use the stub drivers mentioned above. Also, a tVxbIsr task will be spawned which is responsible for dispatching driver interrupt routines.

The following two special APIs are provided for use by RTP application code:

```
STATUS vxbRtpDevAcquire
    (
    char * pName,
    UINT32 unit
    )

STATUS vxbRtpDevRelease
    (
    char * pName,
    UINT32 unit
    )
```

vxbRtpDevAcquire( )

This function is used to borrow a device from the kernel. The name and unit number should match the name and unit number of the device as currently visible in the kernel device tree. If acquiring the device succeeds, then the device will be added to the RTP's internal VxBus tree. If the device is of the class VXB_BUSID_PCI, then it will become a child of the pciBus0( ) node. If the device is of the class VXB_BUSID_FDT, then it will be added as a child of the fdtBus0( ) node. If the device is of any other class, it will become a child of the mainbus0( ) node. If the kernel is unable to loan the device, vxbRtpDevAcquire( ) will fail. This may occur if the device node in the kernel has a driver attached to it and fails to detach.

vxbRtpDevRelease( )

This function can be used to return the device to the kernel's control. This function will first attempt to call vxbDevRemove( ) to delete the child device from the RTP's own device tree. If a driver is bound to the node, this will cause the RTP-resident instance of VxBus to attempt to detach it. If this fails, then vxbRtpDevRelease( ) will fail as well. The RTP-resident driver for the device must implement shutdown and detach methods in order for vxbRtpDevRelease( ) to succeed.

If an RTP exits unexpectedly, any borrowed devices will be returned to the kernel's control by way of an RTP delete hook regardless of whether or not the RTP-resident driver for it implements shutdown/detach methods. Since the RTP is being deleted, the internal state of its private VxBus device tree does not matter.

A special linker set mechanism in the kernel is used to automatically attach all VxBus drivers that are linked into the kernel image, however this linker set mechanism is not implemented for RTPs, so a manual call to vxbDrvAdd( ) is required. To load a driver, you can simply call the vxbDrvAdd( ) API as you would in the kernel. Calling vxbDrvAdd( ) may be done at any time after the RTP starts, even before a corresponding device node is borrowed. Unlike the kernel, you cannot simply link the driver into the RTP executable. If the driver implements shutdown and detach methods, the RTP can also unload the driver by calling vxbDrvRemove( ).

The driver for a borrowed node may create additional child nodes as in the kernel, however it is not possible to arbitrarily assign them register and interrupt resources. It is theoretically possible for the driver to use APIs such as mmap( ) to create memory mapped register resources, but there is no work-around for setting up interrupts.

4. Available VxBus Facilities and Drivers

The RTP-resident version of VxBus implements the same APIs as are available from several kernel modules.

These include:

- **vxbLib.c**
- **vxbShowLib.c**
- **vxbResourceLib.c**
- **vxbParamLib.c**
- **vxbDmaBufLib.c**
- **vxbIntLib.c**

In addition, the **vxbMiiLib.c** bus-specific library is also implemented to support network devices.

All the standard VxBus facilities for manipulating device objects and drivers are available in RTP context, with the following exceptions:

- The APIs related to **vxbIoctlLib** (vxbDevLoan( ), vxbDevReclaim( ) and vxbDevReset( )) are not available. Only the kernel can loan devices.
- The APIs related to device blacklisting, vxbExcludeAdd( ) and vxbExcludeDelete( ), are not available. Since a given RTP explicitly chooses which devices to borrow from the kernel and put in its tree, the exclusion functionality is not needed.
- The APIs related to interrupt controller support such as vxbIntRegister( ) are not available.
- The vxbIntAlloc( ) and vxbIntFree( ) APIs related to dynamic interrupt support are not available.
- Support for automatic registration of drivers is not available. All drivers used in a given RTP must be explicitly registered with VxBus by calling vxbDrvAdd( ). Automatic driver registration relies on linker set support, which is only implemented for the kernel.

The following VxWorks drivers are also currently built for use in RTPs as well as in the kernel:

- **vxbGenericPhy.c**
- **vxbMicrelPhy.c**
- **vxbGei825xxEnd.c**
- **vxbFdtFslImxEnd.c**

While **libvxbus.a** provides access to hardware in an RTP and provides the infrastructure for creating drivers, it does not provide the upper level subsystem code for different classes of devices. So although it is possible for an RTP to claim, for example, a SATA storage controller device, the storage subsystem and filesystem code necessary to access attached storage devices would not be available since that code is only built for the kernel. Similarly, while an RTP may borrow an RS-232 UART device, there is no RTP-resident equivalent of **sioLib** or **tyLib**. This does not preclude the RTP from using the SATA or UART directly, but you are responsible for providing the necessary code to do so.

The one exception is networking: a portion of the MUX and END functionality is provided to allow for the creation of RTP-resident END drivers. This is largely for the benefit of the RTNET TCP/IP stack, which supports both kernel-resident and RTP-resident configurations. This is the only consumer of the RTP-resident VxBus support shipped with VxWorks.

5. RTP-based Driver Design

Drivers intended for use in RTPs are designed exactly the same way as they would be for use in the kernel.

A VXB_DRV structure describing the driver is required, along with the **probe**, **attach**, **shutdown** and **detach** methods. When the driver is attached, it can use the vxbResourceAlloc( ) API to gain access to all of the device's register and interrupt resources. Register ranges will be mapped into the RTP context using private mappings - they are visible only to the borrowing RTP.

Register Access

Device registers can always be accessed using the same vxbReadXX( )/vxbWriteXX( ) access APIs as are available in the kernel. Additionally, when a device that has memory-mapped registers is borrowed by an RTP, each bank will be mapped to the RTP's virtual address space using pmapPrivateMap( ) such that RTP-resident driver code can read and write the register banks. The vxbReadXX( ) and vxbWriteXX( ) APIs perform a system call to have the kernel proxy the register accesses, which imposes additional overhead. Consequently, direct access to the mapped regions should be used when possible if this overhead is unacceptable.

Note: There are a few cases where setting up a direct mapping for memory-mapped registers may not be possible. See VxWorks 7 Release Notes for more information.

DMA Transfers

The **vxbDmaBufLib** library is provided for RTP-resident VxBus drivers and implements the same APIs as the kernel-resident version. It is strongly advised that you use **vxbDmaBufLib** to perform DMA operations since it can handle the necessary address translation, cache synchronization and bounce-buffering operations without requiring you to implement them separately. It also allows existing code based on the kernel-resident version of **vxbDmaBufLib** to be ported to an RTP with less effort.

Some operations may be performed directly:

- mtranslate( ) can be used instead of vmTranslate( ) to perform virtual to physical address translations.
- cacheFlush( ) and cacheInvalidate( ) can be used to perform manual cache synchronization.
- cacheDmaMalloc( ) and cacheDmaFree( ) can be used to allocate uncached buffers where absolutely required.

cacheDmaMalloc( ) and cacheDmaFree( ) are implemented as compatibility routines by the **vxbDmaBufLib** library rather than by the RTP-resident version of **cacheLib**. It is recommended that you select a platform that offers bus snooping for bus master DMA devices when designing an application in which RTP-resident VxBus driver for a device that performs DMA is required, since this reduces CPU overhead.

Note: Be aware that while all multi-core platforms provide cache coherence between CPU cores, not all provide hardware cache coherence between CPU cores and DMA-capable peripheral devices. For example, many multi-core ARM SoCs don't provide bus snooping for PCI peripherals.

Interrupts

The same **vxbIntLib** APIs available in the kernel are available for RTPs, with the exception of the dynamic interrupt APIs such as vxbIntAlloc( ) and vxbIntFree( ). Interrupt handlers are registered using vxbIntConnect( ). The same strategy for creating interrupt service routines in the kernel can be used within the RTP. Note that ISRs will execute in the context of the tVxbIsr task rather than at interrupt context, and are preemptable. However, ISRs should be written to conform to the same restrictions that apply to code executing in interrupt context: ISRs should not call any APIs which would normally be forbidden in interrupt context, such as semTake( ) and taskDelay( ). Even though technically these APIs work, the resulting code would not be portable to the kernel, and could impact interrupt handling for other devices since there is only one tVxbIsr task per RTP.

Because **selectLib** provides the desired RTP/kernel notification functionality and avoids the need to create an additional system call, interrupts are implemented internally using select( ) system calls. Each interrupt resource for a borrowed device is assigned a file descriptor and the tVxbIsr task created for each RTP will pend, waiting for activity to occur. When a device interrupt triggers, the kernel will mask it off in the interrupt controller and perform a selWakeup( ) to un-pend the tVxbIsr task and then dispatch the interrupt service routine associated with the interrupt. Once the ISR returns, the tVxbIsr task will issue an "end of interrupt" ioctl( ) system call to notify the kernel that it can un-mask the vector in the interrupt controller again.

Note: The time required for the kernel to schedule the tVxbIsr task must be taken into consideration as part of the overall time required to execute an ISR when an interrupt is asserted by a device.

8. VXBUS DEVICE DRIVER INFRASTRUCTURE

1. Benefits of the VxBus Driver Infrastructure

The VxBus architecture provides a consistent device driver model that is both architecture- and BSP-independent.

VxWorks 7 introduces the VxBus Generation 2 (or VxBus GEN2) as an update and re-implementation of VxBus intended to improve the original design and implementation at a level that cannot be corrected with simple bug fixes. While the VxBus GEN2 model has certain key differences from VxBus GEN1 and is not directly source code compatible, the improvements in the new model provide significant benefits.

At its core, VxBus is intended to provide device driver developers with the following features:

- A consistent driver framework.
- The ability to organize devices in a simple topology.
- The ability to create drivers that can be loaded and unloaded on the fly.
- A mechanism for managing device resources (BARs, IRQs).
- A mechanism for managing parent/child device relationships.
- Improved driver portability.
- BSP and architecture independence.

Prior to the introduction of VxBus, VxWorks BSP developers were typically required to implement their own device management strategies that, while suitable for their specific application, were highly specific to the hardware for which they were developed. In many cases, developers ended up reinventing the same infrastructure over and over again. Therefore, even if a driver was written for a platform-independent device (for example, a PCI network adapter), the resulting driver code was so tightly coupled with the BSP, it was difficult to re-use the code on other hardware platforms.

By contrast, if you write a driver for a device such as PCI that uses the VxBus model it typically works on any target hardware platform, as long as the underlying VxBus support is correctly implemented.

Basic VxBus Concepts on page 40 There are three parts to a complete VxBus system: the VxBus core library (and some utility libraries), VxBus drivers, and VxBus class-specific helper modules.

1.1. Basic VxBus Concepts

There are three parts to a complete VxBus system: the VxBus core library (and some utility libraries), VxBus drivers, and VxBus class-specific helper modules.

The core library is the heart of VxBus while the other two modules are built on top of VxBus (the core library) and use its facilities, but are actually separate from core VxBus.

VxBus organizes device nodes in a tree, starting from a root node called the *root nexus*, and identified as **mainbus0**. Every platform has one instance of the root nexus, and all devices on the system are children or descendants of the **mainbus0** node.

Bus Versus Function Devices

A device that has children is referred to as a *bus*, while a device that has no children is called a *function* or an *edge*. However, internally VxBus GEN2 does not make any special distinction between bus and function devices. That is, a bus is just a device that has child devices. (This is different from VxBus GEN1, where buses used different data structures and had different semantics compared to devices.) The only real difference lies in the device driver for device (bus or function); a driver for a bus is responsible for creating and (optionally) destroying children.

Parent or bus devices are also required to initialize resources that are later used by their children. These resources include register bank ranges (BARs) and interrupts (IRQs). For example, a parent PCI bus driver is able to scan the resources for a child device on the bus. The parent creates a resource list and adds an entry for each BAR that the device supports, along with at least one resource for an IRQ. (Most PCI devices support at least one INTx interrupt and local bus devices have at least one interrupt vector, therefore, some have more than one.) A driver for a child device allocates the resources from this list depending on its needs. The resource structures provide handles that the driver can use for operating on these resources using various APIs.

Driver Management

Device drivers are managed by VxBus using a simple list. Drivers are matched to devices using their class identification, and using a probe mechanism. Each driver is responsible for providing a probe function for testing whether or not the driver can manage a particular device. VxBus only attempts to attach drivers to device nodes with a matching class (for example, drivers of the PCI class are matched with nodes that are also PCI class), but ultimately it is up to the driver probe function to decide if the driver can claim a device or not.

Devices and drivers have reference counts that are managed internally. Whenever a bus device instantiates a new child device, the bus device's reference count is incremented. Similarly, whenever a driver is bound to a device, the driver's reference count is incremented as well. Neither a device nor a driver can be removed until its reference count reaches 0. This prevents a parent bus device node from being removed until all of its children have also been removed, and prevents a driver from being unloaded unless it can be successfully detached from all devices to which it is currently bound.

Driver Methods

Drivers are responsible for implementing methods. At an absolute bare minimum, a driver must provide probe and attach methods (otherwise, there is no way to associate the driver with a corresponding device node). Wind River strongly recommends that drivers also have shutdown and detach methods. This allows them to be unloaded. Some classes of drivers do not really lend themselves to unloading (for example, interrupt controller drivers), but the methods should be implemented where possible.

Additional methods can be provided to implement class-specific functionality, but currently the probe, attach, shutdown, and detach methods are the only ones referenced by the core VxBus library.

Note: An optional ioctl method can be used by VxBus as well.

The method implementation allows drivers to export functions in a way that does not create explicit linker references between objects. In fact, it is possible to write a driver that has no global symbols in it. In addition to facilitating code isolation, this helps reduce namespace pollution.

Driver Loading

In VxWorks 7, loading of drivers is performed using a special mechanism known as a *linker set*. Unlike VxBus GEN1, it is not necessary for a driver to provide a registration function. Instead, a driver uses a special macro to define its driver structure as part of the VxBus driver linker set. (The component description file (CDF) for the driver ensures that there is a linker reference made to this structure.) The driver's definition structure is automatically added to the linker set by the linker scripts, and is enumerated at system startup. This removes the need to inject an explicit call to a registration function in the **prjConfig.c** file for a project.

Device Tree Population

Lastly, the mechanism used to populate the VxBus device tree is not specifically defined by VxBus. In general, the driver for the **mainbus0** node creates a child bus that creates new child devices based on some information source. This source can be the device tree source (**.dts**) file in the BSP, the ACPI device table (on Intel-based systems), or devices automatically detected on an attached PCI bus. All BSPs typically include a flattened device tree (FDT) **.dts** file which enumerates the on-board peripherals that cannot be auto-probed. A VxBus FDT bus driver creates new nodes based on the **.dts** contents, some of which can in turn create their own child nodes.

WNDRVR

Parent topic: Benefits of the VxBus Driver Infrastructure on page 40

2. VxBus Source Code Structure

Within VxWorks 7, the VxBus code is separated into two primary layers, each of which is also provided in a separate RPM.

This includes the following:

vxbus

This layer contains the following elements. These are each made individually selectable from the VxWorks Source Build (VSB) configuration tool by the layer makefile.

- core. This includes the core **vxbLib** module and utilities.
- buslib. Helper/class libraries for bus drivers.
- subsystem. VxBus subsystem libraries.

drv

Platform-independent drivers. This is a child layer to the vxbus layer.

The vxbus layer core element provides the **vxbLib** library, the generic nexus driver, and some utility libraries. All other VxBus layers and drivers are built around the vxbus layer core.

The **buslib** element includes class libraries for bus drivers, including PCI, I2C and flattened device tree (FDT). In some cases, these libraries include the bus type definition. Typically, they provide helper functions used by bus drivers to enumerate child devices on the bus as well as provide sample match methods for use in driver probe functions.

The subsystem element includes application libraries for use with certain kinds of device drivers. For example, the GPIO subsystem provides a library that GPIO drivers can use to register their device nodes. The subsystem exposes GPIO functionality to user code. This layer also includes the VxBus interrupt subsystem (**vxbIntLib**) which implements APIs used by interrupt controller drivers to register themselves with the interrupt subsystem. The layer also includes APIs for function drivers to use to allow them to make use of interrupts exposed by the interrupt controllers.

3. VxBus Device Handles and the Device Management API

Every device managed by VxBus is assigned a device handle, which is of type **VXB_DEV_ID**. To manage VxBus devices, drivers access this handle through an API provided by the VxBus core library.

The definition for the **VXB_DEV_ID** is not public; it is kept internal to VxBus and treated as an opaque handle. Drivers are not expected to directly reference any of its structure members. Instead, all access is through accessor functions provided by the VxBus library. This is intended to promote binary compatibility (although this measure alone does not guarantee binary compatibility).

An existing VxBus device node (through its driver) creates child nodes and binds them to itself. In this way, a tree of nodes can be constructed such that it reflects the actual hardware topology of the system.

The VxBus core library provides a number of APIs for device handle management. These are primarily intended to be used by drivers themselves, though a few can be used by external code. The basic device management procedure involves creating a device node and adding it to the VxBus tree. This is done with the following APIs:

vxbDevCreate( )

Initializes a node (also, optionally, allocates storage for it).

vxbDevDestroy( )

Destroys a node (and, optionally, releases storage for it.)

vxbDevNameGet( )

Return the unit address string associated with the node.

vxbDevNameAddrGet( )

Returns the unit address string associated with the node.

vxbDevNameSet( )

Set the device name (for example, **sio**, **scsi**, or **eth**).

vxbDevNameAddrSet( )

Set the unit address string associated with the node.

vxbDevPathGet( )

Return the device path for the node.

vxbDevClassSet( )

Set the device class (for example, **VXB_BUSID_FDT**, **VXB_BUSID_PCI**, and so forth).

vxbDevAdd( )

Add the device node to the VxBus tree.

vxbDevRemove( )

Remove the device node from the tree.

Adding a new device is typically only done by a bus node. Such a node has the ability to probe for new devices, and when it finds one, it adds it to the tree as a child of itself. New device nodes are always added as children of existing nodes in the tree. The vxbDevAdd( ) function requires a pointer to an existing node to act as the new device's parent. By contrast, the vxbDevRemove( ) function only requires a pointer to the device node itself (internally, VxBus already knows who its parent is).

If a node is removed with vxbDevRemove( ), and that node is a bus that has child nodes, the parent bus driver is also responsible for destroying any children in its detach method. VxBus does not remove the child nodes automatically. This is because the parent might need to release additional data structures associated with the device that VxBus cannot safely do on its own. If the parent bus is unable to successfully remove its own child devices, it must return **ERROR** from its detach method.

A parent device must use vxbDevNameSet( ) and vxbDevClassSet( ) to assign a name and class to the node prior to adding it to the tree. The class, in particular, is needed to help facilitate the driver binding process.

Be aware that after a parent assigns an initial node name using vxbDevNameSet( ), the name may change after the node is claimed by a driver. The initial name string assigned by a parent is sometimes referred to as the orphan name. The orphan name may fit a pattern specific to the type of bus that the node is on. For example, PCI devices have orhpan names of the form pci device (vvvv:dddd) where vvvv is the vendor ID and dddd is the device ID.

After a driver is attached to a device, its name string changes to match that of the driver. This change persists even after a driver is detached from a node. The vxbDevNameAddrSet( ) and vxbDevNameAddrGet( ) routines operate on what is called the unit address name string. The unit address is assigned by the parent and provides an indication of the device's location on the parent bus. The

format of the node address is a comma separated list of hexadecimal numbers. The meaning and number of values is bus-specific. For example several bus types have conventions for unit address strings as defined below.

FDT Bus Devices

FDT bus devices should have unit address values included in their names. However, VxWorks is not consistent in its use of them. A typical device node may be defined as follows:

```
        serial1: serial@2020000
             {
             compatible = "fsl,imx-sio";
             clock-frequency = <80000000>;
             clocks = <&clk 51>;
             reg = <0x02020000 0x4000>;
             interrupts = <58>;
             interrupt-parent = <&intc>;
             pinmux-0 = <&uart1_2>;
             };
```

The 2020000 portion of the name **serial@2020000** is the unit address. However, some VxWorks BSPs have **.dts** files that do not use this naming convention. There may be inconsistent use of leading zeroes and the 0x prefix. The following combinations are possible:

```
      device@a000
      device@0xa000
      device@0000a000
      device@0x0000a000
      device@0
```

If an FDT node has a unit address (for example, the node has an @ sign with a number after it), then the number is stripped of any leading zeros and the 0x prefix (if present) and the resulting value is used as a hexadecimal string.

If an FDT node does not have a unit address, but does have io= properties, then the first io= property is found and the address is converted into a hexadecimal string for use as the unit address.

If there are no io= properties, then the first reg= property is used.

If there are no reg= properties, then the FDT code falls back to using the offset value (that is, the offset into the FDT blob where the node is located) as the unit address. This approach is less than ideal because the resulting unit address depends entirely on the layout of the **.dts** file, and can change if the file is modified to add new nodes or properties. However, after the system is booted the FDT blob does not change, therefore the unit addresses should remain consistent throughout the system's lifetime.

PCI Bus Devices

PCI devices use the following format:

- DD[,F] where DD is the device number and F is the function number.

The function number will be added only for function numbers greater than 0.

ACPI Bus Devices

ACPI devices are difficult to assign bus identifiers to because the ACPI specification doesn't include an analog to the unit address. The current scheme is to use a string of the following format:

- NL,ADDR where NL is the nesting level value obtained when walking the device table (usually a number in the range of 0 to 5), and ADDR is the value of one of the address resources associated with the device.

If the device has an ACPI_RESOURCE_TYPE_IO resource (for example, an I/O port such as 0x3F8), then this value is used as the unit address. If there is no ACPI_RESOURCE_TYPE_IO resource, then the first ACPI_RESOURCE_TYPE_FIXED_MEMORY32 resource is used instead. If the device has neither kind of resource, then a value based on the order in which the device was discovered is used.

MII Bus Devices

Ethernet physical layers (PHYs) discovered via miiBusCreate( ) are assigned a unit address equal to the MII bus address that the device is strapped for. This will be some value between 0 and 31 (1f), because the MII specification only allows up to 32 devices on a single Management Data Input/Output (MDIO) serial bus.

Fallback/Compatibility

For custom bus controllers or for any cases not covered by the above conventions, **vxbLib** implements a fallback mechanism that attempts to choose a meaningful unit address to avoid collisions or ambiguity. The current scheme is as follows:

If a device is added to the tree with vxbDevAdd( ) which was not assigned a unit address via vxbDevNameAddrSet( ), then a unit address is selected based on the device's location within the parent's list of child device nodes. For example, if a parent already has two child devices, they are assigned unit address strings 0 and 1. If a third device is added, it is given the unit address string 2.

Be aware that this scheme can result in unit address values changing if devices are removed from the parent bus. If two devices are unplugged from the bus and then added back in reverse order, they will exchange unit address strings.

The unit address string and the original orphan name string are used to form a device's path name. The path name for a given node can be retrieved using vxbDevPathGet( ). The intent is that the path name should reflect the device's location in the system's hardware topology and remain relatively constant. The path name remains the same regardless of whether or not a driver is bound to a node. The path name is intended mainly to be used with the device exclusion mechanism.

The following APIs are part of the exclusion scheme:

vxbExcludeAdd( ) Adds a device to the exclusion list by path name.

vxbExcludeDelete( ) Deletes a device from the exclusion list by path name.

vxbDevLoan( ) Loans a device out of the tree when the system is running.

vxbDevReclaim( ) Reclaims a device that was previously loaned.

These APIs are used primarily to support VxWorks running as the root OS when using the virtualization profile. In this scenario, the VxWorks Root OS may be executing simultaneously with one or more additional guest partitions under control of the hypervisor. Some system devices may be allocated to guests other than VxWorks, in which case it's important that VxWorks be configured to not also access the same devices (for example, if a single system has two Ethernet devices where one device is allocated for use by the root OS and another by a guest OS). This can be done while the system is running by using the vxbDevLoan( ) function to mark a device as loaned to a different guest in the system. If the guest exits, the device can be reclaimed later for use by the VxWorks root OS by calling the vxbDevReclaim( ) function.

When in the loaned state, the device is marked as a permanent orphan which means that no driver in the root OS can attach to device even if its probe routine yields a match. If a driver is already attached to a device, calling vxbDevLoan( ) detaches it. If the detach operation fails, the attempt to loan the device to a guest also fails. Detaching a driver only succeeds when the driver correctly implements the shutdown and detach methods. The vxbExcludeAdd( ) API can be used to mark a device as loaned from the moment it is discovered, and puts the device on an exclusion list so that a driver cannot attach to the node. This is useful when the root OS includes a driver that matches the device, but the driver does not support shutdown and detach functionality.

The vxbExcludeAdd( ) function must be provided with the device's path name as returned by vxbDevPathGet( ). The vxbDevShowPaths( ) show routine displays the paths for all the nodes currently in the tree. When a device with a matching path is added to the tree with vxbDevAdd( ), it is automatically placed in the loaned state before any driver can claim it. The exclusion list entry is then removed because it is no longer needed. At this point, it is possible to use vxbDevReclaim( ) to reclaim the device for use by the root OS again, if desired. The vxbExcludeDelete( ) API can be used to manually delete an entry from the exclusion list (if it

has not already been removed automatically). Be aware that vxbExcludeAdd( ) must be called before vxbLibInit( ) in order to have any effect.

4. Driver Structure Definition and Driver Management APIs

Drivers in VxBus are defined using a **VXB_DRV** structure which includes identifying information and a method table. This structure is used to manage drivers in your VxWorks system.

An example of this is as follows:

```
LOCAL VXB_DRV_METHOD sampleMethods[] =
    {
    /* DEVICE API */
    { VXB_DEVMETHOD_CALL(vxbDevProbe), sampleProbe },
    { VXB_DEVMETHOD_CALL(vxbDevAttach), sampleAttach },
    { VXB_DEVMETHOD_CALL(vxbDevShutdown), sampleShutdown },
    { VXB_DEVMETHOD_CALL(vxbDevDetach), sampleDetach },
    { 0, NULL }
    };

VXB_DRV vxbSampleDrv =
    {
        { NULL } ,              /* Linked list header */
        "sample",               /* Name */
        "Sample VxBus driver",  /* Description */
        VXB_BUSID_FDT,          /* Class */
        0,                      /* Flags */
        0,                      /* Reference count */
        sampleMethods           /* Method table */
    };

VXB_DRV_DEF(vxbSampleDrv);
```

Here, a method table is defined containing probe, attach, shutdown, and detach methods. The list is terminated by a blank entry. The **vxbSampleDrv** driver definition includes a pointer to this table. The additional elements are:

name

The name associated for all devices claimed by this device. If a device node has a different name, it is overwritten with the one specified by the driver.

description

A string containing a brief description of the driver. This can be displayed by various show functions.

class

The driver class. When VxBus probes for devices to associate with this driver, it only considers those whose class (as set by vxbDevClassSet( )) matches this value.

flags

Driver flags. These are used to indicate any special features supported by this driver.

reference count

Used internally by VxBus to track how many device nodes are using this driver.

WNDRVR

The VXB_DRV_DEF( ) macro has a special purpose; it declares the driver structure to be part of a linker set. The linker set feature allows data to be aggregated into a special array created using the linker scripts that can be enumerated later using the datasetIterate( ) function. This lets you avoid adding code to **prjConfig.c** (or elsewhere) to manually register the driver with the VxBus system. Driver component description files (CDFs) typically include a linker reference just for the driver object (for example, **vxbSampleDrv**) to ensure that the object code is linked into the project. You do this because the VxBus method calling scheme is such that there may not be any other symbol dependencies on the driver, which would prevent the linker from adding it to the image at all.

There are essentially only two functions available for driver management, They are used to add and remove drivers to the driver list:

vxbDrvAdd( )

Add a new driver.

vxbDrvRemove( )

Remove an existing driver.

You can use these functions in your code to dynamically load new drivers at any point after system startup. This includes drivers loaded as downloadable kernel modules (DKMs). Compiled-in drivers are added automatically through the linker set feature. At boot time, VxBus uses the datasetIterate( ) function to walk the driver list and call vxbDrvAdd( ) for any drivers that are found.

You can unload drivers added at boot in this way later (assuming that they include shutdown and detach methods), though the object code remains compiled into the image. You can also add them back in again, if desired.

The vxbDrvAdd( ) and vxbDrvRemove( ) functions both operate on **VXB_DRV** handles. For example, the following command can be used to dynamically unload the sample driver defined above from a running VxWorks system using the target shell:

```
-> vxbDrvRemove (&vxbSampleDrv)
```

5. VxBus Driver Methods

To facilitate binary compatibility, VxBus uses a method calling mechanism to avoid explicit linker references to driver functions. Some methods that are used by VxBus itself, while others are used by bus-specific class libraries. Drivers can also declare custom methods.

A driver declares a table of methods that it supports. This table is declared as part of the **VXB_DRV** structure.

There are four built-in methods that VxBus always knows about: probe, attach, shutdown, and detach. An optional fifth method, ioctl, is available if the INCLUDE_VXBUS_IOCTL component is included in your VxWorks image. All of these methods return a **STATUS** value, except for the probe method, which returns an integer. This is because the probe mechanism in VxBus supports soft matches, which are described in VxBus Node and Driver Management Behavior on page 49.

Methods are declared in **.m** files, which are processed into C (**.c**) and header (**.h**) files when a VSB is compiled. The core VxBus methods are defined in the following directory:

installDir **/vxworks-7/pkgs_v2/os/drv/vxbus/core/src/vxbDevMethod.m**

Each method declaration is provided in the following format:

```
METHOD STATUS vxbMethodFunc(VXB_DEV_ID pDev) ; default[ERROR] # "Method description"
```

The method header files are added to the VxWorks source build (VSB) project directory under:

krnl/h/public/hwif/methods

In the following example, a VXB_METHOD_FUNC( ) wrapper is created which can be used to invoke the method on device handle:

WNDRVR

```
    STATUS r;

    r = VXB_METHOD_FUNC(pDev);
```

Note: VXB_METHOD_FUNC( ) is a fictitious macro, it does not exist in VxWorks.

The driver for the device that implements the method must declare it in its method table as follows:

```
LOCAL VXB_DRV_METHOD sampleMethods[] =
    {
    /* DEVICE API */
    { VXB_DEVMETHOD_CALL(vxbDevProbe), sampleProbe },
    { VXB_DEVMETHOD_CALL(vxbDevAttach), sampleAttach },
    { VXB_DEVMETHOD_CALL(vxbDevShutdown), sampleShutdown },
    { VXB_DEVMETHOD_CALL(vxbDevDetach), sampleDetach },

    /* EXAMPLE API */

    { VXB_DEVMETHOD_CALL(vxbMethodFunc), sampleMethodFunc },
    { 0, NULL }
    };
```

If your driver fails to implement the desired method, calling it on a device handle to which the driver is bound returns the value declared as the default in the method definition file (usually **ERROR**).

Use the vxbDevMethodFind( ) function provided by VxBus to test to see if a particular handle supports a specified method, and if present, return a pointer to it. For example:

```
    vxbMethodFunc_t * pMethod.
    STATUS r;

    pMethod = vxbDevMethodFind (pDev, VXB_DEVMETHOD_CALL(vxbMethodFunc));

    if (pMethod == NULL)
        {
        printf ("method not found...\n");
        r = OK;
        }
    else
        {
        r = pMethod (pDev);
        }
```

This is useful if the absence of a method in a given handle is not necessarily a fatal error. You can use it to distinguish the case where a method is not implemented versus the case where the method is implemented but calling the method generates an error.

How Callbacks Relate to Methods on page 48 VxBus provides driver-independent functions that are used to call VxBus driver methods. These methods perform the actual hardware operations. The VxWorks project facility helps provide the connection between these functions and the corresponding methods.

5.1. How Callbacks Relate to Methods

VxBus provides driver-independent functions that are used to call VxBus driver methods. These methods perform the actual hardware operations. The VxWorks project facility helps provide the connection between these functions and the corresponding methods.

For example, the vxbDmaChanControl( ) function is provided in **vxbDmaLib.c**. This function calls VXB_DMA_CHAN_CONTROL( ) method to perform the actual DMA operations. However, VXB_DMA_CHAN_CONTROL( ) is not defined directly. Rather, when you create a VxWorks source build (VSB) project, the project facility helps define the connection between the function and the method by creating the following files:

vsbDir **/krnl/h/public/hwif/methods/vxbDmaMethod.h**

vsbDir **/krnl/***arch* **/common/objvxbus/vxbDmaMethod.c**

Where *vsbDir* is the directory where you created your VSB project and *arch* is the hardware architecture you are using for your project.

The following file is used as input data for the project utility so that it knows how to generate the files:

installDir **/vxworks-7/pkgs_v2/os/drv/vxbus/subsystem/src/dma/vxbDmaMethod.m**

You must fill in the necessary code in **vxbDmaMethod.m** to describe the method you want to use.

Again, taking DMA as an example, for VXB_DMA_CHAN_CONTROL( ), fill in the following line:

```
METHOD STATUS vxbDmaChanControl(VXB_DEV_ID pDev, VXB_DMA_CHAN * pDmaChan, VXB_DMA_CTRL_CMD cmd, vo
id * pArg);
default[ERROR] # "VxBus DMA channel control method"
```

The project facility is designed to understand this language. You must take care that the string **"vxbDmaChanControl"** is only used to generate the corresponding capital letter string **"VXB_DMA_CHAN_CONTROL"**. Do not confuse it with the name used in **vxbDmaLib.c**.

Now, if you open **vxbDmaMethod.c**, you see **VXB_DMA_CHAN_CONTROL** is used as follows:

```
VXB_DEVMETHOD_DEF(vxbDmaChanControl, "VxBus DMA channel control method");
STATUS VXB_DMA_CHAN_CONTROL (VXB_DEV_ID pDev, VXB_DMA_CHAN * pDmaChan, VXB_DMA_CTRL_CMD cmd, void
* pArg)
    {
    vxbDmaChanControl_t * func = (vxbDmaChanControl_t *) vxbDevMethodFind (pDev, VXB_DEVMETHOD_CAL
L(vxbDmaChanControl));

    if (func == NULL)
        return (STATUS)ERROR;

    return func (pDev, pDmaChan, cmd, pArg);
    }
```

You can see here that VXB_DMA_CHAN_CONTROL( ) searches the method list from **vxbFslDma.c** to find the driver callback and then calls it. You can also see that **"default[ERROR]"** from the **vxbDmaMethod.m** file is used to check the pointer function.

Parent topic: VxBus Driver Methods on page 47

6. VxBus Node and Driver Management Behavior

The VxBus device driver infrastructure matches devices present in your hardware with drivers available in your VxWorks 7 system and to help you manage those pairings. You should know how the system behaves prior to starting your development.

The primary job of VxBus is to match drivers with devices, and allow devices and drivers to be easily added and removed (if needed, some application designs do not require this functionality, and certified designs may not even permit the device or driver configuration to change after boot). Consequently, VxBus applies a strategy to bind drivers and devices together which works in phases. For binding, there is a probe phase and an attach phase. The attach phase can only be reached if the probe phase is successful, and the driver binding process can only complete if both the probe and attach phases are successful.

The steps taken to bind a newly created device with a driver are outlined as follows:

1. When a new device is added with vxbDevAdd( ), VxBus looks through the list of currently available drivers in search of those with the same class as the new node.
2. When VxBus finds a driver with a matching class, it invokes the driver's probe method, passing it a pointer to the newly added device node.
3. If the driver chooses to claim the device node, it returns OK (or some non-zero positive integer), or it returns **ERROR** if it chooses not to claim it.
4. If the probe returns **OK**, VxBus considers the current driver as an explicit match for the device and the probe phase ends immediately with a successful status.
5. If the probe returns **ERROR**, VxBus moves on to the next suitable driver.
6. If the probe returns a non-zero positive integer, VxBus consider this driver a potential match. VxBus keeps track of which driver returns the lowest non-zero return value. If no driver returns **OK**, the driver with the lowest non-zero return value is considered a successful match.
7. If all probe methods return **ERROR**, the probe phase fails and the device is left as an orphan.
8. If the probe is successful, VxBus invokes the driver's attach method, again supplying a pointer to the device node.
9. The driver must then allocate resources for the device node, perform a device reset, verify that the hardware is working, (optionally) bind to an application or subsystem library, and then return **OK** if all is successful.

If the attach function returns success, the driver is considered bound to the device and the binding process ends. The driver has its reference count incremented internally to indicate that it is in use by the node.

If none of the drivers in the system claim the device (that is, all probe functions return **ERROR**), the device node is left in an orphan state. It is still part of the tree, but it remains in a non-functional state until a suitable driver claims it.

In some cases, the attach method of a driver can fail because it depends on a resource provided by another device that has not yet been initialized. VxBus provides a mechanism to handle this eventuality. Internally, VxBus implements an *attach scheduler* facility. When probing is successful, both the node and the driver are added to the attach schedule list, and the attach scheduler is invoked. The attach process is then carried out on all entries on the list. If the attach function succeeds (that is, it returns **OK**), the corresponding list entry is deleted. If the attach function returns **ERROR**, it is removed from the list (this is considered a hard failure that is not re-tried because it can lead to an infinite loop). If the attach function returns **EAGAIN**, the list entry remains in the scheduler and an attempt is made to re-try the attach process the next time a new device node is added to the tree. Once the desired resources are available, the attach function returns **OK** because it no longer needs to be rescheduled.

The probe mechanism allows for soft matches. That is, the probe method can return **OK (0)** to indicate a definite match, or it can return a positive integer value to indicate a potential match. This allows for the implementation of class drivers that can be optionally overridden by a more specific driver. For example, USB OHCI host controllers from different vendors often implement the same hardware programming interface. Therefore, you can have a single USB OHCI class driver that handles multiple devices from different manufacturers. Such a class driver can return a value of 100 from its probe function. However, if a particular USB OHCI host controller has (as an example) an issue that requires vendor-specific handling, you can provide a driver specific to that device which returns **OK (0)**. When VxBus performs its probe phase, it chooses the driver that returns either 0 or the closest positive integer to it. Therefore, if only the generic USB OHCI class driver is present, that driver claims the device, but if both the USB OHCI driver and a device-specific driver are present, the device-specific driver claims it as it is a better match.

The same probe and attach process is also performed whenever a new driver is added. In this case, the logic is inverted; rather than traversing the driver list in search of a driver that matches a given node, the node list is traversed in search of a node that matches the new driver. As before, a probe attempt is only made in cases where both the device node and the driver are of the same class.

Device Removal

When device nodes are removed using the vxbDevRemove( ) function, VxBus uses the following strategy:

1. If the device has a driver bound to it, VxBus calls the driver's shutdown and detach methods. If the driver has no such methods, or if either one of them fails, vxbDevRemove( ) returns **ERROR**.
2. If the driver is successfully detached from the device, or the device node has no driver associated with it, and its reference count is 0 (that is, it has no children of its own), VxBus removes the device handle from the parent's device list. (This also has the effect of decrementing the parent's reference count.)

For a driver to be removable, it must supply both shutdown and detach methods. A detach method alone is not sufficient. The shutdown method is intended to turn off the hardware and put it into a quiescent state. Ideally, the shutdown method does not use any blocking kernel calls, because it could be called as part of a system reset after an exception. If the device does not require shutdown semantics, the shutdown method is simply an empty function that returns **OK**.

7. Device Classes

Each device can be assigned a class (such as PCI) using the vxbDevClassSet( ) function. When VxBus attempts to bind a driver to a node, it selects only those drivers that are of the same class as the device.

For example, if a device is specified to be of class **VXB_BUSID_PCI**, then VxBus only attempts to match drivers of the same class (in this case, PCI) to the node.

```
#include <hwif/vxBus.h>

#define VXB_BUSID_CUSTOM BUSTYPE_ID(customBus)
```

The most common device classes are **VXB_BUSID_NEXUS** (for children of the **mainbus0** node). **VXB_BUSID_FDT** (for children of the **vxbFdtBus** driver), and **VXB_BUSID_PCI** (for PCI devices). **VXB_BUSID_MII** is also used for PHY drivers for Ethernet interfaces.

When a driver declares itself to be of a particular class, it is specifying that it can only service one type of device. However, in some cases there can be two separate classes of devices that are largely similar, and you may not want to have two substantially similar copies of the same code. In this case, you can use the same driver and it can declare itself to be for two different classes. Do this using two different VXB_DRV_DEF( ) instances.

For example, legacy NE2000-compatible PCI Ethernet cards are substantially compatible with their ISA bus ancestors, so much so that nearly identical same code can be used for each. The only difference can be in the driver probe functions. In this case, your driver can declare itself to be suitable for child nodes of both PCI and ISA buses using the following strategy:

```
/* Declare PCI bus attachment */

VXB_DRV vxbPciSampleDrv =
    {
        { NULL } ,              /* Linked list header */
        "samplePci",            /* Name */
        "Sample VxBus driver",  /* Description */
        VXB_BUSID_PCI,          /* Class */
        0,                      /* Flags */
        0,                      /* Reference count */
        sampleMethods           /* Method table */
    };

VXB_DRV_DEF(vxbPciSampleDrv);
```


WNDRVR

```
/* Declare ISA bus attachment */

VXB_DRV vxbIsaSampleDrv =
    {
        { NULL } ,              /* Linked list header */
        "sampleIsa",            /* Name */
        "Sample VxBus driver",  /* Description */
        VXB_BUSID_ISA,          /* Class */
        0,                      /* Flags */
        0,                      /* Reference count */
        sampleMethods           /* Method table */
    };

VXB_DRV_DEF(vxbIsaSampleDrv);
```

In this example, the same **sampleMethods** method table is used with both driver declarations. This implies that the same probe, attach, shutdown, and detach methods are used in both cases. Obviously, a PCI-based probe function that tests PCI vendor or device IDs does not work for ISA bus devices. However, the attach, shutdown, and detach methods can identical. There are two ways to address this: either provide two different method tables, where each table specifies a different probe function (one for PCI and one for ISA), or code the probe function in such a way that it tests the device node's class and takes the appropriate action. For example:

```
int myProbe
    (
    VXB_DEV_ID pDev
    )
    {
    VXB_BUSTYPE_ID class;

    class = vxbDevClassGet (pDev);

    if (VXB_BUSID_MATCH(class, VXB_BUSID_PCI)
        {
        /* do PCI-based probe */
        }
    else if (VXB_BUSID_MATCH(class, VXB_BUSID_ISA)
        {
        /* do ISA-based probe */
        }
    else
        return (ERROR);
    }
```

Note: This is a hypothetical example only. Currently, no **VXB_BUSID_ISA** class exists in VxBus GEN2, as VxWorks 7 does not support any platforms that include legacy ISA peripherals.

8. Device-Specific Data

Every VxBus device node has a **VXB_DEV_ID** structure associated with it. However, this structure is defined by VxBus itself and is only a generic handle. A given node typically requires additional device-specific context information.

Device-specific context can be allocated by two different entities: the parent driver of the device node and the device driver itself. There are two sets of functions used for attaching and detaching this data with a device handle.

vxbDevSoftcSet( )

Sets a device's software context pointer.

vxbDevSoftcGet( )

Gets a device's software context pointer.

vxbDevIvarsSet( )

Set a device's internal variables pointer.

vxbDevIvarsGet( )

Gets a device's internal variables pointer.

The vxbDevSoftcSet( ) and vxbDevSoftcGet( ) functions are intended to be used by the driver for a given device. For example, your driver for an Ethernet device may need to maintain certain interface states such as descriptor lists and producer/consumer indexes. It is customary for the driver to allocate storage for its device-specific context in its attach method and to associate it with the device handle as follows:

```
DRIVER_CONTEXT * pCtx;

    pCtx = malloc (sizeof(DRIVER_CONTEXT));
    vxbDevSoftcSet (pDev, pCtx);
```

The context handle is then recovered from the VxBus device handle at any time using vxbDevSoftcGet( ). As the driver developer, the layout of the driver-defined context structure is entirely up to you.

If your driver includes shutdown and detach support, the context must be released in the driver's detach method. For example:

```
DRIVER_CONTEXT * pCtx;

    pCtx = vxbDevSoftcGet (pDev);
    vxbDevSoftcSet (pDev, NULL);

    free (pCtx);
```

9. Bus-Specific Data

Bus device drivers are responsible for creating the bus-specific context structure that is required to manage the child devices on the bus, and for associating the context with the device.

The vxbDevIvarSet( ) and vxbDevIvarsGet( ) functions are intended to be used by the driver for a device's parent node. This is also known as *bus-specific context*, or *internal variables*. For example, if a PCI bus controller driver creates a node for a child device on the bus, then the PCI bus driver is responsible for creating the bus-specific context structure and associating it with the device. For example:

```
    BUS_CONTEXT * pCtx;
    VXB_DEV_ID pChild;
    VXB_DEV_ID pBus;

    pChild = NULL;
    vxbDevCreate (&pChild);
    vxbDevClassSet (pChild, VXB_BUSID_PCI);
```


WNDRVR

```
    vxbDevNameSet (pChild, "pcidev");
    pCtx = malloc (sizeof(BUS_CONTEXT));
    vxbDevIvarsSet (pChild, pCtx);

    vxbDevAdd (pBus, pChild);
```

Here, the bus-specific context structure contains things such as the device's PCI bus, device, and function information, as well as other information used by the parent to manage the child device.

The bus-specific context is released once the parent removes the child device node from the VxBus device tree:

```
    BUS_CONTEXT * pCtx;
    VXB_DEV_ID pChild;

    pCtx = vxbDevIvarsGet (pChild);
    vxbDevIvarsSet (pChild, NULL);
    free (pCtx);

    vxbDevRemove (pChild);
```

Wind River strongly recommends that every device of a given class use the same bus-specific context structure. For example, all children of a PCI bus (and hence the **VXB_BUSID_PCI** class) must use the same structure as each other. For the most part, only the parent driver of a node makes use of the bus-specific context. However, there are exceptions to this. For this reason, having the structure vary among different nodes in the same device class can lead to a great deal of confusion.

10. VxBus Device Resources

VxBus GEN2 introduces the concept of a **VXB_RESOURCE** abstraction for handling register banks and interrupts.

Each resource has a common **VXB_RESOURCE** parent structure and either a **VXB_RESOURCE_ADR** or **VXB_RESOURCE_IRQ** child structure. An API is provided for parent devices to manage a child device's resource list and for child devices to acquire access to resource entries.

The **VXB_RESOURCE** parent structure contains an **'id'** field, which specifies the resource's type and its unique index value, an owner field that specifies the **VXB_DEV_ID** of the device with which the resource is associated, and a unique serial number (which is used internally by VxBus). The **'id'** field is a 32-bit quantity and can be referenced using the following macros:

VXB_RES_IDX()

References the resource index value.

VXB_RES_TYPE()

References the resource type field.

VXB_RES_ID()

References the total unique identifier (a combination of the index and type fields).

VXB_RES_ID_CREATE(type, idx)

Creates an ID field using user-supplied type and index values.

The resource type identifier returned by **VXB_RES_TYPE()** is one of three values: **VXB_RES_MEMORY**, **VXB_RES_IO**, or **VXB_RES_IRQ**. The index field allows multiple resources of the same type to be associated with a single device. For example, if a device has two **VXB_RES_IRQ** resources, the first has an index value of 0 while the second has an index value of 1.

Each **VXB_RESOURCE** structure also has a **pRes** pointer that points to either a **VXB_RESOURCE_ADR** structure (if the resource type is **VXB_RES_MEMORY** or **VXB_RES_IO**) or a **VXB_RESOURCE_IRQ** structure (if the resource type is **VXB_RES_IRQ**). These sub-structures define the actual resource values.

Each device must have a resource list, which is initialized and populated by the device's parent. The parent device driver decides where the resource list is stored. However, it is typically part of the bus-specific, internal variable (**Ivar**) structure that is created by the parent bus driver. The parent initializes the resource list using the vxbResourceInit( ) function, and then adds new resources to the list using vxbResourceAdd( ). If the parent bus driver supports device removal, it can remove a given resource with vxbResourceRemove( ). VxBus provides a vxbResourceIterate( ) function that allows the parent to safely perform a list traversal.

Once resources are initialized, a child device can acquire access to them using the vxbResourceAlloc( ) function, and relinquish access using vxbResourceFree( ). A child can also acquire a pointer to the head of its resource list using vxbResourceListGet( ). These functions invoke methods that the device's parent bus driver must implement (because it is the parent that manages a child's resource list). For example, if a child device wants to access its 0th interrupt resource, it can do so as follows:

```
VXB_DEV_ID pDev;
VXB_RESOURCE * pRes;

pRes = vxbResourceAlloc (pDev, VXB_RES_IRQ, 0);
```

Device register banks come in two forms: memory mapped (which is the most common) and I/O space. I/O space registers are unique to the IA-32 and Intel 64 architectures. Intel processors support the notion of I/O **ports** that exist in a private 64 KB address space, independent of any memory-mapped addressing, and which are accessed using a special set of machine instructions. The PCI bus architecture also allows peripherals to include both memory-mapped and I/O space BARs, and the PCI specification defines both memory-mapped and I/O space transaction types that can take place between a PCI host controller and child devices. However, on non-Intel platforms that have PCI support, I/O space registers are also implemented using memory mapping because non-Intel CPUs do not support any special machine instructions for I/O. The VxBus resource model still supports the notion of I/O space resources on non-Intel platforms, which allows PCI device drivers to be implemented in a platform-independent fashion.

The **VXB_RESOURCE_ADR** structure used with register bank resources contains the physical start address of the register bank, its virtual address, and its size. The two types of BAR resources are **VXB_RES_MEMORY** and **VXB_RES_IO**.

For **VXB_RES_MEMORY** resources, the **'start'** field specifies the offset within the CPU's physical address space where the register bank is mapped, and the **'size'** field defines the bank's size in bytes. The **'virtAddr'** field defines the virtual address where the register bank is mapped. This field is only valid when the **VXB_RES_FLAG_MAPPED** bit is set in the **'id'** field of the parent **VXB_RESOURCE** structure. (This is generally set when the driver calls vxbRegMap( ).)

Due to hardware constraints, **VXB_RES_IO** resources are defined differently depending on whether the host architecture is Intel or not. For the Intel architecture, both the **'start'** and **'virtAddr'** fields are the same, and refer to the I/O port base for the device. For example, the COM1 serial port on many Intel-based machines is, by long-standing convention, typically mapped to I/O port 0x3F8. Because I/O ports exist in a non-memory space, they do not require any virtual to physical translation, hence both **'start'** and **'virtAddr'** can be equal.

For non-Intel platforms (ARM, PowerPC, and RISC-V), I/O space BARs only occur if the system supports PCI, and one or more PCI devices on the bus implements I/O space BARs. These BARs are also memory mapped on the host, even though the PCI host controller (or root complex) translates them into I/O transactions on the bus. Consequently, on non-Intel platforms, **VXB_RES_IO** resources for PCI devices have the same semantics as **VXB_RES_MEMORY** resources; the **'start'** field specifies the bank's location within the CPU's physical address space and the **'virtAddr'** field specifies its virtual mapping address.

The index assignment for the **VXB_RES_MEMORY** and **VXB_RES_IO** resources is different for PCI devices compared to others. For PCI devices, the index represents the BAR index itself. For example, PCI devices may have up to 6 BARs, which can be either memory-mapped or I/O space. The exact BAR layout depends on the specific device, and is known to the driver developer (because you have the necessary hardware documentation). As an example, assume that a given device has the following BARs:

BAR	Offset	Description
bar0	0x10	memory mapped BAR 1
bar1	0x14	I/O space BAR 1
bar2	0x18	memory mapped BAR 2
bar3	0x1C	unused
bar4	0x20	unused
bar5	0x24	unused

In order to obtain the resource handles for these BARs, you must use the following code:

```
pMemBar1 = vxbResourceAlloc (pDev, VXB_RES_MEMORY, 0);
pIoBar1  = vxbResourceAlloc (pDev, VXB_RES_IO, 1);
pMemBar2 = vxbResourceAlloc (pDev, VXB_RES_MEMORY, 2);
```

The vxbResourceAlloc( ), vxbResourceFree( ), and vxbResourceListGet( ) functions are actually wrappers around the **VXB_RESOURCE_ALLOC**, **VXB_RESOURCE_FREE**, and **VXB_RESOURCE_GET** method calls. The parent bus driver for a given function device must implement these methods. Much of the method code is likely to be common among all bus drivers. However, individual drivers may need special-case code depending on hardware requirements. The basic logic for implementing the resource allocation method is as follows:

```
LOCAL VXB_RESOURCE * sampleResourceAlloc
    (
    VXB_DEV_ID pBus,
    VXB_DEV_ID pChild,
    UINT32 id
    )
    {
    BUS_IVARS * pIvars;

    /* Get internal variables (stashed earlier, during device creation). */
    pIvars = vxbDevIvarsGet (pChild);

    /* Look up the requested resource via its ID. */
    pRes = vxbResourceFind (&pIvars->resourceList, id);

    /* No such resource */
    if (pRes == NULL)
        return (NULL);

    switch (VXB_RES_TYPE(pRes->id))
        {
        case VXB_RES_MEMORY:
        case VXB_RES_IO:
            vxbRegMap (pRes);
            break;
        case VXB_RES_IRQ:
            vxbIntMap (pRes);
            break;
```

```
        default:
            break;
        }

    return (pRes);
    }
```

A sample resource free method implementation reverses anything done by the allocation method. For example:

```
LOCAL STATUS sampleResourceFree
    (
    VXB_DEV_ID pBus,
    VXB_DEV_ID pChild,
    VXB_RESOURCE * pRes
    )
    {
    /* No such resource */
    if (pRes == NULL)
        return (ERROR);

    switch (VXB_RES_TYPE(pRes->id))
        {
        case VXB_RES_MEMORY:
        case VXB_RES_IO:
            vxbRegUnmap (pRes);
            break;
        case VXB_RES_IRQ:
            /* Currently nothing to do for IRQs. */
            break;
        default:
            break;
        }

    return (OK);
    }
```

The vxbRegMap( ) and vxbIntMap( ) functions (which are described in more detail in Device Register Access on page 57 and VxBus Interrupt Handling on page 59, respectively) are used to perform setup for the resources. In the case of registers, vxbRegMap( ) creates any virtual to physical mappings as needed (and vxbRegUnmap( ) destroys them). For interrupts, vxbIntMap( ) finalizes the linkage between the IRQ resource and the VxBus interrupt subsystem.

11. Device Register Access

VxBus provides a utility facility for reading and writing device registers. It is designed to be used in conjunction with VxBus resource handles.

The VxBus utility supports memory-mapped registers and I/O space registers (on the Intel platform). The facility provides the following functions:

vxbRegMap( )

Maps a register region to a virtual address range.

vxbRegUnmap( )

Destroys a previously-created virtual mapping.

vxbRead8( )

Reads an 8-bit register.

vxbRead16( )

Reads a 16-bit register.

vxbRead32( )

Reads a 32-bit register.

vxbWrite8( )

Writes an 8-bit register.

vxbWrite16( )

Writes a 16-bit register.

vxbWrite32( )

Writes a 32-bit register.

> Note: There are no 64-bit access functions. This is because, in general, I/O peripherals do not have 64-bit registers. (There are some timer devices on the Intel architecture that use 64-bit registers. However, it is only possible to read all 64 bits at once with an Intel 64 CPU in long mode.)

The vxbRegMap( ) and vxbRegUnmap( ) functions are provided to create virtual mappings for memory-mapped register banks. Typically, the mappings are specified in terms of physical ranges in the BSP (or in PCI BARs). Previous versions of VxWorks used an identity-mapped memory mode, where there was typically a one-to-one correspondence between a device's physical register bank address and its virtual address. This meant that drivers could use the two values interchangeably. Also, mappings were statically defined in a table in the BSP. However, VxWorks 7 introduces a new memory model where mappings are created dynamically. The OS no longer uses a simple identity mapping scheme. Consequently, the vxbRegMap( ) function must be used to create a virtual mapping window for a device's register bank before driver code can access it. It also assigns the **pHandle** value that is appropriate for the resource (**VXB_HANDLE_MEM** or **VXB_HANDLE_IO**).

> Note: On the Intel platform, I/O space register banks are not memory-mapped resources. This means they can be accessed before vxbRegMap( ) is called. However, this practice is not portable to other target hardware platforms. On the ARM and PowerPC architectures, PCI I/O space BARs are memory-mapped and you must still create a virtual mapping for them.

The read/write functions are meant to be used in place of BSP-defined functions such as sysInLong( ) and sysOutLong( ). Where possible, each of the register read/write functions is coded in platform-specific assembly language to provide the best performance. In cases where byte-swapping is needed, the functions are performed using machine-specific acceleration methods (such as load with swap and store with swap instructions on PowerPC).

The read and write methods all accept a handle that defines the type of register being accessed. This handle is set by the vxbRegMap( ) function, which is used by a parent bus driver in its vxbResourceAlloc( ) method. Similarly, vxbRegUnmap( ) is used in vxbResourceFree( ).

As an example, if you have a device that has a memory-mapped register bank and your driver wants to access it using its device handle, you can use something similar to the following code:

WNDRVR

```
    VXB_DEV_ID pDev;
    VXB_RESOURCE * pRes;
    VXB_RESOURCE_ADR * pAdr;
    UINT32 regVal;

    /* Obtain the resource handle for memory mapped BAR 0 */
    pRes = vxbResourceAlloc (pDev, VXB_RES_MEMORY, 0);
    /* Obtain the BAR info */
    pAdr = pRes->pRes;

    regVal = vxbRead32 (pAdr->pHandle, pAdr->virtAddr);
    regVal |= SOME_BIT;
    vxbWrite32 (pAdr->pHandle, pAdr->virtAddr, regVal);
```

Note: Using this register access function is recommended for maximum driver portability. This is especially useful with drivers for PCI devices, which can be used on any target architecture.

In cases where a driver is unique to a specific hardware platform, you can access register banks directly, using the **virtAddr** pointer (which is just a virtual address) as follows:

```
    UINT32 * pRegBase;
    UINT32 regVal;

    pRegBase = pAdr->virtAddr;

    regVal = pRegBase[0];
    regVal |= SOME_BIT;
    pRegBase[0] = regVal;
```

In this case, a memory barrier may be needed depending on your circumstances.

Note: This sample code assumes that no endianness fix ups are needed.

12. VxBus Interrupt Handling

Most devices generate interrupt events that must be serviced. Interrupt handling is not a core part of VxBus. However, VxBus provides a subsystem to assist with this servicing.

The VxBus interrupt subsystem includes three basic parts:

- An internal ISR management scheme and table.
- An API for interrupt controller drivers to advertise interrupts.
- An API for function (non-bus) drivers to consume interrupts.

Interrupt handling on a given platform can, in some cases, be a complicated affair with several levels. The CPU core itself may only be the last stop in a complex chain of events. The way a CPU architecture handles interrupts can vary, as can the type and number of peripheral interrupt controllers. In some cases, several interrupt controllers are cascaded together in stages. This means that while the functions provided for interrupt handling are fairly standardized, the underlying semantics depend somewhat on how interrupt controller drivers are written for a given platform.

On the PowerPC architecture, for example, the CPU has an exception handler table, of which one slot is reserved for device interrupts. (Others are used for data access traps, floating-point traps, machine checks, and so on.) When a device interrupt trap occurs, it is initially fielded by the PowerPC architecture code. At this point, the architecture layer does not know what device has

asserted the interrupt; this can only be determined by querying the interrupt controller hardware. On most PowerPC platforms, this is the EPIC or MPIC device integrated into the SoC.

The architecture trap handler must therefore invoke the interrupt controller driver's interrupt service routine (ISR). This service routine determines which interrupt vector has been triggered. From there, the interrupt controller's ISR invokes the ISR for the triggering device.

Note: The triggering device itself can actually be a second interrupt controller that is cascaded to the first one. In this case, another level of indirection is added. (One example of this is the QUICCEngine on some of the Freescale PowerPC processors.)

The architecture code always provides an interface to bind handlers to CPU traps. However, this interface varies somewhat, depending on the architecture. In VxWorks 7, the following functions are the most common situations:

- ARM: excIntConnect( )
- Intel: intArchConnect( ), intArchDisconnect( )
- PowerPC: excIntConnect( ) (or excVecSet( ))
- RISC-V: excIntConnect( )

For more information, see the VxWorks 7 architecture guide for your target processor.

For function drivers, the following functions are available, all of which make use of VxBus interrupt resource handles:

vxbIntConnect( )

Binds an ISR to a device interrupt resource.

vxbIntDisconnect( )

Unbinds an ISR from a device interrupt resource.

vxbIntEnable( )

Unmasks an interrupt resource (at the PIC level).

vxbIntDisable( )

Masks off an interrupt resource (at the PIC level).

vxbIntReroute( )

In a multicore system, routes an interrupt to a specific core.

For parent/bus drivers, the following function is also provided:

vxbIntMap( )

Creates linkage between an interrupt resource and the VxBus interrupt subsystem's ISR table.

In order for these functions to work, your driver must first acquire a handle to the desired interrupt resource. For example, the following code illustrates how to set up a single interrupt for a device:

```
VXB_DEV_ID pDev;
VXB_RESOURCE * pRes;

pRes = vxbResourceAlloc (pDev, VXB_RES_IRQ, 0);

vxbIntConnect (pDev, pRes, myIsrFunc, myIsrContext);
vxbIntEnable (pDev, pRes);
```

The converse is performed in the driver's detach function. For example:

```
vxbIntDisable (pDev, pRes);
vxbIntDisconnect (pDev, pRes);
vxbResourceFree (pDev, pRes);
```

VxBus assumes that the device itself has some mechanism for enabling and disabling generation of interrupt events. This mechanism is used to mask interrupts off and on, rather than relying on vxbIntEnable( ) and vxbIntDisable( ), because the latter only masks the interrupt on and off at the interrupt controller.

VxBus Interrupt Re-routing on page 61 By default, interrupts are triggered on CPU 0 in a multicore system. However, the interrupt can be re-routed to a different core.
VxBus Interrupt Controller Driver Management on page 61 VxBus provides an API to help you manage interrupt controller drivers and devices.

12.1. VxBus Interrupt Re-routing

By default, interrupts are triggered on CPU 0 in a multicore system. However, the interrupt can be re-routed to a different core.

The strategy for this re-routing is as follows:

```
VXB_RESOURCE * pRes; /* Interrupt resource */
cpuset_t cpus;

CPUSET_ZERO(cpus);
CPUSET_SET(cpus, 1); /* re-route to core 1 */

vxbIntReroute (pRes, cpus);
```

Currently, the vxbIntReroute( ) interface supports static interrupts on all platforms but only supports re-routing dynamically allocated interrupts such as MSI/MSI-X on the Intel architecture. In some cases, re-routing of MSI/MSI-X interrupts is not possible due to hardware constraints. For example, on the Xilinx Zynq platform (which is based on ARMv7), there is only one interrupt vector available for all PCI-based interrupts, including those triggered using MSI or MSI-X. Because of this constraint, you cannot re-route MSI/MSI-X vectors individually.

Parent topic: VxBus Interrupt Handling on page 59

12.2. VxBus Interrupt Controller Driver Management

VxBus provides an API to help you manage interrupt controller drivers and devices.

For interrupt controller drivers, the two functions provided by VxBus are vxbIntRegister( ) and VXB_INT_ISR_CALL( ). The vxbIntRegister( ) function is used during the interrupt controller driver's attach phase to allocate a portion of the interrupt subsystem's ISR table. This is usually done as follows:

```
VXB_DEV_ID pDev; /* interrupt controller device handle */
UINT32 node;     /* unique interrupt controller identifier */
UINT32 irqCount; /* number of requested IRQ slots */
UINT32 ipiCount; /* number of requested IPI slots */
UINT32 offset;   /* returned ISR table base offset */

vxbIntRegister (pDev, node, irqCount, ipiCount, &offset);
```

The returned offset value indicates the location within the interrupt subsystem's internal table where this particular controller's interrupt service routine (ISR) slots are allocated. (The table is an array, and the offset indicates the location within the array that **irqCount** + **ipiCount** slots are reserved for the controller.)

> Note: The number of IRQs to allocate is entirely dependent on your hardware. If the interrupt controller supports 256 interrupt vectors, then the driver must allocate 256 IRQ slots.

When a function driver uses vxbIntConnect( ) to bind to an interrupt, its device ISR function pointer and context pointer are saved in the ISR table slot that corresponds to its assigned interrupt vector. The interrupt controller driver is responsible for providing its own service routine that determines the hardware vector number of the triggering device and then uses VXB_INT_ISR_CALL( ) to invoke the corresponding device ISR.

For example, if you have a machine that has an interrupt controller with 256 vectors (and no IPIs), the hardware reference manual for this machine defines how the hardware vector numbers are assigned to individual devices. The hardware-defined vector numbers do not themselves have to actually be 0 to 255. They could be any values, just as long as they fall into a contiguous range of 256. (For example, the vector values could be 512 to 767.)

For this example, assume that the hardware vectors do in fact range from 0 to 255, and that vector 117 corresponds to the interrupt for the on-board network device. For this example, you want to bind an interrupt to that vector. In addition, assume that the interrupt controller has a register that can be read to determine the hardware vector number of the currently interrupting device.

In this example, the interrupt controller driver registers itself with the VxBus interrupt subsystem as follows:

```
    MY_PIC_CONTEXT * pCtx;
    UINT32 offset;

    pCtx = malloc (sizeof(MY_PIC_CONTEXT));
    vxbDevSoftcSet (pDev, pCtx);
    vxbIntRegister (pDev, node, 256, 0, &offset);

    /* Save the returned ISR table offset */
    pCtx->offset = offset;

    /* Get a handle to our register bank */
    pCtx->pRes = vxbResourceAlloc (pDev, VXB_RES_MEMORY, 0);
    pCtx->pAdr = (VXB_RESOURCE_ADR *)pRes->pRes;

    /* Attach our ISR to the CPU interrupt trap */
    excIntConnect (myControllerIsr, pDev);
```

Now, assume that the interrupt subsystem assigns this controller 256 slots starting at offset value 64. (If it is the first and only controller in the system it is more likely to be assigned offset 0, but this example assumes a non-zero value to fully illustrate the subsystem behavior.) This means that slots 64 through 320 (64 + 256) are allocated to this controller. It also means that slot 181 (64 + 117) corresponds to the network controller's interrupt.

The controller driver can then implement an ISR that looks similar to the following:

```
LOCAL void myControllerIsr
    (
    VXB_DEV_ID pDev
    )
    {
    MY_PIC_CONTEXT * pCtx;
    VXB_RESOURCR_ADR * pAdr;
    UINT32 hardwareVector;
```

```
    pCtx = vxbDevSoftcGet (pDev);
    pAdr = pCtx->pAdr;

    /*
     * Keep reading the current pending vector register until there
     * are no more interrupting devices. This assumes that interrupts
     * are stacked.
     */

    while (1)
        {
        /* Get the vector of the triggering device */
        hardwareVector = vxbRead32 (pAdr->pHandle, PIC_CURRENT_VECTOR);
        if (hardwareVector == VEC_INVALID)
            break;
        VXB_INT_ISR_CALL(pCtx->offset + hardwareVector);
        /* If end-of-interrupt action required, do it here. */
        }

    return;
    }
```

In the above code, you add the **hardwareVector** value to the base offset returned earlier by vxbIntRegister( ). You then use the result with VXB_INT_ISR_CALL( ). If **hardwareVector** is 117, and the base offset is 64, this results in the driver calling the ISR function at slot 181.

Note: An interrupt controller driver can also be a consumer of the function device API. That is, an interrupt controller can also call vxbIntConnect( ) in order to attach itself to a previously registered interrupt controller. This allows VxBus to support cascaded interrupt controllers.

In the example, assume that vector 32 is actually connected to an external I/O hub device, which itself contains an interrupt controller for its own internal devices. The driver for this secondary controller does not call excIntConnect( ) to bind to the CPU trap handler. Instead, the driver uses vxbIntConnect( ) to bind to vector 32, as supplied by the upper level PIC. It then uses vxbIntRegister( ) to allocate its own block of ISR table slots.

If the secondary PIC has 16 of its own interrupts, then the driver can do the following:

```
    MY_CASCADED_PIC_CONTEXT * pCtx;
    VXB_DEV_ID pDev;
    UINT32 offset;

    pCtx = malloc (sizeof(MY_CASCADED_CONTEXT));
    vxbDevSoftcSet (pDev, pCtx);
    vxbIntRegister (pDev, node, 16, 0, &offset);
    pCtx->offfset = offset;

    /* Get a handle for our parent interrupt */
    pCtx->pIrq = vxbResourceAlloc (pDev, VXB_RES_IRQ, 0);

    /* Connect our cascaded handler to the parent interrupt */
    vxbIntConnect (pDev, pCtx->pIrq, myCascadedIsr, pDev);
```

If the cascaded controller is initialized immediately after the parent controller, the ISR table offset is likely to be 321 (the next free slot immediately after the previously allocated block). The cascaded controller driver can then implement its own service routine, which looks similar to that of the parent controller, except that it determines the vector number of the currently interrupting device in a different manner.

In this scenario, when a device attached to the I/O hub triggers an interrupt, the following steps occur:

1. The architecture trap handler code runs.
2. The trap handler code calls the parent myControllerIsr( ) function.
3. The myControllerIsr( ) function calls the myCascadedIsr( ) function.
4. The myCascadedIsr( ) function calls the ISR for the attached device.

In addition, the VxBus subsystem includes optional support for dynamic interrupts. Most devices support at least one static interrupt (some devices have more than one). However, other devices allow for additional interrupt delivery mechanisms to be connected on the fly at run time. The most common case of this is message signaled interrupts (MSI) or extended message signaled interrupts (MSI-X) for PCI class devices. All PCI devices support the use of legacy INTx interrupts. However, those with MSI or MSI-X capability can be configured to use one or more additional message signaled interrupts. The driver for such a device enables or disables MSI/MSI-X interrupts depending on platform support and the driver design.

Adding a dynamic interrupt for a device involves two steps: allocating it from the dynamic interrupt function and creating a new **VXB_RESOURCE** structure for it. The VxBus interrupt subsystem supports two functions for dynamic interrupt handling:

vxbIntAlloc( )

Allocates one or more dynamic interrupts.

vxbIntFree( )

Releases the previously allocated dynamic interrupt(s).

The following example illustrates how to request two additional dynamic interrupt vectors for a device:

```
    VXB_DEV_ID pDev;
    VXB_RESOURCE_LIST * pList;
    VXB_RESOURCE * pRes;
    VXB_RESOURCE_IRQ * pIrq;
    VXB_DYNC_INT_ENTRY * pDynList;
    int vecCount;
    int i;

    /* We want two new vectors */
    vecCount = 2;

    pList = vxbResourceListGet (pDev);

    /* allocate the new vectors */
    pDynList = malloc (sizeof(VXB_DYNC_INT_ENTRY) * vecCount);
    if (vxbIntAlloc (vecCount, pDynList) != vecCount)
        {
        free (pDynList);
        return (ERROR);
        }

    for (i = 0; i < vecCount; i++)
        {
        pRes = malloc (sizeof(VXB_RESOURCE));
        pIrq = malloc (sizeof(VXB_RESOURCE_IRQ));

        pIrq->flag |= VXB_INT_FLAG_DYNAMIC;
        pIrq->hVec = pDynList[i].hVec;
        pIrq->lVec = pDynList[i].lVec;
        pIrq->pVxbIntrEntry = (void *)&pDynList[i];
```

```
        /* all new vectors start at index 1 */
        pRes->id = VXB_RES_ID_CREATE(VXB_RES_IRQ, 1 + i);
        pRes->pRes = pIrq;

        vxbResourceAdd (pList, pRes);
        }
```

The driver can now use vxbResourceAlloc( ) to acquire the handle for its new dynamic interrupt vectors. If the driver is designed to be unloadable, it must also use vxbResourceRemove( ) and vxbIntFree( ) to tear down and release the additional vectors as the parent bus driver does not know how to do it itself. (The parent bus can only manage static resources. Resources dynamically created by the child device must also be destroyed by the child device.)

Note: Certain subsystem helper libraries can provide utility functions for allocating and releasing dynamic interrupts. For example, the PCI library includes an MSI helper module for managing MSI and MSI-X interrupts.

Parent topic: VxBus Interrupt Handling on page 59

13. PCI Support Library

Because PCI bus support is available on many of the platforms supported by VxWorks, VxBus provides a dedicated PCI bus library.

This library provides support for probing devices on a PCI bus, for auto-configuring device BARs and IRQs (for platforms where there is no system bootstrap firmware that performs this task), and for handling MSI/MSI-X dynamic interrupt support.

While the PCI bus operation is defined by the PCI specification, the PCI controller implementation varies depending on the platform.

In order for PCI to work, you must provide a PCI bus driver. This driver is responsible for providing methods for doing PCI configuration reads and writes as well as a method for translating PCI interrupt pin values into platform IRQs. Because a PCI bus driver is also a parent of child devices, it must provide the **VXB_RESOURCE_ALLOC**, **VXB_RESOURCE_FREE**, and **VXB_RESOURCE_LIST_GET** methods.

Exactly how a PCI controller driver implements configuration space read and write operations depends on the underlying hardware. Most Intel-based systems use the same basic mechanism. However, ARM, PowerPC, and RISC-V platforms have a wide variety of differing PCI implementations. Sometimes configuration accesses are done using *keyhole* registers, while in other cases the configuration space is memory-mapped. The Intel architecture uses a keyhole register scheme for accessing the first 256 bytes of configuration space and a memory-mapped scheme for accessing the 4096-byte extended configuration space for PCIe devices.

Parent PCI bus drivers for platforms that do not include any bootstrap firmware to configure PCI devices (for example, a BIOS) must use the PCI auto-configuration kernel component. In order for this component to function, the driver must specify user-defined ranges for PCI outbound window mappings. These are the ranges used for accessing MMIO and I/O-space registers on PCI devices. To do this, your driver populates a **barRes** table with all applicable ranges and supplies it to the PCI subsystem using the vxbPciSetBarResPool( ) function.

Window range information, bus range information, and interrupt mapping information is typically specified in the BSP device tree source (**.dts**) file and can vary depending on the target board. The vxbFdtPciRanges( ), vxbFdtPciIntrInfo( ), and vxbFdtPciGetBusRange( ) functions are provided to collect this information.

Once this information is gathered, the bus is initialized by calling vxbPciAutoConfig( ). This function probed for all devices directly attached to the bus and instantiates VxBus nodes for them. If one of those devices is a PCI to PCI bridge, or a PCIe switch, the PCI bridge driver is bound to that device and any child devices of the bridge or switch are also detected and attached. This process continues until all leaf nodes of the PCI bus are reached.

Note: Currently, the PCI subsystem code assumes that the driver software context for PCI bus devices is a **PCI_ROOT_CHAIN**. PCI bus parent drivers can define additional elements in their driver-specific contexts if needed, but a **PCI_ROOT_CHAIN** structure must be the first element.

A sample PCI driver specification is as follows:

```
LOCAL VXB_DRV_METHOD myPciMetodList[] =
    /* DEVICE API */

    { VXB_DEVMETHOD_CALL(vxbDevProbe),        myProbe},
    { VXB_DEVMETHOD_CALL(vxbDevAttach),       myAttach},
    { VXB_DEVMETHOD_CALL(vxbDevIoctl),        vxbPciBusIoctl},

    /* PCI BUS API */

    { VXB_DEVMETHOD_CALL(vxbPciCfgRead),      myCfgRead},
    { VXB_DEVMETHOD_CALL(vxbPciCfgWrite),     myCfgWrite},
    { VXB_DEVMETHOD_CALL(vxbPciIntAssign),    myIntAssign},

    /* RESOURCE API */

    { VXB_DEVMETHOD_CALL(vxbResourceAlloc),   (FUNCPTR)myResAlloc},
    { VXB_DEVMETHOD_CALL(vxbResourceFree),    myResFree},
    { VXB_DEVMETHOD_CALL(vxbResourceListGet), (FUNCPTR)myResListGet},
    VXB_DEVMETHOD_END
    };

VXB_DRV myPciDrv =
    {
    { NULL } ,
    "myPciBus",                   /* Name */
    "MY PCI host bridge driver",  /* Description */
    VXB_BUSID_FDT,                /* Class */
    0,                            /* Flags */
    0,                            /* Reference count */
    myMethodList                  /* Method table */
    };

VXB_DRV_DEF(myPciDrv);
```

The following shows an example attach function for a driver for a PCI bus controller on a platform that uses a flattened device tree (FDT) for configuration:

```
typedef struct myDrvCtrl
    {
    PCI_ROOT_CHAIN    myRootChain;
    UINT32            myDrvState;
    } MY_DRV_CTRL;

STATUS myPciAttach
    (
    VXB_DEV_ID pDev
    )
    {
    struct barRes bres[PCI_ROOT_RES_MAX_IDX];
    struct fdtPciRange iospace;
```


WNDRVR

```
    struct fdtPciRange memspace;
    struct fdtPciRange preMemspace;
    UINT32 baseBus;
    UINT32 endBus;
    MY_DRV_CTRL * pDrvCtrl;
    PCI_ROOT_CHAIN * pChain;
    PCI_ROOT_RES * pRes;
    VXB_FDT_DEV * pFdt;

    pDrvCtrl = malloc (sizeof(MY_DRV_CTRL));
    pRes = malloc (sizeof(PCI_ROOT_RES));

    vxbDevSoftcSet (pDev, pDrvCtrl);

    pChain = &pDrvCtrl->myRootChain;
    pChain->pRootRes = pRes;

    /*
     * Some code is needed here to acquire the VXB_RESOURCE handle
     * for this device's register bank, which will likely be used
     * to implement the configuration read/write methods.
     */

    pDrvCtrl->myDrvState = SOME_STATE;

    /* Obtain the window range and start/end bus info from FDT/DTS */

    pFdt = vxbDevIvarsGet (pDev);

    vxbFdtPciRanges (pFdt, FDT_PCI_SPA_MEM, &memspace);
    vxbFdtPciRanges (pFdt, FDT_PCI_SPA_MEM_PRE, &preMemspace);
    vxbFdtPciRanges (pFdt, FDT_PCI_SPA_IO,  &iospace);
    vxbFdtPciGetBusRange (pFdt, &baseBus, &endBus);

    /* Set up I/O outbound window */

    bres[PCI_ROOT_RES_IO_IDX].base = iospace.basePci;
    bres[PCI_ROOT_RES_IO_IDX].baseParent = iospace.baseParent;
    bres[PCI_ROOT_RES_IO_IDX].len = iospace.len;
    bres[PCI_ROOT_RES_IO_IDX].adjust =
        iospace.baseParent - iospace.basePci;

    /* Set up MMIO outbound window */

    bres[PCI_ROOT_RES_MEM32_IDX].base = memspace.basePci;
    bres[PCI_ROOT_RES_MEM32_IDX].baseParent = memspace.baseParent;
    bres[PCI_ROOT_RES_MEM32_IDX].len = memspace.len;
    bres[PCI_ROOT_RES_MEM32_IDX].adjust =
        memspace.baseParent - memspace.basePci;

    /* Set up prefetchable MMIO outbound window */

    bres[PCI_ROOT_RES_PREMEM32_IDX].base = preMemspace.basePci;
    bres[PCI_ROOT_RES_PREMEM32_IDX].baseParent = preMemspace.baseParent;
    bres[PCI_ROOT_RES_PREMEM32_IDX].len = preMemspace.len;
    bres[PCI_ROOT_RES_PREMEM32_IDX].adjust =
        preMemspace.baseParent - preMemspace.basePci;
```

```
    /* Set up the BAR pools using the window range info */

    vxbPciSetBarResPool (pciRootRes, bres);

    /* Set up the start/end bus numbers */

    pRes->baseBusNumber = baseBus;
    pRes->endBusNumber = endBus;

    /* We need to perform autoconfiguration of all PCI devices */

    pChain->autoConfig = TRUE;
    pChain->segCount = 1;

    /* Discover devices */

    vxbPciAutoConfig (pDev);

    return (OK);
    }
```

The majority of the code in the above example is needed so that the PCI library code can assign BAR values to PCI devices for a case where there is no pre-boot firmware to perform this task.

On an Intel platform, the BIOS or UEFI firmware typically assign BARs and VxWorks can simply inherit this configuration rather than assign new values. In this case, the example is similar to the following:

```
STATUS myPciAttach
    (
    VXB_DEV_ID pDev
    )
    {
    MY_DRV_CTRL * pDrvCtrl;
    PCI_ROOT_CHAIN * pChain;
    PCI_ROOT_RES * pRes;

    pDrvCtrl = malloc (sizeof(MY_DRV_CTRL));
    pRes = malloc (sizeof(PCI_ROOT_RES));

    vxbDevSoftcSet (pDev, pDrvCtrl);

    /*
     * Some code is needed here to acquire the VXB_RESOURCE handle
     * for this device's register bank, which will likely be used
     * to implement the configuration read/write methods.
     */

    pDrvCtrl->myDrvState = SOME_STATE;

    pChain = &pDrvCtrl->myRootChain;
    pChain->pRootRes = pRes;

    pRes->baseBusNumber = 0;
    pRes->endBusNumber = 255;

    pChain->autoConfig = FALSE;
```

```
    pChain->segCount = 1;

    /* Discover devices */

    vxbPciAutoConfig (pDev);

    return (OK);
    }
```

14. Interaction between VxBus Nodes

Due to hardware complexity, some nodes on the VxBus device tree are inter-dependent including parent-child, and sibling relationships. VxBus provides support for this inter-dependence and you must account for this behavior in your VxWorks system.

Some platforms include entire device subsystems that are made up of several modules that must operate together in order to function correctly. A good example of this is the Freescale DPAA networking subsystem found in some Freescale QorIQ processors. The DPAA subsystem includes queue manager, buffer manager, and frame manager aspects which must all be configured and linked together in order to send and receive network traffic. Each of these functions is complex enough to warrant its own driver. However, this implies that there are several inter-dependent nodes in the VxBus tree.

Nodes that have a direct parent-child relationship can interact easily because a parent provides methods that its children can invoke, and vice-versa. The following APIs are provided so that parent and child nodes can interact:

vxbDevParent( )

Gets a handle to the parent of the specified node.

vxbDevFirstChildGet( )

Gets the first child device of the current node.

vxbDevNextChildGet( )

Gets the next child device of the current node.

A child can always obtain a handle for its parent node using the vxbDevParent( ) function. This function provides no locking. The assumption is that as long as a child node exists, its parent exists as well. Therefore, a child can always safely refer to its parent.

Similarly, parent nodes always maintain valid references to their child device nodes, right up until they destroy them. Until that point, the parent can use the vxbDevFirstChildGet( ) and vxbDevNextChildGet( ) functions to iterate over their list of children. (You can also chose to maintain an internal list of child nodes as part of the parent node's driver as well, though you must be careful to keep it synchronized with the actual list of children as maintained by VxBus itself.)

Nodes that do not have direct parent-child relationships, such as sibling nodes, present more of a challenge. In theory, you can design your system such that there are no dependencies between sibling nodes. If one node has a dependency on another, then you can try to place the dependent node further down the device tree. In practice, this is not always possible due to the design of the hardware. For example, with the Freescale DPAA subsystem, the **Qman**, **Bman** and **Fman** components are all child nodes of the flattened device tree (FDT) bus. Also, the **Fman** depends on both the **Qman** and the **Bman** components, and it cannot be a child of both.

Consequently, three additional functions are provided for sibling nodes:

vxbDevAcquire( )

Acquires an extra reference to a VxBus node.

vxbDevAcquireByName( )

Locates a VxBus node in the tree using its name and unit number, and returns a pointer to it, acquiring an extra reference in the process.

vxbDevAcquireBySerial( )

Locates a VxBus node in the tree using its serial number and returns a pointer to it, acquiring an extra reference in the process. The serial number for a node can be obtained using vxbDevSerialGet( ).

vxbDevAcquireByPath( )

Locates a VxBus node in the tree using its path name, and returns a pointer to it, acquiring an extra reference in the process. The path name for a node can be obtained using vxbDevPathGet( ).

vxbDevRelease( )

Releases a reference to a VxBus node.

The vxbDevAcquire( ) function simply increments the reference count on a VxBus node handle. VxBus itself increments and decrements reference counts on nodes internally as child devices are added to it, but this function is used to tell VxBus that there are external dependencies on the node as well.

For example, a driver for a storage device must register the device node with the storage subsystem in its attach function so that it can be used in later I/O requests. The vxbDevAcquire( ) function must be used to indicate that the storage subsystem also has a pointer to the device. If this is not done, the device can be removed from the VxBus tree, and the storage module then has a stale pointer. Incrementing the reference count on the node prevents it from being removed.

The complementary vxbDevRelease( ) function decrements the reference count. This is done once the additional external reference to the device node is cleared.

The vxbDevAcquireByName( ) function locates a particular device within the VxBus tree, using its name and unit number. For example:

```
pDev = vxbDevAcquireByName ("gei", 0);
```

This safely locates the matching device node and increments its reference count.

The vxbDevAcquireBySerial( ) and vxbDevAcquireByPath( ) functions can also be used to look up nodes in the tree, and also incrememt a node's reference count when a match is found.

Note: These functions are analogous to the vxbInstByNameFind( ) function in VxBus GEN1, except every call to vxbDevAcquireByName( ), vxbDevAcquireBySerial( ) or vxbDevAcquireByPath( ) must eventually be accompanied by a call to vxbDevRelease( ).

These functions must be used sparingly because VxBus is not able to manage the resulting external reference counts that they create. In the worst case, overuse can cause a driver with unload functionality to become non-unloadable due to unforeseen dependencies between devices.

15. Static Device Configuration

VxBus GEN 2 uses the flattened device tree (FDT) model as a starting point from which to begin its discovery of devices. If your hardware permits it, device discovery can be driven by auto-configuration but, initially, a static configuration mechanism is required.

The FDT model is specified at http://devicetree.org. VxWorks 7 includes the FDT library for parsing FDT *blobs* as well as the host side **dtc** compiler for converting device tree source (**.dts**) sources into binary (**.dtb**) files. The FDT information can be obtained from U-Boot or can be compiled directly into VxWorks. (The case where the FDT information is directly linked into VxWorks is analagous to the use of the **hwconf.c** file in VxBus GEN1.)

VxWorks includes a **vxbFdtBus** driver that is used as the starting point for device discovery. The **vxbFdtBus** driver creates a child node for every device specified in the FDT structure, which is of class **VXB_BUSID_FDT**. The device name and unit number are set according to the name and unit number specified in the FDT. Driver probe functions match against nodes using their name, or by using the vxbFdtDevMatch( ) helper function, which checks for the **compatible** property in the underlying FDT node.

16. VxBus Driver Tunables

In some cases, you want to have the ability to customize driver behavior on a per-device basis from your BSP. VxBus provides a driver parameter system that allows drivers to advertise tunable parameters that can be altered on the fly at run time

The defaults for these tunables can be configured from the BSP using the device tree source (**.dts**) file.

Your driver can optionally provide a parameter table as part of its **VXB_DRV** structure. The driver must also set the VXB_DRVFLAG_PARAM parameter in its flags field to indicate that the parameter table is present. The following example shows the **vxbSampleDrv** driver with a single optional parameter:

```
LOCAL VXB_DRV_METHOD sampleMethods[] =
    {
    /* DEVICE API */
    { VXB_DEVMETHOD_CALL(vxbDevProbe), sampleProbe },
    { VXB_DEVMETHOD_CALL(vxbDevAttach), sampleAttach },
    { VXB_DEVMETHOD_CALL(vxbDevShutdown), sampleShutdown },
    { VXB_DEVMETHOD_CALL(vxbDevDetach), sampleDetach },
    { 0, NULL }
    };

    LOCAL VXB_PARAMS sampleParams[] =
        {
        { "example", VXB_PARAM_INT32, { (void *)1 } },
        { NULL, VXB_PARAM_END_OF_LIST, { NULL } }
        };

VXB_DRV vxbSampleDrv =
    {
        { NULL } ,              /* Linked list header */
        "sample",               /* Name */
        "Sample VxBus driver",  /* Description */
        VXB_BUSID_FDT,          /* Class */
        VXB_DRVFLAG_PARAM,      /* Flags */
        0,                      /* Reference count */
        sampleMethods,          /* Method table */
        sampleParams            /* Parameters */
    };

VXB_DRV_DEF(vxbSampleDrv);
```

Two functions are provided for accessing parameters at run time: vxbParamGet( ) and vxbParamSet( ). Your driver can use vxbParamGet( ) to access the current value of a given parameter as follows:

```
VXB_PARAM_VALUE val;

vxbParamGet (pDev, "example", VXB_PARAM_INT32, &val);
printf ("parameter value is: %d\n", val.int32);
```

The default value for a parameter can be overridden from the BSP by creating a **devparam** section in the **chosen** clause of the **.dts** file. For example:

```
chosen {
    bootargs = "<default bootargs string>";
    devparam {
        sample@0 {
            example = <99>;
        };
    };
}
```

Multiple parameters can be specified in the same block for a given device node.

9. DEVICE-SPECIFIC DRIVERS

1. About Device-Specific Drivers

All VxBus device drivers include certain common features such as implementation of the four basic VxBus methods, a common detection, probing, and initialization sequence, access to common resources, and so forth. However, device drivers that support certain device types have unique requirements.

Every device class has certain unique details. When developing a new device driver for a specific device type, you must be aware of the specific driver requirements for that device type. You must also be aware of the common driver requirements (see Benefits of the VxBus Driver Infrastructure on page 40).

The following device classes have specific requirements:

- Clock on page 73
- DMA on page 82
- GPIO on page 111
- I2C on page 119
- Interrupt Controller on page 124
- PCI and PCIe on page 147
- Pin Mux on page 180
- Serial on page 185
- SPI on page 189
- Storage on page 193
- Timer on page 211

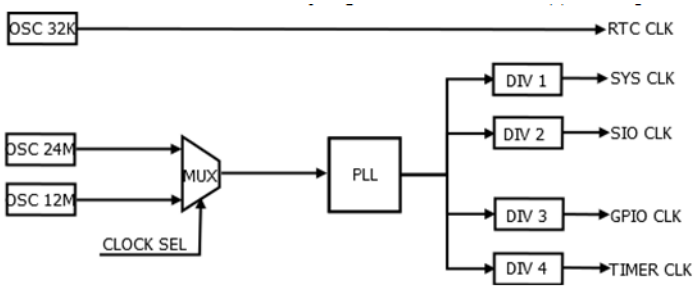
Note: Network device drivers are documented in the VxWorks 7 Network Driver Developer's Guide.

2. Clock Drivers

2.1. Overview

In a typical SOC (system on chip), devices and I/O are normally driven by various clock sources.

Those clock sources are normally organized as clock tree (s) as shown below:



Each OSC can act as a root of a tree. Once the clock source selector (MUX) selects a parent clock source, a new sub clock tree is created.

Related information
Header Files on page 74
Device Tree Binding on page 74
Initialization on page 75
Structures and Required Methods on page 75
Supplied Clock Functions on page 78
Clock Controller Drivers on page 79
Client Device Driver Usage on page 81

2.2. Header Files

VxBus clock driver header files are in the **vxbClkLib.h** file.

The header file for clock drivers is located here: *installdir* **/vxworks-7/pkgs_v2/os/drv/vxbus/subsystem/h/subsys/clk/vxbClkLib.h**.

Related information
Overview on page 73
Device Tree Binding on page 74
Initialization on page 75
Structures and Required Methods on page 75
Supplied Clock Functions on page 78
Clock Controller Drivers on page 79
Client Device Driver Usage on page 81

2.3. Device Tree Binding

Clock devices are part of the device tree.

There are various types of clock devices that require a different list of properties. An example of a 'clock source' device (a clock that drives another device/devices) description in a DTS file is shown below:

```
clk: prcm@0x44E00000
{
  /* this device provide clk control, device control */
  compatible = "ti,am3prcm";
  reg = <0x44E00000 0x2000>;
  #address-cells = <1>;
  #size-cells = <0>;
  #clock-cells = <1>;
  ……
}
```

The following property is required when defining a VxBus clock source device in the DTS file:

- **#clock-cells**: the number of clocks it supplies. The value is 0 if it only provides a single clock. The value is 1 if it provides more than a single clock.

The following property is optional when defining a VxBus clock source device in the DTS file:

- **clock-output-names**: a list of strings of clock output signal names from this clock source. This describes to the user what clock outputs are supported by this clock.

  clock-output-names should NEVER be directly referenced by user the node.

'clock users' are devices that use a 'clock source'. A clock user can be any device node, and it can be a clock user and a clock source simultaneously (it is common that a clock consumes a clock and outputs other clocks). An example of a 'clock user' device description in a DTS file is shown below:

```
dmtimer3: dmtimer@48042000
{
  ……
  clocks = <&clk 113>, <&clk 114>;
  clock-names = "ick", "fck";
  ……
};
```

In this example **dmtimer3** uses two clock inputs, one named **ick**, with a clock index of 113, the other named **fck** , with a clock index of 114.

The following property is required when defining a VxBus clock user device in the DTS file:

- **clocks**: a list of clock source phandle and clock parameter pairs; one pair represents one clock. If the **#clock-cell** property from the clock source is 0, then there is no parameter present in the list.

The following properties are optional when defining a VxBus clock user device in the DTS file:

- **clock-names**: a list of strings present in the same order as the 'clock' property. A device driver may use this **clock-names** property to get the clock handle.

  If there is only one clock referenced, there is no need to specify the **clock-names** property, the clock system will automatically select that one clock.
- **clock-ranges**: indicates which child nodes can inherit the clocks and **clock-names** property from this node.

Related information

Overview on page 73
Header Files on page 74
Initialization on page 75
Structures and Required Methods on page 75
Supplied Clock Functions on page 78
Clock Controller Drivers on page 79
Client Device Driver Usage on page 81

2.4. Initialization

Since many devices require a clock source, the clock device has to be initialized before the devices that will use it as a source.

Be sure to carefully arrange your device node order so that any required clocks are initialized before the devices that will use them as a source.

Related information

Overview on page 73
Header Files on page 74
Device Tree Binding on page 74
Structures and Required Methods on page 75
Supplied Clock Functions on page 78
Clock Controller Drivers on page 79
Client Device Driver Usage on page 81

2.5. Structures and Required Methods

This chapter lists the required data structures and methods required by VxBus and by the clock subsystem

Clock Structures

The clock subsystem uses the following clock structure to manage a clock:

```
typedef struct vxb_clk
    {
    DL_NODE                     clkNode;
    DL_NODE                     rootClkNode;
    char*                       clkName;
    UINT32                      clkStatus;
    UINT64                      clkRate;
    UINT32                      clkRefs;
    VXB_CLK_TYPE                clkType;      /* type of this clock */
    char*                       parentName;   /* dynamic register need this */
    char**                      parentNames;  /* dynamic register need this */
    struct vxb_clk*             parentClock;
    struct vxb_clk**            parentClocks;
    UINT32                      parentNum;    /* how many parent */
    UINT32                      parentIdx;    /* current parent's index */
    DL_NODE                     parentNode;
    DL_LIST                     childClkList;
    struct vxb_clk_domain*      clkDomain;
    struct vxb_clk_funcs *      clkFuncs;
    VXB_DEV_ID*                 pDev;         /* clock controller’s device handle */
    }VXB_CLK;
```

For drivers that use statically defined clocks as their clock source, the clock library also provides the following statically defined macro:

```
#define DEFINE_VXB_CLOCK(name, rate, parent, clkdom, funcs, ctx)\
LOCAL struct vxb_clk name =             \
    {                                   \
    {NULL, NULL},  /* clkNode */        \
    {NULL, NULL},  /* rootClkNode */    \
    #name,         /* clkName */        \
    rate,          /* clkRate */        \
    0,             /* clkStatus*/       \
    0,             /* clkRefs */        \
    0,             /* clkType */        \
    NULL,          /* parentName */     \
    NULL,          /* parentNames */    \
    parent,        /* parentClock */    \
    NULL,          /* parentClocks */   \
    1,             /* parentNum*/       \
    0,             /* parentIdx*/       \
    {NULL, NULL},  /* parentNode */     \
    {NULL, NULL},  /* childClkList */   \
    clkdom,        /* clkDomain */      \
    funcs,         /* clkFuncs */       \
    (void *)ctx,   /* clkContext */     \
    NULL           /* Pdev */           \
    };
```

Note: For clock drivers that use a device tree to represent the clock topology, the driver can call vxbClkRegister( ) instead of using this macro.

Required Methods

The following structure defines all required methods the driver should supply:

```
typedef struct vxb_clk_funcs
    {
    STATUS (*clkEnable)          /*enable the clock hardware */
        (
        struct vxb_clk * clk
        );
    STATUS (*clkDisable)         /* disable the clock hardware */
        (
        struct vxb_clk * clk
        );
    STATUS (*clkRateSet)         /* set the clock rate */
        (
        struct vxb_clk * clk,
        UINT64 rate
        );
    UINT64 (*clkRateGet)         /* get the clock rate */
        (
        struct vxb_clk * clk,
        UINT64 parentRate
        );
    UINT32 (*clkStatusGet)       /* get the clock status */
        (
        struct vxb_clk * clk
        );
    STATUS (*clkParentSet)       /* choose a parent clock */
        (
        struct vxb_clk * cclk,
        struct vxb_clk * pclk
        );
    STATUS (*clkInit)            /* hardware initialization */
        (
        struct vxb_clk * clk
        );
    STATUS (*clkExtCtrl)         /* clock exit call back */
        (
        struct vxb_clk * clk,
        UINT32           ctrl,
        void *           context
        );
    }VXB_CLK_FUNC;
```

Related information

Overview on page 73
Header Files on page 74
Device Tree Binding on page 74
Initialization on page 75
Supplied Clock Functions on page 78
Clock Controller Drivers on page 79
Client Device Driver Usage on page 81

2.6. Supplied Clock Functions

There are several functions available to you for use in your clock drivers and in other drivers that require a clock source.

Functions for Clock Drivers

The following functions are available for use in your clock driver:

vxbClkRegister( )

Registers a clock that uses a device tree to represent relationships. In contrast to static clocks, the clock topology is defined in the device tree and is initialized when the clock registers.

vxbClkDomainRegister( )

Registers a clock domain that uses a device tree to represent relationships. In contrast to static clock domain, the domain contents are defined in the device tree and initialized when the domain registers.

vxbClksInit( )

Initializes statically defined clocks. The topology is decided when the clocks are defined. All clocks must be present as an array and end with a NULL clock pointer.

vxbClkDomainInit( )

Initializes statically defined clock domains.

vxbClkStaticDataSet( )

Sets the statically defined clock data pointers so that static clock data can be defined by the BSP instead of binding to the clock driver.

vxbClkStaticDataGet( )

Gets the statically defined clock data pointers. These pointers are used by vxbClkRegister( ) and vxbClkDomainRegister( )

Functions for Drivers that Require a Clock Source

The following functions are available to drivers that require a clock source:

vxbClkGet( )

Gets a clock handle by using the user's device instance and clock name. If the clock name is NULL, this function gets the first clock in the **clocks** property. (See Device Tree Binding on page 74.)

vxbClkEnable( )

Enables a clock by handle. This function also enables the parent clock(s), if necessary.

vxbClkDisable( )

Disables a clock by handle. This function also disables the parent clock(s), if necessary.

vxbClkStatusGet( )

Gets a clock status using the handle. For example, it shows if a clock is enabled.

vxbClkRateGet( )

Gets the specified clock's rate. The clock rate is dynamically calculated by the clock driver.

vxbClkEnableAll( )

Enables all clocks present in the device node. In contrast to the standard vxbClkEnable( ) function, this function does not require a clock handle as a parameter. This function is useful for devices that enable all clocks when they start.

vxbClkDisableAll( )

Disables all clocks present in the device node. In contrast to the standard vxbClkDisable( ) function, this function does not require a clock handle as a parameter. This function is useful for devices that disable all clocks when they shut down.

Related information

Overview on page 73
Header Files on page 74
Device Tree Binding on page 74
Initialization on page 75
Structures and Required Methods on page 75
Clock Controller Drivers on page 79
Client Device Driver Usage on page 81

2.7. Clock Controller Drivers

The VxWorks clock system supports two types of clock driver. The *static clock driver* is a clock that is defined in code. The *dynamic clock driver* drives all clocks that are present in the device tree.

This section provides guidelines on how to implement clock controller drivers (both static and dynamic). The clock driver parses the device tree and registers clocks according to the configurations available in the device tree at run time.

Static Clock Driver

For the static clock driver, all clocks and domains must be statically defined using the macros **DEFINE_VXB_CLOCK** and **DEFINE_VXB_CLOCK_DOMAIN**.

The following is an example of a static clock definition:

```
DEFINE_VXB_CLOCK (dpll_mpu_ck, 0, &sys_clkin_ck, NULL, NULL, &dpll_mpu_ck_data);
```

All static clocks should be added into the following **NULL** terminated clock pointer array:

```
static VXB_CLK * clocks [] =
{   ……
    &dpll_mpu_ck       ,
    ……
    NULL
};
```

Once all clocks and domains are defined, a driver can connect them by calling the vxbClksInit( ) routine. This call is typically a part of the driver's attach routine. For example:

```
myDrvAttch()
{
  ....
  ....
  vxbClkDomainInit (domains);
  vxbClksInit(pDev, clocks, NULL);
```

```
  ....
}
```

Dynamic Clock Driver

For the dynamic clock driver, all clock information is provided in the device tree. The following information is required:

Clock Name

The clock's name, which may be omitted by using the name of the flattened device tree (FDT) node for the clock.

Parent List

The list of all possible parents for the clock.

Domain (optional)

If the clock belongs to a domain, present its domain name.

Type

Indicate which type of clock this (for example, a MUX clock, a divider, or a fixed rate clock). This information is parsed from the clock's **compatible** property.

Data

Different clocks need different data. For example, for a fixed rate clock, the required data is rate. For a divider, it is the ratio. Data can also include the clock's register address and so forth.

The following example is for various clock definitions in a device tree (**.dts** file):

```
osc: refclk
        {
          #clock-cells = <0>;
           compatible ="fixed-clock";
           clock-frequency = <100000000>;
           clock-output-names = "refclk-main";
        };
       pllclk: pllclk@1100000
        {
           #clock-cells = <0>;
           compatible = "someone, pll-clk";
           clocks = <&osc>;
           reg = <0x1100000 4>;
           mask = <0x3f>;
           value = <2>;
        };
       muxclk: muxclk@1200000
        {
          #clock-cells = <0>;
          compatible = "someone,pll-mux-clk";
          clocks = <&pllclk>, <&osc>;
          reg = <0x1200000 4>;
          shift = <23>;
          width = <1>;
        };
```

The **osc** is a fixed rate clock source with a rate of 1000 Mhz. The **pllclk** uses the **osc** as its parent (source clock). The **pllclk** outputs a clock with a different rate (this is a PLL clock).

The **muxclk** is a MUX (a clock selector). It has two possible parents, **pllclk** or **osc**.

A clock driver decides which parent to use. Once the driver parses the clock node, it connects it into the clock system by calling vxbClkRegister( ). The parameter is a structure of type **VXB_CLK_REG**. The driver fills the structure with the information from the FDT as shown in the following code snippet:

```
VXB_CLK_REG pClkReg = vxbDevMalloc(sizeof (*pClkReg));
  /* do some FDT magic to fill thispClkReg */
  vxbClkReg (pClkReg);
```

Related information

Overview on page 73
Header Files on page 74
Device Tree Binding on page 74
Initialization on page 75
Structures and Required Methods on page 75
Supplied Clock Functions on page 78
Client Device Driver Usage on page 81

2.8. Client Device Driver Usage

Once a clock device driver is loaded and running, other device drivers can call the clock system functions (in the clock subsystem) to operate the clocks.

In Device Tree Binding on page 74, the **dmtimer3** clock was used as an example. It has two clocks **ick** and **fck**. In the **dmtimer** device driver, you can enable both clocks as follows:

```
VXB_CLK* pClk;
pClk = vxbClkGet(pDev, "ick");     /* get the clock handle */
vxbClkEnable(pClk);               /* enable the clock*/
pClk = vxbClkGet (pDev, "fck");
vxbClkEnable(pClk);
```

Alternatively, the clock user driver can call vxbClkEnableAll(pDev);( ) to enable all clocks in this device node (where **pDev** is the device node).

If your clock user driver wants to obtain the clock rate, use the following code:

```
UINT64 freq;
freq = vxbClkRateGet (pClk);
```

Related information

Overview on page 73
Header Files on page 74
Device Tree Binding on page 74
Initialization on page 75
Structures and Required Methods on page 75
Supplied Clock Functions on page 78
Clock Controller Drivers on page 79

3. DMA Drivers

WNDRVR

3.1. DMA Driver Overview

General-purpose direct memory access (DMA) engines that handle DMA accesses from, or to, external devices or from and to memory are included in some hardware designs and are often integrated in system-on-chip CPU designs. VxBus provides support for DMA devices.

DMA Overview

The DMA driver class provides a standard method for presenting the services of DMA engines to other drivers in the VxWorks system. It is composed of three elements:

- the DMA controller driver interfaces to the hardware DMA engine.
- the DMA client driver may be, for example, a serial or network or file system driver. It requests DMA actions to be performed that will ultimately be actioned by the appropriate DMA controller driver.
- the DMA library acts as a bridge between the DMA client and controller drivers.

The controller driver registers a DMA device instance using the DMA library API. The client driver requests service using the library API, and the library subsequently instructs the controller driver to take specific actions.

The DMA library is implemented in the file *installDir* **/vxworks-7/pkgs_v2/os/drv/vxbus-***a.b.c.d* **/subsystem/src/dma/vxbDmaLib.c**. Two types of API is implemented here. One API is used by DMA controller driver, the other by the DMA client driver.

Note: The VxBus layer version number is represented by *a.b.c.d*.

DMA Controller Driver

The controller driver registers an instance into the DMA driver by calling the library API function vxbDmaRegister( ). DMA instances are also unregistered by calling vxbDmaUnregister( ).

The DMA controller driver either allows the DMA library to manage channel resources, or it manages them itself. To manage resources via the DMA library, the controller driver must call vxbDmaChanAdd( ) to register the channel resources with the DMA library. Alternatively, to manage resources in the controller driver itself, the driver must implement a **VXB_DMA_CHAN_ALLOC** method in the DMA controller driver source code. The DMA controller driver must then be coded to manage channel resources without help from the DMA library.

Related information

DMA Library API for DMA Controller Drivers on page 82
DMA Library API for DMA Client Drivers on page 84
DMA Buffer Library API on page 90
VxBus Driver Methods for DMA Drivers on page 103
DMA Driver Header Files on page 105
DMA Driver Configuration on page 106
Available Utility Functions for DMA Drivers on page 107
DMA Driver Initialization on page 107
Initiating a DMA Transfer In a Client Driver on page 107
DMA Client Driver on page 108
DMA Client Driver With Error Status Check on page 109
Debugging DMA Drivers on page 111

3.2. DMA Library API for DMA Controller Drivers

The DMA library implements APIs that are used by DMA controller drivers to register and unregister DMA instances, and their individual DMA channels.

The functions provided by **vxbDmaLib** are as follows:

- vxbDmaRegister( )
- vxbDmaUnregister( )
- vxbDmaChanAdd( )

DMA controller drivers register, unregister and configure DMA channels using this API.

vxbDmaRegister( )

The vxbDmaRegister( ) function is called by the DMA controller driver to register a DMA instance into the DMA library. The prototype is as follows:

```
void vxbDmaRegister
    (
    VXB_DMA_DEV * pVxbDmaDev
    );
```

In this prototype, pVxbDmaDev refers to the DMA instance to be registered.

vxbDmaUnregister( )

The vxbDmaUnregister( ) function is called by the DMA controller driver to remove a DMA instance that was registered previously with the DMA library. The prototype is as follows:

```
void vxbDmaUnregister
    (
    VXB_DMA_DEV * pVxbDmaDev
    );
```

vxbDmaChanAdd( )

The vxbDmaChanAdd( ) method is used by the DMA controller driver to add a DMA channel to a corresponding registered DMA instance. The prototype is as follows:

```
void vxbDmaChanAdd
    (
    VXB_DMA_DEV * pVxbDmaDev,
    VXB_DMA_CHAN * pVxbDmachan
    )
```

In this prototype, the parameters are as follows:

pVxbDmaDev

Points to the DMA instance that the channel will be added to.

pVxbDmachan

Points to the DMA channel that must be added.

Related information

DMA Driver Overview on page 82
DMA Library API for DMA Client Drivers on page 84

DMA Buffer Library API on page 90
VxBus Driver Methods for DMA Drivers on page 103
DMA Driver Header Files on page 105
DMA Driver Configuration on page 106
Available Utility Functions for DMA Drivers on page 107
DMA Driver Initialization on page 107
Initiating a DMA Transfer In a Client Driver on page 107
DMA Client Driver on page 108
DMA Client Driver With Error Status Check on page 109
Debugging DMA Drivers on page 111

3.3. DMA Library API for DMA Client Drivers

The DMA library implements APIs that are used by DMA client drivers to allocate, free, configure and control DMA channels.

The client driver functions provided by **vxbDmaLib** are as follows:

- vxbDmaChanAlloc( )
- vxbDmaChanAllocAdvance( )
- vxbDmaChanFree( )
- vxbDmaChanConfig( )
- vxbDmaChanControl( )
- vxbDmaChanPreMemCpy( )
- vxbDmaChanPreScgUio( )
- vxbDmaChanPreEx( )
- vxbDmaChanStart( )

vxbDmaChanAlloc( )

The vxbDmaChanAlloc( ) function is called by the DMA client driver to allocate a channel within the DMA library. The prototype is as follows:

```
VXB_DMA_CHAN * vxbDmaChanAlloc
    (
    UINT32 uCap
    );
```

The parameter uCap matches the client driver's DMA requirement to the supported capability of the DMA controller. The possible values are defined in *installDir* **/vxworks-7/pkgs_v2/os/drv/vxbus-***a.b.c.d* **/subsystem/h/subsys/dma/vxbDmaLib.h**. Three options exist:

- **VXB_DMA_MEMCPY**: Memory-to-memory DMA transfer requested by DMA client.
- **VXB_DMA_INTERRUPT**: Interrupt on DMA completion requested by DMA client.
- **VXB_DMA_SG**: Scatter/gather DMA transfer requested by DMA client.

Note: If the DMA controller driver implements the method **VXB_DMA_CHAN_ALLOC**, then the channel resources will be managed by the controller driver rather than the DMA library. In that case, the parameter uCap will be ignored on the allocating channel.

vxbDmaChanAllocAdvance( )

The vxbDmaChanAllocAdvance( ) function is an advanced version of vxbDmaChanAlloc( ). The client driver uses it to allocate a channel in the DMA library based on the VxBus name, the unit number, the channel number and capability. To use this function, the DMA controller driver must implement the method **VXB_DMA_CHAN_ALLOC_ADVANCE**. The prototype is as follows:

```
VXB_DMA_CHAN * vxbDmaChanAllocAdvance
    (
    char * pName,
    UINT32 unit,
    int chanNum,
    UINT32 uCap
    );
```

In this prototype, the parameters are as follows:

pName

VxBus name of the DMA controller driver.

unit

VxBus unit number of the DMA controller driver.

chanNum

The DMA channel number required by the DMA client. If chanNum is less than 0, the client driver will not allocate a fixed channel number.

uCap

The parameter uCap matches the client driver's DMA requirement to the supported capability of the DMA controller. The possible values are defined in *installDir* **/vxworks-7/pkgs_v2/os/drv/vxbus-***a.b.c.d* **/subsystem/h/subsys/dma/vxbDmaLib.h**. Three options exist:

- **VXB_DMA_MEMCPY**: Memory-to-memory DMA transfer requested by DMA client.
- **VXB_DMA_INTERRUPT**: Interrupt on DMA completion requested by DMA client.
- **VXB_DMA_SG**: Scatter/gather DMA transfer requested by DMA client.

Note: If the DMA controller driver implements the method **VXB_DMA_CHAN_ALLOC**, then the channel resources will be managed by the controller driver rather than the DMA library. In that case, the parameter uCap will be ignored on the allocating channel.

vxbDmaChanFree( )

The vxbDmaChanFree( ) function is called by the DMA client driver to release a channel previously allocated by a call to vxbDmaChanAlloc( ). The prototype is as follows:

```
void vxbDmaChanFree
    (
    VXB_DMA_CHAN * pDmaChan
    );
```

The parameter pDmaChan points to the channel that must be released.

WNDRVR

vxbDmaChanConfig( )

The vxbDmaChanConfig( ) function is called by the DMA client driver to configure a channel. This function is used to instruct the DMA channel hardware how to transfer data. The prototype is as follows:

```
STATUS vxbDmaChanConfig
    (
    VXB_DMA_CHAN * pDmaChan,
    VXB_DMA_SLAVE_CONFIG *pConfig
    );
```

In this prototype, the parameters are as follows:

pDmaChan

Points to the DMA channel that must be configured.

pConfig

Points to a structure named VXB_DMA_SLAVE_CONFIG. It controls how the channel must work and is defined as follows:

```
typedef struct vxbDmaSlaveConfig {
    VXB_DMA_TRANS_DIR    direction;
    VIRT_ADDR            src;
    VIRT_ADDR            dest;
    UINT32               srcAddrWidth;
    UINT32               destAddrWidth;
    UINT32               srcMaxBurst;
    UINT32               destMaxBurst;
    int                  dmaEvent;
    UINT32               txFrame;
} VXB_DMA_SLAVE_CONFIG;
```

direction

Sets the direction of data transfer. Must be set for all drivers. This value is defined by the enumeration **VXB_DMA_TRANS_DIR**:

```
typedef enum vxbDmaTransDir {
    VXB_DMA_MEM_TO_MEM,        /* transfer from memory to memory */
    VXB_DMA_MEM_TO_DEV,        /* transfer from memory to device */
    VXB_DMA_DEV_TO_MEM,        /* transfer from device to memory */
    VXB_DMA_DEV_TO_DEV         /* transfer from device to device */
} VXB_DMA_TRANS_DIR;
```

src

Must be set to NULL.

dest

Must be set to NULL.

srcAddrWidth

Sets the data bus width of the DMA transfer source device. For 32bit source devices this parameter must be set to 4. If the transfer is from device to memory, this parameter must be set correctly. For other types of transfer, the value may be irrelevant. To determine whether this value is important or not, refer to the DMA controller driver you intend to use.

destAddrWidth

Sets the data bus width of the DMA transfer destination device. For 32bit destination devices this parameter must be set to 4. If the transfer is from memory to device, this parameter must be set correctly. For others types of transfer, the value may be irrelevant. To determine whether this value is important or not, refer to the DMA controller driver you intend to use.

srcMaxBurst

Indicates the burst size of the source device. If srcAddrWidth is configured, this parameter must also be set correctly.

destMaxBurst

Indicates the burst size of the destination device. If destAddrWidth is configured, this parameter must also be set correctly.

dmaEvent

Indicates the hardware DMA event number. This value will be defined in the SOC hardware reference manual. Must be set to -1 if this channel does not need a hardware DMA request. For example, when transferring data from memory to memory, set this parameter to -1.

txFrame

Must be set to 0.

vxbDmaChanControl( )

The vxbDmaChanControl( ) function is called by the DMA client driver to control a channel's action. This function is used to terminate, pause or resume the activity of a DMA channel. The prototype is as follows:

```
STATUS vxbDmaChanControl
    (
    VXB_DMA_CHAN *    pDmaChan,
    VXB_DMA_CTRL_CMD  cmd,
    void *            pArg
    );
```

In this prototype, the parameters are as follows:

pDmaChan

Points to the channel that must be controlled.

cmd

Specifies the required action. This parameter can be set to:

- **VXB_DMA_TERMINATE_ALL**
- **VXB_DMA_PAUSE**
- **VXB_DMA_RESUME**
- **VXB_DMA_CONFIG**

pArg

The parameter setting depends on the setting of the cmd parameter:

- **VXB_DMA_TERMINATE_ALL**, **VXB_DMA_PAUSE**, **VXB_DMA_RESUME**

pArg must either be set to NULL, or to point to a customer defined structure.

- **VXB_DMA_CONFIG**

  pArg must be set to pConfig.

vxbDmaPreMemCpy( )

The vxbDmaPreMemCpy( ) function is used by the DMA client driver to specify the job details. The DMA controller driver uses this information when setting up the hardware DMA engine. This function does not initiate the DMA transfer. Rather, it prepares the hardware for the transfer. The prototype is as follows:

```
VXB_DMA_TX * vxbDmaChanPreMemCpy
    (
    VXB_DMA_CHAN *   pDmaChan,
    VIRT_ADDR        src,
    VIRT_ADDR        dest,
    size_t           len,
    int              flag
    )
```

In this prototype, the parameters are as follows:

pDmaChan

Points to the DMA channel to be used in the transfer.

src

Source address for the DMA transfer. Must be a virtual address.

dest

Destination address for the DMA transfer. Must be a virtual address.

len

Number of bytes to transfer.

flag

Must be set to zero unless a return status is required following the completion of the DMA transfer. In this case, flag must be set to **VXB_DMA_RETURN_STATUS**.

> Note: DMA return status is only supported on Power-PC architecture and particular BSPs. Refer to DMA Client Driver With Error Status Check on page 109 for more details.

This function returns a pointer to the structure **VXB_DMA_TX**. This pointer must be passed to the function vxbDmaChanStart( ).

vxbDmaPreScgUio( )

The vxbDmaPreScgUio( ) function is used by the DMA client driver to specify a scatter-gather type transfer on a DMA channel. This can be used to transfer from non-contiguous memory fragments to a single buffer or peripheral registers. The source pointers must reference the base address and the length of the fragments you wish to transfer. When the transfer is to a peripheral register, the destination pointer must be NULL. This function does not initiate the DMA transfer. Rather, it prepares the hardware for the transfer. The prototype is as follows:

```
VXB_DMA_TX * vxbDmaChanPreScgUio
    (
    VXB_DMA_CHAN *   pDmaChan,
    struct uio   *   pUioSrc,
    struct uio   *   pUioDst,
    UINT32           flags,
    void         *   pContext
    )
```

In this prototype, the parameters are as follows:

pDmaChan

Points to the DMA channel to be used in the transfer.

pUioSrc

Source fragments for the DMA transfer. Must define base address and length of each source fragment. The structure is defined in *installDir* **/vxworks-7/pkgs_v2/os/core-***a.b.c.d* **/io/shareh/sys/uio.h**.

Note: *a.b.c.d* refers to the current version of the VxWorks core layer.

```
struct iovec
    {
    void *iov_base;    /* Base address */
    size_t iov_len;    /* Length */
    }

struct uio
    {
    struct iovec *uio_iov;    /* sets base address and length
                                 of this fragment */
    int uio_iovcnt;
#ifdef _WRS_KERNEL
    off64_t uio_offset;
#else
    off_t uio_offset;
#endif
    size_t uio_resid;
    enum uio_seg uio_segflg;
    enum uio_rw uio_rw;
    struct proc *uio_procp;
    }
```

pUioDst

Destination address for the DMA transfer. Must be a specified in the same format as the source fragments.

flags

Must be set to zero.

pContext

Must be set to NULL.

This function returns a pointer to the structure **VXB_DMA_TX**. This pointer must be passed to the function vxbDmaChanStart( ).

WNDRVR

Note: The API function vxbDmaChanPreScg( ) is deprecated, and vxbDmaChanPreScgUio( ) must be used instead.

vxbDmaChanPreEx( )

The vxbDmaChanPreEx( ) function is called by the DMA client driver to prepare an extended DMA transfer. The function does not initiate the DMA transfer. The prototype is as follows:

```
VXB_DMA_TX * vxbDmaChanPreEx
    (
    VXB_DMA_CHAN * pDmaChan,
    VOID * pCustomChanInfo
    );
```

While pDmaChan selects which DMA channel must perform the extended DMA transfer, pCustomChanInfo points to user provided information to send to the DMA channel.

vxbDmaChanStart( )

The vxbDmaChanStart( ) function is called by the DMA client driver to initiate the DMA transfer immediately. The function parameter is the value returned from vxbDmaChanPreMemCpy( ). Before calling this API function, the callback routine and parameters must be setup in the DMA controller driver. The prototype is as follows:

```
STATUS vxbDmaChanStart
    (
    VXB_DMA_TX * pVxbDmaTx
    );
```

Related information

DMA Driver Overview on page 82
DMA Library API for DMA Controller Drivers on page 82
DMA Buffer Library API on page 90
VxBus Driver Methods for DMA Drivers on page 103
DMA Driver Header Files on page 105
DMA Driver Configuration on page 106
Available Utility Functions for DMA Drivers on page 107
DMA Driver Initialization on page 107
Initiating a DMA Transfer In a Client Driver on page 107
DMA Client Driver on page 108
DMA Client Driver With Error Status Check on page 109
Debugging DMA Drivers on page 111

3.4. DMA Buffer Library API

The DMA buffer library API provides functions that handle allocation of buffer memory, translation between virtual and physical addressing, cache synchronization and bounce buffering.

The vxbDmaBufLib( ) API saves the DMA driver author from the complexity of managing cache coherent buffers that are accessible by both DMA controller and client drivers. A bounce buffer is used when the devices address a lower region of memory during the DMA transfer, but the application or middleware subsequently accesses the contents of that buffer at a different address.

The vxbDmaBufLib( ) API is available both in kernel and user mode, supporting DMA drivers within RTPs.

Summary of vxbDmaBufLib API on page 91 The DMA buffer library API is available for use by both controller and client drivers.

Examples of DMA Buffer Library API Use on page 100 When a client driver initiates DMA transfers with the peripheral device, it may use the DMA buffer library API to manage transmit and receive buffers.

Related information

DMA Driver Overview on page 82
DMA Library API for DMA Controller Drivers on page 82
DMA Library API for DMA Client Drivers on page 84
VxBus Driver Methods for DMA Drivers on page 103
DMA Driver Header Files on page 105
DMA Driver Configuration on page 106
Available Utility Functions for DMA Drivers on page 107
DMA Driver Initialization on page 107
Initiating a DMA Transfer In a Client Driver on page 107
DMA Client Driver on page 108
DMA Client Driver With Error Status Check on page 109
Debugging DMA Drivers on page 111

Summary of vxbDmaBufLib API

The DMA buffer library API is available for use by both controller and client drivers.

This **vxbDmaBufLib** API contains the following driver accessible functions:

- vxbDmaBufTagParentGet( )
- vxbDmaBufTagCreate( )
- vxbDmaBufTagDestroy( )
- vxbDmaBufMapCreate( )
- vxbDmaBufMemAlloc( )
- vxbDmaBufMapDestroy( )
- vxbDmaBufMemFree( )
- vxbDmaBufMapLoad( )
- vxbDmaBufMapMblkLoad( )
- vxbDmaBufMapIoVecLoad( )
- vxbDmaBufMapUnLoad( )
- vxbDmaBufMapFlush( )
- vxbDmaBufMapInvalidate( )
- vxbDmaBufSync( )
- vxbDmaBufMapSync( )

vxbDmaBufTagParentGet()

Use this function to retrieve a DMA tag which is associated with the device that will be involved in the DMA transfer. A DMA tag describes the characteristics and constraints of a DMA transfer with this device. The function prototype is as follows:

```
VXB_DMA_TAG_ID vxbDmaBufTagParentGet
    (
    VXB_DEV_ID  pInst,
    UINT32      regBaseIndex
    );
```

The parameter pInst is the pointer to the device instance that this driver is using. The regBaseIndex parameter is used when referring to some devices.

The function returns the parent DMA tag.

vxbDmaBufTagCreate()

This function allows you to create a new DMA tag that defines a set of constraints for a future DMA transfer with an existing parent device. These constraints are specified in the function parameters.

```
VXB_DMA_TAG_ID vxbDmaBufTagCreate
    (
    VXB_DEV_ID  pInst,
    VXB_DMA_TAG_ID  parent,
    bus_size_t  alignment,
    bus_size_t  boundary,
    bus_addr_t  lowAddr,
    bus_addr_t  highAddr,
    bus_dma_filter_t *  filter,
    void *  filterArg,
    bus_size_t  maxSize,
    int  nSegments,
    bus_size_t  maxSegSz,
    int  flags,
    bus_dma_lock_t *lockFunc,
    void *  lockFuncArg,
    VXB_DMA_TAG_ID *ppDmaTag
    )
```

In this prototype, the parameters are as follows:

pInst

Pointer to the device instance that the DMA transfer will operate on.

parent

Original DMA tag defined for the parent device.

alignment

Specified cache alignment for the memory transfer window.

boundary

Alignment boundary for the memory transfer window.

lowAddr

Lower address bounds for the memory transfer window.

highAddr

Higher address bounds for the memory transfer window.

filter

Optional address filter function. This function allows for devices that have DMA restrictions that cannot be specified by a single window.

filterArg

This argument will be passed to every invocation of the filter function.

maxSize

Maximum mapping size.

nSegments

Number of memory segments in the memory window.

maxSegSz

Maximum permitted memory segment size.

flags

This parameter may be set to the following individual values, or a bitwise combination of them:

- **VXB_DMABUF_NOCACHE** indicates a cache coherent buffer. This is ignored on Intel architecture, where cache-snooping is always enabled instead.
- **VXB_DMABUF_CACHESNOOP_BUS** selects cache-snooping mode.
- **VXB_DMABUF_CACHEALIGNCHECK** is valid on non-Intel architectures. This enables bounce buffering, where the device addresses a lower region of memory during the DMA transfer, but the application or middleware subsequently accesses the contents of that buffer at a different address. The buffer passed to vxbDmaBufLoad( ) or vxbDmaBufMapIoVecLoad( ) must be cacheable and not aligned on a cache-line when you specify **VXB_DMABUF_MAPLOAD_CACHEALIGNCHECK** in the load routine flags argument.

lockFunc

If a mutex must be manipulated as part of the transfer, specify the mutex or lock function here.

lockFuncArg

This argument will be passed to every invocation of the lock function.

ppDmaTag

If you require a pointer to the newly created DMA tag, it is returned here.

The function returns a pointer to the newly created DMA tag.

vxbDmaBufTagDestroy()

This function parmits the destruction of a DMA tag once you are finished with it. Usually this will be called by a driver that is unloading.

```
STATUS vxbDmaBufTagDestroy
    (
    VXB_DMA_TAG_ID  dmaTagID
    )
```

Pass the DMA tag to be destroyed as dmaTagID.

The function returns **OK** if the tag was valid and the memory freed, and **ERROR** otherwise.

vxbDmaBufMapCreate()

Use this function to create a mapping between physical and virtual memory. Before you can load a map, you must create one.

```
VXB_DMA_MAP_ID vxbDmaBufMapCreate
    (
```

```
    VXB_DEV_ID  pInst,
    VXB_DMA_TAG_ID  dmaTagID,
    int  flags,
    VXB_DMA_MAP_ID *pMap
    )
```

In this prototype, the parameters are as follows:

pInst

Pointer to the device instance that the DMA transfer will operate on.

dmaTagID

ID of the DMA tag that describes this DMA transfer.

flags

This parameter can either be set to NULL or **VXB_DMABUF_MAPCREATE_NOBOUNCE_BUF**. This macro disables the creation of a bounce buffer.

pMap

A pointer to the created map.

The function returns a pointer to the newly created map ID.

vxbDmaBufMemAlloc()

This function allocates DMA-able memory that conforms to the constraints defined in the associated DMA tag.

```
void * vxbDmaBufMemAlloc
    (
    VXB_DEV_ID  pInst,
    VXB_DMA_TAG_ID  dmaTagID,
    void **  vaddr,
    int  flags,
    VXB_DMA_MAP_ID *pMap
    )
```

The parameters in this prototype are as follows:

pInst

Pointer to the device instance that the DMA transfer will operate on.

dmaTagID

ID of the DMA tag that describes this DMA transfer.

vaddr

Pointer to the allocated memory.

flags

This parameter can either be set to NULL or **VXB_DMABUF_MEMALLOC_CLEAR_BUF**. This macro requests that the allocated buffer be zeroed out before being returned.

pMap

Pointer to the related map.

If the allocation succeeds, the function returns a pointer to the memory. Otherwise, it returns **ERROR**.

vxbDmaBufMapDestroy()

Once the DMA transfer has completed, use this function to destroy the mapping that was used.

```
STATUS vxbDmaBufMapDestroy
    (
    VXB_DMA_TAG_ID  dmaTagID,
    VXB_DMA_MAP_ID  map
    )
```

Pass both the related DMA tag, dmaTagID, and the map to be destroyed, map.

If the specified map was valid and the memory could be freed, then the function returns **OK**. Otherwise, it returns **ERROR**.

vxbDmaBufMemFree()

Use this function to release the DMA-able memory allocated by vxbDmaBufAlloc( ).

```
STATUS vxbDmaBufMemFree
    (
    VXB_DMA_TAG_ID  dmaTagID,
    void *  vaddr,
    VXB_DMA_MAP_ID  map
    )
```

The parameters in this prototype are as follows:

dmaTagID

ID of the DMA tag that describes this DMA transfer.

vaddr

Pointer to the memory that is to be released.

map

The map containing the memory to be released.

The function returns **OK** if it succeeds, and **ERROR** if it fails.

vxbDmaBufMapLoad()

Use this function to use a created map and to setup an address translation between a virtual buffer and a physical memory region.

```
STATUS vxbDmaBufMapLoad
    (
    VXB_DEV_ID  pInst,
    VXB_DMA_TAG_ID  dmaTagID,
    VXB_DMA_MAP_ID  map,
    void *  buf,
    bus_size_t  bufLen,
```


WNDRVR

```
    int  flags
    )
```

The parameters in this prototype are as follows:

pInst

Pointer to the device instance that the DMA transfer will operate on.

dmaTagID

ID of the DMA tag that describes this DMA transfer.

map

The previously created map.

buf

Buffer pointer.

bufLen

Buffer length.

flags

This parameter can be set to NULL or **VXB_DMABUF_MAPLOAD_CACHEALIGNCHECK**. This macro enables bounce buffering on non-Intel architectures as long as the related DMA tag is created with the flag **VXB_DMABUF_CACHEALIGNCHECK** and the buffer is cacheable but not aligned on a cache-line.

The function returns **OK** if it succeeds, and **ERROR** if it fails.

vxbDmaBufMapMblkLoad()

This function is similar to vxbDmaBufMapLoad( ). However, it is intended to be used in network end drivers that use mBlk tuples.

```
STATUS vxbDmaBufMapMblkLoad
    (
    VXB_DEV_ID  pInst,
    VXB_DMA_TAG_ID  dmaTagID,
    VXB_DMA_MAP_ID  map,
    M_BLK_ID  pMblk,
    int  flags
    )
```

The parameters in this prototype are as follows:

pInst

Pointer to the device instance that the DMA transfer will operate on.

dmaTagID

ID of the DMA tag that describes this DMA transfer.

map

The previously created map.

pMblk

Pointer to the Mblk.

flags

This parameter can be set to NULL or **VXB_DMABUF_MAPLOAD_CACHEALIGNCHECK**. This macro enables bounce buffering on non-Intel architectures as long as the related DMA tag is created with the flag **VXB_DMABUF_CACHEALIGNCHECK** and the buffer is cacheable but not aligned on a cache-line.

The function returns **OK** if it succeeds, and **ERROR** if it fails.

vxbDmaBufMapIoVecLoad()

This function must be used instead of vxbDmaBufMapLoad( ) when a scatter/gather array of virtual buffers are being worked on. This approach is useful for transferring multiple data blocks, or disk transfers.

```
STATUS vxbDmaBufMapIoVecLoad
    (
    VXB_DEV_ID  pInst,
    VXB_DMA_TAG_ID  dmaTagID,
    VXB_DMA_MAP_ID  map,
    struct uio *  uio,
    int  flags
    )
```

The parameters in this prototype are as follows:

pInst

Pointer to the device instance that the DMA transfer will operate on.

dmaTagID

ID of the DMA tag that describes this DMA transfer.

map

The previously created map.

uio

Pointer to segment pointers for this transfer.

flags

This parameter can either be set to NULL or **VXB_DMABUF_MAPLOAD_CACHEALIGNCHECK**. This macro enables bounce buffering on non-Intel architectures as long as the related DMA tag is created with the flag **VXB_DMABUF_CACHEALIGNCHECK** and the buffer is cacheable but not aligned on a cache-line.

The function returns **OK** if it succeeds, and **ERROR** if it fails.

vxbDmaBufMapUnLoad()

Use this function to destroy a previously created virtual buffer mapping. This function is typically used after a DMA transfer has completed, and the associated buffer memory must be freed.

```
STATUS vxbDmaBufMapUnLoad
    (
    VXB_DMA_TAG_ID  dmaTagID,
```


WNDRVR

```
    VXB_DMA_MAP_ID  map
    )
```

Pass both the related DMA tag, dmaTagID, and the map to be unloaded, map.

The function always returns **OK**.

vxbDmaBufMapFlush()

Use this function to flush the cache for the specified DMA map.

```
STATUS vxbDmaBufMapFlush
    (
    VXB_DEV_ID  pInst,
    VXB_DMA_TAG_ID  dmaTagID,
    VXB_DMA_MAP_ID  map
    )
```

The parameters in this prototype are as follows:

pInst

Pointer to the device instance that the DMA transfer will operate on.

dmaTagID

ID of the DMA tag that describes this DMA transfer.

map

The related map.

The function returns **OK** if it succeeds, and **ERROR** if it fails.

vxbDmaBufMapInvalidate()

Use this function to invalidate the cache for the specified DMA map.

```
STATUS vxbDmaBufMapInvalidate
    (
    VXB_DEV_ID  pInst,
    VXB_DMA_TAG_ID  dmaTagID,
    VXB_DMA_MAP_ID  map
    )
```

The parameters in this prototype are as follows:

pInst

Pointer to the device instance that the DMA transfer will operate on.

dmaTagID

ID of the DMA tag that describes this DMA transfer.

map

The related map.

The function returns **OK** if it succeeds, and **ERROR** if it fails.

vxbDmaBufSync()

Use this function to setup the bounce buffer after DMA reception is complete, or to initiate the DMA transmit. It also invalidates the cache prior to reception, and flushes the cache prior to transmission.

```
STATUS vxbDmaBufSync
    (
    VXB_DEV_ID  pInst,
    VXB_DMA_TAG_ID  dmaTagID,
    VXB_DMA_MAP_ID  map,
    bus_dmasync_op_t  op
    )
```

The parameters in this prototype are as follows:

pInst

Pointer to the device instance that the DMA transfer will operate on.

dmaTagID

ID of the DMA tag that describes this DMA transfer.

map

The related map.

op

This parameter sets the operation mode, and can be set individually or as a combination of activities:

1. **_VXB_DMABUFSYNC_DMA_PREWRITE** copies the transmit data to the bounce buffer and/or flushes the DMA cache if necessary.
2. **_VXB_DMABUFSYNC_DMA_PREREAD** invalidates the DMA cache prior to receiving data.
3. **_VXB_DMABUFSYNC_DMA_POSTREAD** copies the received data from the bounce buffer.
4. **_VXB_DMABUFSYNC_DMA_POSTWRITE** invalidates the DMA cache prior to the read of the DMA status and/or copies the DMA status from the bounce buffer.

> Note: If multiple options are combined, they apply in the order presented. For example, the first activity is **_VXB_DMABUFSYNC_DMA_PREWRITE**.

The function returns **OK** if it succeeds, and **ERROR** if it fails.

vxbDmaBufMapSync()

You must use this function instead of vxbDmaBufSync( ) when you intend to act on a portion of the DMA map, and not the whole map. It will setup the bounce buffer after DMA reception is complete, or to initiate the DMA transmit. It also invalidates the cache prior to reception, and flushes the cache prior to transmission.

```
STATUS vxbDmaBufMapSync
    (
    VXB_DEV_ID  pInst,
    VXB_DMA_TAG_ID  dmaTagID,
```


WNDRVR

```
    VXB_DMA_MAP_ID  map,
    off_t  offset,
    size_t  length,
    bus_dmasync_op_t  op
    )
```

The parameters in this prototype are as follows:

pInst

Pointer to the device instance that the DMA transfer will operate on.

dmaTagID

ID of the DMA tag that describes this DMA transfer.

map

The related map.

offset

This parameter defines the start address for the sync.

length

This parameter defines the window size in the map that will be synced.

op

This parameter sets the operation mode, and can be set individually or as a combination of activities:

1. **_VXB_DMABUFSYNC_DMA_PREWRITE** copies the transmit data to the bounce buffer and/or flushes the DMA cache if necessary.
2. **_VXB_DMABUFSYNC_DMA_PREREAD** invalidates the DMA cache prior to receiving data.
3. **_VXB_DMABUFSYNC_DMA_POSTREAD** copies the received data from the bounce buffer.
4. **_VXB_DMABUFSYNC_DMA_POSTWRITE** invalidates the DMA cache prior to the read of the DMA status and/or copies the DMA status from the bounce buffer.

> Note: If multiple options are combined, they apply in the order presented. For example, the first activity is **_VXB_DMABUFSYNC_DMA_PREWRITE**.

The function returns **OK** if it succeeds, and **ERROR** if it fails.

Parent topic: DMA Buffer Library API on page 90

Examples of DMA Buffer Library API Use

When a client driver initiates DMA transfers with the peripheral device, it may use the DMA buffer library API to manage transmit and receive buffers.

Different stages of a client device driver will makes use of the DMA buffer library API in particular ways. Example use of the API is shown below.

Driver Attach Routine

```
/* initialise device pDev */

/* initialise driver buffers for DMA receive and transmit */
parentTag = vxbDmaBufTagParentGet ( pDev, … );

/* setup tx descriptor */
txDescriptor.Tag = vxbDmaBufTagCreate ( pDev, parentTag, … );

txDescriptor.Mem = vxbDmaBufMemAlloc ( pDev, txDescriptor.Tag, … );

/* setup rx descriptor */
rxDescriptor.Tag = vxbDmaBufTagCreate ( pDev, parentTag, … );

rxDescriptor.Mem = vxbDmaBufMemAlloc ( pDev, rxDescriptor.Tag, … );

/* setup rx frame */
rxFrame.Tag = vxbDmaBufTagCreate ( pDev, parentTag, … );

rxFrame.Frame = vxbDmaBufMemAlloc ( pDev, rxFrame.Tag, … );

/* setup mBlks */
mBlk.Tag = vxbDmaBufTagCreate ( pDev, parentTag, … );

mBlk.Frame = vxbDmaBufMemAlloc ( pDev, MblkTag, … );

/* Setup mapping for mBlks */
vxbDmaBufMapCreate ( pDev, MblkTag, 0, &mblkMap );
```

Setup for Transmit and Receive

```
/* setup receive buffer ring */

/* map the buffer */
vxbDmaBufMapMblkLoad ( pDev, MblkTag, &mblkMap, pMblk, 0 );

/* setup the transmit buffer ring */

/* map the buffer */
vxbDmaBufMapLoad ( pDev, MblkTag, … );
```

Stop Device Routine

```
/* stop device */

/* shutdown device interrupts */

/* release driver resources */
vxbDmaBufMapUnload ( rxDescriptor.Tag, … );
vxbDmaBufMapUnload ( txDescriptor.Tag, … );
```

Receive Interrupt Service Routine

```
/* do DMA receive into buffer */

/* sync the received packet bounce buffer so the middleware can read the packet */
vxbDmaBufSync ( pDev, MblkTag, pMap, VXB_DMABUFSYNC_PREREAD );

/* unload the map */
vxbDmaBufMapUnload ( MblkTag, pMap );

/* acknowledge interrupt */
```

Prepare Packet for Transmission

```
/* load the DMA map so the transmit packet can be constructed */
vxbDmaBufMapMblkLoad ( pDev, MblkTag, pMap, pMblk, 0 );

/* construct the packet in the buffer */

/* sync the bounce buffer */
vxbDmaBufSync ( pDev, MblkTag, pMap, VXB_DMABUFSYNC_POSTWRITE );

/* trigger DMA transmit */
```

Polled Mode Send Routine

```
/* construct packet in buffer and trigger the send */

/* unload the DMA map once the transmit has completed */
vxbDmaBufMapUnload ( MblkTag, pMap );
```

Polled Mode Receive Routine

```
/* receive the packet into the buffer */

/* sync the bounce buffer so the middleware can read the packet */
vxbDmaBufSync ( pDev, MblkTag, pMap, VXB_DMABUFSYNC_PREREAD );
```

Driver Shutdown

```
/* stop device and detach from it */

/* tidy up all memory areas used for DMA configuration and communication */
vxbDmaBufMemFree ( rxDescriptor.Tag, memPtr, … );

vxbDmaBufMemFree ( txDescriptor.Tag, memPtr, … );

vxbDmaBufMapDestroy ( MblkTag, &mblkMap );

vxbDmaBufTagDestroy (rxDescriptor.Tag );
```

```
    vxbDmaBufTagDestroy (txDescriptor.Tag );

    vxbDmaBufTagDestroy (rxFrame.Tag );

    vxbDmaBufTagDestroy (mBlk.Tag );
```

Parent topic: DMA Buffer Library API on page 90

3.5. VxBus Driver Methods for DMA Drivers

VxBus device driver methods are provided for allocating, freeing, and controlling direct memory access (DMA) channels.

The functions provided by **vxbDmaLib** make use of four VxBus driver methods. The methods are:

- {VXB_DMA_CHAN_ALLOC}( )
- {VXB_DMA_CHAN_FREE}( )
- {VXB_DMA_CHAN_PRE_MEM_CPY}( )
- {VXB_DMA_CHAN_CONTROL}( )

DMA drivers provide access to device services by associating functions with these methods.

{VXB_DMA_CHAN_ALLOC}( )

The {VXB_DMA_CHAN_ALLOC}( ) method is used by the DMA library to allocate a DMA channel on the device managed by the DMA driver. The prototype is as follows:

```
VXB_DMA_CHAN * VXB_DMA_CHAN_ALLOC
    (
    VXB_DEV_ID pDev
    );
```

In this prototype, pDev refers to the DMA controller itself.

{VXB_DMA_CHAN_FREE}( )

The {VXB_DMA_CHAN_FREE}( ) method is used by the DMA library to free a DMA channel on the device managed by the DMA driver. The prototype is as follows:

```
STATUS VXB_DMA_CHAN_FREE
    (
    VXB_DEV_ID pDev,
    VXB_DMA_CHAN * pDmaChan
    );
```

In most cases, the only requirement for the driver is to free the particular DMA channel allocated to the device. Parameter pDev refers to the VxBus device ID of the DMA device, and pDmaChan refers to the DMA channel to be freed.

{VXB_DMA_CHAN_PRE_MEM_CPY}( )

The {VXB_DMA_CHAN_PRE_MEM_CPY}( ) method is used by the DMA library to queue a transaction from the buffer or register, to a buffer or register in system memory or on the device. The prototype is as follows:

```
VXB_DMA_TX * VXB_DMA_CHAN_PRE_MEM_CPY
    (
    VXB_DEV_ID pDev,
    VXB_DMA_CHAN * pDmaChan,
    VIRT_ADDR src,
    VIRT_ADDR dest,
    size_t len,
    int flag
    )
```

In this prototype, the parameters are as follows:

pDev

The VxBus device ID of the DMA device.

pDmaChan

The DMA channel requsted by the device.

src

The source address transaction attribute for the DMA channel.

dest

The destination address transaction attribute for the DMA channel.

len

The transaction length for the DMA channel.

Note: For this method, the source (src) and destination (dest) addresses are virtual addresses rather than physical addresses.

{VXB_DMA_CHAN_CONTROL}( )

The {VXB_DMA_CHAN_CONTROL}( ) method is used by the DMA library to control a DMA channel. If you have hardware or software constraints, or other considerations, you can use this method to ensure that specific channels can be paused, resumed, configured, and terminated. The prototype is as follows:

```
STATUS VXB_DMA_CHAN_CONTROL
    (
    VXB_DEV_ID pDev,
    VXB_DMA_CHAN * pDmaChan,
    VXB_DMA_CTRL_CMD cmd,
    void * pArg
    )
```

In this prototype, the parameters are as follows:

pDev

The VxBus device ID of the DMA device.

pDmaChan

The DMA channel requested by the device.

cmd

Describes the control attributes used during the DMA channel transaction.

The **VXB_DMA_CTRL_CMD** structure is defined in the following file:

installDir **/vxworks-7/pkgs_v2/os/drv/vxbus-***a.b.c.d* **/subsystem/h/subsys/dma /vxbDmaLib.h**

Note: *a.b.c.d* refers to the current version number of the VxBus layer.

The structure is defined as follows:

```
typedef enum vxbDmaCtrlCmd {
    VXB_DMA_TERMINATE_ALL,
    VXB_DMA_PAUSE,
    VXB_DMA_RESUME,
    VXB_DMA_CONFIG
} VXB_DMA_CTRL_CMD;
```

Related information

DMA Driver Overview on page 82
DMA Library API for DMA Controller Drivers on page 82
DMA Library API for DMA Client Drivers on page 84
DMA Buffer Library API on page 90
DMA Driver Header Files on page 105
DMA Driver Configuration on page 106
Available Utility Functions for DMA Drivers on page 107
DMA Driver Initialization on page 107
Initiating a DMA Transfer In a Client Driver on page 107
DMA Client Driver on page 108
DMA Client Driver With Error Status Check on page 109
Debugging DMA Drivers on page 111

3.6. DMA Driver Header Files

Interrupt controller drivers (as well as other drivers that access DMA functions) must include the direct memory access (DMA) library header file (vxbDmaLib.h( )) to use the available library functions.

DMA drivers must include the following header file:

installDir **/vxworks-7/pkgs_v2/os/drv/vxbus-***a.b.c.d* **/subsystem/h/subsys/dma /vxbDmaLib.h**

Note: *a.b.c.d* refers to the current version of the VxBus layer.

Other drivers that want to make use of DMA functions must also include this header file. In order to use the dedicated channel functionality provided by the DMA driver, these drivers must also include the header files for specific DMA drivers.

Related information

DMA Driver Overview on page 82
DMA Library API for DMA Controller Drivers on page 82
DMA Library API for DMA Client Drivers on page 84
DMA Buffer Library API on page 90
VxBus Driver Methods for DMA Drivers on page 103
DMA Driver Configuration on page 106
Available Utility Functions for DMA Drivers on page 107
DMA Driver Initialization on page 107

Initiating a DMA Transfer In a Client Driver on page 107
DMA Client Driver on page 108
DMA Client Driver With Error Status Check on page 109
Debugging DMA Drivers on page 111

3.7. DMA Driver Configuration

DMA drivers are configured using different sources of information in your system, depending on your target hardware architecture.

In general, DMA drivers do not require configuration information from the BSP beyond the normal device-specific information provided for all drivers. Configuration information is derived from the device tree source (DTS) files. The following example is for a DMA-specific DTS file for a Freescale P2020 rdb board:

```
dma@21300 {
         #address-cells = <1>;
         #size-cells = <1>;
         compatible = "fsl,eloplus-dma";
         reg = <0x21300 0x4>;
         ranges = <0x0 0x21100 0x200>;
         cell-index = <0>;

         dma-channel@0 {
             compatible = "fsl,eloplus-dma-channel";
             reg = <0x0 0x80>;
             cell-index = <0>;
             interrupts = <20 0 0 0>;
         };

         dma-channel@80 {
             compatible = "fsl,eloplus-dma-channel";
             reg = <0x80 0x80>;
             cell-index = <1>;
             interrupts = <21 0 0 0>;
         };

         dma-channel@100 {
             compatible = "fsl,eloplus-dma-channel";
             reg = <0x100 0x80>;
             cell-index = <2>;
             interrupts = <22 0 0 0>;
         };

         dma-channel@180 {
             compatible = "fsl,eloplus-dma-channel";
             reg = <0x180 0x80>;
             cell-index = <3>;
             interrupts = <23 0 0 0>;
         };

       };
```

Related information

DMA Driver Overview on page 82
DMA Library API for DMA Controller Drivers on page 82
DMA Library API for DMA Client Drivers on page 84
DMA Buffer Library API on page 90
VxBus Driver Methods for DMA Drivers on page 103

DMA Driver Header Files on page 105
Available Utility Functions for DMA Drivers on page 107
DMA Driver Initialization on page 107
Initiating a DMA Transfer In a Client Driver on page 107
DMA Client Driver on page 108
DMA Client Driver With Error Status Check on page 109
Debugging DMA Drivers on page 111

3.8. Available Utility Functions for DMA Drivers

There are no class-specific utility functions required for DMA drivers.

There are no VxBus utility functions available to DMA drivers.

Related information

DMA Driver Overview on page 82
DMA Library API for DMA Controller Drivers on page 82
DMA Library API for DMA Client Drivers on page 84
DMA Buffer Library API on page 90
VxBus Driver Methods for DMA Drivers on page 103
DMA Driver Header Files on page 105
DMA Driver Configuration on page 106
DMA Driver Initialization on page 107
Initiating a DMA Transfer In a Client Driver on page 107
DMA Client Driver on page 108
DMA Client Driver With Error Status Check on page 109
Debugging DMA Drivers on page 111

3.9. DMA Driver Initialization

DMA device driver initialization is generally device-specific.

Initialization must be completed before or during the VxBus initialization attach (vxbDevAttach( )) phase. This guarantees that **vxbDmaLib** is available to other drivers.

Related information

DMA Driver Overview on page 82
DMA Library API for DMA Controller Drivers on page 82
DMA Library API for DMA Client Drivers on page 84
DMA Buffer Library API on page 90
VxBus Driver Methods for DMA Drivers on page 103
DMA Driver Header Files on page 105
DMA Driver Configuration on page 106
Available Utility Functions for DMA Drivers on page 107
Initiating a DMA Transfer In a Client Driver on page 107
DMA Client Driver on page 108
DMA Client Driver With Error Status Check on page 109
Debugging DMA Drivers on page 111

3.10. Initiating a DMA Transfer In a Client Driver

About This Task

These steps assume the DMA client must prepare and then initiate a memory to memory DMA transfer.

Procedure

1. Apply for the appropriate DMA channel to carry out the transfer.

   This step is completed by a call to either vxbDmaChanAlloc( ) or vxbDmaChanAllocAdvance( ). For example:

   ```
   pDmaChan = vxbDmaChanAlloc(uCap);
   ```

   Alternatively:

   ```
   pDmaChan = vxbDmaChanAllocAdvance(pName, unit, chanNum, uCap);
   ```

   Note: Refer to DMA Library API for DMA Client Drivers on page 84 for a description of uCap.

2. Configure the DMA channel that has been allocated for use in the transfer.

   ```
   vxbDmaChanConfig(pDmaChan, VXB_DMA_SLAVE_CONFIG * pConfig);
   ```

3. Prepare the memory copy DMA transfer.

   ```
   pVxbDmaTx = vxbDmaChanPreMemCpy(pDmaChan, src, dest, len, flag);
   ```

4. Set the callback function and parameter in the DMA client.

   ```
   pVxbDmaTx->fn = fn;
   pVxbDmaTx->pArg = pArg;
   ```

5. Initiate the DMA memory transfer.

   ```
   vxbDmaChanStart(pVxbDmaTx);
   ```

Related information

DMA Driver Overview on page 82
DMA Library API for DMA Controller Drivers on page 82
DMA Library API for DMA Client Drivers on page 84
DMA Buffer Library API on page 90
VxBus Driver Methods for DMA Drivers on page 103
DMA Driver Header Files on page 105
DMA Driver Configuration on page 106
Available Utility Functions for DMA Drivers on page 107
DMA Driver Initialization on page 107
DMA Client Driver on page 108
DMA Client Driver With Error Status Check on page 109
Debugging DMA Drivers on page 111

3.11. DMA Client Driver

The following example shows how to make use of the functions and methods described in the DMA library:

```
VXB_DMA_SLAVE_CONFIG config;

# allocate DMA channel
pDmaChan = vxbDmaChanAlloc(uCap);
```

```
# configure the DMA channel - (optional)
memset ((void*)&config, 0, sizeof(VXB_DMA_SLAVE_CONFIG));

config.direction      = VXB_DMA_DEV_TO_MEM;
config.src            = DEVICE_RECEIVE_REG;
config.dst            = receive_buffer;
config.srcAddrWidth   = burst_size;
config.srcMaxBurst    = max_burst;

vxbDmaChanConfig(pDmaChan, (void *)&config);


# prepare a memory copy DMA transfer
pVxbDmaTx = vxbDmaChanPreMemCpy(pDmaChan,src, dest, len, flag);


# set the callback function and parameter
pVxbDmaTx->fn = fn;
pVxbDmaTx->pArg = pArg;


# start the DMA transfer
vxbDmachanStart(pVxbDmaTx);
```

Note: Refer to DMA Library API for DMA Client Drivers on page 84 for a description of uCap.

The DMA channel status will not be returned on the completion of the transfer. On some CPU architectures, it is possible to interrogate the DMA channel status after the transfer has completed. This status indicates whether the DMA transfer succeeded or failed. For further details, refer to DMA Client Driver With Error Status Check on page 109.

Related information

DMA Driver Overview on page 82
DMA Library API for DMA Controller Drivers on page 82
DMA Library API for DMA Client Drivers on page 84
DMA Buffer Library API on page 90
VxBus Driver Methods for DMA Drivers on page 103
DMA Driver Header Files on page 105
DMA Driver Configuration on page 106
Available Utility Functions for DMA Drivers on page 107
DMA Driver Initialization on page 107
Initiating a DMA Transfer In a Client Driver on page 107
DMA Client Driver With Error Status Check on page 109
Debugging DMA Drivers on page 111

3.12. DMA Client Driver With Error Status Check

On particular Power-PC board support packages (BSPs), the status of the DMA transfer is available to the application once it has completed.

DMA status reporting is available on the following BSPs using the **DRV_FSL_DMA** driver component:

- fsl_p1p2
- fsl_p3p4p5
- fsl_t1

- fsl_t2t4

The following example shows how to make use of the functions and methods described in the DMA library:

```
VXB_DMA_SLAVE_CONFIG config;
VXB_DMA_TX_FN_ARGS usrDmaTxFnArgs;

# allocate DMA channel
pDmaChan = vxbDmaChanAlloc(uCap);


# configure the DMA channel - (optional)
memset ((void*)&config, 0, sizeof(VXB_DMA_SLAVE_CONFIG));

config.direction      = VXB_DMA_DEV_TO_MEM;
config.src            = DEVICE_RECEIVE_REG;
config.dst            = receive_buffer;
config.srcAddrWidth   = burst_size;
config.srcMaxBurst    = max_burst;

vxbDmaChanConfig(pDmaChan, (void *)&config);

# set the callback function and parameter
pVxbDmaTx = vxbDmaChanPreMemCpy(pDmaChan, src, dest, len, VXB_DMA_RETURN_STATUS);

usrDmaTxFnArgs.pArgs = pArg;
pVxbDmaTx->fn = fn;
pVxbDmaTx->pArg = (void *)&usrDmaTxFnArgs;


# start the DMA transfer
vxbDmachanStart(pVxbDmaTx);
```

Once the DMA transfer completes, and the user call back routine has been called, the requested DMA channel status is found in the structure member **usrDmaTxFnArgs.xfrStatus**. A status of 0 means success, while a status of 1 indicates an error occurred during the DMA transfer.

Note:

The hardware NLWR (no last write with response) bit controls how the DMA status is returned by the hardware. For details, refer to your NXP Processor Reference Manual. By default, NLWR is cleared before the transfer takes place. If you require NLWR to be set before the DMA transfer takes place, the call to vxbDmaChanPreMemCpy( ) must be modified as follows:

```
pVxbDmaTx = vxbDmaChanPreMemCpy(pDmaChan, src, dest, len, VXB_DMA_RETURN_STATUS | VXB
_DMA_VENDOR_DEFINED0);
```

Note:

Refer to DMA Library API for DMA Client Drivers on page 84 for a description of uCap.

Related information

DMA Driver Overview on page 82
DMA Library API for DMA Controller Drivers on page 82
DMA Library API for DMA Client Drivers on page 84
DMA Buffer Library API on page 90
VxBus Driver Methods for DMA Drivers on page 103
DMA Driver Header Files on page 105
DMA Driver Configuration on page 106
Available Utility Functions for DMA Drivers on page 107

DMA Driver Initialization on page 107
Initiating a DMA Transfer In a Client Driver on page 107
DMA Client Driver on page 108
Debugging DMA Drivers on page 111

3.13. Debugging DMA Drivers

DMA drivers are tested when your VxWorks system is fully initialized. For this reason, debugging can be performed using the full debug capabilities of VxWorks in addition to conventional instrumentation techniques such as logMsg( ).

When debugging DMA drivers, the only complicating factor is that DMA drivers cannot be tested alone. Because they provide a service to other drivers in the system, they must be tested with another driver. For debugging purposes, you can write a dummy driver that calls the functions in **vxbDmaLib** to allocate a DMA channel and initiate mock DMA transfers.

Related information

DMA Driver Overview on page 82
DMA Library API for DMA Controller Drivers on page 82
DMA Library API for DMA Client Drivers on page 84
DMA Buffer Library API on page 90
VxBus Driver Methods for DMA Drivers on page 103
DMA Driver Header Files on page 105
DMA Driver Configuration on page 106
Available Utility Functions for DMA Drivers on page 107
DMA Driver Initialization on page 107
Initiating a DMA Transfer In a Client Driver on page 107
DMA Client Driver on page 108
DMA Client Driver With Error Status Check on page 109

4. GPIO Drivers

4.1. GPIO Driver Overview

A general-purpose input/output (GPIO) driver is a designed to control a generic pin on a chip. The pin behavior is controlled by the user at run time. This driver type is supported for ARM and PowerPC-based systems.

GPIO devices can be configured to act as input or output, read/write values, or can be used as an IRQ pin.

Related information

VxBus Driver Methods for GPIO Drivers on page 111
Driver Header Files for GPIO on page 112
Device Tree Binding for GPIO Drivers on page 112
GPIO Driver Initialization on page 113
GPIO Driver Controller Structure on page 114
GPIO Driver Functions on page 116
GPIO Controller Driver Implementation on page 118
GPIO Client Device Driver Implementation on page 119

4.2. VxBus Driver Methods for GPIO Drivers

General-purpose input/output (GPIO) drivers do not require any device-specific VxBus driver methods.

GPIO drivers do not require a method to interact with the clock system. Instead, these drivers use a series of callbacks to implement the interaction with the clock system. For more information, see GPIO Driver Controller Structure on page 114 and GPIO Driver Functions on page 116.

Related information

GPIO Driver Overview on page 111
Driver Header Files for GPIO on page 112
Device Tree Binding for GPIO Drivers on page 112
GPIO Driver Initialization on page 113
GPIO Driver Controller Structure on page 114
GPIO Driver Functions on page 116
GPIO Controller Driver Implementation on page 118
GPIO Client Device Driver Implementation on page 119

4.3. Driver Header Files for GPIO

General-purpose input/output (GPIO) drivers must include the **vxbGpioLib.h** header file.

All VxBus GPIO drivers must include a single header file to provide the data types required for the driver:

```
#include <subsys/gpio/vxbGpioLib.h>
```

This header file is located in:

installDir **/vxworks-7/pkgs_v2/os/drv/vxbus/subsystem/h/subsys/gpio/vxbGpioLib.h**

Related information

GPIO Driver Overview on page 111
VxBus Driver Methods for GPIO Drivers on page 111
Device Tree Binding for GPIO Drivers on page 112
GPIO Driver Initialization on page 113
GPIO Driver Controller Structure on page 114
GPIO Driver Functions on page 116
GPIO Controller Driver Implementation on page 118
GPIO Client Device Driver Implementation on page 119

4.4. Device Tree Binding for GPIO Drivers

The properties of a general-purpose input/output (GPIO) device can be configured using a device tree for systems that support the flattened device tree (FDT) model.

The properties of a GPIO device can be managed using a device tree source (DTS) file. When a DTS file is used, and depending on the device type, certain device tree properties are supported.

GPIO Controllers

For GPIO controllers, the following property is supported:

#gpio-cells

This property indicates how many parameters the GPIO controller requires to choose a GPIO pin. The parameters are defined by the GPIO controller driver. They can be an offset in the GPIO bank, directions, or any other property the driver supports.

When the GPIO user node is referring to the GPIO pin, it must read the GPIO controller driver's device tree source file to obtain the parameters.

An example is as follows:

```
gpio0: gpio@44e07000
    {
    compatible = "someone,gpio";
    gpio-controller;
    #gpio-cells = <2>;
    ...
    };
```

In this example, the GPIO controller requires two parameters to identify the GPIO pin.

GPIO Users

A GPIO user is any device node that wants to make use of a GPIO pin. A GPIO user can be any device node.For GPIO users, the following property is supported:

any_gpio_property

This property contains the list of GPIO controller **phandle** and GPIO parameter pairs; each pair represents a single GPIO pin that can be used.

An example is as follows:

```
node
    {
    ...
    row-gpios = <&gpio0 12 0>, <&gpio0 13 0>;
    column-gpios = <&gpio0 0 0>, <&gpio0 1 0>;
    ...
    };
```

In this example, the node requires four GPIO pins; two for row GPIOs and two for column GPIOs.

Related information

GPIO Driver Overview on page 111
VxBus Driver Methods for GPIO Drivers on page 111
Driver Header Files for GPIO on page 112
GPIO Driver Initialization on page 113
GPIO Driver Controller Structure on page 114
GPIO Driver Functions on page 116
GPIO Controller Driver Implementation on page 118
GPIO Client Device Driver Implementation on page 119

4.5. GPIO Driver Initialization

General-purpose input/output (GPIO) drivers use the generic VxBus GEN2 initialization process.

You driver must also call the vxbGpioAddCtlr( ) function to register the GPIO subsystem during the VxBus initialization attach phase (vxbDevAttach( )). This step guarantees that the VxBus GPIO library (**vxbGpioLib**) is available to other device drivers in the system.

Related information

GPIO Driver Overview on page 111
VxBus Driver Methods for GPIO Drivers on page 111
Driver Header Files for GPIO on page 112

Device Tree Binding for GPIO Drivers on page 112
GPIO Driver Controller Structure on page 114
GPIO Driver Functions on page 116
GPIO Controller Driver Implementation on page 118
GPIO Client Device Driver Implementation on page 119

4.6. GPIO Driver Controller Structure

The VxBus GPIO subsystem uses a GPIO controller structure to manage GPIO controllers.

The general-purpose input/output (GPIO) controller structure contains variables and function pointers to manage the GPIO controller. When implementing a GPIO controller driver, you must fill this structure according to your implementation.

The GPIO controller structure is defined as follows:

```
typedef struct vxbGpioCtrl {
    SL_NODE gpioNode;    /* this node */
    VXB_DEV_ID  pDev;    /* pointer to the controller instance */
    UINT32  firstPin;    /* the first pin id for this controller */
    UINT32  length;      /* how many gpio in this controller */
    UINT32  *pValidBmp;  /* this is a valid bitmap for this controller.
                                bit 0 means free, 1 means used. */
UINT32  gpioCells;   /* parameter numbers when called by other module */

    /* allocate a pin. */
    STATUS (*gpioAlloc)
        (
        struct vxbGpioCtrl *pCtrl,
        UINT32 id                   /* pin to allocate */
        );
    /* free a pin */
    STATUS (*gpioFree)
        (
        struct vxbGpioCtrl *pCtrl,
        UINT32 id                   /* pin to free */
        );
    /* get direction of a pin */
    UINT32 (*gpioGetDir)
        (
        struct vxbGpioCtrl *pCtrl,
        UINT32 id
        );
    /* set direction of a pin */
    STATUS (*gpioSetDir)
        (
        struct vxbGpioCtrl *pCtrl,
        UINT32 id,
        UINT32 dir
        );
    /* get value of a pin */
    UINT32 (*gpioGetValue)
        (
        struct vxbGpioCtrl *pCtrl,
        UINT32 id
        );
    /* set value to a pin */
    STATUS (*gpioSetValue)
```

```
        (
        struct vxbGpioCtrl *pCtrl,
        UINT32 id,
        UINT32 value
        );
    /* set debounce time in micro second (us) */
    STATUS (*gpioSetDebounce)
        (
        struct vxbGpioCtrl *pCtrl,
        UINT32 id,
        UINT32 us
        );
    /* find the global GPIO id for given GPIO arg list */
    int  (*gpioFindByFdt)
        (
        struct vxbGpioCtrl *pCtrl,
        UINT32* pArgs
        );
    /* bind the GPIO */
    int  (*gpioBind)
        (
        struct vxbGpioCtrl *pCtrl,
        UINT32* pArgs
        );
    /* config interrupt trigger mode */
    STATUS (*gpioIntConfig)
        (
        struct vxbGpioCtrl *pCtrl,
        UINT32 id,
        INTR_TRIGER trig,
        INTR_POLARITY pol
        );
    /* connect interrupt to this gpio */
    STATUS (*gpioIntConnect)
        (
        struct vxbGpioCtrl *pCtrl,
        UINT32 id,
        VOIDFUNCPTR     pIsr,       /* ISR */
        void *          pArg        /* parameter */
        );
    /* disconnect interrupt from this gpio */
    STATUS (*gpioIntDisConnect)
        (
        struct vxbGpioCtrl *pCtrl,
        UINT32 id,
        VOIDFUNCPTR     pIsr,       /* ISR */
        void *          pArg        /* parameter */
        );
    /* enable interrupt to this gpio */
    STATUS (*gpioIntEnable)
        (
        struct vxbGpioCtrl *pCtrl,
        UINT32 id,
        VOIDFUNCPTR     pIsr,       /* ISR */
        void *          pArg        /* parameter */
        );
    /* disable interrupt to this gpio */
```

```
    STATUS (*gpioIntDisable)
        (
        struct vxbGpioCtrl *pCtrl,
        UINT32 id,
        VOIDFUNCPTR     pIsr,       /* ISR */
        void *          pArg        /* parameter */
        );
    /* show info */
    VOID (*gpioShow)
        (
        struct vxbGpioCtrl *pCtrl,
        UINT32 verbose
        );
} VXB_GPIOCTRL;
```

Note: Depending on your specific target hardware, not all functions provided in the example are implemented. If your hardware does not support a particular function, define it as **NULL**.

Related information

GPIO Driver Overview on page 111
VxBus Driver Methods for GPIO Drivers on page 111
Driver Header Files for GPIO on page 112
Device Tree Binding for GPIO Drivers on page 112
GPIO Driver Initialization on page 113
GPIO Driver Functions on page 116
GPIO Controller Driver Implementation on page 118
GPIO Client Device Driver Implementation on page 119

4.7. GPIO Driver Functions

Several functions are available for use by general-purpose input/output (GPIO) controller drivers and GPIO users. Most functions are based on the concept of a GPIO ID.

GPIO ID

Generally, the GPIO device is organized as a series of pin banks with each bank containing a certain number of pins. The GPIO subsystem assumes that each bank contains the same number of pins. The number of the first pin of the first bank is **0**, and the first pin of the second bank as *bank_length*.

For example, if a chip has four GPIO banks and each bank has four pins (that is, the *bank_length* is **4**), the GPIO IDs for the pins are as follows:

Bank	Pins			
Bank A	0	1	2	3
Bank B	4	5	6	7
Bank C	8	9	10	11
Bank D	12	13	14	15

Note: The GPIO ID is a global ID. The GPIO user does not need to know in which bank the pin is located.

GPIO Controller Driver Functions

The following function is available to your GPIO controller driver:

vxbGpioAddCtlr( )

This function registers a GPIO controller with VxBus.

> Note: The memory designated in the **VXB_GPIOCTRL** structure must be allocated by the controller driver.

GPIO User Functions

The following functions are available to GPIO users:

vxbGpioAlloc( )

Allocates a GPIO pin. The accepted parameter is the GPIO ID. This function must be called before any other GPIO functions that require the GPIO ID parameter.

vxbGpioFree( )

Frees a GPIO pin.

vxbGpioIsFree( )

Checks if a GPIO pin is available.

vxbGpioSetDir( )

Sets the direction of the GPIO pin.

vxbGpioGetDir( )

Gets the direction of the GPIO pin.

vxbGpioGetValue( )

Reads the value of the GPIO pin.

vxbGpioSetValue( )

If the GPIO pin direction is set to output, this function can be used to set its value.

vxbGpioSetDebounce( )

If the GPIO controller supports setting a debounce time, this function is used to set the debounce time in microseconds.

vxbGpioIntConfig( )

If the GPIO device can be used as an IRQ, use this routine to configure the interrupt properties such as trigger polarity, level trigger, or edge and so forth.

vxbGpioIntConnect( )

Connect the GPIO interrupt to the GPIO user ISR.

vxbGpioIntDisconnect( )

Disconnect the GPIO interrupt from the GPIO user ISR.

vxbGpioIntEnable( )

Enable the GPIO interrupt.

vxbGpioIntDisable( )

Disable the GPIO interrupt.

Functions for GPIO Devices Running in an FDT-Based System

GPIO devices running in a VxWorks system that uses a flattened device tree (FDT) for configuration (ARM and PowerPC-based systems) can also use the following functions:

vxbGpioGetByFdtIndex( )

Gets the GPIO ID by parsing the GPIO user node in the device tree. (For more information, see Device Tree Binding for GPIO Drivers on page 112.)

vxbGpioBind( )

If the GPIO controller driver supports it, the GPIO subsystem can supply this bind function. The function allows the GPIO user to perform simple tasks. For example, it can execute user code that performs a simple operation on the GPIO device such as outputting a level signal from the GPIO to power up a device. This function obtains the GPIO configuration from the FDT node and then tells the GPIO controller driver to operate based on the configuration information provided. The required FDT property is similar to the following:

```
gpio_binds = <&gpio2 25 1 1 100
              &gpio2 26 1 0 100
              &gpio2 27 0 0 100>;
```

In this example, **&gpio2** is the GPIO 2 controller's **phandle**. The numbers that follow are driver-specified parameters that are set based on the **#gpio-cells**. The parameter can be set similar to the following:

localID direction value delay

As seen in the example, each row represents one GPIO pin. Therefore, in this case, there are three GPIO pins that are bound. Note that by calling this routine, the GPIO pin will automatically be allocated.

Note: When calling this function, the GPIO pins are automatically allocated.

vxbGpioUnBind( )

Unbinds a previously bound GPIO device and frees the GPIO pin.

Related information

GPIO Driver Overview on page 111
VxBus Driver Methods for GPIO Drivers on page 111
Driver Header Files for GPIO on page 112
Device Tree Binding for GPIO Drivers on page 112
GPIO Driver Initialization on page 113
GPIO Driver Controller Structure on page 114
GPIO Controller Driver Implementation on page 118
GPIO Client Device Driver Implementation on page 119

4.8. GPIO Controller Driver Implementation

To implement a general-purpose input/output (GPIO) controller device driver, you must implement certain functions and provide configuration for your device.

The general workflow for implementing a GPIO controller driver is as follows:

1. Decide what functions must be implemented for your target hardware system.
2. Set the GPIO device parameters for your target system in the device tree source (DTS) for flattened device tree (FDT)-based systems (PowerPC and ARM).
3. Implement the required routines and structures. (See GPIO Driver Controller Structure on page 114 and GPIO Driver Functions on page 116.)
4. Call the vxbGpioAddCtlr( ) function to add the GPIO controller to the GPIO subsystem.

Related information

GPIO Driver Overview on page 111
VxBus Driver Methods for GPIO Drivers on page 111
Driver Header Files for GPIO on page 112
Device Tree Binding for GPIO Drivers on page 112
GPIO Driver Initialization on page 113
GPIO Driver Controller Structure on page 114
GPIO Driver Functions on page 116
GPIO Client Device Driver Implementation on page 119

4.9. GPIO Client Device Driver Implementation

For general-purpose input/output (GPIO) users, you must get the GPIO ID, allocate it, and set its direction and value parameters.

For GPIO users in a flattened device tree (FDT)-based environment (ARM and PowerPC targets), the GPIO subsystem API usage is similar to the following:

```
gpioId = vxbGpioGetByFdtIndex (pDev, <gpio_prop_string>, 0);
                vxbGpioAlloc(gpioId);
                vxbGpioSetDir(gpioId, GPIO_DIR_OUTPUT);
                vxbGpioSetValue(gpioId, 1);
```

In the above example, you determine the GPIO ID, allocate the GPIO pin, and set its direction and value.

Related information

GPIO Driver Overview on page 111
VxBus Driver Methods for GPIO Drivers on page 111
Driver Header Files for GPIO on page 112
Device Tree Binding for GPIO Drivers on page 112
GPIO Driver Initialization on page 113
GPIO Driver Controller Structure on page 114
GPIO Driver Functions on page 116
GPIO Controller Driver Implementation on page 118

5. I2C Drivers

5.1. I2C Driver Overview

Wind River provides support for I2C devices. The VxBus I2C library provides a common read/write interface between the I2C controller and various I2C interface devices.

Each I2C controller has a different way to access the device. For example, the Freescale I2C controller is split into several parts according to the I2C timing sequence including the start, read, write, restart, ack, and stop cycles. However, some controllers do this in a completely different way. The software must first fill several registers. Then, the hardware starts the bus transaction which cannot be split precisely to the I2C timing order.

Considering the many available controllers, the VxBus I2C library is used to provide a common I2C read/write interface (vxbI2cDevXfer( )) between the hardware I2C controller and the various I2C interface devices. I2C device drivers use vxbI2cDevXfer( ) to access the I2C device directly. However, the I2C controller must implement the vxbI2cDevXfer( ) function using the {vxbI2cXfer}( ) method.

The **vxbI2cLib** library provides several functions to simplify access to I2C client devices. For more information, see Device Drivers for I2C Devices on page 123.

Related information

VxBus Driver Methods for I2C Drivers on page 120
Driver Header Files for I2C on page 121
I2C Driver Configuration on page 121
I2C Driver Initialization on page 122
Implementing Driver Service Functions on page 122
Device Drivers for I2C Devices on page 123

5.2. VxBus Driver Methods for I2C Drivers

All VxBus I2C drivers publish a single method, {vxbI2cXfer}( ). This method is used to provide a common read/write interface between I2C controllers and various I2C interface devices.

Within an I2C controller driver, the {vxbI2cXfer}( ) method is implemented using a driver-provided function with the following prototype:

```
STATUS vxbI2cXfer
(
VXB_DEV_ID pDev,
I2C_MSG * msgs,
int num
)
```

pDev

Describes the specific I2C controller instance within the system.

msgs

Contains the I2C device information, such as slave address, data length, read/write buffer, and so forth. For the type definition for **I2C_MSG**, see the following header file:

installDir **/vxworks-7/pkgs_v2/os/drv/vxbus/buslib/h/hwif/buslib/vxbI2cLib.h**

num

Describes how many messages need to be transferred.

Note: Your I2C driver, like all VxBus drivers, must also publish probe and attach methods. Typically, the attach method of the I2C controller device driver creates the downstream I2C devices as its children.

Related information

I2C Driver Overview on page 119
Driver Header Files for I2C on page 121
I2C Driver Configuration on page 121
I2C Driver Initialization on page 122
Implementing Driver Service Functions on page 122
Device Drivers for I2C Devices on page 123

5.3. Driver Header Files for I2C

VxBus I2C drivers must include the **vxbI2cLib.h** header file.

All VxBus I2C drivers must include a single header file to provide the data types required for the driver:

```
#include <hwif/buslib/vxbI2cLib.h>
```

This header file is located in:

installDir **/vxworks-7/pkgs_v2/os/drv/vxbus/buslib/h/hwif/buslib/vxbI2cLib.h**

Related information

I2C Driver Overview on page 119
VxBus Driver Methods for I2C Drivers on page 120
I2C Driver Configuration on page 121
I2C Driver Initialization on page 122
Implementing Driver Service Functions on page 122
Device Drivers for I2C Devices on page 123

5.4. I2C Driver Configuration

I2C drivers are configured using different sources of information in your system, depending on your target hardware architecture.

In general, I2C drivers do not require configuration information from the BSP beyond the normal device-specific information provided for all drivers. The configuration information is derived from the device tree source (DTS) files. The following example is for a I2C-specific DTS file for a Freescale P2020 rdb board:

```
i2c@3000 {
        #address-cells = <1>;
        #size-cells = <0>;
        cell-index = <0>;
        compatible = "fsl-i2c";
        reg = <0x3000 0x100>;
        clock-frequency = <250000000>;
        interrupts = <43 2 0 0>;
        interrupt-parent = <&mpic>;

            adt7461:temp-sensor@4c {
                compatible = "adi,adt7461";
                reg = <0x4c>;
            };

            at24c256:eeprom@50 {
                compatible = "at24, at24c256";
                reg = <0x50>;
            };

            ds1339:rtc@68 {
                compatible = "dallas, ds1339";
                reg = <0x68>;
                interrupts = <1 1 0 0>;
            };

        };
```

In this example, the **fsl-i2c** represents the Freescale I2C controller driver. The macro **DRV_FSL_I2C** must be defined in your project. The sub-table lists the I2C devices connected on this bus. The relevant I2C device driver must also be defined in your image. For EEPROM, the macros **DRV_I2C_EEPROM** and **INCLUDE_EEPROMDRV** must be defined in your project. For RTC, the macro **DRV_I2C_RTC** must also be defined in your project.

Related information

I2C Driver Overview on page 119
VxBus Driver Methods for I2C Drivers on page 120
Driver Header Files for I2C on page 121
I2C Driver Initialization on page 122
Implementing Driver Service Functions on page 122
Device Drivers for I2C Devices on page 123

5.5. I2C Driver Initialization

I2C drivers are initialized during the VxBus probe and attach phases.

Probe Phase

The vxbDevProbe( ) function matches the I2C device with an existing driver. The flattened device tree (FDT) describes an I2C device with compatible properties. The I2C driver also includes a table that lists supported I2C devices. This function compares the compatible strings and returns the minimum order number of matched compatible strings.

Attach Phase

During the VxBus attach initialization phase (vxbDevAttach( )), I2C drivers must initialize all of their internal data structures, perform any required initialization of the I2C hardware, and connect their driver-specific ISR to the interrupt source. For I2C controller drivers, an I2C device instance is created and connected within the bus by calling the vxbDevCreate( ) function.

Typically, during this phase, the I2C controller driver creates semaphores (typically two) for future use. The first semaphore is a mutex semaphore that is used to protect a shared resource. For example, if you have two I2C devices that want to access the same bus controller, one must be held until the other is done accessing the bus controller.

The second semaphone is used to keep the ISR and the task in sync. In this case, using semTake( ) to take a binary sync semaphore when the transfer is done. Then, waiting until a hardware interrupt occurs and giving the semaphone (using semGive( )) in an ISR context. This tells the task that the transfer is complete.

Related information

I2C Driver Overview on page 119
VxBus Driver Methods for I2C Drivers on page 120
Driver Header Files for I2C on page 121
I2C Driver Configuration on page 121
Implementing Driver Service Functions on page 122
Device Drivers for I2C Devices on page 123

5.6. Implementing Driver Service Functions

All I2C controller drivers must implement a transfer function with the requested I2C timing.

The transfer function handles read and write operations on a given I2C target. The prototype for this function is as follows:

```
LOCAL STATUS xxxI2cTransfer
    (
    VXB_DEV_ID pDev,
```

```
    I2C_MSG * msgs,
    int num
    )
```

Typically, this function has the following implementation:

1. The function uses a mutual semaphore to lock the bus, allowing only one device to be accessed at a time.
2. The function starts the transaction on a given target. It follows the I2C protocol by issuing I2C start, send target address, read, and write commands.
3. The function releases the mutual semaphore, unlocking the bus.

Related information

I2C Driver Overview on page 119
VxBus Driver Methods for I2C Drivers on page 120
Driver Header Files for I2C on page 121
I2C Driver Configuration on page 121
I2C Driver Initialization on page 122
Device Drivers for I2C Devices on page 123

5.7. Device Drivers for I2C Devices

Wind River provides device drivers for several EEPROM and real-time clock (RTC) I2C devices, which can be used as references for using the I2C library APIs to access I2C client devices.

The vxbI2cDevXfer( ), interface, which is implemented by the I2C controller device driver's {vxbI2cXfer}() method, can be used by all device drivers for I2C client devices.

The prototype for this function is:

```
STATUS vxbI2cDevXfer
    (
    VXB_DEV_ID pDev,
    I2C_MSG  * msgs,
    int num
)
```

pDev

Describes the accessed I2C device instance within the system.

msgs

Contains the I2C device information, such as slave address, data length, read/write buffer, and so forth. For the type definition for **I2C_MSG**, see the following header file:

installDir **/vxworks-7/pkgs_v2/os/drv/vxbus/buslib/h/hwif/buslib/vxbI2cLib.h**

num

Describes how many messages need to be transferred.

The VxBus I2C library provides the following functions to simplify access to I2C client devices:

- vxbI2cDevRead( )
- vxbI2cDevWrite( )
- vxbI2cByNameRead( )

- vxbI2cByNameWrite( )

These functions are based on the vxbI2cDevXfer( ) interface. For more information, see the **vxbI2cLib** API reference entry.

Related information

I2C Driver Overview on page 119
VxBus Driver Methods for I2C Drivers on page 120
Driver Header Files for I2C on page 121
I2C Driver Configuration on page 121
I2C Driver Initialization on page 122
Implementing Driver Service Functions on page 122

6. Interrupt Controller Drivers

6.1. Interrupt Controller Driver Overview

VxBus provides support for interrupt controller hardware management. Interrupt controller drivers are used to configure and manage interrupt controller hardware in your system.

All interrupt controller drivers must be VxBus compliant.

When writing a VxBus interrupt controller driver, you must have a thorough understanding of the VxBus device driver framework (see Basic VxBus Concepts on page 40). Wind River strongly recommends that you review the VxBus interrupt controller drivers provided with your product to find the one that most closely matches the hardware you are working with and then use that driver as a model for your development.

Interrupt Identification

Within the context of VxBus, an interrupt is considered to be an entity specific to the device that generates the interrupt. That is, all interrupts are identified by the VxBus device and an interrupt index. This uniquely identifies every interrupt source on the system based on what generates the interrupt.

From the perspective of an interrupt controller, you must also refer to interrupts by the input pins on which the interrupt is connected. When discussing interrupt controllers, this is referred to as the interrupt input. Interrupt identification is discussed further in Interrupt Inputs Structure on page 142.

Interrupt Controller Driver Responsibilities

The interrupt controller driver is responsible for enabling, disabling, and routing the interrupt to a specific core. It also manages interrupt input characteristics such as trigger type (edge trigger or level trigger), trigger value (active high or active low), and other characteristics of the interrupt source.

The interrupt library is responsible for maintaining the interrupt controller and ISRs for each interrupt input. It is also responsible for the argument that is passed to each ISR.

The **vxbIsrHandler** library provides functions to help manage the ISR connections. It serves as the base library for **vxbIntLib**, **vxbDyncIntLib**, and **vxIpiLib**. The library attempts to dispatch ISRs in the most efficient manner possible. When a single ISR is connected, the ISR is dispatched directly. When multiple ISRs are connected, the library creates a chain of ISR handlers to call when an interrupt occurs. The vxbIsrHandler( ) function should only be used by the static, dynamic, and interprocessor interrupt (IPI) libraries. The interrupt controller driver should not use it, except for in the ISR dispatch routine (**VXB_INT_ISR_CALL**).

The following libraries are available for use by your interrupt controller driver:

vxbIntLib

The **vxbIntLib** library provides routines to manager static-type interrupt controllers. There can be multiple interrupt controller in system, such as when the interrupt controllers are cascaded. When a device needs to connect or enable an interrupt, this library is used to help the device find the correct interrupt controller and call its method, if needed.

vxbDyncIntLib

The **vxbDyncIntLib** library provides functions to manage a dynamic interrupt controller. It also supplies an API to support dynamically allocating or freeing interrupt resources. Some hardware platforms allow dynamic assignment of interrupt identifiers. Individual bus types can define a bus-specific mechanism for handling dynamic vectors. For a PCI bus, this includes MSI and MSI-X.

vxIpiLib

The **vxIpiLib** library provides routines to manage IPI-type interrupt controllers. It also supplies the IPI API for the kernel system. When the system needs to connect, enable, or emit an IPI interrupt, this library is used to help manage the IPI ISR and invoke the corresponding method in IPI device driver.

Related information

Driver Methods for Interrupt Controller Drivers on page 125
Interrupt Controller Header Files on page 130
Interrupt Controller Driver Configuration on page 131
Interrupt Controller Driver Initialization on page 134
Interrupt Controller Topology on page 136
Cascading Interrupt Controller Support on page 137
ISR Dispatch on page 139
Dynamic Interrupt Vectors on page 141
Interrupt Inputs Structure on page 142
VxWorks SMP Considerations on page 144
Debugging Considerations on page 146

6.2. Driver Methods for Interrupt Controller Drivers

VxBus device driver methods are provided for handling basic functionality, dynamic vector assignment, and multiprocessor systems.

There are three groups of driver methods relevant to interrupt controller drivers. The first group is required for basic interrupt controller functionality. The second group deals with issues related to dynamic vector assignment. The last group deals with issues related to multiprocessor systems.

Basic Methods

There are four driver methods related to basic interrupt functionality: {vxbIntConfig}( ), {vxbIntUnconfig]( ), {vxbIntEnable}( ), and {vxbIntDisable}( ).

Method	Description

{vxbIntConfig}( )	The {vxbIntConfig}( ) method configures the hardware for the specified interrupt. That is, it sets the interrupt's vector number, polarity, sense and priority, and so forth.
	`STATUS func {vxbIntConfig}`
	`(`
	`VXB_DEV_ID pDev,`
	`UINT32 vector,`

Method	Description

	`VXB_INTR_ENTRY * pVxbIntrEntry`
	`)`
	
	The parameters are as follows:
	
	pDev
	The device instance that wants to use the interrupt.
	vector
	The interrupt vector number.
	pVxbIntrEntry
	Points to the structure that stores the specifier definitions for the interrupt pin.

{vxbIntUnconfig}( )	The {vxbIntUnconfig}( ) method can be used by the interrupt controller to reverse any setup performed by the complementary {vxbIntConfig}( ) method. Not every interrupt controller may need this functionality, but it's provided in case a given platform requires it.
	
	`STATUS func {vxbIntUnconfig}`
	`(`
	`VXB_DEV_ID pDev,`
	`UINT32 vector,`
	`VXB_INTR_ENTRY * pVxbIntrEntry`
	`)`
	
	The parameters are as follows:
	
	pDev
	The device instance that wants to use the interrupt.
	vector
	The interrupt vector number.
	pVxbIntrEntry
	Points to the structure that stores the specifier definitions for the interrupt pin.

{vxbIntEnable}( )	The {vxbIntEnable}( ) method enables the interrupt input and marks the specified ISR as enabled.
	
	`STATUS func {vxbIntEnable}`
	`(`
	`VXB_DEV_ID pDev,`
	`UINT32 vector,`
	`VXB_INTR_ENTRY * pVxbIntrEntry`
	`)`
	
	Note that if an interrupt vector is shared (that is, two devices are tied to the same pin), this method will only be called when

Method	Description
	vxbIntEnable( ) is invoked the first time, since enabling the pin twice would be redundant. Also, running the enable process a second time for a new device might interfere with interrupt delivery for an existing device that has already been turned on.

{vxbIntDisable}( )	The {vxbIntDisable}( ) method marks the specified ISR as disabled. If there are no other enabled ISRs chained to the same interrupt input, the routine disables the input.
	`STATUS func {vxbIntDisable}`
	`( VXB_DEV_ID pDev, UINT32 vector, VXB_INTR_ENTRY * pVxbIntrEntry )`
	Note that if an interrupt vector is shared (that is, two devices are tied to the same pin), this method will only be called when vxbIntDisable( ) is invoked for the last time. If device A and device B are both connected to the same vector, and the driver for device A calls vxbIntDisable( ), it would be incorrect to mask the interrupt off in the PIC since that would prevent interrupt delivery to device B as well, which may still need to service events. Therefore, the {vxbIntDisable}( ) method is only called when the last device that is still expecting interrupts on a given vector calls vxbIntDisable( ).

{vxbIntVecSet}( )	The {vxbIntVecSet}( ) method should be provided by drivers for interrupt controllers that implement the vectored interrupt model. In the vectored model case, the driver does not need to use VXB_INT_ISR_CALL( ) to dispatch ISRs. This is done by the hardware itself. However, some platform-dependent setup must typically be performed in order to bind an ISR to an entry in the hardware interrupt dispatch table. The interrupt subsystem uses {vxbIntVecSet}( ) to provide the controller driver with the ISR routine and argument for a given vector that it manages, so that the driver can perform this platform-dependent setup. Most VxWorks CPU architecture libraries implement a routine such as intArchConnect( ) or excVecSet( ) for this purpose. Note that the VxBus interrupt system handles chaining of shared interrupt vectors internally, so the controller driver does not need to implement chaining itself. Drivers for interrupt controllers that implement the polled model (for example, ARM and PowerPC) are not required to implement this method. Instead, they should perform software dispatching of interrupts using VXB_INT_ISR_CALL( ). If a driver does not implement this method, the VxBus interrupt subsystem will simply skip the callback. A polled model driver may still include this methods, if the driver designer chooses to monitor the vector setup and teardown events.
	`STATUS func {vxbIntVecSet}`
	`( VXB_DEV_ID pDev, UINT32 vector,`

Method	Description

	`VXB_INTR_ENTRY * pVxbIntrEntry,`
	`VOIDFUNCPTR pFunc,`
	`void * pArg`
	`)`

{vxbIntVecClear}( )	The status{vxbIntVecClear}( )
	`STATUS func {vxbIntVecClear}`
	`(`
	`VXB_DEV_ID pDev,`
	`UINT32 vector,`
	`VXB_INTR_ENTRY * pVxbIntrEntry,`
	`VOIDFUNCPTR pFunc,`
	`void * pArg`
	`)`
	The {vxbIntVecClear}( ) method is complementary to {vxbIntVecSet}( ). Like {vxbIntVecSet}( ), it is only required for drivers that handle vectored model interrupt controllers. This method is called by the VxBus interrupt system to unbind an ISR handler from a vector in the hardware's interrupt dispatch table. This method should undo any setup performed by {vxbIntVecSet}( ). In the case where a single vector is shared by multiple devices, {vxbIntVecSet}( ) may be called to disconnect a specific device's ISR from the vector, and replace it with the VxBus interrupt subsystem's chained interrupt handler instead. A typical implementation of this method might call a CPU architecture routine such as intArchDisconnect( ), or possibly call excVecSet( ) a second time with an NULL ISR pointer. As with {vxbIntVecSet}( ), {vxbIntVecClear}( ) is optional for drivers that support polled interrupt model controllers. If the method is not implemented, the VxBus interrupt subsystem will simply skip the callback.

{vxbIntMap}( )	When the standard DTB is used, the **hVec** is parsed by the interrupt controller using this method which will override the library assigned value (with the first cell). For more information, see Interrupt Controller Driver Configuration on page 131.
	Note: If the DTB mode is legacy, don't override the **hVec** and return ERROR.
	`STATUS VXB_INT_MAP`
	`(`
	`VXB_DEV_ID pDev,`
	`UINT32 * hVec,`
	`VXB_INTR_ENTRY * pVxbIntrEntry`
	`)`

Dynamic Vector Methods

There are two driver methods related to dynamic vectors: {vxbIntAlloc}( ) and {vxbIntFree}( ).

Note: These methods are only required for support of PCI devices with multiple MSI and MSI-X support.

Method	Description

{vxbIntAlloc}( )	The {vxbIntAlloc}( ) method allocates specified interrupt resources for the VxBus device. The device driver, or its parent driver, implement the actual allocate function and fill in the interrupt information for the device.
	`int {vxbIntAlloc}`
	`(`
	`VXB_DEV_ID pDev,`
	`int count,`
	`VXB_DYNC_INT_ENTRY * pVxbDyncIntEntry`
	`)`

{vxbIntFree}( )	The {vxbIntFree}( ) method releases all allocated dynamic interrupt resources for the device. The device driver, or its parent driver, implement the actual allocate function and fill in the interrupt information for the device.
	`void {vxbIntFree}`
	`(`
	`VXB_DEV_ID pDev,`
	`int count,`
	`VXB_DYNC_INT_ENTRY * pVxbDyncIntEntry`
	`)`

Multiprocessor Methods

There are two driver methods related to dynamic vectors: {vxbIntBind}( ) and {vxbIntIpi}( ).

Method	Description

{vxbIntBind}( )	The {vxbIntBind}( ) method reroutes a specified interrupt from the CPU to which it is currently routed, to the CPU specified by the **destCpu** argument.
	`STATUS func {vxbIntBind}`
	`(`
	`VXB_DEV_ID pDev,`
	`UINT32 vector,`
	`UINT32 cpu,`
	`VXB_INTR_ENTRY * pVxbIntrEntry`
	`)`
	The interrupt is specified by the device and index indicated in the arguments. All interrupts connected to the same input are rerouted together.

{vxbIntIpi}( )	Interprocessor interrupts (IPIs) are used for various purposes in multiprocessor systems. The {vxbIntIpi}( )

Method	Description
	method is used to generate an IPI interrupt on the target CPU. `STATUS func {vxbIntIpi} ( VXB_DEV_ID pDev, UINT32 vector, phys_cpuset_t phys_cpus )`

Related information

Interrupt Controller Driver Overview on page 124
Interrupt Controller Header Files on page 130
Interrupt Controller Driver Configuration on page 131
Interrupt Controller Driver Initialization on page 134
Interrupt Controller Topology on page 136
Cascading Interrupt Controller Support on page 137
ISR Dispatch on page 139
Dynamic Interrupt Vectors on page 141
Interrupt Inputs Structure on page 142
VxWorks SMP Considerations on page 144
Debugging Considerations on page 146

6.3. Interrupt Controller Header Files

Interrupt controller drivers must include the interrupt controller library header file (vxbIntLib.h( )) in order to use the available library functions.

The interrupt controller library header file, vxbIntLib.h( ), is the only header file available to interrupt controller drivers. This header file provides access to the **vxbIntLib** functions. Wind River strongly recommends that you use these functions.

This header file is located in:

installDir **/vxworks-7/pkgs_v2/os/drv/vxbus/subsystem/h/subsys/int**

Your interrupt controller driver can include it as follows:

```
#include <subsys/int/vxbIntLib.h>
```

For dynamic interrupt controllers, your driver must also include the dynamic library header file, **vxbDyncIntLib.h**. Include the file as follows:

```
#include <subsys/int/vxbDyncIntLib.h>
```

Related information

Interrupt Controller Driver Overview on page 124
Driver Methods for Interrupt Controller Drivers on page 125
Interrupt Controller Driver Configuration on page 131
Interrupt Controller Driver Initialization on page 134
Interrupt Controller Topology on page 136
Cascading Interrupt Controller Support on page 137
ISR Dispatch on page 139
Dynamic Interrupt Vectors on page 141

Interrupt Inputs Structure on page 142
VxWorks SMP Considerations on page 144
Debugging Considerations on page 146

6.4. Interrupt Controller Driver Configuration

Interrupt controller drivers are configured using various sources of information in your system, depending on your target hardware architecture.

Interrupt controller drivers are configured using device tree source (DTS) files included in your BSP.

For Intel Architecture, interrupt controller drivers are configured using information from multiple sources. In addition to FDT, information comes from the BIOS, derived from either MultiProcessor (MP) tables or Advanced Configuration and Power Interface (ACPI), depending on your hardware.

The device registers for almost all interrupt controllers are located logically on the processor bus.

Interrupt controller drivers must be described in the DTS files in the target hardware BSP. For more information on how the DTS files are organized, see the following standards documentation:

- Power.org™ Standard for Embedded Power Architecture™ Platform Requirements (ePAPR)
- Open Firmware 3 Recommended Practice: Interrupt Mapping

Interrupt Controller Node

The interrupt controller device is described as a node in a DTS file. Any configuration information required by the driver is stored in the node.

The following example is taken from the **fsl_p1p2** BSP. In this example, the interrupt controller is an MPIC and the interrupt pin of the I2C controller is 27. The IPI interrupt pins are 224 to 232. This information is described in the **P2020ds.dts** as follows:

```
/* mpic interrupt controller with 4 cells per specifier */

    mpic: pic@40000 {
            interrupt-controller;
            #address-cells = <0>;
            #interrupt-cells = <4>;
            reg = <0x40000 0x40000>;
            };

/*
* The P2020 I2C controller node has an internal
* interrupt number of 27.  As per the reference manual
* this corresponds to interrupt source configuration
* registers at 0x5_0560.
*
* The interrupt source configuration registers begin
* at 0x5_0000.
*
* To compute the interrupt specifier interrupt number
*
*       0x560 >> 5 = 43
*
* The interrupt source configuration registers begin
* at 0x5_0000, and so the i2c vector/priority registers
* are at 0x5_0560.
```

```
*/

    i2c@3000 {
            #address-cells = <1>;
            #size-cells = <0>;
            cell-index = <0>;
            compatible = "fsl-i2c";
            reg = <0x3000 0x100>;
            clock-frequency = <250000000>;
            interrupts = <43 2 0 0>;
            interrupt-parent = <&mpic>;
            }


    msi@41600 {
            compatible = "fsl,mpic-msi";
            reg = <0x41600 0x80>;
            msi-available-ranges = <0 0x100>;
            interrupts = <
                0xe0 0 0 0
                0xe1 0 0 0
                0xe2 0 0 0
                0xe3 0 0 0
                0xe4 0 0 0
                0xe5 0 0 0
                0xe6 0 0 0
                0xe7 0 0 0>;
            interrupt-parent = <&mpic>;
            };
```

The example includes two interrupt controllers; one is the MIPC, the other is the MSI.

The MPIC device is a static interrupt controller which manages all of the devices that connect interrupts directly, such as the I2C bus device. The interrupt-parent property designates that its interrupt controller is **mpic**, and the interrupts property provides the interrupt and interrupt-specific information. This information is passed to the interrupt controller and the input pin is configured after the information is parsed by the interrupt controller.

The MSI device is a dynamic interrupt controller. In this example, it is the second level interrupt controller. This device takes eight vectors in the EPIC interrupt controller and supplies 256 dynamic vectors for other devices, such as PCIe devices.

A device could also use multiple interrupts which could be static or dynamic. For static interrupts, the interrupts property list all interrupts. Using the etsec driver as an example, the driver takes three interrupts as follows:

```
interrupts = <31 2 0 0 32 2 0 0 33 2 0 0>
```

For dynamic interrupts, the property must specify the interrupt number that is needed when a call to vxbIntAlloc( ) is made.

When the device needs to connect an interrupt, the second parameter for the vxbIntConnect( ), vxbIntDisconnect( ), vxbIntEnable( ), or vxbIntDisable( ) function is used to index the interrupt. The first interrupt is interrupt 0. The second interrupt is interrupt 1.

Interrupt-Specific Information

Some interrupt controllers support CPU routing, interrupt priority, trigger mode, and so forth. Some interrupt controllers do not support this functionality. The VxBus infrastructure does not supply a uniform framework to support these features but rather leaves it to the driver to handle this functionality. The interrupt controller can support this functionality together with the DTS file. The

interrupts property can be defined arbitrarily with the exception of the first parameter. The first parameter must be the interrupt vector internal to the interrupt controller.

For example, in the MPIC, the interrupts property is defined as follows:

vectorgrouptriglevelpriorityCPU

Therefore, if you want to assign the priority for an input pin, you can set it using the fifth parameter. This is a shared rule between the DTS and the interrupt controller. Because this rule is shared, only changing the DTS is not effective. The interrupt controller must parse the values using the same rules.

The interrupt controller should apply these rules in the {vxbIntConfig}( ) method. The third parameter (**VXB_INTR_ENTRY** * pVxbIntrEntry) contains the entries for the interrupts property in an array. For more detail, refer to the definition of **VXB_INTR_ENTRY**.

Note: When adding or deleting an entry for the interrupts property, the #interrupt-cells of the interrupt controller node should also be changed to align with the size of interrupts. All of the interrupts properties in the DTS should have the same size. That is, if you add a priority option for a device, then all devices should set this option. If a specific device does not need to change the property, it can pass an invalid number. The interrupt controller can then exam the invalid number and ignore it as needed.

In the following example (based on the MPIC interrupt controller driver), the DMA controller's interrupt pin is routed to CPU1 and the priority is set to 15.

```
/* MPIC node */

mpic: pic@40000 {
        interrupt-controller;
        #address-cells = <0>;
        #interrupt-cells = <6>;
        reg = <0x40000 0x40000>;
        compatible = "fsl,mpic","chrp,open-pic";
        };

/* DMA controller node */

dma-channel@80 {
compatible = "fsl,eloplus-dma-channel";
reg = <0x80 0x80>;
cell-index = <1>;
interrupts = <21 0 0 0 15 1>;
};
```

Interrupt Cell Definitions

Refer to your interrupt controller driver source code to determine how interrupts are defined including the number of cells required (*#interrupt-cells* property). This is required because the interrupt cell format may be different depending on the device. For example, for the **ARM Generic Interrupt Controller (GIC)** (see **InstallDir\vxworks-7\pkgs_v2\os\arch\arm-2.0.4.0\kernel\drv\src\vxbFdtArmGenIntCtlr.c**), the format is:

```
interrupts = <34 0 4>;
```

Where:

- The first number (34) is the interrupt number

- The second number (0) is the interrupt type (SPI)
- The third number (4) is the priority (active high level trigger)

However, for **Xilinx Vivado 2017.4**, the format is:

```
interrupts = <0 34 4>;
```

Where:

- The first number (0) is the interrupt type (SPI)
- The second number (34) is the interrupt number
- The third number (4) is the priority (active high level trigger)

Setting hVec When Using Standard DTB

The VxWorks legacy DTB always assumes the first cell of the interrupts property is **hVec** because the interrupt library handles it this way (not the interrupt controller driver). However, you can use the standard DTB and keep legacy DTB support in your BSP. When the standard DTB is used, the **hVec** should be parsed by the interrupt controller using the vxbIntMap( ) method which will override the library assigned value (with the first cell).

The interrupt controller can dynamically check the DTB type and set the **pProp** field in **VXB_INTR_ENTRY** as follows:

Legacy DTB - vxFdtStdDtb( ) returns `FALSE`

> This means the **hVec** is the first cell. the vxbIntMap( ) method should not handle it in the standard DTB way and returns `ERROR`. The prop points to the second cell using the interrupt library and assumes the first cell is **hVec** (no parsing is required by the interrupt controller).

Standard DTB - vxFdtStdDtb( ) returns `TRUE`

> This means the **hVec** must be parsed by the interrupt controller using the vxbIntMap( ) method. The prop points to the first cell of interrupts property because vxbIntMap( ) method also needs the first cell to calculate the **hVec**. Other corresponding APIs which use this prop pointer also need to be enhanced to support the standard DTB mode (for example, the vxbIntConfig( ) method).

Related information

Interrupt Controller Driver Overview on page 124
Driver Methods for Interrupt Controller Drivers on page 125
Interrupt Controller Header Files on page 130
Interrupt Controller Driver Initialization on page 134
Interrupt Controller Topology on page 136
Cascading Interrupt Controller Support on page 137
ISR Dispatch on page 139
Dynamic Interrupt Vectors on page 141
Interrupt Inputs Structure on page 142
VxWorks SMP Considerations on page 144
Debugging Considerations on page 146

6.5. Interrupt Controller Driver Initialization

Initialization of the interrupt controller driver must be finished in the attaching phase.

The {vxbDevAttach}( ) method governs the attach phase. For more information, see VxBus Driver Methods on page 47.

The interrupt subsystem (interrupt library) supplies functions for both interrupt controllers and devices that want to make use of interrupts. For the more information about the API that allows devices to use interrupts, see VxBus Interrupt Handling on page 59.

For static interrupt controllers, the interrupt subsystem can also integrate interprocessor interrupt (IPI) support (as is done in the MPIC interrupt controller). The interrupt subsystem supplies a register function for this purpose, vxbIntRegister( ). When a device is a static interrupt controller, it must call vxbIntRegister( ) to register itself with the interrupt subsystem. For example:

```
STATUS vxbIntRegister
    (
    VXB_DEV_ID    pDev, /* interrupt controller id */
    UINT32        node, /* interrupt node in FDT */
    UINT32        irqs, /* number of interrupt */
    UINT32        ipis, /* number of ipi interrupt */
    UINT32      * pBase /* interupt vector base */
    )
```

In the above example, the parameters are as follows:

pDev

The device ID.

node

The interrupt device node offset in the flat device table (FDT). This offset is used to mark the interrupt controller in the interrupt subsystem. When another device wants to connect an interrupt, the interrupt subsystem uses this offset to do the matching. In the device tree source (DTS) file, each device has an interrupt-parent property. The value of that parameter is the interrupt controller offset. If the interrupt-parent value is equal to the node offset of the interrupt controller, the device connects the interrupt to that interrupt controller. The interrupt subsystem then calls its method to configure, enable, or disable the interrupt.

irqs

The number of supported vectors.

ipis

The number of supported IPIs.

pBase

This parameter is used to return a logic base that is allocated from the system. The value for pBase is returned when the interrupt controller registers itself with the interrupt subsystem using vxbIntRegister( ).The value of this parameter is used by the interrupt controller to dispatch the interrupt. The VXB_INT_ISR_CALL( ) function is the ISR dispatch function. This function must pass the logic vector.

For dynamic interrupt controllers, the driver must call vxbDyncIntRegister( ) to register itself with the interrupt subsystem. For example:

```
STATUS vxbDyncIntRegister
    (
    VXB_DEV_ID    pDev,         /* interrupt controller id */
    UINT32        irqs,
    UINT32      * pBase         /* allocate base vector */
    )
```

In the above example, the parameters are as follows:

pDev

The device ID.

irqs

The number of supported vectors.

pBase

This parameter is used to return a logic base that is allocated from the system. The value of this parameter is used by the interrupt controller to dispatch the interrupt. The **VXB_INT_ISR_CALL** macro is supplied to the interrupt controller to dispatch the interrupt.

Related information

Interrupt Controller Driver Overview on page 124
Driver Methods for Interrupt Controller Drivers on page 125
Interrupt Controller Header Files on page 130
Interrupt Controller Driver Configuration on page 131
Interrupt Controller Topology on page 136
Cascading Interrupt Controller Support on page 137
ISR Dispatch on page 139
Dynamic Interrupt Vectors on page 141
Interrupt Inputs Structure on page 142
VxWorks SMP Considerations on page 144
Debugging Considerations on page 146

6.6. Interrupt Controller Topology

To handle interrupts effectively in your VxWorks system, you must understand the topology of the interrupt controller device tree.

Every interrupt controller has some number of interrupt inputs. The number of inputs can range from one to several inputs. In addition to interrupt inputs, each interrupt controller has one or more interrupt outputs. The outputs are where interrupts are generated. Most interrupt controllers treat their interrupt outputs in the same manner that other drivers handle interrupt generation. That is, the controllers connect an ISR using the vxbIntConnect( ) function. When a vxbIntConnect( ) call is made, an interrupt controller in the system claims the interrupt. This response is the same, whether the caller to vxbIntConnect( ) is an interrupt controller instance or an instance from some other device class. This behavior implies that interrupt controllers have a hierarchy of connectivity, where the inputs of some interrupt controllers are connected to the outputs of other interrupt controllers. The management of each interrupt controller device is separated, because each interrupt controller is represented by a separate VxBus instance.

Within this hierarchy, each CPU can be considered to be an interrupt controller device at the top of the interrupt controller device tree. CPU interrupt controller devices are special in a number of ways. Although these drivers handle interrupt inputs in a manner similar to other drivers, they handle interrupt outputs in a special manner. The drivers do not try to connect interrupt outputs using the VxBus interrupt connection mechanism. Instead, they connect to the architecture-specific code that is provided for interrupt connection.

Interrupts can be delivered as messages rather than values on a physical wire. This may be the case for interrupt handling on the CPU. In some cases, an external bus controller and interrupt controller are included on the same device, such as with the PCI-X and PCIe bus controller devices used on some PowerPC processors. Typically, there are several things that the interrupt controller instance must do differently when interrupts are delivered as messages versus when they are hard-wired interrupts. This can include assignment of a vector (which in this context is simply an identification number) for each device that generates interrupt messages.

Related information

Interrupt Controller Driver Overview on page 124
Driver Methods for Interrupt Controller Drivers on page 125
Interrupt Controller Header Files on page 130
Interrupt Controller Driver Configuration on page 131
Interrupt Controller Driver Initialization on page 134
Cascading Interrupt Controller Support on page 137
ISR Dispatch on page 139
Dynamic Interrupt Vectors on page 141

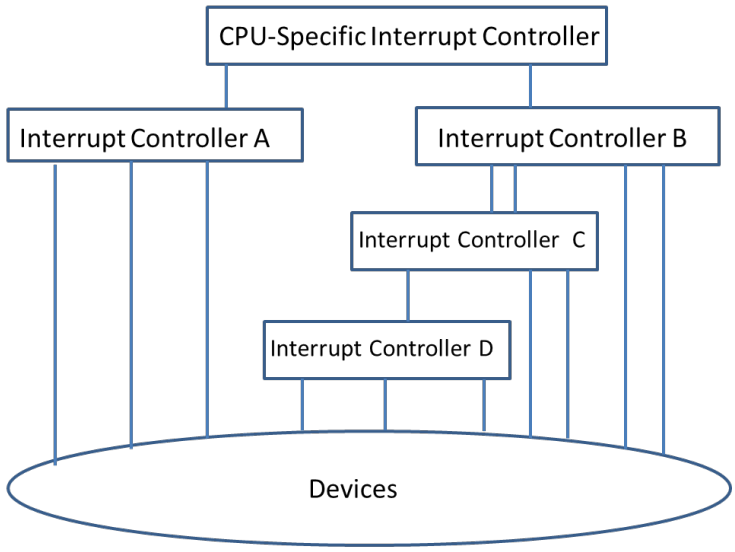
Interrupt Inputs Structure on page 142
VxWorks SMP Considerations on page 144
Debugging Considerations on page 146

6.7. Cascading Interrupt Controller Support

When working with a large number of interrupt inputs, you are sometimes required to use a cascade of several interrupt controllers. When your system is properly configured, VxBus supports this arrangement.

The VxBus interrupt subsystem supports cascading of interrupt controllers as shown in the following figure:

Figure 5. Cascading Interrupt Controllers



Each interrupt controller uses a VxBus device driver that matches it. Each interrupt controller must have a flattened device tree (FDT) node which, along with other details, states it's parent controller.

FDT Configuration

The following DTS example (from an IBM Ebony board) shows the nodes of the interrupt controller (**UIC-C**) and it's parent controller (**UIC-B**):

```
UIC-B: interrupt-controller0 {
    compatible = "ibm,uic-440gp", "ibm,uic";
    interrupt-controller;
    cell-index = <0>;
    dcr-reg = <0x0c0 0x009>;
    #address-cells = <0>;
    #size-cells = <0>;
    #interrupt-cells = <2>;
};

UIC-C: interrupt-controller1 {
    compatible = "ibm,uic-440gp", "ibm,uic";
    interrupt-controller;
    cell-index = <1>;
```

```
    dcr-reg = <0x0d0 0x009>;
    #address-cells = <0>;
    #size-cells = <0>;
    #interrupt-cells = <2>;
    interrupts = <0x1e 0x4
                  0x1f 0x4>; /* cascade */
    interrupt-parent = <&UIC-B>;
};
```

The node for interrupt controller, **UIC-C**, specifies the parent to which the controller is cascaded (this is used in the driver attach method). It also specifies two hardware vectors (**0x1e** and **0x1f**). These are the interrupts on the parent controller (**UIC-B**) to which **UIC-C** is connected.

Writing the Attach Method for the Cascaded (Child) Controller

In the driver attach method, call the vxbIntRegister( ) function to register the interrupt controller to the VxBus interrupt subsystem. In addition, call the vxbIntConnect( ) function for each interrupt line (in the above example, it is called two times).

The calls to vxbIntConnect( ) bind the interrupt service routines (ISRs) to the interrupts provided by the parent interrupt controller. When an interrupt is triggered, the ISR in the child controller discovers which of its interrupt sources triggered the event and then uses VXB_INT_ISR_CALL( ) to invoke the handlers for that event. In turn, this causes the ISRs for the devices bound to that interrupt to be executed. That is, the parent interrupt controller calls the child interrupt controller's ISR, and that ISR then calls the underlying device ISR.

Other Required Methods

At a minimum, you must implement the following methods in the device driver for a cascaded interrupt controller system. (These methods are called by the down stream device connected to the interrupt controller):

vxbYourIntCtrlIntConfig(VXB_DEV_ID pDev, UINT32 vector, VXB_INTR_ENTRY * pVxbIntrEntry)

> Configures the interrupt pin for priority, edge/level, low/high trigger, and so forth. The information is pointed to by the pVxbIntrEntry parameter.

vxbYourIntCtrIntEnable(VXB_DEV_ID pDev, UINT32 vector, VXB_INTR_ENTRY * pVxbIntrEntry)

> Enables the interrupt.

vxbYourIntCtrIntDisable(VXB_DEV_ID pDev, UINT32 vector, VXB_INTR_ENTRY * pVxbIntrEntry)

> Disables the interrupt.

The vxbIntEnable( ) and vxbIntDisable( ) methods in the cascaded child controller are used to mask or unmask the cascaded controller's events as needed.

Presumably, the cascaded controller has a way of enabling or disabling each of its event sources (which is chip-specific). This can be a single bit-mask register (where each bit controls a different event source), or there can be a separate control/configuration register for each source. Disabling the cascaded event prevents it from ever triggering an interrupt until it is re-enabled.

Related information

Interrupt Controller Driver Overview on page 124
Driver Methods for Interrupt Controller Drivers on page 125
Interrupt Controller Header Files on page 130
Interrupt Controller Driver Configuration on page 131
Interrupt Controller Driver Initialization on page 134
Interrupt Controller Topology on page 136
ISR Dispatch on page 139

Dynamic Interrupt Vectors on page 141
Interrupt Inputs Structure on page 142
VxWorks SMP Considerations on page 144
Debugging Considerations on page 146

6.8. ISR Dispatch

Dispatching of interrupt service routines may be handled in one of two ways, depending on the hardware architecture: using the polled model or the vectored model.

With the polled model, the various I/O peripherals have their interrupt lines routed to an interrupt controller device, each with a unique vector number assigned to them. The mapping between vector number and the corresponding I/O device is usually described in the hardware documentation, and varies depending on the platform. When an I/O device triggers its interrupt, the controller triggers a CPU trap.

To field interrupts in this situation, the interrupt controller driver must install a handler for the trap. When the trap handler runs, it must poll the controller to determine which device interrupt vectors may currently be pending (which indicates which peripherals have events waiting to be serviced). This is usually done by reading one of the controller's registers. The vector number for the currently pending device interrupt is typically used as an index into a lookup table, to locate the interrupt service routine provided by the driver for the I/O device. Because it is possible that more than one device may signal an interrupt at the same time, the interrupt controller software must continue polling until there are no more pending events. Depending on the controller design, the driver may also need to write a register to indicate that the event has been serviced (that is, signal an "end of interrupt").

The main hallmark of the polled model is that dispatching of interrupt service routines is done with the assistance of software. The CPU is notified when an interrupt occurs, but does not implicitly know which device triggered it. It is up to the interrupt controller driver to decode the interrupt vector and dispatch the appropriate ISR.

By contrast, with the vectored model, interrupt service routines may be directly dispatched by hardware. Platforms that support the vectored model also use a dispatch table, but the table format is dictated by the hardware design. There is usually some number of entries in the table, and each entry corresponds to a particular interrupt vector. (The association between a slot in the vector table and a given peripheral may be hardwired, or there may be an intermediate arbiter that allows the association to be configured through software.) The interrupt controller software is responsible for initializing each slot in the table. However, when an interrupt does occur, the hardware automatically branches to the interrupt service routine specified in the corresponding table slot.

This gives the vectored model an advantage over the polled model, in that while it may require some additional setup, ISRs may be dispatched with less latency, since no additional CPU instructions are needed to perform the vector decoding and ISR lookup.

The ARM and PowerPC platforms are often implemented as polled model systems. The Intel platform is the most common example of a vectored model system. Note that while there are some ARM and PowerPC variants which support the vectored model as well, those currently supported for VxWorks 7 use polling.

The VxBus interrupt subsystem is intended to support both the polled and vectors models in a way that allows the platform dependencies to be isolated within the interrupt controller driver, and that provides the best performance.

Vectored Model Dispatching

For the vectored interrupt model, the interrupt controller driver is not required to perform any dispatching functions as the hardware can dispatch individual interrupt service routines itself. The driver is responsible for initializing the hardware dispatch table such that each vector will trigger the correct interrupt service routine. This is usually done with an architecture API such as intArchConnect( ). A driver for a vectored model system must implement the {vxbIntVecSet}( ) and {vxbIntVecClear}( ) methods so that it can be notified when an ISR is ready to be bound to (or disconnected from) a particular vector.

Note that for vectored model systems, the VxBus interrupt subsystem still maintains a software vector table. However it is used primarily to maintain internal state, and to handle chaining in the event that two devices share the same interrupt vector.

For polled model systems, the vector table is actually used as part of the software interrupt dispatch process as described below.

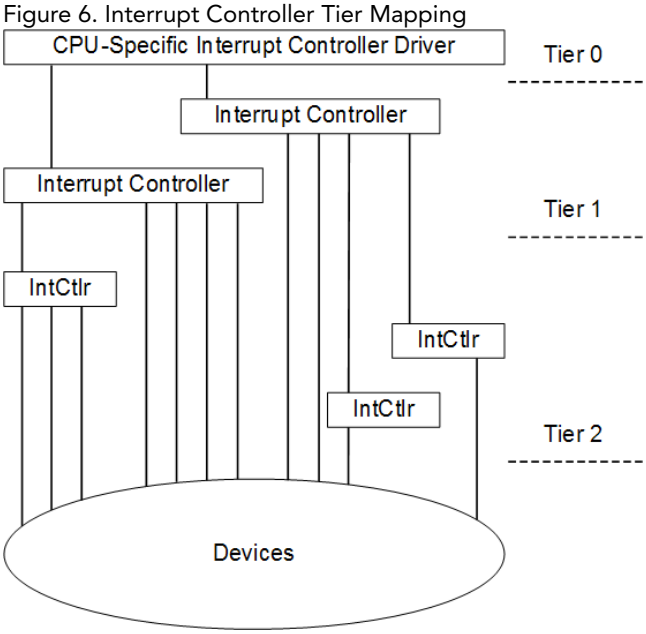
Polled Model Dispatching

To understand how ISR dispatch works, you must understand the interrupt controller hierarchy involved including how the various tiers of the hierarchy interact and the terminology associated with them.

In this document, the CPU-specific driver is referred to as tier 0, the interrupt controller driver(s) connected directly to the CPU-specific interrupt controller are referred to as tier 1, interrupt controllers connected to tier 1 are referred to as tier 2, and so on.

Tier 0 interrupt controller drivers are always architecture-dependent or CPU-dependent. Devices used as tier 1 interrupt controllers are typically, though not necessarily, used only on a single processor architecture. Devices used as tier 2 and lower interrupt controllers are not typically architecture specific.

Figure 6. Interrupt Controller Tier Mapping



Every interrupt controller driver, regardless of the tier on which it resides, dispatches downstream ISRs by invoking the **VXB_INTCTLR_ISR_CALL** macro.

In general, the tier 1 interrupt controller ISR must do the following:

1. Mask off interrupts from the source that generated the interrupt.
2. Re-enable interrupts with a call to intCpuUnlock( ).
3. Invoke **VXB_INTCTLR_ISR_CALL**.
4. Disable interrupts with a call to intCpuLock( ).

Because there is no previous intCpuLock( ) call to return the appropriate lock value, the system cannot easily determine what argument to use for intCpuUnlock( ). Because tier 1 interrupt controller devices are typically used with only a single architecture, interrupt controller drivers for those devices can use architecture-specific information for the argument to intCpuUnlock( ). However, per good programming practice, the value of the argument should be available as a macro that can be set differently according to the CPU macro, and an error generated if the driver is compiled for any unsupported architecture.

For example:

```
#if CPU==PPC32
IMPORT int vxPpcIntMask;
```

```
#define SAMPLE_INTCTLR_INTMASK vxPpcIntMask
#else /* CPU==PPC32 */
#error vxbSampleIntCtlr not available for this architecture
#endif /* CPU==PPC32 */
```

To minimize the number of interrupts that occur, interrupt controller drivers should process all pending unmasked interrupts whenever the interrupt controller driver ISR is called. Often, this means that the driver reads a register to determine which inputs have pending interrupts, and processes each interrupt source in a loop.

The following example is modified from the MPIC interrupt controller driver:

```
while (TRUE)
    {
    /* clear int, return the vec for highest IRQ */

    mpicIntVec = MPIC_REG_READ(MPIC_INT_ACK_REG) & 0xffff;

    if (mpicIntVec == MPIC_SPUR_VEC)
        break;

    CPU_INT_UNLOCK (intMask);

    VXB_INT_ISR_CALL((pDrvCtrl->intBase + mpicIntVec));

    CPU_INT_LOCK (&intMask);

    MPIC_REG_WRITE(MPIC_EOI_REG, 0x0);
    } /* while */
```

By using the intCpuUnlock( ) and intCpuLock( ) calls in the tier 1 interrupt controller ISR, the system provides priority dispatching of interrupts for devices connected directly to tier 1 interrupt controller devices. Currently, VxWorks does not provide a mechanism to perform similar priority dispatching at other tiers. If your application requires priority dispatching at other tiers, see the Wind River Online Support website for supplemental documentation and for the most recent interrupt controller drivers.

Related information

Interrupt Controller Driver Overview on page 124
Driver Methods for Interrupt Controller Drivers on page 125
Interrupt Controller Header Files on page 130
Interrupt Controller Driver Configuration on page 131
Interrupt Controller Driver Initialization on page 134
Interrupt Controller Topology on page 136
Cascading Interrupt Controller Support on page 137
Dynamic Interrupt Vectors on page 141
Interrupt Inputs Structure on page 142
VxWorks SMP Considerations on page 144
Debugging Considerations on page 146

6.9. Dynamic Interrupt Vectors

VxBus provides the ability for interrupt controller drivers to manage dynamically generated interrupts.

Some bus types allow dynamic assignment of interrupt values, often referred to as vectors. For example, variants of the PCI bus can provide message signaled interrupts (MSIs), which require firmware or software to assign the vector. There is also an MSI-X variant, which is a different representation of dynamic interrupt vectors on variants of the PCI bus type.

In addition, some interrupt controller devices reside on multifunction chips. Multifunction chips can include an interrupt controller device in addition to other devices, and may have a register containing the interrupt vector to use when the device generates an interrupt. The driver software must often write a valid vector to this register in order for the device to generate an interrupt. And the vector used must be generated somehow, possibly dynamically at runtime.

Configuring and Programming Dynamic Vectors Using the Service Driver Routines

The service driver (that is, the driver for the device that requires use of the interrupt such as the Ethernet or serial device) can call vxbIntAlloc( ) and vxbIntFree( ) to request and free an interrupt pin. The driver can also call vxbIntConnect( ) and vxbIntDisConnect( ) to connect or disconnect the ISR from the interrupt pins. These functions are available from the **vxbIntLib** library.

Note: For clarity, the vxbPciMsiAlloc( ) alias is available for MSIs on the PCI bus.

The dynamic interrupt controller must implement the vxbIntAlloc( ) and vxbIntFree( ) functions. It must also maintain the interrupt resource internally. This can be done using a bitmap or an array which records the allocated or freed interrupt resource.

Note: For the dynamic interrupt controller, the vxbIntAlloc( ) function should always return the continuous interrupt when the caller wants to apply a multi-interrupt resource. Otherwise, some device management, such as that needed for the MSI interrupt, cannot work. (The MSI-X interrupt does not have this limitation.)

Determining Dynamic Vector Values

The interrupt controller driver must choose the dynamic vector according to constraints in the hardware. Keep the following in mind:

- The best system performance is obtained when ISRs are not chained. Therefore, dynamically assigned vectors should be unassigned to other devices whenever possible.
- When multiple dynamically assigned vectors are available, they should be sequential. The interrupt controller driver may be able to scatter multiple dynamically assigned vectors throughout the range of acceptable vectors, but VxWorks does not support this functionality.

Related information

Interrupt Controller Driver Overview on page 124
Driver Methods for Interrupt Controller Drivers on page 125
Interrupt Controller Header Files on page 130
Interrupt Controller Driver Configuration on page 131
Interrupt Controller Driver Initialization on page 134
Interrupt Controller Topology on page 136
Cascading Interrupt Controller Support on page 137
ISR Dispatch on page 139
Interrupt Inputs Structure on page 142
VxWorks SMP Considerations on page 144
Debugging Considerations on page 146

6.10. Interrupt Inputs Structure

When interrupt controller drivers use the **vxbIntib** library functionality, the interrupt inputs must be represented by a structure.

There are two structures used to describe interrupt resources: **VXB_RESOURCE_IRQ** and **VXB_INTR_ENTRY**. Both are used to define information about the interrupt itself and the linkage between the I/O device and its interrupt controller. The structures are intentionally defined in a platform-independent fashion, although in some cases the fields may provide information that is interpreted in a platform-specific way by an interrupt controller driver. The two structures are defined as follows:

```
typedef struct vxbResourceIrq
    {
    UINT32           hVec;      /* hardware vector */
    UINT32           lVec;      /* logic vector */
    VOIDFUNCPTR      pIsr;      /* ISR */
    void *           pArg;      /* parameter */
    UINT32           flag;
    struct vxbIntrEntry * pVxbIntrEntry;   /* store the bus specific
                                              Interrupt information */
    } VXB_RESOURCE_IRQ;
```

```
typedef struct vxbIntrEntry
    {
    UINT32        node;
    UINT32 *      pProp;
    UINT32        numProp;
    } VXB_INTR_ENTRY;
```

The **VXB_RESOURCE_IRQ** structure contains a pointer to the ISR routine that is dispatched when an interrupt triggers, along with an optional argument. These fields are set when a driver calls vxbIntConnect( ). The **flags** field is also manipulated by the VxBus interrupt subsystem and typically specifies whether the interrupt is statically configured (**VXB_INT_FLAG_STATIC**) or dynamically configured (**VXB_INT_FLAG_DYNAMIC**).

The **VXB_RESOURCE_IRQ** structure also contains the **hVec** and **lVec** values. These are used by the interrupt subsystem and the interrupt controller driver for dispatching. The VxBus interrupt subsystem manages an internal dispatch table comprised of **VXB_INTR_VEC** structures. The size of the table is configurable with the **VXB_MAX_INTR_VEC** parameter, and defaults to 1024. An interrupt controller calls vxbIntRegister( ) to request that a portion of the dispatch table be allocated for its exclusive use.

The **lVec** value in the **VXB_RESOURCE_IRQ** structure is used to indicate the absolute index value of the slot in the dispatch table that corresponds to this particular interrupt resource. There may be more than one interrupt controller device present in the VxBus device tree, although in many cases there is just one. In this configuration, a set of table slots is allocated starting at **lVec** index 0, and extending to **nIrqs**, where **nIrqs** is the number of interrupts requested in the call to vxbIntRegister( ).

The **hVec** value represents a relative index, and is intended to be used by the interrupt controller driver. For example, assume that a given interrupt controller implements a range of 32 vectors. The driver can request that 32 dispatch table slots be allocated from the dispatch table when calling vxbIntRegister( ). The resulting **hVec** values will be in the range of 0 to 31, regardless of their absolute location in the table. **vxbIntRegister()** will also fill in the **intBase** value with the absolute base offset of those 32 vectors.

When the interrupt controller dispatches the interrupt using VXB_INT_ISR_CALL( ), it can specify the exact absolute offset by adding **hVec** to the **intBase** value returned by vxbIntRegister( ).

The **hVec** value is typically filled in by the parent bus controller that creates a device. If a device is described by an FDT entry in a BSP's device tree file (**.dts**), then it will usually include an interrupt property of some kind. This property is considered to be an interrupt vector attached to a particular interrupt controller. In the example above, in which the controller has 32 vectors, the interrupt property will be specified as some value from 0 to 31.

The **lVec** value is filled in later, when the resource is allocated. The parent of a given device must implement the {vxbResourceAlloc}( ), {vxbResourceFree}( ) and {vxbResourceListGet}( ) methods. The {vxbResourceAlloc}( ) method is responsible for calling vxbIntMap( ), which creates the linkage between the **VXB_RESOURCE** handle for the interrupt and the parent interrupt controller.

The linkage between the resource and the controller is established using the **node** field in the **VXB_INTR_ENTRY** structure. When an interrupt controller driver calls vxbIntRegister( ), it must provide an offset value as one of the arguments. The **node** field in the **VXB_INTR_ENTRY** structure must match this offset value.

The meaning of these fields is derived from the typical use case, which involves FDT. In the **.dts** file, an I/O device includes an interrupt property that defines its interrupt vector and an interrupt parent property that points to another entry in the **.dts**. When the device tree source file is compiled into the binary FDT blob, this reference is converted to an offset pointer, that is, the offset within

the FDT blob where the FDT structure for the interrupt parent begins. The FDT bus controller driver fills in the **node** field in the **VXB_INTR_ENTRY** structure with this value when it creates the device's resource handles. When the FDT node for the interrupt controller itself is enumerated, it will call vxbIntRegister( ) with the same offset value. When a driver finally calls vxbResourceAlloc( ) to request the interrupt resource handle, the parent calls vxbIntMap( ), which searches for a registered interrupt controller with an offset value that matches the specified **node** value in the resource. Once the match is found, the **lVec** field in the **VXB_RESOURCE_IRQ** structure is filled in.

If no match is found right away, vxbIntMap( ) defers the linkage creation. The resource is temporarily placed on a deferral list, until such time as the corresponding interrupt controller registers itself with the interrupt subsystem. Calls to vxbIntConnect( ) and vxbIntEnable( ) appear to succeed in the interim, however no interrupts occur until the interrupt controller is registered, at which point the linkage and connect/enable actions are finalized.

This strategy is used to handle the case in which the FDT bus controller code instantiates an I/O device before its corresponding interrupt parent device. Without the deferral mechanism, this out-of-order device enumeration would lead to I/O driver initialization failures (for example, vxbIntConnect( ) would fail because there would not be an interrupt controller instance available yet to complete the interrupt setup). It may be possible to limit the occurrences of this through careful layout of the **.dts** file, however the FDT specification does not forbid I/O devices to be listed before interrupt controllers, and many **.dts** file writers will order things this way. Note that this is avoided in some operating systems by using a separate software framework for managing interrupt controller hardware and using a two-pass scan of the device tree blob (the first pass looks for interrupt controller nodes and initializes their resources, and the second pass initializes I/O devices). Since VxWorks places interrupt controller and I/O device instances in the same tree, some strategy for handling out-of-order instantiation is required.

The offset and node values are just 32-bit quantities. For buses that use a different enumeration strategy, the parent controller drivers can use whatever allocation strategy that is convenient, as long as the offset and node values can eventually be paired up by vxbIntMap( ).

The **numProp** and **pProp** fields in the **VXB_INTR_ENTRY** structure are used to contain additional implementation-dependent properties related to the interrupt resources. This may include things such as triggering (edge versus level) or polarity (active low versus active high). The interpretation of these properties is defined by the interrupt controller driver itself. For devices specified by **.dts** files in a BSP, there may be a variable number of properties. The first is always assumed to be the vector number, and the FDT bus controller driver will always place this in the **hVec** field. The **pProp** pointer points to the remaining property values in the device tree blob and the **numProp** field indicates how many are present.

The implementation-dependent fields are specified as an array on UINT32 values. For FDT class devices, the fields must be decoded to host-native endian format using **vxFdt32ToCpu()**. For other bus types, the fields may be encoded in other ways (or simply not encoded at all) as dictated by software design requirements.

Related information

Interrupt Controller Driver Overview on page 124
Driver Methods for Interrupt Controller Drivers on page 125
Interrupt Controller Header Files on page 130
Interrupt Controller Driver Configuration on page 131
Interrupt Controller Driver Initialization on page 134
Interrupt Controller Topology on page 136
Cascading Interrupt Controller Support on page 137
ISR Dispatch on page 139
Dynamic Interrupt Vectors on page 141
VxWorks SMP Considerations on page 144
Debugging Considerations on page 146

6.11. VxWorks SMP Considerations

There are some aspects of multiprocessor (MP) systems that require special handling from the interrupt controller driver.

A multiprocessor (MP) system is a computer system with more than one processor. There are several common configurations of MP systems. When all of the processors in a system are identical, and a single OS is running on all processors at the same time, the system is called a symmetric multiprocessing (SMP) system.

Routing Interrupt Inputs to Individual CPUs

In a VxWorks SMP system, individual interrupts can be routed to processors other than the bootstrap processor. When VxWorks SMP boots, each device interrupt pin is routed to the CPU which is specified by the device tree source (DTS) file (for ARM, PowerPC, RISC-V) or the BIOS and Intel Architecture device table (IADT) (for Intel Architecture). If the destination is not specified, the interrupt is routed to the bootstrap CPU.

Interprocessor Interrupts

On multiprocessor systems, there are several OS modules that must be able to interrupt individual processors on the system. The OS modules that require this functionality include the scheduler (and any other module that manages tasks), the OS debug support module, and, potentially, every module that requires management of cache and MMU. The mechanism to interrupt individual processors is called interprocessor interrupts, or IPIs.

> Note: What the kernel modules do when they invoke IPIs is beyond the scope of this document. This document focuses only on generating IPIs, which is the responsibility of interrupt controller drivers.

In SMP, kernel debug support modules use **vxIpiLib** to generate IPIs. The routines in **vxIpiLib** rely on VxBus interrupt controller drivers to perform the actual work. Other kernel modules, such as the scheduler, use an internal library to perform interprocessor interactions, which resolve to **vxIpiLib** calls. In all of these cases, the system ends up calling a function provided by an interrupt controller driver in order to generate the IPI.

The mechanism used to support IPIs relies on a single driver method, {vxIpiControlGet}( ). This method returns a pointer to a structure that describes the kinds of IPIs that the interrupt controller driver can generate. The structure contains:

- a list of CPUs that this interrupt controller device can interrupt
- a count of the number of different IPIs that this interrupt controller can generate

The structure is the **VXIPI_CTRL** structure, which is defined in:

installDir **/vxworks-7/pkgs_v2/os/drv/vxbus/subsystem/h/subsys/int/vxIpiLib.h**

The structure is defined as follows:

```
typedef struct vxIpiCtrl
    {
    SL_NODE          ipiList;    /* Next IPI structure */
    phys_cpuset_t    pCpus;      /* destination CPUs interruptable */
    VXB_DEV_ID       pCtlr;      /* Interrupt Controller */
    INT32            ipiCount;   /* Number of IPIs available */
    VXIPI_VEC *      pIpiVec;    /* IPI vector */
    } VXIPI_CTRL, * VXIPI_CTRL_ID;
```

The function library that supports IPIs is located in:

installDir **/vxworks-7/pkgs_v2/os/drv/vxbus/subsystem/src/int/vxIpiLib.c**

Within the VxBus interrupt controller design, IPIs are represented outside the interrupt controller driver by a simple integer value. This value is an index of the IPI. The value may reflect some vector information. However, any relationship between the IPI ID and any vector should be hidden in the interrupt controller driver. An interrupt controller device that can generate eight distinct IPIs has ipiID values ranging from zero to seven.

Depending on the system configuration, one or more **ipiID** values are reserved for system use. The remainder may be available for application use. **ipiID 1** is always reserved for debug support, and is not available to applications. When VxWorks SMP support is

used, **ipiID 0** is reserved for CPC calls used by the OS, and all remaining IPIs are reserved and therefore not available for application use.

In addition, there may be special considerations on some BSP or hardware platforms that require the interrupt controller to reserve additional interrupts for other purposes. These interrupts are an exception to the reserved interrupts for VxWorks SMP. This situation is rare and should not be required in most cases.

Limitations in a Multiprocessor System

For certain limitations in multiprocessor systems, interrupt controllers may be required to assist in a workaround. The category of limitation described here contains those issues related to the use of SMP on hardware that is not truly symmetric. That is, the system includes some devices that cannot be managed equally by all processors.

The system is configured and all peripheral devices are initialized before any additional processors are brought online. However, in some systems there are devices that the bootstrap processor does not have access to. These devices cannot be brought online until after some other processor is brought up. In order to support these devices, a special BSP configuration can be used to allow the devices to be started. However, when the ISRs for the devices are connected, the interrupt controller must be able to route the ISRs to one of the processors to which the device is connected. If SMP must be used on this type of asymmetric hardware platform, the interrupt controller can choose to have special-purpose resources provided by the BSP to indicate restrictions of this nature.

A similar situation can occur for devices connected directly to the bootstrap processor and unavailable to other processors. However, in this situation, the problem is reversed. The system works fine as long as those devices are not rerouted to other processors. In this case, the best solution is to ignore the issue. If applications attempt to reroute those devices to processors that do not have access to the device registers, the device simply fails. Application developers should consider this situation during their development.

Related information

Interrupt Controller Driver Overview on page 124
Driver Methods for Interrupt Controller Drivers on page 125
Interrupt Controller Header Files on page 130
Interrupt Controller Driver Configuration on page 131
Interrupt Controller Driver Initialization on page 134
Interrupt Controller Topology on page 136
Cascading Interrupt Controller Support on page 137
ISR Dispatch on page 139
Dynamic Interrupt Vectors on page 141
Interrupt Inputs Structure on page 142
Debugging Considerations on page 146

6.12. Debugging Considerations

Interrupt controller drivers are one of the most difficult driver classes to debug. For this reason, Wind River recommends that you debug interrupt controller drivers using a hardware debugger.

Because the serial console and network interfaces are not available until the interrupt controller driver is available, you generally cannot defer driver registration. Therefore, you cannot establish a debug session with a working VxWorks system until after the interrupt controller driver is working correctly.

When your hardware debugger is combined with a suitable graphical interface that includes knowledge of source code, interrupt controller drivers can be debugged more efficiently.

Related information

Interrupt Controller Driver Overview on page 124
Driver Methods for Interrupt Controller Drivers on page 125
Interrupt Controller Header Files on page 130
Interrupt Controller Driver Configuration on page 131
Interrupt Controller Driver Initialization on page 134
Interrupt Controller Topology on page 136

Cascading Interrupt Controller Support on page 137
ISR Dispatch on page 139
Dynamic Interrupt Vectors on page 141
Interrupt Inputs Structure on page 142
VxWorks SMP Considerations on page 144

7. PCI and PCIe Drivers

7.1. Overview

The VxBus driver framework supports device drivers for PCI and PCI Express (PCIe) bus controllers.

The PCI/PCIe bus controller library detects PCI bus 0, determines the topology of the PCI tree, locates all PCI devices on the bus, and creates device instances for those devices in the VxBus tree. Before calling the library, your PCI bus controller driver must prepare the device parameters (for example, PCI space or a bus number) and store them in the **PCI_ROOT_CHAIN** structure. The bus controller driver must also supply the device configuration information, the reading and writing interface, and a method for assigning interrupts.

The PCI/PCIe bus controller is treated as a local device, similar to a serial or Ethernet on-chip type device. For the ARM, PowerPC, and RISC-V architectures, which use a flattened device tree (FDT), the bus controller driver gets the bus controller configuration from the device tree blob (DTB). The PCI/PCIe bus controller device is generally located under the **fdt** root bus node. Therefore, the **vxbFdtBus** driver creates the PCI/PCIe bus controller device. For Intel architecture, the PCI/PCIe bus controller device is created by **acpiBus** and **vxbFdtBus**.

The PCI/PCIe devices on the bus are scanned, configured, and created by the PCI/PCIe bus controller driver and inserted into VxBus as a child of the controller device. The device driver is then assigned as **VXB_BUSID_PCI**. The **VXB_BUSID_PCI** type class driver is then used to probe the PCI device.

Note: Devices attached to the PCI bus (such as a PCI network interface card (NIC) or a PCI serial card) are controlled by drivers that conform to the device type, such as a network driver for a NIC or a serial driver for the serial card.

Related information

Guidelines for Writing PCI and PCIe Device Drivers on page 147
Driver Methods for PCI Bus Controller Drivers on page 149
Driver Methods for PCI and PCIe Drivers on page 151
Header Files on page 153
PCI Bus Library on page 153
Device Configuration on page 156
PCI Infrastructure Initialization on page 158
PCI Show Functions on page 160

7.2. Guidelines for Writing PCI and PCIe Device Drivers

To write a PCI or PCIe device driver, you must understand some of the basic requirements for this driver type.

Set the VxBus Device Class

Your PCI/PCIe device driver must set the **VXB_BUSID_PCI** entry in the **vxbClass** member of the the **VXB_DRV** structure. This structure is located in the **vxbLib.h** header file.

```
typedef struct vxbDrv
    {
    SL_NODE           vxbNode; /* must always be first element */
```

```
    char *            pVxbName;
    char *            pVxbDesc;
    VXB_BUSTYPE_ID    vxbClass;  // for PCI device drvier, vxbClass must be set VXB_BUSID_PCI
    UINT32            vxbFlags;
    int               vxbRefCnt;
    VXB_DRV_METHOD *  pVxbMethods;
    } VXB_DRV;
```

This example is taken from the following:

installDir **/vxworks-7/pkgs_v2/os/drv/vxbus/drv/src/sio/vxbPciNs16550Sio.c**

```
VXB_DRV vxbPciNs16550Drv =
    {
    { NULL } ,
    "ns16550",                      /* Name */
    "ns16550 serial PCI driver",    /* Description */
    VXB_BUSID_PCI,                  /* Class */
    0,                              /* Flags */
    0,                              /* Reference count */
    pciNs16550MethodList           /* Method table */
    };
```

Use the PCI Configuration Access API

PCI and PCIe device drivers must use the PCI configuration access API to access PCI configuration space as follows:

```
VXB_PCI_BUS_CFG_READ(pDev, byteOffset, transactionSize, data)
VXB_PCI_BUS_CFG_WRITE(pDev, byteOffset, transactionSize, data)
```

Match Devices and Drivers

PCI and PCIe device drivers must use the vxbPciDevMatch( ) function to perform a quick match with devices and drivers by comparing the vendor or device ID.

Each PCI driver has one supported PCI device ID table. The probe function in each PCI device driver compares the device ID with the supported PCI device ID table. If a match is found, the probe function returns **OK**. Otherwise, the probe function fails and moves on to the next PCI device driver.

The following example is from:

installDir **/vxworks-7/pkgs_v2/net/end/drv/src/end/vxbGei825xxEnd.c**

```
/* List of supported device IDs.*/

LOCAL const VXB_PCI_DEV_MATCH_ENTRY geiPciDevIDList[] =
    {
        { INTEL_DEVICEID_82543GC_COPPER, INTEL_VENDORID , NULL},
        { INTEL_DEVICEID_82543GC_FIBER, INTEL_VENDORID , NULL},
        { INTEL_DEVICEID_82544EI_COPPER, INTEL_VENDORID , NULL},
        { INTEL_DEVICEID_82544GC_COPPER, INTEL_VENDORID , NULL},
        { INTEL_DEVICEID_82544GC_LOM, INTEL_VENDORID , NULL},
        { INTEL_DEVICEID_82544EI_FIBER, INTEL_VENDORID , NULL},
        ...
```


WNDRVR

Manage Resources

PCI and PCIe devices use register and interrupt resources similar to other generic drivers. However, the PCI bus controller driver helps manage the resources when the device driver allocates a resource. The only difference is that the PCI and PCIe devices can use the MSI/MSI-X interrupt.

For example:

```
    /* get interrupt resource */

    if (vxbPciMsiAlloc(pDev, 1) < 0)
        XXX_LOGMSG("vxbPciMsiAlloc failed, INTX running\r\n",0,0,0,0,0,0);

    if ((pRes = vxbResourceAlloc(pDev, VXB_RES_IRQ, 0)) == NULL)
        return NULL;

    pDrvCtrl->intRes = pRes;

    vxbIntConnect (pDev, pDrvCtrl->intRes, pDrvCtrl->geiIntFunc, pDrvCtrl);
    vxbIntEnalbe(pDev, pDrvCtrl->intRes, pDrvCtrl->geiIntFunc, pDrvCtrl);
```

Related information

Overview on page 147
Driver Methods for PCI Bus Controller Drivers on page 149
Driver Methods for PCI and PCIe Drivers on page 151
Header Files on page 153
PCI Bus Library on page 153
Device Configuration on page 156
PCI Infrastructure Initialization on page 158
PCI Show Functions on page 160

7.3. Driver Methods for PCI Bus Controller Drivers

VxBus PCI/PCIe bus controller drivers must supply basic VxBus methods, such as probe, attach, detach, and shutdown. Because PCI/PCIe is a bus-type device, the driver must also supply the {vxbResourceAlloc}( ), {vxbResourceFree}( ), and {vxbResourceListGet}( ) methods.

PCI Read Method

The {vxbPciCfgRead}( ) method performs whatever device-specific operations are required in order to perform the requested read operation from the bus configuration space.

The PCI/PCIe-specific method prototype used in the controller driver is as follows:

```
STATUS vxbPciCfgRead
       (
        VXB_DEV_ID pDev,        /* device info */
        PCI_HARDWARE * pPciDev, /* PCI device info */
        UINT32 byteOffset,      /* offset into cfg space */
        UINT32 width,           /* transaction size, in bytes */
        void * pData            /* buffer to read-from/write-to */
        )
```

pDev

The **VXB_DEV_ID** for the bus controller instance.

pPciDev

The PCI device information for the target hardware.

byteOffset

The offset into the configuration space where the read is performed. Because non-aligned configuration accesses are not allowed, byteOffset must be an even multiple of the transaction size.

width

The data size to read, in bits. Valid values are 8, 16, and 32.

pData

A pointer to the buffer used to store the value that is read from the configuration space.

PCI Write Method

The {vxbPciCfgWrite}( ) method performs any device-specific operations are required in order to perform the requested write operation from the bus configuration space.

```
STATUS vxbPciCfgWrite
        (
        VXB_DEV_ID pDev,        /* device info */
        PCI_HARDWARE * pPciDev, /* PCI device info */
        UINT32 byteOffset,      /* offset into cfg space */
        UINT32 width,           /* transaction size, in bytes */
        UINT32 data             /* data write to the offset */
        )
```

pDev

The **VXB_DEV_ID** for the bus controller instance.

pPciDev

The PCI device information for the target hardware.

byteOffset

The offset into the configuration space where the write is performed. Because non-aligned configuration accesses are not allowed, byteOffset must be an even multiple of the transaction size.

width

The data size to write, in bits. Valid values are 8, 16, and 32.

data

The data that will be written to configuration space.

Interrupt Assignment Method

The {vxbPciIntAssign}( ) method performs any device-specific operations required to assign interrupts for the device.

The PCI bus controller driver implements this method in the following way:

1. The controller must be able to handle any devices connected to the controller directly.
2. If the bus controller can handle all interrupt operations, including interrupt swizzling, {vxbPciIntAssign}( ) returns **OK** when it is called first time and the PCI library is not used. The driver can also handle a different board by using the compatible option in FDT for ARM and PowerPC.
3. If the bus controller can only handle devices that are directly connected, when a device connects using a bridge, calls to the {vxbPciIntAssign}( ) method fail. In this case, the PCI library does the swizzling, and then calls the {vxbPciIntAssign}( ) method again. The library passes the root device and swizzling interrupt pin, **intPin**. The bus controller driver can now handle the interrupt and returns **OK**.

The PCI/PCIe-specific method prototype used in the controller driver is as follows:

```
STATUS vxbPciIntAssign
        (
        VXB_DEV_ID           pDev,
        PCI_HARDWARE *       pciDev,
        int                  pin,
        UINT8 *              interrupt,
        VXB_INTR_ENTRY   *   pIntrEntry
        )
```

pDev

The **VXB_DEV_ID** for the bus controller instance.

pPciDev

The PCI device information for the target hardware.

pin

The interrupt pin derived from the device configuration space.

interrupt

The assigned interrupt lines.

pIntrEntry

The interrupt entry pointer. This function fills the interrupt vector and property for this entry. The interrupt subsystem uses this entry to connect interrupts. For more information, refer to the interrupt subsystem.

Related information

Overview on page 147
Guidelines for Writing PCI and PCIe Device Drivers on page 147
Driver Methods for PCI and PCIe Drivers on page 151
Header Files on page 153
PCI Bus Library on page 153
Device Configuration on page 156
PCI Infrastructure Initialization on page 158
PCI Show Functions on page 160

7.4. Driver Methods for PCI and PCIe Drivers

There are methods that you must implement in PCI and PCIe leaf device drivers (not PCI Controller or PCI Bridge device drivers).

Methods to Implement in Your Device Driver

PCI device drivers must provide public functional methods that are expected by the subsystem. For example, if the PCI device is an interrupt controller driver, the interrupt subsystem is expecting certain interrupt-related driver methods such as {vxbIntConfig}( ), {vxbIntEnable}( ), and so forth.

Device-class-specific methods for each device driver class are described in About Device-Specific Drivers on page 73. Every PCI device driver (as well as every other driver) must, at a minimum, implement the probe and attach methods.

Probe Method for PCI Devices

The probe method compares the device ID and vendor ID for the PCI device against the device ID and vendor ID that the device driver supports. A typical probe method includes the following code:

```
LOCAL STATUS myDevProbe
    (
    VXB_DEV_ID pInst
    )
    {
      return vxbPciDevMatch(pInst, myDevPciMatchList, NULL);
    }
```

A typical match list is similar to the following:

```
LOCAL const VXB_PCI_DEV_MATCH_ENTRY myDevPciMatchList[] =
    {
        { DEVICE_ID_0, VENDOR_ID_0, NULL },
        { DEVICE_ID_1, VENDOR_ID_0, NULL },
        ....
        ....
        { DEVICE_ID_n, VENDOR_ID_m, NULL },
        {} /* Empty terminated list */
    };
```

Attach Method for PCI devices

The attach method for a PCI leaf device is similar to an attach method for any other leaf device as described in VxBus Driver Methods on page 47. The **pDev** structure that is passed as the only parameter to the attach method already contains the PCI bus address information (base address and range). Therefore, resource allocation for memory (or IO) is done in an identical manner to non-PCI devices. This is also true for the allocation of interrupt resources. Interrupts can be connected and enabled in exactly the same way as they are for non-PCI devices.

PCI configuration registers can be accessed using the VXB_PCI_BUS_CFG_READ( ) and VXB_PCI_BUS_CFG_WRITE( ) functions. If the device requires DMA, it is set here as well. The DMA setup is identical to the DMA setup for non-PCI devices. (For more information on DMA drivers, see DMA Driver Overview on page 82.)

Related information

Overview on page 147
Guidelines for Writing PCI and PCIe Device Drivers on page 147
Driver Methods for PCI Bus Controller Drivers on page 149
Header Files on page 153
PCI Bus Library on page 153
Device Configuration on page 156
PCI Infrastructure Initialization on page 158

WNDRVR

PCI Show Functions on page 160

7.5. Header Files

PCI bus controller drivers and PCI device drivers must include a bus library header file. The header file provides the data types and prototype functions required for the driver.

Include the **vxbPciLib.h** file as follows:

```
#include hwif/buslib/vxbPciLib.h
```

For controller drivers that use a flattened device tree (FDT) (ARM and PowerPC), you must also include the FDT PCI library header as follows:

```
#include hwif/buslib/vxbFdtPci.h
```

Related information

Overview on page 147
Guidelines for Writing PCI and PCIe Device Drivers on page 147
Driver Methods for PCI Bus Controller Drivers on page 149
Driver Methods for PCI and PCIe Drivers on page 151
PCI Bus Library on page 153
Device Configuration on page 156
PCI Infrastructure Initialization on page 158
PCI Show Functions on page 160

7.6. PCI Bus Library

The PCI bus library supplies a set of functions to control PCI and PCIe bus controllers and devices. Your driver can call these functions directly.

Available Functions for PCI and PCIe Bus Controllers

The following functions are available for PCI and PCIe bus controllers:

- vxbPciAutoConfig( )
- vxbPciSetBarResPool( )

Syntax for the vxbPciAutoConfig( ) function is as follows:

```
STATUS vxbPciAutoConfig
    (
    VXB_DEV_ID      busCtrlID
    )
```

This function configures all PCI headers and announces any PCI devices to the VxBus infrastructure.

Syntax for the vxbPciSetBarResPool( ) function is as follows:

WNDRVR

```
void vxbPciSetBarResPool
    (
    PCI_ROOT_RES  * pciRootRes,
    struct barRes * res
    )
```

Available Functions for PCI and PCIe Devices

The following functions are available for PCI and PCIe devices:

- vxbPciDevMatch( )
- vxbPciMsiAlloc( )
- vxbPciMsiFree( )
- vxbPciMsiXAlloc( )
- vxbPciMsiXFree( )

Syntax for the vxbPciDevMatch( ) function is as follows:

```
STATUS vxbPciDevMatch
    (
    VXB_DEV_ID                      pDev,           /* device to do match */
    const VXB_PCI_DEV_MATCH_ENTRY * pMatchTbl,      /* pointer to match table */
    VXB_PCI_DEV_MATCH_ENTRY **      pMatchedEntry   /* best matched entry */
    )
```

Syntax for the vxbPciMsiAlloc( ) function is as follows:

```
int vxbPciMsiAlloc
    (
    VXB_DEV_ID      pDev,
    UINT32          cnt
    )
```

This function allocates one or more PCI-X or PCIe MSI interrupts. (Only PCI-X and PCIe devices support MSI capability. Older PCI devices do not.) If the allocation succeeds, the function returns the number of allocated MSI vectors. If the allocation fails, it returns **ERROR**. While the PCI specification allows devices to support multiple MSI vectors, not all devices do. The number of available vectors can be 1, 2, 4, 8, 16, or 32. The PCI specification also requires that multiple vectors be allocated contiguously (that is, the device uses the MSI data value as a base and may add a value of 1 to 31 internally to calculate the additional data values). As a result, when implementing your system, you must be aware that even if a device supports multiple MSI vectors, there can be platform-dependent considerations that limit the total number of vectors that can be allocated. For example, if a device supports 32 vectors, but the platform interrupt controller driver only reserves 16 vectors for dynamic interrupt use, it is not possible to request 32 MSI vectors.

If the MSI vector allocation succeeds, the device has an additional **VXB_RESOURCE** entry added to its resource list for each MSI vector. Resource 0 continues to represent the device's legacy INTX interrupt, though the legacy interrupt is disabled once vxbPciMsiAlloc( ) returns successfully. MSI vectors begin at index 1. For driver development, use vxbResourceAlloc( ) to acquire handles to these additional resources as needed.

In many cases, if a device supports only one MSI vector, it can simply be used in place of the device's INTX interrupt (all device-specific interrupt handling logic remains the same). If it supports more than one vector, you may need to program additional device-specific I/O registers in order to designate what events are associated with each vector.

It is not possible to allocate both MSI and MSI-X vectors simultaneously.

Syntax for the vxbPciMsiFree( ) function is as follows:

```
STATUS vxbPciMsiFree
    (
    VXB_DEV_ID      pDev,
    UINT32          cnt
    )
```

This function releases multiple MSI vectors at the same time. Because MSI vectors cannot be released individually, this function disables all MSI vectors. Once MSI is disabled, the legacy INTX interrupt for the device is enabled again. All **VXB_RESOURCE** entries representing MSI interrupts are removed from the device's resource list. For driver development, use vxbIntDisconnect( ) and vxbIntDisable( ) on all MSI interrupt resources prior to using vxbPciMsiFree( ).

Syntax for the vxbPciMsiXAlloc( ) function is as follows:

```
int vxbPciMsiXAlloc
    (
    VXB_DEV_ID      pDev,
    UINT32          cnt
    )
```

This function allocates one or more PCIe MSI-X interrupts. If the allocation succeeds, the function returns the number of allocated MSI-X vectors. If the allocation fails, it returns **ERROR**. Unlike MSI, the PCI specification allows a device with MSI-X capability to support up to 2048 MSI-X vectors. In practice, a given device is likely to support much fewer. As with MSI interrupts, there may be platform-specific limitations on just how may MSI-X vectors can be allocated for a given device.

As with MSI, a new **VXB_RESOURCE** entry is added to the device's resource list for each newly allocated MSI-X vector. For driver development, you must allocate them and bind service routines to them as required. Exactly what device events are associated with a given MSI-X vector varies depending on the device design, and device-specific configuration is required.

It is not possible to allocate both MSI and MSI-X vectors simultaneously.

Note: For optimal portability, your driver must implement legacy INTX interrupt support so that it can still function if MSI or MSI-X interrupt allocation fails.

Syntax for the vxbPciMsiXFree( ) function is as follows:

```
STATUS vxbPciMsiFree
    (
    VXB_DEV_ID      pDev,
    UINT32          cnt
    )
```

This function releases multiple MSI-X vectors at the same time. The number of vectors released must be the same as the number previously allocated. Once MSI-X is disabled, the legacy INTX interrupt for the device is enabled again. As with MSI, all **VXB_RESOURCE** entries representing MSI-X interrupts are removed from the device's resource list. For driver development, use vxbIntDisconnect( ) and vxbIntDisable( ) on all MSI-X interrupt resources prior to using vxbPciMsiFree( ).

Related information

Overview on page 147
Guidelines for Writing PCI and PCIe Device Drivers on page 147
Driver Methods for PCI Bus Controller Drivers on page 149
Driver Methods for PCI and PCIe Drivers on page 151
Header Files on page 153
Device Configuration on page 156
PCI Infrastructure Initialization on page 158

PCI Show Functions on page 160

7.7. Device Configuration

For ARM and PowerPC, PCI and PCIe bus controller drivers typically get device configuration information from the device tree blob (DTB) file.

PCI and PCIe bus controller drivers do not typically require configuration information from a BSP. Rather, all device information is provided by the DTB (for ARM and PowerPC). The flattened device tree (FDT) specification defines various PCI resources. The FDT file must be customized for your specific hardware. For more information on the FDT, see the FDT specification.

The following is an example PCI device tree source (DTS) file for the Freescale P2020 rdb target hardware:

```
pci0: pcie@ffe0a000 {
        compatible = "fsl,mpc8548-pcie";
        device_type = "pci";
        #size-cells = <2>;
        #address-cells = <3>;
        #interrupt-cells = <1>;
        bus-range = <0 255>;
        clock-frequency = <33333333>;
        interrupts = <26 2 0 0>;
        reg = <0xffe0a000 0x1000>;
        ranges = <0x2000000  0 0xe0000000 0x80000000  0 0x20000000
                  0x1000000  0 0x00000000 0xffc00000  0 0x10000>;
        interrupt-map-mask = <0xfff800 0 0 7>;
        interrupt-map = <
            0000 0x0 0x0 0x1 &mpic 0x0 0x1 0 0
            0000 0x0 0x0 0x2 &mpic 0x1 0x1 0 0
            0000 0x0 0x0 0x3 &mpic 0x2 0x1 0 0
            0000 0x0 0x0 0x4 &mpic 0x3 0x1 0 0
            >;
     };
```

As noted in the previous section, the bus library must prepare input for the library. Device information is stored in the **PCI_ROOT_CHAIN** structure. The **PCI_ROOT_CHAIN** type variable must be in the first entry of the **pDrvCtrl** pointer so that the bus library can use it to scan, configure, and create a PCI device.

The **PCI_ROOT_CHAIN** structure is defined as follows:

```
typedef struct pciRootRes
        {
        PCI_RES   barRes[PCI_ROOT_RES_MAX_IDX];
        DL_LIST   pciDevList;
        UINT32    baseBusNumber;
        UINT32    endBusNumber;
        struct pciRootChain * pRChan;
        } PCI_ROOT_RES;

typedef struct pciRootChain
        {
        PCI_ROOT_RES * pRootRes;
        UINT32         segCount;
        BOOL           autoConfig;
        } PCI_ROOT_CHAIN;
```

The FDT PCI library provides functions that can be used to help the controller driver to parse the information in the FDT and fill in the **PCI_ROOT_CHAIN** structure. The functions are as follows:

- vxbFdtPciGetBusRange( )
- vxbFdtPciRanges( )
- vxbFdtPciIntrInfo( )
- vxbFdtPciIntrGet( )

Syntax for the vxbFdtPciGetBusRange( ) function is as follows:

```
STATUS vxbFdtPciGetBusRange
        (
        VXB_FDT_DEV * node,
        UINT32 * start,
        UINT32 * end
        )
```

This function gets the start and end bus number.

Syntax for the vxbFdtPciRanges( ) function is as follows:

```
STATUS vxbFdtPciRanges
        (
        VXB_FDT_DEV * node,
        UINT32        type,
        struct fdtPciRange *space
        )
```

This function gets the PCI I/O and memory or pre-fetch memory space.

Syntax for the vxbFdtPciIntrInfo( ) function is as follows:

```
STATUS vxbFdtPciIntrInfo
        (
        VXB_FDT_DEV *       node,
        struct fdtPciIntr * intrInfo
        )
```

This function gets interrupt information. The **intrInfo** parameter is used to assign an interrupt in the interrupt assignment method of bus controller driver.

Syntax for the vxbFdtPciIntrGet( ) function is as follows:

```
STATUS vxbFdtPciIntrGet
        (
        struct fdtPciIntr *  intrInfo,
        int                  bus,
        int                  slot,
        int                  func,
        int                  pin,
        UINT8 *              interrupt,
        VXB_INTR_ENTRY   *   pIntrEntry
        )
```

This function is used by the interrupt assignment function of the bus controller driver. It provides the interrupt information (obtained using the vxbFdtPciIntrInfo( ) function) to the bus controller driver.

Example Code

```
    VXB_FDT_DEV * pFdtDev = vxbDevIvarsGet(pDev);
    struct fdtPciRange iospace, memspace;
    XXX_PCI_DRV_CTRL * pDrvCtrl = vxbDevSoftcGet(pDev);
    PCI_HARDWARE pciDev;
    UINT16 ltssm = 0;
    UINT32 baseBus, endBus;
    PCI_ROOT_CHAIN * pciRChain;
    PCI_ROOT_RES   * pciRootRes;
    struct barRes bres[PCI_ROOT_RES_MAX_IDX];

    pciRChain = &pDrvCtrl->rc;
    pciRootRes = vxbMemAlloc(sizeof(PCI_ROOT_RES));

    if (pciRootRes == NULL)
        return ERROR;

    pciRChain->pRootRes = pciRootRes;

    memset (&pciDev, 0, sizeof(PCI_HARDWARE));
    memset (bres, 0, sizeof(struct barRes)*PCI_ROOT_RES_MAX_IDX);

    vxbFdtPciRanges(pFdtDev, FDT_PCI_SPA_MEM, &memspace);
    vxbFdtPciRanges(pFdtDev, FDT_PCI_SPA_IO,  &iospace);

    vxbFdtPciIntrInfo (pFdtDev, &pDrvCtrl->pciIntr);

    if (vxbFdtPciGetBusRange(pFdtDev, &baseBus, &endBus) != OK)
        return ERROR;
```

Related information

Overview on page 147
Guidelines for Writing PCI and PCIe Device Drivers on page 147
Driver Methods for PCI Bus Controller Drivers on page 149
Driver Methods for PCI and PCIe Drivers on page 151
Header Files on page 153
PCI Bus Library on page 153
PCI Infrastructure Initialization on page 158
PCI Show Functions on page 160

7.8. PCI Infrastructure Initialization

The PCI infrastructure is initialized during PCI controller initialization. As with any other device, the PCI controller is initialized according to the initialization phases of VxBus.

VxBus Phase 1

The archNamePciProbe( ) function matches the PCI bus controller device with the existing device driver.

For ARM, PowerPC, and RISC-V, the flattened device tree (FDT) describes a PCI device with properties that are compatible with the device driver. The match mechanism is the same as that used for other FDT-compatible devices. It supplies a supported device list and then calls the vxbFdtDevMatch( ) function to match the device with the driver.

For Intel Architecture, the iaPciProbe( ) function calls vxbDevMatch( ) to match the PCI controller device driver to the PCI controller device description in the Intel Architecture ACPI and FDT.

VxBus Phase 2

If the device binds to the driver in phase 1, the archNameAttach( ) function is invoked. This function allocates register and interrupt resources from the parent bus and uses them to operate the PCI controller hardware. In addition, the archNameAttach( ) function prepares resources for the PCI library and invokes the vxbPciAutoConfig( ) function to scan, configure, and create child devices.

For example:

```
    pciRChain->segCount = 1;
    pciRChain->autoConfig = TRUE;
    pciRootRes->baseBusNumber = baseBus;
    pciRootRes->endBusNumber = endBus;

    bres[PCI_ROOT_RES_IO_IDX].base = iospace.basePci;
    bres[PCI_ROOT_RES_IO_IDX].adjust = iospace.baseParent - iospace.basePci;
    bres[PCI_ROOT_RES_IO_IDX].len = iospace.len;
    bres[PCI_ROOT_RES_MEM32_IDX].base = memspace.basePci;
    bres[PCI_ROOT_RES_MEM32_IDX].len = memspace.len;
    bres[PCI_ROOT_RES_MEM32_IDX].adjust = memspace.baseParent - memspace.basePci;

    vxbPciSetBarResPool (pciRootRes, bres);

    /* PCI autoconfiguration and announce device */

    vxbPciAutoConfig (pDev);
```

The vxbPciAutoConfig( ) function configures all of the PCI headers and calls vxbPciDeviceScan( ) which scans the devices under the PCI bus controller and stores the device information for all scanned devices in the PCI device list, **pciDevList**.

Note: The vxbPciDeviceScan( ) function only obtains and stores PCI-related information such as bus, device, function, vendor ID, PCI memory address and range, and so forth in the **pciDevList**. Device-specific information that is not PCI-related is obtained from the FDT or ACPI (depending on your architecture) at a later stage.

Finally, the vxbPciAutoConfig( ) function calls the vxbPciAddDevice( ) function.

Adding Devices with vxbPciAddDevice( )

The vxbPciAddDevice( ) function loops through the **pciDevList** and, for each device in the list, calls vxbDevAdd( ) as shown in the following example:

```
for (pNode = DLL_FIRST (pciDevList); pNode != NULL;  pNode = DLL_NEXT (pNode)) // The list of pciD
evList
                   contains all the devices(including bridge and pci device) connected on the PCI
controller, the pciDevList was filled in vxbPciDeviceScan function.
{
```

```
      if(vxbDevCreate (&pVxbDev) != OK)    // create vxBus tree device for each node in pciDevLis
t
         continue;

      VXB_PCI_CFG_READ(busCtrlID, &hardInfo, PCI_CFG_VENDOR_ID, 2, &pPciDevInfo->pciVendId);  // s
ave the PCI device verdor and device ID
      VXB_PCI_CFG_READ(busCtrlID, &hardInfo, PCI_CFG_DEVICE_ID, 2, &pPciDevInfo->pciDevId);

      vxbDevClassSet(pVxbDev, VXB_BUSID_PCI);  // set the BUS IS is PCI

      /* allocate resources */
      {allocate mem/io/premme resources for each PCI device(include bridge and class device) }

      /* allocate  INTx interrupt  */
      {assign INT for each PCI device (bridge and class device)}

      (void)vxbDevAdd (busCtrlID, pVxbDev);   // Add the device into busCtrlID, the busCtrlID repr
esents the PCI controller deviceID.
       continue; // go for the next pci device
}
```

The vxbDevAdd( ) function calls the probe and attach methods for the individual devices. For each new device node, vxbDevAdd( ) searches for the best matched driver by calling the probe method in all of the drivers included in the same class as the device. The probe method returns a score (on ARM, PowerPC, and RISC-V) or **OK**/**ERROR** (on Intel Architecture). VxBus then selects the highest scoring driver or a driver that returns an **OK** match (depending on your architecture) and assigns it to the device.

Once the driver for the device is assigned, vxbDevAdd( ) calls the attach method of the selected driver. The attach method obtains additional device information from the FDT or ACPI and then allocates resources and initializes the device hardware.

Note: If the vxbDevAdd( ) function is called for a PCI bridge device, the PCI bridge device driver (in its attach method) scans the devices on its subordinate bus and calls vxbDevAdd( ) for each one of those devices. This continues until the driver has iterated over the entire tree (that is, all the leaf nodes are reached).

Related information

Overview on page 147
Guidelines for Writing PCI and PCIe Device Drivers on page 147
Driver Methods for PCI Bus Controller Drivers on page 149
Driver Methods for PCI and PCIe Drivers on page 151
Header Files on page 153
PCI Bus Library on page 153
Device Configuration on page 156
PCI Show Functions on page 160

7.9. PCI Show Functions

PCI show functions are made available using the **INCLUDE_PCI_SHOW** component. These functions can be used to gain information about the PCI bus and any PCI devices in the system.

The **INCLUDE_PCI_SHOW** component provides the following PCI show functions:

- vxbPciCtrlShow( )
- vxbPciDeviceShow( )
- vxbPciFuncShow( )
- vxbPciTopoShow( )

The syntax for the vxbPciCtrlShow( ) is as follows:

```
void vxbPciCtrlShow (void)
```

This function lists all registered PCI and PCIe bus controller device IDs. The IDs can then be used call other vxbPciXXXShow( ) functions.

The syntax for vxbPciDeviceShow( ) is as follows:

```
STATUS vxbPciDeviceShow
    (
    VXB_DEV_ID busCtrlID,
    UINT8      busNo       /* bus number */
    )
```

This function prints information about the PCI devices on a given segment of the PCI bus. The segment is specified by busNo).

The syntax for vxbPciFuncShow( ) is as follows:

```
STATUS vxbPciFuncShow
    (
    VXB_DEV_ID busCtrlID,
    UINT8      bus,        /* bus */
    UINT8      device,     /* device */
    UINT8      function    /* function */
    )
```

This function reads and displays information from the specified bus, device, and function.

The syntax for vxbPciTopoShow( ) is as follows:

```
void vxbPciTopoShow
    (
    VXB_DEV_ID busCtrlID
    )
```

This function traverses the PCI bus and prints assorted information about each device found. The information is intended to present the topology of the PCI bus. It includes the following:

- device type
- command and status words
- memory and I/O space configuration (for PCI to PCI bridges)
- values of all implemented BARs

Related information

Overview on page 147
Guidelines for Writing PCI and PCIe Device Drivers on page 147

Driver Methods for PCI Bus Controller Drivers on page 149
Driver Methods for PCI and PCIe Drivers on page 151
Header Files on page 153
PCI Bus Library on page 153
Device Configuration on page 156
PCI Infrastructure Initialization on page 158

8. PCI Express (PCIe) Endpoint Drivers

8.1. Overview

The VxBus framework supports PCI Express (PCIe) endpoint (EP) drivers.

The PCIe system implements point-to-point connections between a root complex and an EP. The root complex communicates to an EP, and the EP communicates to a root complex. VxBus supports the use of a target PCIe hardware controller in root complex mode. Refer to Overview on page 147 for more details.

The PCIe controller on some SoCs can be configured in either root complex or EP mode. VxBus supports the use of a PCIe controller in EP mode. When used in this manner, you will setup the VxWorks system to behave as a PCIe EP and respond correctly to any external PCIe root complex it is attached to.

Related information

Introduction to the PCI Express Endpoint Driver Class on page 162
PCI Express Endpoint Library Device Driver Methods on page 163
Endpoint Library Methods for PCI Express Client Drivers on page 165
General Endpoint Library Methods on page 167
VxBus Driver Methods for the Endpoint Device Driver on page 174
PCI Express Endpoint Library Header File on page 176
Endpoint Device Driver Configuration Values on page 176
Implementation of the Endpoint Device Driver on page 176

8.2. Introduction to the PCI Express Endpoint Driver Class

The PCI Express (PCIe) endpoint driver class is the standard way to present PCIe endpoint (EP) devices to VxWorks clients.

The EP driver class is composed of three elements:

- the EP device driver interfaces to the PCIe hardware controller.
- the PCIe client driver implements a service that is presented to the external PCIe root complex. For example, the client could implement serial, network or file system services.
- the EP library bridges the PCIe client and the EP device driver.

The EP device driver registers an EP device instance using the EP library API. The PCIe client driver requests EP function using the EP library, and the EP library then subsequently instructs the EP device driver to take specific actions.

The EP library source file is located at *installDir* **/vxworks-7/pkgs_v2/os/drv/vxbus-***a.b.c.d* **/buslib/src/pci/vxbPciEpLib.c**. Two different APIs are implemented here. The first is the API used by the EP device driver, the second is used by the PCIe client driver.

Note: The VxBus layer version is represented by *a.b.c.d*.

Related information

Overview on page 162
PCI Express Endpoint Library Device Driver Methods on page 163

Endpoint Library Methods for PCI Express Client Drivers on page 165
General Endpoint Library Methods on page 167
VxBus Driver Methods for the Endpoint Device Driver on page 174
PCI Express Endpoint Library Header File on page 176
Endpoint Device Driver Configuration Values on page 176
Implementation of the Endpoint Device Driver on page 176

8.3. PCI Express Endpoint Library Device Driver Methods

The PCI Express (PCIe) endpoint library implements APIs that are used by the endpoint (EP) device driver.

The EP device driver must call the API to register and unregister EP device instances, create and destroy the PCIe address region, and perform a notify callback function.

The following functions are available within **vxbPciEpLib.c**:

- vxbPciEpDevRegister( )
- vxbPciEpDevUnregister( )
- vxbPciEpDevPoolCreate( )
- vxbPciEpDevPoolDestroy( )
- vxbPciEpDevNotify( )

Syntax for the vxbPciEpDevRegister( ) function is as follows:

```
STATUS vxbPciEpDevRegister
    (
    VXB_PCIEP_DEV *  pEpDev
    )
```

The EP device driver calls this function to register an EP device instance into the EP library. The argument **pEpDev** refers to the EP device instance being registered.

Syntax for the vxbPciEpDevUnregister( ) function is as follows:

```
void vxbPciEpDevUnregister
    (
    VXB_PCIEP_DEV *  pEpDev
    )
```

The EP device driver calls this function to remove an EP device instance that was previously registered with the EP library.

Syntax for the vxbPciEpDevPoolCreate( ) function is as follows:

```
void vxbPciEpDevPoolCreate
    (
    VXB_PCIEP_DEV *  pEpDev,
    VIRT_ADDR        virtBase,
    PHYS_ADDR        physBase,
    size_t           size,
    void *           pHandle
    )
```


WNDRVR

pEpDev

The EP device instance to be added to.

virtBase

The mapped virtual address.

physBase

The mapped physical address.

size

The number of bytes in the PCIe address region.

pHandle

The VxBus access handle pointer.

This method returns a structure called **VXB_PCIEP_POOL**. The EP library uses it to manage the PCIe EP controller device's PCIe address. The structure is defined as follows:

```
typedef struct vxbPciEpPool {
    PHYS_ADDR         physBase;  /* physical base address */
    VIRT_ADDR         virtBase;  /* virtual base address */
    void *            pHandle;
    size_t            size;      /* number of bytes in the PCIe address region */
    size_t            pageSize;  /* MMU page size */
    size_t            pages;     /* total number of pages in this pool */
    UINT32 *          pBitmap;   /* bitmap for allocated page */
    SEM_ID            lock;      /* mutex to protect pool operation */
} VXB_PCIEP_POOL;
```

Syntax for the vxbPciEpDevPoolDestroy( ) function is as follows:

```
void vxbPciEpDevPoolDestroy
    (
    VXB_PCIEP_DEV *  pEpDev
    )
```

The EP device driver calls this function to remove the PCI address region added previously with vxbPciEpDevPoolCreate( ). The argument **pEpDev** refers to the EP device instance that is related to the memory pool being destroyed.

Syntax for the vxbPciEpDevNotify( ) function is as follows:

```
void vxbPciEpDevNotify
    (
    VXB_PCIEP_DEV *  pEpDev,
    UINT8            funcldx
    )
```

The EP device driver calls this function to perform its notify callback function. The argument **pEpDev** refers to the EP device instance performing the notify function. The argument **funcldx** is the function number. Set it to **VXB_PCIEP_FUNC_ALL** to perform all the notify callbacks.

Related information

Overview on page 162
Introduction to the PCI Express Endpoint Driver Class on page 162
Endpoint Library Methods for PCI Express Client Drivers on page 165

General Endpoint Library Methods on page 167
VxBus Driver Methods for the Endpoint Device Driver on page 174
PCI Express Endpoint Library Header File on page 176
Endpoint Device Driver Configuration Values on page 176
Implementation of the Endpoint Device Driver on page 176

8.4. Endpoint Library Methods for PCI Express Client Drivers

The PCI Express (PCIe) endpoint library implements APIs that are used by the PCIe client driver to allocate and free functions resources, and also perform general endpoint (EP) operations.

The following PCIe client driver functions are available within **vxbPciEpLib.c**:

- vxbPciEpFuncAlloc( )
- vxbPciEpFuncAllocByFunc( )
- vxbPciEpFuncFree( )
- vxbPciEpBarResAlloc( )
- vxbPciEpBarResFree( )
- vxbPciEpNotifySet( )

Syntax for the vxbPciEpFuncAlloc( ) function is as follows:

```
VXB_PCIEP_FUNC * vxbPciEpFuncAlloc
    (
    void
    )
```

The client driver calls this function to allocate a new function within the EP library.

Syntax for the vxbPciEpFuncAllocByFunc( ) function is as follows:

```
void vxbPciEpFuncAllocByFunc
    (
    char *       pName,
    UINT32       unit,
    UINT8        funcIdx
    )
```

pName

The VxBus name of the EP device driver.

unit

The VxBus unit number of the EP device driver.

funcIdx

The EP function number as required by the PCIe client driver. Set to **VXB_PCIEP_FUNC_ALL** to force the client driver not to allocate a fixed function number.

This is an advanced version of the vxbPciEpFuncAlloc( ) API function. The PCIe client driver uses this API function to allocate a function within the EP library based on the EP device driver VxBus name, unit and function number.

Syntax for the vxbPciEpFuncFree( ) function is as follows:

```
void vxbPciEpFuncFree
    (
    VXB_PCIEP_FUNC *  pFunc
    )
```

The parameter pFunc points to the function that must be released. The client driver calls this method to release a previously allocated function.

Syntax for the vxbPciEpBarResAlloc( ) function is as follows:

```
VXB_PCIEP_BAR_RES * vxbPciEpBarResAlloc
    (
    UINT8             barNo,
    size_t            barSize,
    size_t            align,
    UINT8             flags
    )
```

barNo

The BAR number must be set to between 0 and 5.

barSize

The BAR register size. It must be aligned.

align

Memory alignment.

flags

The BAR flags. For example, set to **PCI_BAR_MEM_ADDR32** for 32-bit memory decoding, memory request type, and non-prefetchable.

The PCIe client driver calls this function to allocate an aligned and coherent region of system memory for use with the EP BAR register. This method allocates the EP BAR resource memory by calling cacheDmaMalloc( ) and then saves the resulting physical and virtual addresses to the structure called **VXB_PCIEP_BAR_RES**. It then returns the structure, defined as follows:

```
typedef struct vxbPciEpBarRes {
    UINT8             flags;         /* BAR space flag */
    UINT8             barNo;         /* BAR index 0 to 5 */
    PHYS_ADDR         physAddr;      /* aligned physical CPU address */
    VIRT_ADDR         virtAddr;      /* aligned virtual CPU address */
    VIRT_ADDR         virtAddrOrig;  /* original unaligned virtual CPU address */
    size_t            size;          /* register's size of byte */
} VXB_PCIEP_BAR_RES;
```

Syntax for the vxbPciEpBarResFree( ) function is as follows:

```
void vxbPciEpBarResFree
    (
    VXB_PCIEP_BAR_RES * pBarRes
    )
```

The PCIe client driver calls this function to free a BAR resource. The argument **pBarRes** points to the BAR register resource that was allocated by a call to vxbPciEpBarResAlloc( ).

WNDRVR

Syntax for the vxbPciEpNotifySet( ) function is as follows:

```
VXB_PCIEP_BAR_RES * vxbPciEpNotifySet
    (
    VXB_PCIEP_FUNC *     pFunc,
    VXB_PCIEP_NOTIFY_FN fn,
    void *               pArg
    )
```

pFunc

The function instance that was allocated by either vxbPciEpFuncAlloc( ) or vxbPciEpFuncAllocByFunc( ).

fn

The notify call back function.

pArg

The notify callback function parameter.

The PCIe client driver calls this function to setup the notify callback function.

Note: The notify callback function type must be defined as follows:

```
typedef void (* VXB_PCIEP_NOTIFY_FN)(VXB_PCIEP_FUNC * pFunc, void * pArg);
```

Related information

Overview on page 162
Introduction to the PCI Express Endpoint Driver Class on page 162
PCI Express Endpoint Library Device Driver Methods on page 163
General Endpoint Library Methods on page 167
VxBus Driver Methods for the Endpoint Device Driver on page 174
PCI Express Endpoint Library Header File on page 176
Endpoint Device Driver Configuration Values on page 176
Implementation of the Endpoint Device Driver on page 176

8.5. General Endpoint Library Methods

The PCI Express (PCIe) endpoint library implements a general API used by the client driver.

The following methods are available in the general API:

- vxbPciEpBarConfig( )
- vxbPciEpMsiGet( )
- vxbPciEpMsixGet( )
- vxbPciEpMsiSet( )
- vxbPciEpMsixSet( )
- vxbPciEpIrqRaise( )
- vxbPciEpIrqLower( )
- vxbPciEpMap( )
- vxbPciEpUnmap( )
- vxbPciEpStart( )

- vxbPciEpHeaderSet( )
- vxbPciEpLinkCheck( )
- vxbPciEpCfgRead( )
- vxbPciEpCfgWrite( )

Syntax for the vxbPciEpBarConfig( ) function is as follows:

```
STATUS vxbPciEpBarConfig
    (
    VXB_PCIEP_FUNC *    pFunc,
    VXB_PCIEP_BAR_RES * pBarRes,
    BOOL                enable
    )
```

pFunc

The function instance configured by a previous call to either vxbPciEpFuncAlloc( ) or vxbPciEpFuncAllocByFunc( ).

pBarRes

The BAR resource that was allocated by a call to vxbPciEpBarResAlloc( ).

enable

When TRUE, enable this BAR. Disable the BAR when FALSE.

This method is called to configure the endpoint (EP) BAR register. It checks BAR parameters with device capability and executes EP device configuration. Once the BAR is enabled, the information is updated in the EP configure space. Also, the BAR local address is mapped to its inbound address translation unit (ATU).

Syntax for the vxbPciEpMsiGet( ) function is as follows:

```
void vxbPciEpMsiGet
    (
    VXB_PCIEP_FUNC *   pFunc
    )
```

pFunc

The function instance configured by a previous call to either vxbPciEpFuncAlloc( ) or vxbPciEpFuncAllocByFunc( ).

This method is used to get the EP MSI vector numbers that are allocated by the host.

Syntax for the vxbPciEpMsixGet( ) function is as follows:

```
void vxbPciEpMsixGet
    (
    VXB_PCIEP_FUNC *   pFunc
    )
```

pFunc

The function instance configured by a previous call to either vxbPciEpFuncAlloc( ) or vxbPciEpFuncAllocByFunc( ).

This method is used to get the EP MSI-x vector numbers that are allocated by the host.

Syntax for the vxbPciEpMsiSet( ) function is as follows:

WNDRVR

```
void vxbPciEpMsiSet
    (
    VXB_PCIEP_FUNC *   pFunc,
    UINT8              numVectors
    )
```

pFunc

The function instance configured by a previous call to either vxbPciEpFuncAlloc( ) or vxbPciEpFuncAllocByFunc( ).

numVectors

The required MSI vector number. Valid values are 1, 2, 4, 8, 16 and 32.

This method is used to setup the EP MSI vector number.

Syntax for the vxbPciEpMsixSet( ) function is as follows:

```
void vxbPciEpMsixSet
    (
    VXB_PCIEP_FUNC *   pFunc,
    UINT16             numVectors,
    UINT8              bir,
    UINT32             offset
    )
```

pFunc

The function instance configured by a previous call to either vxbPciEpFuncAlloc( ) or vxbPciEpFuncAllocByFunc( ).

numVectors

The required MSI-x vector. Valid values are between 1 and 2048.

bir

The BAR indicator. Valid values are between **BAR0** and **BAR5**.

offset

The MSI-x table offset.

This method is used to set the EP MSI-x vector numbers, bir and offset parameters.

Syntax for the vxbPciEpIrqRaise( ) function is as follows:

```
void vxbPciEpIrqRaise
    (
    VXB_PCIEP_FUNC *  pFunc,
    PCI_EP_IRQ_TYPE   type,
    UINT16            vector
    )
```

pFunc

The function instance configured by a previous call to either vxbPciEpFuncAlloc( ) or vxbPciEpFuncAllocByFunc( ).

type

The required interrupt type. Valid values are:

- **PCI_EP_IRQ_LEGACY** to select the legacy PCI interrupt type
- **PCI_EP_IRQ_MSI** to select the PCI MSI interrupt type
- **PCI_EP_IRQ_MSIX** to select the PCI MSI-x interrupt type

vector

The interrupt vector to be raised. Valid values are:

Table 1. Valid Vector Numbers

Interrupt Type	Valid Vector Range
PCI_EP_IRQ_LEGACY	0 to 3
PCI_EP_IRQ_MSI	1 to 32
PCI_EP_IRQ_MSIX	1 to 2048

This method raises an INTX/MSI/MSI-x interrupt to the PCIe host that the EP is attached to.

Note: When using the type **PCI_EP_IRQ_LEGACY**, the interrupt is level triggered and this method asserts the interrupt. The interrupt must be deasserted by the software by calling the method vxbPciEpIrqLower( ). Both **PCI_EP_IRQ_MSI** and **PCI_EP_IRQ_MSIX** are edge triggered, and so no manual interrupt deassertion is required.

Syntax for the vxbPciEpIrqLower( ) function is as follows:

```
STATUS vxbPciEpIrqLower
    (
    VXB_PCIEP_FUNC *  pFunc,
    PCI_EP_IRQ_TYPE   type,
    UINT16            vector
    )
```

pFunc

The function instance configured by a previous call to either vxbPciEpFuncAlloc( ) or vxbPciEpFuncAllocByFunc( ).

type

Must be set to **PCI_EP_IRQ_LEGACY**.

vector

The interrupt vector number to be lowered.

This method is used to lower a **PCI_EP_IRQ_LEGACY** level triggered interrupt type to the PCIe host.

Syntax for the vxbPciEpMap( ) function is as follows:

```
void vxbPciEpMap
    (
    VXB_PCIEP_FUNC *  pFunc,
    VIRT_ADDR *       pVirtAddr,
    PHYS_ADDR *       pPhysAddr,
    UINT64            pciAddr,
    size_t            size
    )
```

pFunc

The function instance configured by a previous call to either vxbPciEpFuncAlloc( ) or vxbPciEpFuncAllocByFunc( ).

pVirtAddr

Returned EP local virtual address for the mapped PCI address region.

pPhysAddr

Returned EP local physical address for the mapped PCI address region.

pciAddr

The PCIe host physical address to be mapped.

size

The number of bytes to be mapped.

This method is used to map a specified host PCI address range to an EP local address. This is done by setting the EP outbound ATU. The method returns the mapped EP local virtual address as pVirtAddr.

Syntax for the vxbPciEpUnmap( ) function is as follows:

```
STATUS vxbPciEpUnmap
    (
    VXB_PCIEP_FUNC *  pFunc,
    VIRT_ADDR *       pVirtAddr,
    PHYS_ADDR *       pPhysAddr,
    UINT64            pciAddr,
    size_t            size
    )
```

pFunc

The function instance configured by a previous call to either vxbPciEpFuncAlloc( ) or vxbPciEpFuncAllocByFunc( ).

pVirtAddr

The EP local virtual address to be unmapped.

pPhysAddr

The EP local physical address to be unmapped.

pciAddr

The PCIe host physical address to be unmapped.

size

The number of bytes to be unmapped.

This method is used to unmap the host memory range previously mapped by a call to vxbPciEpMap( ).

Syntax for the vxbPciEpStart( ) function is as follows:

```
STATUS vxbPciEpStart
    (
    VXB_PCIEP_FUNC *  pFunc,
```

```
    BOOL              start
    )
```

pFunc

The function instance configured by a previous call to either vxbPciEpFuncAlloc( ) or vxbPciEpFuncAllocByFunc( ).

start

When TRUE, the specified function must start. When FALSE, the specified function must stop.

This method is used to start or stop an EP function.

> **Note:** Before an EP function can be started, a number of methods must already have been completed:
>
> - vxbPciEpBarConfig( )
> - vxbPciEpMsiSet( ) or vxbPciEpMsixSet( )

Syntax for the vxbPciEpHeaderSet( ) function is as follows:

```
STATUS vxbPciEpStart
    (
    VXB_PCIEP_FUNC *   pFunc,
    VXB_PCIEP_HEADER * pHeader
    )
```

pFunc

The function instance configured by a previous call to either vxbPciEpFuncAlloc( ) or vxbPciEpFuncAllocByFunc( ).

pHeader

The function's configuration header to be defined. The configuration header type is the following:

```
typedef struct vxbPciEpHeader
{
    UINT16         vendorId;        /* PCI vendor ID */
    UINT16         deviceId;        /* PCI device ID */
    UINT8          revId;           /* PCI revision ID */
    UINT8          progifCode;      /* PCI programming interface */
    UINT8          subclassCode;    /* PCI subclass code */
    UINT8          baseclassCode;   /* PCI baseclass code */
    UINT8          cacheLineSize;   /* PCI cacheline size */
    UINT16         subsysVendorId;  /* PCI subsys vendor ID */
    UINT16         subsysId;        /* PCI subsys ID */
    UINT8          interruptPin;    /* PCI interrupt pin */
} VXB_PCIEP_HEADER
```

This method is used to set a function's standard configuration header.

> **Note:** Some fields in the EP header field are always read only. For example, **PCI_CFG_VENDOR_ID** and **PCI_CFG_SUB_VENDOR_ID**.

Syntax for the vxbPciEpLinkCheck( ) function is as follows:

```
STATUS vxbPciEpLinkCheck
    (
    VXB_PCIEP_FUNC *  pFunc
    )
```

pFunc

The function instance configured by a previous call to either vxbPciEpFuncAlloc( ) or vxbPciEpFuncAllocByFunc( ).

This method is used to check the EP link status. It will return **OK** if the link is up, and **ERROR** otherwise.

Syntax for the vxbPciEpCfgRead( ) function is as follows:

```
STATUS vxbPciEpCfgRead
    (
    VXB_PCIEP_FUNC *  pFunc,
    UINT32            offset,
    UINT32            width,
    void *            pData
    )
```

pFunc

The function instance configured by a previous call to either vxbPciEpFuncAlloc( ) or vxbPciEpFuncAllocByFunc( ).

offset

The offset within EP configure space to be read from.

width

The width to be read. Valid values are 1, 2 or 4.

pData

The pointer to be used when reading the configure space.

This method is used to read endpoint device's configure space.

Syntax for the vxbPciEpCfgWrite( ) function is as follows:

```
STATUS vxbPciEpCfgWrite
    (
    VXB_PCIEP_FUNC *  pFunc,
    UINT32            offset,
    UINT32            width,
    UINT32            data
    )
```

pFunc

The function instance configured by a previous call to either vxbPciEpFuncAlloc( ) or vxbPciEpFuncAllocByFunc( ).

offset

The offset within EP configure space to be written to.

width

The width to be written. Valid values are 1, 2 or 4.

data

The data to be written.

This method is used to write to the EP device configure space.

Related information

Overview on page 162
Introduction to the PCI Express Endpoint Driver Class on page 162
PCI Express Endpoint Library Device Driver Methods on page 163
Endpoint Library Methods for PCI Express Client Drivers on page 165
VxBus Driver Methods for the Endpoint Device Driver on page 174
PCI Express Endpoint Library Header File on page 176
Endpoint Device Driver Configuration Values on page 176
Implementation of the Endpoint Device Driver on page 176

8.6. VxBus Driver Methods for the Endpoint Device Driver

The endpoint (EP) device driver has common VxBus methods and structure definitions.

Common VxBus Driver Methods

The following methods are available in the general API:

- vxbDevProbe(VXB_DEV_ID pDev)
- vxbDevAttach(VXB_DEV_ID pDev)

Device Driver VXB_PCIEP_DEV Structure

All EP device driver functionality is accessed through the **VXB_PCIEP_DEV** structure. It is the job of the EP device driver to update all the structure fields to describe the capabilities of this EP device that it controls.

Locate the structure type definitions for **VXB_PCIEP_DEV** in the header file *installDir* **/vxworks-7/pkgs_v2/os/drv/vxbus-***a.b.c.d***/buslib/h/hwif/buslib/vxbPciEpLib.h**.

SL_NODE node

Used to maintain the EP library device list.

VXB_DEV_ID pDev

Used to point to the EP device VxBus ID.

BOOL msiCap

Describes the MSI capability of the EP device.

BOOL msixCap

Describes the MSI-x capability of the EP device.

BOOL notifyCap

Describes the notify capability of the EP device.

UINT8 bitMapReservedBar

A bitwise value describing the EP device reserved BARs For example, a value of 0x30 means that BAR4 and BAR5 are reserved.

UINT8 bitMapFixed64bitBar

A bitwise value describing the EP device fixed 64-bit BARs. For example, a value of 0x30 means that BAR4 and BAR5 only support 64-bit.

UINT64 barSizeFixed[6]

A value describing the EP device fixed BAR size. A barSizeFixed[x] of 0 means the BARx is resizeable. For example, a value of 0x100000 means BARx has a fixed size of 0x100000 bytes.

UINT64 addrAlign

A value describing the EP device BAR alignment. For example:

- PCIEP_ADDR_ALIGN_WITH_SIZE means the BAR address must be aligned with the BAR size.
- 0x100000 means the BAR address must be aligned with 0x100000.

UINT8 maxFunction

A value describing the EP device max physical function. Valid values are between 1 and 8.

Pointers to general EP methods

The structure contains pointers to the methods described in General Endpoint Library Methods on page 167.

- **STATUS (*epDevBarConfig)**
- **STATUS (*epDevMsiSet)**
- **UINT8 (*epDevMsiGet)**
- **STATUS (*epDevMsixSet)**
- **UINT16 (*epDevMsixGet)**
- **STATUS (*epDevIrqRaise)**
- **STATUS (*epDevIrqLower)**
- **STATUS (*epDevMap)**
- **STATUS (*epDevUnmap)**
- **STATUS (*epDevStart)**
- **STATUS (*epDevHeaderSet)**
- **STATUS (*epDevLinkCheck)**
- **STATUS (*epDevCfgRead)**
- **STATUS (*epDevCfgWrite)**

Related information

Overview on page 162
Introduction to the PCI Express Endpoint Driver Class on page 162
PCI Express Endpoint Library Device Driver Methods on page 163
Endpoint Library Methods for PCI Express Client Drivers on page 165
General Endpoint Library Methods on page 167
PCI Express Endpoint Library Header File on page 176
Endpoint Device Driver Configuration Values on page 176
Implementation of the Endpoint Device Driver on page 176

8.7. PCI Express Endpoint Library Header File

The PCI Express (PCIe) endpoint (EP) and client drivers must both include the PCIe endpoint library header file.

This header file is located at *installDir* **/vxWorks-7/pkgs_v2/os/drv/vxbus-***a.b.c.d* **/buslib/h/hwif/buslib/vxbPciEpLib.h**.

Note: The VxBus layer version is represented by *a.b.c.d*.

Related information

Overview on page 162
Introduction to the PCI Express Endpoint Driver Class on page 162
PCI Express Endpoint Library Device Driver Methods on page 163
Endpoint Library Methods for PCI Express Client Drivers on page 165
General Endpoint Library Methods on page 167
VxBus Driver Methods for the Endpoint Device Driver on page 174
Endpoint Device Driver Configuration Values on page 176
Implementation of the Endpoint Device Driver on page 176

8.8. Endpoint Device Driver Configuration Values

When implementing an endpoint (EP) device driver, certain important hardware values must be established and assigned to the driver configuration.

Device configuration values are often located in the board support package (BSP) flattened device tree (FTD) file. Refer to Detecting and Configuring Devices with a Device Tree on page 21 for further details. Alternatively, the hardware advanced configuration and power interface (ACPI) settings are a useful source of information. Also, you can identify hardware values by performing a PCI query on the host workstation attached to the EP VxWorks target.

Related information

Overview on page 162
Introduction to the PCI Express Endpoint Driver Class on page 162
PCI Express Endpoint Library Device Driver Methods on page 163
Endpoint Library Methods for PCI Express Client Drivers on page 165
General Endpoint Library Methods on page 167
VxBus Driver Methods for the Endpoint Device Driver on page 174
PCI Express Endpoint Library Header File on page 176
Implementation of the Endpoint Device Driver on page 176

8.9. Implementation of the Endpoint Device Driver

The implementation of a PCI Express (PCIe) endpoint (EP) driver involves a series of device-specific initialization steps.

The EP driver initialization must be completed and the driver registered using vxbPciEpDevRegister( ) before the vxbDevAttach( ) phase begins.

Each EP device has a PCIe address region determined by the particular SoC hardware design. The EP device driver must map this region and add it to the EP device instance by calling vxbPciEpDevPoolCreate( ).

The EP device driver must have at least one function. A maximum of eight functions are permitted. The EP device driver must update the device instance structure **maxFunction** field with the number of driver functions in the EP before calling vxbPciEpDevRegister( ). Each EP device function is identified on the PCIe bus by a separate bus/device/function tuple.

Registering the Endpoint Device Driver on page 177 The PCI Express (PCIe) Endpoint (EP) driver must follow these steps to register the EP device with the EP library.
Preparing the Client Driver to Call Endpoint Functions on page 178 The client driver must follow these steps before it calls PCI Express (PCIe) Endpoint (EP) driver functions.

Examples of General Client Driver Operations on page 179A client driver that has initialised its interface to the EP device driver can perform many general operations.

Related information

Overview on page 162
Introduction to the PCI Express Endpoint Driver Class on page 162
PCI Express Endpoint Library Device Driver Methods on page 163
Endpoint Library Methods for PCI Express Client Drivers on page 165
General Endpoint Library Methods on page 167
VxBus Driver Methods for the Endpoint Device Driver on page 174
PCI Express Endpoint Library Header File on page 176
Endpoint Device Driver Configuration Values on page 176

Registering the Endpoint Device Driver

The PCI Express (PCIe) Endpoint (EP) driver must follow these steps to register the EP device with the EP library.

About This Task

These steps assume that you have already implemented and tested your individual EP driver functions.

Note: The example given is based on the **ti_tda4** board support package (BSP).

Procedure

1. Populate the **VXB_PCIEP_DEV** driver structure with EP device and driver details.

```
pEpDev = &(pEpDrvCtrl -> epDev);
pEpDev -> pDev = pDev;
pEpDev -> msiCap = TRUE;
pEpDev -> msixCap = TRUE;
pEpDev -> notifyCap = FALSE;
pEpDev -> bitMapReservedBar = 0;
pEpDev -> bitMapFixed64bitBar = 0;
memset_s ((void *) &pEpDev -> barSizeFixed[0], sizeof (pEpDev -> barSizeFixed), 0, sizeof (pEpD
ev -> barSizeFixed));
pEpDev -> addrAlign = PCIEP_ADDR_ALIGN_WITH_SIZE;
pEpDev -> maxFunction = (UINT8) maxFunc;
```

2. Setup the EP PCIe structure for this EP device.

```
vxbPciEpDevPoolCreate (pEpDev, virtPci, physPci, pciSize, pciHandle);
```

3. Populate the **VXB_PCIEP_DEV** driver structure with the EP library general functions.

```
pEpDev -> epDevCfgRead = tiTda4PcieEpCfgRead;
pEpDev -> epDevCfgWrite = tiTda4PcieEpCfgWrite;
pEpDev -> epDevBarConfig = tiTda4PcieEpBarConfig;
pEpDev -> epDevMsiGet = tiTda4PcieEpMsiGet;
pEpDev -> epDevMsiSet = tiTda4PcieEpMsiSet;
pEpDev -> epDevMsixGet = tiTda4PcieEpMsixGet;
pEpDev -> epDevMsixSet = tiTda4PcieEpMsixSet;
pEpDev -> epDevIrqRaise = tiTda4PcieEpIrqRaise;
pEpDev -> epDevIrqLower = tiTda4PcieEpIrqLower;
```


WNDRVR

```
pEpDev -> epDevMap = tiTda4PcieEpMap;
pEpDev -> epDevUnmap = tiTda4PcieEpUnmap;
pEpDev -> epDevStart = tiTda4PcieEpStart;
pEpDev -> epDevHeaderSet = tiTda4PcieEpHeaderSet;
pEpDev -> epDevLinkCheck = tiTda4PcieEpLinkCheck;
```

4. Register the EP device driver with the EP library.

```
vxbPciEpDevRegister (pEpDev);
```

Parent topic: Implementation of the Endpoint Device Driver on page 176

Preparing the Client Driver to Call Endpoint Functions

The client driver must follow these steps before it calls PCI Express (PCIe) Endpoint (EP) driver functions.

About This Task

These steps assume you are implementing an EP client driver which must call at least one EP driver function to do its job.

Procedure

1. Apply for the appropriate EP function.

   This is done either on a function-by-function basis, or for a particular EP driver function.

   To apply for a single EP device driver function:

```
pFunc = vxbPciEpFuncAlloc ();
```

   To apply for a particular valid EP device driver function:

```
pFunc = vxbPciEpFuncAllocByFunc ("tda4-pcie-ep", 0, VXB_PCIEP_FUNC_ALL);
```

2. Allocate and configure the BAR registers.

```
for (bar = 0; bar < PCIEP_BAR_TOTAL; bar++)
{
    pBarRes[bar] = vxbPciEpBarResAlloc (bar, barSize, alignment, PCI_BAR_MEM_ADDR32);
    vxbPciEpBarConfig (pFunc, pBarRes [bar], TRUE);
}
```

3. Configure the standard header.

```
vxbPciEpHeaderSet (pFunc, &testEpHeader);
```

4. Setup the EP MSI and MSI-x capability.

```
vxbPciEpMsiSet (pFunc, 32);
vxbPciEpMsixSet (pFunc, 2048, bir, offset);
```

5. Start the required function.

```
vxbPciEpStart (pFunc, TRUE);
```

Parent topic: Implementation of the Endpoint Device Driver on page 176

Examples of General Client Driver Operations

A client driver that has initialised its interface to the EP device driver can perform many general operations.

Examples of client driver operations include the following:

1. Reading the BAR register to receive the content that has been written there by the host workstation.

```
pBarReg [bar] = (void *) pBarRes [bar] -> virtAddr;
pBarSize [bar] = pBarRes [bar] -> size;
memcpy ((void *) pBarReg [bar], dstbuf, pBarSize [bar]);
```

2. Writing to the BAR register in response to the host workstation.

```
pBarReg [bar] = (void *) pBarRes [bar] -> virtAddr;
pBarSize [bar] = pBarRes [bar] -> size;
memcpy (srcbuf, (void *)pBarReg[bar], pBarSize[bar]);
```

3. Triggering a legacy interrupt to the host workstation.

```
vxbPciEpIrqRaise (pFunc, PCI_EP_IRQ_LEGACY, 0);
vxbPciEpIrqLower (pFunc, PCI_EP_IRQ_LEGACY, 0);
```

4. Triggering an MSI interrupt to the host workstation.

```
vxbPciEpIrqRaise (pFunc, PCI_EP_IRQ_MSI, 1);
```

5. Triggering an MSI-x interrupt to the host workstation.

```
vxbPciEpIrqRaise (pFunc, PCI_EP_IRQ_MSIX, 1);
```

6. Mapping the host PCI address space to access the host workstation memory.

```
vxbPciEpMap (pFunc, &virtAddr, &physAddr, pciAddr, size);
memcpy ((void *)virtAddr, dstbuf, size);
memcpy (srcbuf, (void *)virtAddr, size);
```

Parent topic: Implementation of the Endpoint Device Driver on page 176

9. Pin Mux Drivers

WNDRVR

9.1. Pin Mux Driver Overview

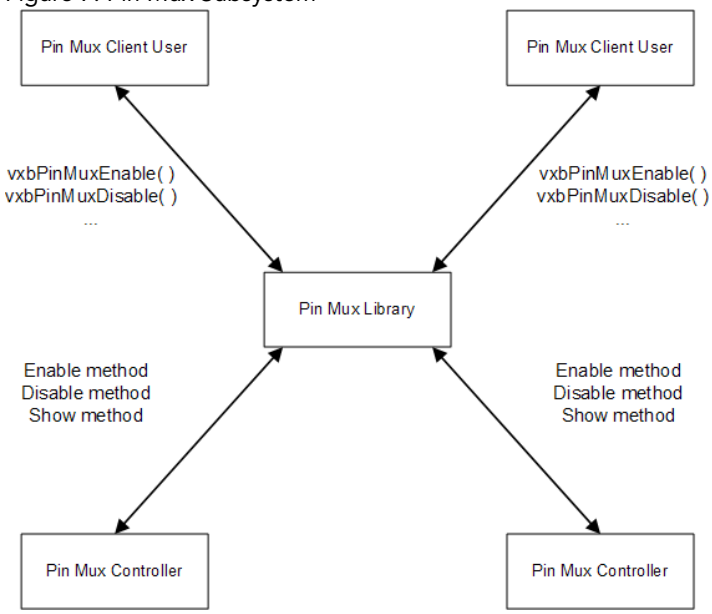
Wind River provides support for pin multiplexer (pin mux) controller drivers and devices for ARM and PowerPC target systems.

A pin mux is a group of pins that is used for multiple mutually exclusive functions on many SOC systems. The pin functions can be selected by a pin mux controller by setting pin multiplexing or configuration parameters such as pull-up/down, tri-state, drive-strength, and so forth.

The pin mux subsystem includes a pin mux library, pin mux controller drivers, and pin mux client users. The pin mux library provides generic interfaces which are used to interact with the pin mux controllers and the client users. A pin mux controller driver must be registered with the pin mux subsystem and the driver must provide pin mux hardware control functions. A pin mux client is normally a device that needs a special group of pins to be selected as a designated function.

An overview of the organization of the pin mux subsystem is illustrated in the following figure:

Figure 7. Pin Mux Subsystem



Related information

VxBus Driver Methods for Pin Mux Drivers on page 180
Driver Header Files for Pin Mux on page 181
Pin Mux Driver Configuration on page 182
Available Functions for Pin Mux Client Users on page 183
Pin Mux Controller Driver Implementation on page 184

9.2. VxBus Driver Methods for Pin Mux Drivers

VxBus pin mux drivers can publish methods for enabling or disabling the hardware pin mux configuration, and for printing information to the console. Some methods are optional, depending on your device.

Pin mux drivers use the following driver methods to interact with the pin mux subsystem:

{vxbPinMuxEnable}( )

The {vxbPinMuxEnable}( ) method enables pin mux configuration in the hardware. This method is required for a pin mux controller device driver.

The prototype is as follows:

```
STATUS vxbPinMuxEnable (VXB_DEV_ID pDev, INT32 offset);
```

pDev

The VxBus device instance for the pin mux controller device.

offset

The device table source (DTS) node offset of the specific pin mux configuration data.

{vxbPinMuxDisable}( )

The {vxbPinMuxDisable}( ) method disables pin mux configuration in the hardware. This method is optional for a pin mux controller device driver because it is not supported by, or may not be relevant for, all pin mux hardware.

The prototype is as follows:

```
STATUS vxbPinMuxDisable (VXB_DEV_ID pDev, INT32 offset);
```

pDev

The VxBus device instance for the pin mux controller device.

offset

The device table source (DTS) node offset of the specific pin mux configuration data.

{vxbPinMuxShow}( )

The {vxbPinMuxShow}( ) method prints specific information on the console provided by the pin mux controller driver. This method is optional for a pin mux controller device driver.

The prototype is as follows:

```
void vxbPinMuxShow (VXB_DEV_ID pDev, INT32 verbose);
```

pDev

The VxBus device instance for the pin mux controller device.

verbose

Sets verbose mode. Use a value of 0 for basic output.

Related information

Pin Mux Driver Overview on page 180
Driver Header Files for Pin Mux on page 181
Pin Mux Driver Configuration on page 182
Available Functions for Pin Mux Client Users on page 183
Pin Mux Controller Driver Implementation on page 184

9.3. Driver Header Files for Pin Mux

VxBus pin mux drivers must include the **vxbPinMuxLib.h** header file.

WNDRVR

All VxBus pin mux drivers must include a single header file to provide the data types required for the driver:

```
#include <subsys/pinmux/vxbPinMuxLib.h>
```

This header file is located in:

installDir **/vxworks-7/pkgs_v2/os/drv/vxbus/subsystem/h/subsys/pinmux/vxbPinMuxLib.h**

Related information

Pin Mux Driver Overview on page 180
VxBus Driver Methods for Pin Mux Drivers on page 180
Pin Mux Driver Configuration on page 182
Available Functions for Pin Mux Client Users on page 183
Pin Mux Controller Driver Implementation on page 184

9.4. Pin Mux Driver Configuration

Pin mux drivers are configured using different sources of information in your system, depending on your target hardware architecture.

The device configuration information for the pin mux subsystem is binded to the device tree nodes of the corresponding board. Some client users require a single static pin mux configuration that can be set up during initialization while others need to reconfigure at runtime. For this reason, each client user can define a set of named states. The number and names of those states is defined by the client user. Each pin mux state is assigned an integer ID (started from 0), and is contiguous. For each state ID, a unique property exists to define the pin mux configuration. Each state can be optionally assigned a name and that name has a one-to-one mapping to the state integer ID.

The required properties of a client user that uses the pin mux subsystem is as follows:

pinmux-0

This property is the list of **phandles**, each pointing at a pin mux configuration node. The referenced pin mux configuration nodes must be child nodes of the pin mux controller device that they configure. Multiple entries can exist in this list so that a state can be built from multiple nodes.

This is the default state of a client's pin mux configuration.

The optional properties of a client user that uses the pin mux subsystem is as follows:

pinmux-1

This property is the list of **phandles**, each pointing at a pin mux configuration node for state 1.

pinmux-n

This property is the list of **phandles**, each pointing at a pin mux configuration node for state *n*. The value of *n* must be less than 128.

pinmux-names

This property is the list of names to assign to the states. List entry 0 defines the name for interger state ID 0, list entry 1 defines the name for integer state ID1, and so forth.

The following is an example of a device tree source (DTS) binding for a pin mux client device:

```
enet
    {
    pinmux-names = "active", "sleep";
    pinmux-0 = <&enet_active>;
```

```
    pinmux-1 = <&enet_sleep1 &enet_sleep2>;
        };
uart
    {
    pinmux-names = "default";
    pinmux-0 = <&uart1_default >;
        };
```

Pin mux controller devices must contain the pin mux configuration nodes that are referenced by the client users. Also, how the pin mux configuration is set according to the pin mux configuration nodes depends on the design of your pin mux controller driver.

The following is an example of the DTS binding for a pin mux controller device:

```
iomux
    {
    ...
    enet_active
        {
        ...
        };
    enet_sleep1
        {
        ...
        };
    enet_sleep2
        {
        ...
        };
    uart1_default
        {
        ...
        };
    };
```

Related information

Pin Mux Driver Overview on page 180
VxBus Driver Methods for Pin Mux Drivers on page 180
Driver Header Files for Pin Mux on page 181
Available Functions for Pin Mux Client Users on page 183
Pin Mux Controller Driver Implementation on page 184

9.5. Available Functions for Pin Mux Client Users

The pin mux library provides several functions that client users can access to set the pin mux configuration without direct access to the pin mux control registers.

The following functions are available for client users:

vxbPinMuxEnable( )

This function enables the pin mux for the default state of a specific client device. Typically, this function is only required by a client device when it must enable its default pin mux configuration during the initialization at system start up.

vxbPinMuxDisable( )

This function disables the pin mux for the default state of a specific client device.

vxbPinMuxEnableById( )

This function enables the pin mux for a specific device and state using the state ID.

vxbPinMuxDisableById( )

This function disables the pin mux for a specific device and state using the state ID.

vxbPinMuxGetIdByName( )

This function gets the pin mux state ID using the state name.

Related information

Pin Mux Driver Overview on page 180
VxBus Driver Methods for Pin Mux Drivers on page 180
Driver Header Files for Pin Mux on page 181
Pin Mux Driver Configuration on page 182
Pin Mux Controller Driver Implementation on page 184

9.6. Pin Mux Controller Driver Implementation

Your pin mux controller device must be properly added to the pin mux subsystem before any client user can set its pin mux implementation.

A generic structure, **VXB_PINMUX_CTRL**, is provided by the pin mux library to represent a pin mux controller:

```
typedef struct vxbPinMuxCtrl
    {
    VXB_DEV_ID  pDev;           /* pointer to the controller instance */
    } VXB_PINMUX_CTRL;
```

The pin mux controller driver must provide the callback methods to set the corresponding hardware as follows:

```
LOCAL VXB_DRV_METHOD xxxPinMuxMethodList[] =
    {
    /* DEVICE API */
     …
    { VXB_DEVMETHOD_CALL(vxbPinMuxEnable),      xxxPinMuxEnable  },
    { VXB_DEVMETHOD_CALL(vxbPinMuxDisable),     xxxPinMuxDisable  },
    { VXB_DEVMETHOD_CALL(vxbPinMuxShow),        xxxPinMuxShow  },
    …
    };
```

The pin mux library provides the following functions for a pin mux controller driver to register with, and unregister from, the pin mux subsystem:

```
STATUS vxbPinMuxRegister (VXB_PINMUX_CTRL * pCtrl);
STATUS vxbPinMuxUnregister (VXB_PINMUX_CTRL * pCtrl);
```

The typical calling sequences for a pin mux controller driver during initialization include the following:

```
pPmxCtrl = (VXB_PINMUX_CTRL *) vxbMemAlloc (sizeof (VXB_PINMUX_CTRL));
pPmxCtrl->pDev = pDev;
vxbPinMuxRegister (pPmxCtrl);
vxbPinMuxEnable (pDev);
```

Related information

Pin Mux Driver Overview on page 180
VxBus Driver Methods for Pin Mux Drivers on page 180
Driver Header Files for Pin Mux on page 181
Pin Mux Driver Configuration on page 182
Available Functions for Pin Mux Client Users on page 183

10. Serial Drivers

10.1. Overview

VxBus serial drivers provide an interface to the I/O system similar to the pre-VxBus multi-mode (SIO) serial drivers, but also provide an enhanced initialization mechanism and a simplified interface to the BSP.

These drivers provide an interface for setting hardware options, such as the number of start bits, stop bits, data bits, parity, and so on. In addition, they provide an interface for polled communication that can provide external mode debugging (such as ROM-monitor style debugging) over a serial line. Each serial port is associated with an **SIO_CHAN** structure. When a single device supports more than one port, the device must be able to provide a separate **SIO_CHAN** structure for each supported port. The contents of the **SIO_CHAN** structure are defined in: *installDir* **/vxworks-7/pkgs_v2/os/core/io/h/sioLib.h**. Many drivers append additional fields to the end of the **SIO_CHAN** structure to contain additional information required by the driver.

Related information

VxBus Driver Methods on page 185
Header Files on page 186
Device Configuration on page 186
Available Utility Functions on page 186
Initialization on page 186
SIO_CHAN and SIO_DRV_FUNCS on page 187

10.2. VxBus Driver Methods

The serial device driver only requires the common VxBus methods.

The required driver methods are:

- vxbDevProbe(VXB_DEV_ID pDev)
- vxbDevAttach(VXB_DEV_ID pDev)
- vxbDevShutdown(VXB_DEV_ID pDev)
- vxbDevDetach(VXB_DEV_ID pDev)

As part of registering the device with VxBus , the serial device driver will call ttyDevCreate( ) in attach method (vxbDevAttach( )), and will call ttyDevDelete( ) in detach method (vxbDevDetach( )). For example:

```
r= ttyDevCreate (tyName, (SIO_CHAN *)pChan, 512, 512);
```

Related information

Overview on page 185
Header Files on page 186
Device Configuration on page 186
Available Utility Functions on page 186
Initialization on page 186

WNDRVR

SIO_CHAN and SIO_DRV_FUNCS on page 187

10.3. Header Files

There are specific serial driver header files that should be included in VxBus serial drivers.

In addition to the generic VxBus and other system header files, as well as any driver-specific header files, VxBus serial drivers should include the following serial driver header files:

```
#include <sioLib.h>
#include <hwif/drv/sio/vxbSioUtil.h>
```

Related information

Overview on page 185
VxBus Driver Methods on page 185
Device Configuration on page 186
Available Utility Functions on page 186
Initialization on page 186
SIO_CHAN and SIO_DRV_FUNCS on page 187

10.4. Device Configuration

All the required configuration information is available in the FDT/ACPI or in some cases PCI query.

Please refer to Detecting and Configuring Devices with a Device Tree on page 21 for information on how to obtain device configuration values from FDT.

Related information

Overview on page 185
VxBus Driver Methods on page 185
Header Files on page 186
Available Utility Functions on page 186
Initialization on page 186
SIO_CHAN and SIO_DRV_FUNCS on page 187

10.5. Available Utility Functions

There are no class-specific utility functions required for serial drivers.

However, some of the inline macros defined in *installDir* **/vxworks-7/pkgs_v2/os/core/io/h/sioLib.h** may be useful for your development. For a complete list of available functions, see the **sioLib.h** file.

Related information

Overview on page 185
VxBus Driver Methods on page 185
Header Files on page 186
Device Configuration on page 186
Initialization on page 186
SIO_CHAN and SIO_DRV_FUNCS on page 187

10.6. Initialization

The VxBus GEN2 serial driver can only be used by TTY functionality. No other module can access its channel structure.

In VxBus GEN2 only interrupt mode is supported. The initialization process and sequence is the generic initialization process for VxBus GEN2 devices. The only unique initialization step is the call to ttyDevCreate()/ttyDevDelete( ) during registration/unregistration. For additional information on serial driver initialization, refer to VxBus Driver Methods on page 185.

Related information

Overview on page 185
VxBus Driver Methods on page 185
Header Files on page 186
Device Configuration on page 186
Available Utility Functions on page 186
SIO_CHAN and SIO_DRV_FUNCS on page 187

10.7. SIO_CHAN and SIO_DRV_FUNCS

The standard SIO structures are defined in **SIO_CHAN** and **SIO_DRV_FUNCS**.

Every SIO port is associated with an **SIO_CHAN** structure. When an instance only provides one port, serial drivers typically put the **SIO_CHAN** structure at the beginning of the driver-specific data structure **pDrvCtrl**. This allows the **pDrvCtrl** structure pointer to be identical to the **SIO_CHAN** structure. This structure contains a single member, a pointer to an **SIO_DRV_FUNCS** structure. These structures are defined in: *installDir* **/vxworks-7/pkgs_v2/os/core/io/h/sioLib.h** The structures are defined as follows:

```
typedef struct sio_drv_funcs SIO_DRV_FUNCS;
typedef struct sio_chan /* a serial channel */
{
  SIO_DRV_FUNCS * pDrvFuncs;
  /* device data */
} SIO_CHAN;
struct sio_drv_funcs /* driver functions */
{
  int (*ioctl)
  (
     SIO_CHAN * pSioChan,
     int cmd,
     void * arg
   );
   int (*txStartup)
   (
     SIO_CHAN * pSioChan
   );
   int (*callbackInstall)
   (
     SIO_CHAN * pSioChan,
     int callbackType,
     STATUS (*callback)(void *, ...),
     void * callbackArg
   );
   int (*pollInput)
   (
     SIO_CHAN * pSioChan,
     char * inChar
   );
   int (*pollOutput)
   (
     SIO_CHAN * pSioChan,
     char outChar
   );
};
```

The members of the **SIO_DRV_FUNCS** structure function as follows:

ioctl( )

Points to the standard I/O control interface routine. This routine provides the primary control interface for the driver. To access the I/O control services for a standard SIO device, use the following symbolic constants:

SIO_BAUD_SET

SIO_BAUD_GET

Sets and retrieves the port baud rate.

SIO_HW_OPTS_SET

SIO_HW_OPTS_GET

Sets and retrieves the port hardware options. The available options are: CLOCAL, HUPCL, CREAD, CSIZE, PARENB, and PARODD.

For more information on these options, see:

installDir **/vxworks-7/pkgs_v2/os/core/io/shareh/sioLibCommon.h**

SIO_MODE_SET

SIO_MODE_GET

SIO_AVAIL_MODES_GET

Sets and retrieves the port mode to switch between polled mode and interrupt driven mode, and find which modes are available. Polled mode is specified as **SIO_MODE_POLL** and interrupt driven mode is specified with **SIO_MODE_INT**. When **SIO_AVAIL_MODES_GET** is used, the values of **SIO_MODE_POLL** and **SIO_MODE_INT** are logically or-d together as follows:

```
*(int *)arg = SIO_MODE_INT | SIO_MODE_POLL;
```

SIO_OPEN

Sets modem control lines (**RTS** and **DTR**) to **TRUE** if not already set, and initializes the device for user operation. Only valid if S **IO_HUP** is supported.

SIO_HUP

Resets **RTS** and **DTR** signals.

Other ioctl( ) commands can be supported as well. For a more complete list of ioctl( ) commands that can be supported by serial drivers (such as keyboard modes and keyboard LED states), see: *installDir* **/vxworks-7/pkgs_v2/os/core/io/shareh/sioLibCommon.h**.

txStartup( )

Provides a pointer to the routine that the system calls when new data is available for transmission. Typically, this routine is called only from the **ttyDrv.o** module. This module provides a level of functionality that allows a raw serial channel to behave with line control and canonical character processing.

callbackInstall( )

Provides the driver with pointers to callback routines that the driver can call asynchronously to handle character puts and gets. The driver is responsible for saving the callback routines and arguments that it receives from the callbackInstall( ) routine. The available callbacks are **SIO_CALLBACK_GET_TX_CHAR** and **SIO_CALLBACK_PUT_RCV_CHAR** .

- Define **SIO_CALLBACK_GET_TX_CHAR** to point to a routine that fetches a new character for output. The driver calls this callback routine with the supplied argument and an additional argument that is the address to receive the new output character (if any). The called routine returns **OK** to indicate that a character was delivered or **ERROR** to indicate that no more characters are available.
- Define **SIO_CALLBACK_PUT_RCV_CHAR** to point to a routine the driver can use to pass characters to the system. For each incoming character, the callback routine is called with the supplied argument, and the new character as a second argument. Drivers normally do not care about the return value from this call. In most cases, there is nothing that a driver can do but drop a character if the I/O system is not able to receive it.

pollInput( ) and pollOutput( )

Provide an interface to polled mode operations of the driver. These routines are not called unless the device has already been placed into polled mode by an **SIO_MODE_SET** operation. To avoid blocking and improve performance, once **TxFIFO** is available, pollOutput( ) will place the character into **TxFIFO** but not wait for the transmitter to finish the last send.

Related information

Overview on page 185
VxBus Driver Methods on page 185
Header Files on page 186
Device Configuration on page 186
Available Utility Functions on page 186
Initialization on page 186

11. SPI Drivers

11.1. Overview

VxBus serial peripheral interface (SPI) library drivers are used to provide a common SPI transfer routine (vxbSpiDevXfer ( )) between the SPI controller and the various SPI interface devices.

Typically, the SPI controller creates an SPI device instance for those devices that connect on the bus at the VxBus vxbDevAttach( ) phase. After that, the SPI controller is ready and the user can call vxbSpiDevXfer ( ) to transfer data between the SPI device and the corresponding controller.

To add a new SPI module driver, the user should create two drivers: the SPI controller driver and an SPI device driver. The device driver organizes the transfer data according to the corresponding datasheet, then calls vxbSpiDevXfer ( ) to transmit the data to the controller handle. The bus controller driver provides basic initialization, enables the controller, and handles SPI data operations.

Related information

VxBus Driver Methods on page 189
Header Files on page 190
DTS Configuration on page 191
Initialization on page 191
Implementing Driver Service Routines on page 192
Device Driver on page 192

11.2. VxBus Driver Methods

All SPI drivers written in accordance with the VxBus model publish a single driver method, { VXB_SPI_XFER }( ).

Within an SPI controller driver, the { vxbSpiXfer }( ) method is implemented using a driver-provided routine with the following prototype:

```
STATUS VXB_SPI_XFER
(
VXB_DEV_ID pDev,
SPI_HARDWARE * pHardware,
SPI_TRANSFER * transfer
)
```

The pDev parameter describes the specific SPI controller instance within the system. The pHardware parameter refers to the SPI device feature, such as **chipSelect**, **mode**, **max frequency**, etc. The structure is defined as follows:

```
typedef struct vxbSpiDevInfo
{
char * deviceName;
UINT8  chipSelect;
UINT8  bitWidth;
UINT32 devFreq;
UINT32 mode;
}SPI_HARDWARE;
```

deviceName: Indicates SPI device name, used to match the corresponding driver.

chipSelect: Connect on which CS.

bitWidth: The bit width per words.

devFreq: The maximum work frequency of the SPI device.

mode: The mode field holds a bit wise value that describes the capabilities of the SPI device. This value is constructed by performing a logical OR operation on the appropriate values from the values defined in: **vxworks-7/pkgs_v2/os/drv/vxbus/buslib/h/hwif/buslib/vxbSpiLib.h**. The bit wise options are shown below:

```
#define SPI_MODE_0 (0 | 0)
#define SPI_MODE_1 (0 | SPI_CKPHA)
#define SPI_MODE_2 (SPI_CKPOL | 0)
#define SPI_MODE_3 (SPI_CKPOL | SPI_CKPHA)
#define SPI_CSPOL_HIGH 0x04 /* if defined CS active high otherwise active-low */
#define SPI_LSB_FIRST 0x08 /* if defined LSB first otherwise MSB first */
#define SPI_LOOP 0x10 /* if defined LOOP mode otherwise normal mode */
#define SPI_FULL_DUPLEX 0x20
```

Related information

Overview on page 189
Header Files on page 190
DTS Configuration on page 191
Initialization on page 191
Implementing Driver Service Routines on page 192
Device Driver on page 192

11.3. Header Files

All SPI drivers written in accordance with the VxBus model need to include a single header file to provide the data types required for the driver.

The following is an example of including the required header file:

WNDRVR

```
#include <hwif/buslib/vxbSpiLib.h>
```

Related information

Overview on page 189
VxBus Driver Methods on page 189
DTS Configuration on page 191
Initialization on page 191
Implementing Driver Service Routines on page 192
Device Driver on page 192

11.4. DTS Configuration

SPI drivers do not typically require configuration information from the BSP beyond the normal device-specific information provided for all drivers.

Below is an SPI DTS example for the Freescale P2020 rdb board:

```
spi@7000 {
                #address-cells = <1>;
                #size-cells = <0>;
                cell-index = <0>;
                compatible = "fsl-espi";
                reg = <0x7000 0x1000>;
                clock-frequency = <250000000>;
                interrupts = <59 2 0 0>;
                interrupt-parent = <&mpic>;
        };
```

When VxBus SPI drivers are used, the BSP must be configured to include the specified SPI controller and SPI device driver.

Related information

Overview on page 189
VxBus Driver Methods on page 189
Header Files on page 190
Initialization on page 191
Implementing Driver Service Routines on page 192
Device Driver on page 192

11.5. Initialization

SPI drivers perform their initialization during the phases of VxBus initialization.

SPI driver initialization takes part in two phases.

Phase 1 - vxbDevProbe

This function will match the FDT type SPI device with the SPI driver. The FDT describes the SPI device with a compatible property. The SPI driver also has an identity table that lists supported SPI devices. The vxbDevProbe( ) routine compares compatible strings and returns the minimum order number of matched compatible strings.

Phase 2 - vxbDevAttach

During VxBus initialization phase 2 vxbDevAttach ( ) is invoked. The SPI driver initializes all of its internal data structures, performs any required initialization of the SPI hardware, and connects its driver-specific ISR to the interrupt source. For an SPI controller driver, an SPI device is then created and connected within the bus by calling vxbDevCreate ( ).

Typically, the SPI controller driver will create two semaphores at this stage. The first semaphore is a mutex semaphore that is used to protect a shared resource. Consider the case where there are two SPI devices that are trying to access the same bus controller. In this situation, one must be held until the other is done accessing the bus controller. The second semaphore is used to synchronize between the ISR and the task. In this case, using semTake( ) to acquire the binary sync semaphore before the one byte transfer is done. Then, waiting until a hardware interrupt occurs and then giving (using semGive( )) the semaphore in an ISR context in order to signal the task that the transfer is complete.

Related information

Overview on page 189
VxBus Driver Methods on page 189
Header Files on page 190
DTS Configuration on page 191
Implementing Driver Service Routines on page 192
Device Driver on page 192

11.6. Implementing Driver Service Routines

This chapter documents service routines specific to SPI.

The service routines that can be supported by an SPI driver are described here:

All SPI controller drivers should implement a transfer routine which handles data transfer between the device and the controller. This routine should also handle the SPI timing requirements.

The transfer routine handles read and write operation on a given SPI target. The prototype for this routine is as follows:

```
LOCAL STATUS xxxspiTransfer
(
VXB_DEV_ID pDev, /* controller instance */
SPI_HARDWARE * pSpiDev, /* device hardware info */
SPI_TRANSFER * transfer /* device transfer data */
)
```

The controller driver retrieves the SPI device hardware parameter from pSpiDev and gets the transfer data from parameter transfer, then handles the register to finish data transfer.

Related information

Overview on page 189
VxBus Driver Methods on page 189
Header Files on page 190
DTS Configuration on page 191
Initialization on page 191
Device Driver on page 192

11.7. Device Driver

Currently, VxBus supports SPI interfaces for SPI serial flash devices from most vendors. The corresponding component is **DRV_SPIFLASH_SP25**.

The SPI library supplies the vxbSpiDevXfer ( ) routine to access all kinds of SPI devices. The prototype for this routine is as follows:

```
STATUS vxbSpiDevXfer
(
VXB_DEV_ID     pDev,
SPI_HARDWARE * pHardWare,
SPI_TRANSFER * transfer
)
```

The **VXB_DEV_ID** parameter pDev describes the accessed SPI device instance within the system. The corresponding controller driver must be ready at this time.

For more information, refer to the flash drivers.

Related information

Overview on page 189
VxBus Driver Methods on page 189
Header Files on page 190
DTS Configuration on page 191
Initialization on page 191
Implementing Driver Service Routines on page 192

12. Storage Drivers

12.1. Overview

Storage device drivers are the interface between your file system and the storage hardware on your board.

VxWorks provides support for several types of storage devices including:

- SATA devices
- SD/MMC devices
- NAND devices
- NOR devices

Related information

SATA Devices on page 193
SD/MMC Host Controller Devices on page 197
FLASH (NOR) Devices on page 203
NVMe Devices on page 207
Migrating from a Previous Version of VxWorks on page 208

12.2. SATA Devices

SATA storage device drivers include code to connect the driver to VxBus and to the flattened device tree (FDT) infrastructure or ACPI for the target hardware. The drivers also include specific driver methods.

Header Files

Each storage driver must include the following header files in order to use the higher-level block device and event handling utilities:

```
#include <drv/xbd/xbd.h>
#include <drv/erf/erfLib.h>
#include "vxbSataLib.h"
#include "vxbSataXbd.h"
#include <hwif/vxbus/vxbLib.h>
```

Including the Driver in Your VxWorks Image

To include the FSL SATA driver in your VxWorks project, include theFDT_DRV_STORAGE_FSLSATA component.

For Intel-based systems, Wind River provides a PCI-based driver for the Silicon Image 31XX SATA controller. To include this driver in your VxWorks image project, include the DRV_STORAGE_SI31XX component.

XBD Ioctl Interface for SATA Driver

The SATA driver uses an XBD ioctl interface. By using theioctl( ) command in this interface, the file system is able to send the SATA driver. The SATA driver provides basic XBD commands such as XBD_GETGEOMETRY, XBD_GETBASENAME, and XBD_GETINFO. In addition, the SATA driver provides SATA specific commands such as the S.M.A.R.T. (Self-Monitoring Analysis and Reporting Technology) command, and the command to identify information.

For more information about the SATA XBD ioctl interface, please refer to thevxbSataXbd API reference.

Note: BD_SATA_PARMSGETuses SATA to identify any requested information. When reading data from hardware, this command does not begin byte swaping. You need to manually swap data to decode readable information.

Device Tree Source (DTS) Requirements

For the FSL SATA driver, you must include the following DTS bindings (this example is for the fsl_p3p4p5 BSP):

```
sata@220000 {
    compatible = "fsl,sata";
    reg = <0x220000 0x1000>;
    interrupts = <68 0x2 0 0>;
    interrupt-parent = <&mpic>;
};
```

This example shows the minimum information that the DTS must provide including the compatibility string, the physical address, and interrupt information.

Because some SATA controllers require additional configuration parameters, Wind River recommends that you provide them as part of the DTS node rather than hard coding the values in the device initialization code.

Writing a SATA Driver on page 195 When writing a SATA device driver, you must provide specific code and implement certain driver methods.

Related information

Overview on page 193
SD/MMC Host Controller Devices on page 197
FLASH (NOR) Devices on page 203
NVMe Devices on page 207
Migrating from a Previous Version of VxWorks on page 208

Writing a SATA Driver

When writing a SATA device driver, you must provide specific code and implement certain driver methods.

About This Task

The following steps provide the basic requirements for writing a VxBus SATA device driver:

Procedure

1. Create the driver files.

   Your driver must reside under the following directory:

   installDir **/vxworks-7/pkgs_v2/storage/bdm/sata/**

   Provide a header file, a source file, and a component description file (CDF) for the driver.

2. Add the corresponding device tree source (DTS) description to the DTS file in the BSP folder.

   Note: This step is only required for ARM, RISC-V, and PowerPC-based systems. Intel-based systems use ACPI.

3. In the driver source file, add a method table that defines, at a minimum, the driver probe and attach methods.

   For example:

```
LOCAL VXB_DRV_METHOD MySataDeviceMethods[] =
{
  { VXB_DEVMETHOD_CALL (vxbDevProbe),  MySataCtrlProbe},
  { VXB_DEVMETHOD_CALL (vxbDevAttach), MySataCtrlAttach},
    VXB_DEVMETHOD_END
};
```

4. In the driver source file, add a driver declaration.

   For example:

```
VXB_DRV vxbFdtmySataDrv =
{
   { NULL },
   "mySata", /* Name */
   "My Sata controller description",   /* Description */
   VXB_BUSID_FDT,   /* or VXB_BUSID_PCI or VXB_BUSID_IADT */
   0,    /* Flags */
   0,     /* Reference count */
   &MySataDeviceMethods     /* Method table */
};
VXB_DRV_DEF(vxbFdtMySataDrv)
```

5. Create an attach method similar to the following:

```
LOCAL STATUS  MySataCtrlAttach
    (
    VXB_DEV_HANDLE pDev
    )
```

```
{
SATA_HOST *    pDrvCtrl;
VXB_RESOURCE * pRes;
int            i;

pDrvCtrl = (SATA_HOST *)vxbMemAlloc (sizeof(SATA_HOST));
if (pDrvCtrl == NULL)
    return ERROR;

pDrvCtrl->pDev = pDev;
vxbDevSoftcSet (pDev, (void *) pDrvCtrl);

/* Get memory resource */

for (i = 0; i < VXB_MAXBARS; i++)
    {
    pRes = vxbResourceAlloc (pDev, VXB_RES_MEMORY, i);
    if ((pRes !=NULL) && (pRes->pRes != NULL))
        {
        break;
        }
    }

if (i == VXB_MAXBARS)
    {
    goto errExit;
    }

pDrvCtrl->pAddrRes = pRes;
pDrvCtrl->regHandle = (void *)((ULONG)((VXB_RESOURCE_ADR *)pRes->pRes)->pHandle);
pDrvCtrl->regBase[0] = (void *)((VXB_RESOURCE_ADR *)pRes->pRes)->virtual;

/* Get interrupt resource */

pRes = vxbResourceAlloc (pDev, VXB_RES_IRQ, 0);
if (pRes == NULL)
    {
    goto errExit;
    }

pDrvCtrl->pIntRes = pRes;

if(OK != mySataCtrlInit(pDrvCtrl)) /* a non VxBus specific SATA controller Init */
    {
    goto errExit;
    }
/* connect the SATA interrupt and enable it */
if (OK != vxbIntConnect(pDev, pDrvCtrl->pIntRes,
                        (VOIDFUNCPTR)mySataIsr, (void *)pDrvCtrl))
    {
    goto errExit;
    }
if (OK != vxbIntEnable(pDev, pDrvCtrl->pIntRes))
    {
    goto errExit;
    }
return OK;
```

```
errExit:
    if (pDrvCtrl->pAddrRes!= NULL)
        (void)vxbResourceFree(pDev, pDrvCtrl->pAddrRes);

    if (pDrvCtrl->pIntRes != NULL)
        (void)vxbResourceFree(pDev, pDrvCtrl->pIntRes);
    vxbMemFree (pDrvCtrl);
    return ERROR;

    }
```

6. Implement the following methods, which are expected by the SATA storage subsystem:

   SataCmdIssue( )

   This method sends a SATA command to the SATA controller. This method is not VxBus specific and is heavily dependent on your SATA controller. Use the reference code provided with your hardware as a baseline.

   XferReq( )

   This method handles a transfer request. This method is not VxBus specific and is heavily dependent on your SATA controller. Use the reference code provided with your hardware as a baseline.

7. Implement the required functions.

   At a minimum, your driver must implement the following functions:

   mySataIsr( )

   This is an interrupt service routine (ISR). This ISR is connected using the vxbIntConnect( ) function from the driver attach method.

   mySataCtrlInit( )

   This function initializes the SATA controller hardware. This function is called by the driver attach method.

Parent topic: SATA Devices on page 193

12.3. SD/MMC Host Controller Devices

SD/MMC storage device drivers include code to connect the driver to VxBus and to the flattened device tree (FDT) infrastructure for the target hardware. The drivers also include specific driver methods.

Header Files

Each storage driver must include the following header files in order to use the higher-level block device and event handling utilities:

```
#include <drv/xbd/xbd.h>
#include <drv/erf/erfLib.h>
#include <vxbSdMmcLib.h>
#include <hwif/vxBus.h>
#include <hwif/buslib/vxbFdtLib.h>
```


WNDRVR

Including the Driver in Your VxWorks Image

To include an SD or MMC device driver in your project, include the component for your desired driver in your VxWorks image project (VIP). The following table describes the available components and their associated device types.

Component Name	Description
DRV_FSL_SDHC_CTRL	FSL SD host controller driver. This component requires the SD bus component, INCLUDE_SD_BUS.
DRV_IMX_SDHC_CTRL	FSL IMX SD host controller driver
DRV_TI_MMCHS_CTRL	TI MMC host controller driver
DRV_SDSTORAGE_CARD	SD card target driver
DRV_MMCSTORAGE_CARD	MMC card target driver

Device Tree Source (DTS) Requirements

The following DTS bindings are required for the FS SD host controller driver (this example is for the fsl_p3p4p5 BSP):

```
sdhc@114000 {
    compatible = "fsl,sdhc";
    reg = <0x114000 0x1000>;
    clock-frequency = <375000000>;
    interrupts = <48 2 0 0>;
    interrupt-parent = <&mpic>;
};
```

This example shows the minimum information that the DTS must provide including the compatibility string, the physical address, and interrupt information.

Because some SD host controllers require additional configuration parameters, Wind River recommends that you provide them as part of the DTS node rather than hard coding the values in the device initialization code.

In addition to the basic bindings, the following example (for the IMX SD host controller) also provides a number of configuration parameters:

```
sdhc0: sdhc@0x02190000 {
        compatible = "imx6Ai,sdhc";
        reg = <0x02190000 0x4000>;
        clock-frequency = <198000000>;
        clocks = <&clk 52>;
        interrupts = <54>;
        interrupt-parent = <&intc>;
        cd_type = <2>;
        wp_type = <2>;
        cd_pin = <1>;
        wp_pin = <148>;
        pinmux-0 = <&usdhc1_2>;
};
```

In the above example, the following configuration parameters are provided

- card detect type (cd_type)
- write protect type (wp_type)
- card detect GPIO pin index (cd_pin)
- write protect GPIO pin index (wp_pin)
- pin mux handle (pinmux-0)

Note: Unlike the host controller device, the actual SD or MMC card does not have a DTS representation.

Writing an SD/MMC Driver on page 199 When writing an SD/MMC device driver, you must provide specific code and implement certain driver methods.

Related information

Overview on page 193
SATA Devices on page 193
FLASH (NOR) Devices on page 203
NVMe Devices on page 207
Migrating from a Previous Version of VxWorks on page 208

Writing an SD/MMC Driver

When writing an SD/MMC device driver, you must provide specific code and implement certain driver methods.

About This Task

The following steps provide the basic requirements for writing a VxBus SD/MMC device driver:

Procedure

1. Create the SD/MMC host controller driver files.

   Your SD/MMC host controller driver must reside under one of the following directories:

   installDir **/vxworks-7/pkgs_v2/storage/bdm/sdmmc/host/sdhc**

   installDir **/vxworks-7/pkgs_v2/storage/bdm/sdmmc/host/timmchs**

   Provide a header file, a source file, and a component description file (CDF) for the driver.

2. Create the SD/MMC card driver files.

   Your SD/MMC card driver must reside under the following directory:

   installDir **/vxworks-7/pkgs_v2/storage/bdm/sdmmc/device/storage**

   Provide a header file, a source file and a CDF file for the driver.

3. Add the corresponding device tree source (DTS) description to the DTS file in the BSP folder.

   Note: This step is only required for ARM, RISC-V, and PowerPC-based systems. Intel-based systems use ACPI.

4. For an SD host controller driver, add a method table in the source file that defines, at a minimum, the {vxbDevProbe}( ), {vxbDevAttach}( ), and {vxbSdSpecInfoGet}( ) driver methods.

   For example:

```
LOCAL VXB_DRV_METHOD MySdhcCtrl_methods[] =
{
  {VXB_DEVMETHOD_CALL(vxbDevProbe), MySdhcCtrlProbe },
  {VXB_DEVMETHOD_CALL(vxbDevAttach), MySdhcCtrlAttach },
  {VXB_DEVMETHOD_CALL(vxbSdSpecInfoGet), MySdhcSpecInfoGet },
   VXB_DEVMETHOD_END
};
```

5. In the source file, add the driver declaration for the host controller driver.

   For example:

```
VXB_DRV vxbFdtmySdhcDrv =
{
   { NULL },
   "mySdhc",   /* Name */
   "My Sata controller description",   /* Description */
   VXB_BUSID_FDT,   /* or VXB_BUSID_PCI or VXB_BUSID_IADT */
   0,     /* Flags */
   0,        /* Reference count */
   &MySdhcCtrl_methods     /* Method table */
};
VXB_DRV_DEF(vxbFdtmySdhcDrv)
```

6. Write the command issue function.

   This function takes (as parameters) the device to which the command is targeted, the command opcode, the desired sector offset, a pointer to a data buffer, and so forth. The function puts this information into the format required by the controller. Finally, the function completes the necessary register or descriptor accesses to queue this command on the controller. All of the parameters (aside of the device handle) are passed in the *SD_CMD* structure as follows:

```
/* The data of SD cmd define */

typedef struct
    {
    void *      buffer;
    UINT32      blkNum;
    UINT32      blkSize;
    BOOL        isRead;
    } SDDATA;

/* SD command define */

typedef struct
    {
    UINT32      cmdIdx;
    UINT32      cmdArg;
    UINT32      rspType;
    UINT32      cmdRsp[4];
    UINT32      cmdErr;
    UINT32      dataErr;
    BOOL        hasData;
```


WNDRVR

```
    SDDATA     cmdData;
    } SD_CMD;
```

Following is an example:

```
STATUS imxSdhcCmdIssue
    (
    VXB_DEV_HANDLE pDev,
    SD_CMD * pSdCmd
    )
    {
    SDHC_DEV_CTRL * pDrvCtrl;
    SD_HOST_CTRL * pSdHostCtrl;
    STATUS rc;

    pDrvCtrl = (SDHC_DEV_CTRL *)GET_DRVCTRL(pDev);
    if (pDrvCtrl == NULL)
        return ERROR;

    pSdHostCtrl = (SD_HOST_CTRL *)(&(pDrvCtrl->sdHostCtrl));
    if (pSdHostCtrl == NULL)
        return ERROR;

    if (pSdHostCtrl->polling)
        rc = sdhcCtrlCmdIssuePoll(pDev, pSdCmd);
    else
        rc = sdhcCtrlCmdIssue(pDev, pSdCmd);

    return (rc);
    }
```

7. Write the SD, MMC, or SDIO host controller initialization function.

   This function/s provide/s basic initialization for clock set, bus width set, host controller initialization, and so forth. This function is called at the end of the driver attach method. In the following example, there are three separate functions to perform this basic initialization:

```
LOCAL STATUS imxSdhcBusWidthSetup
    (
    VXB_DEV_HANDLE pDev,
    UINT32 width
    )
    {
    if (pDev == NULL)
        return ERROR;

    CSR_CLRBIT_4 (pDev, SDHC_PROCTL, PROCTL_DTW_4BIT | PROCTL_DTW_8BIT_FSL);
    switch (width)
        {
        case SDMMC_BUS_WIDTH_1BIT:
            CSR_SETBIT_4 (pDev, SDHC_PROCTL, PROCTL_DTW_1BIT);
            break;
        case SDMMC_BUS_WIDTH_4BIT:
            CSR_SETBIT_4 (pDev, SDHC_PROCTL, PROCTL_DTW_4BIT);
            break;
        case SDMMC_BUS_WIDTH_8BIT:
```

```
            CSR_SETBIT_4 (pDev, SDHC_PROCTL, PROCTL_DTW_8BIT_FSL);
            break;
        default:
            break;
        }
    return OK;
    }

LOCAL STATUS imxSdhcClkFreqSetup
    (
    VXB_DEV_HANDLE pDev,
    UINT32 clock
    )
    {
    SDHC_DEV_CTRL * pDrvCtrl;

    int divmax, ddrMode;
    int prescaler;
    int div = 1;
    unsigned int val;
    UINT32 clkSrc;

    pDrvCtrl = (SDHC_DEV_CTRL *)GET_DRVCTRL(pDev);
    if (pDrvCtrl == NULL)
        return ERROR;

    clkSrc = pDrvCtrl->sdHostCtrl.curClkFreq;
    ....
    ....
val = CSR_READ_4 (pDev, SDHC_SYSCTL);
    val |= (div << SYSCTL_DVS_SHIFT) |
           (prescaler << SYSCTL_DIV_SHIFT);

    CSR_WRITE_4 (pDev, SDHC_SYSCTL, val);

    vxbUsDelay (2000);

    CSR_CLRBIT_4 (pDev, SDHC_VEND_SPEC, SDHC_VEND_SPEC_FRCCLK);
    ....
}

LOCAL STATUS imxSdhcInit
    (
    VXB_DEV_HANDLE pDev
    )
    {
    STATUS rc;
    SDHC_DEV_CTRL * pDrvCtrl;

    pDrvCtrl = (SDHC_DEV_CTRL *)GET_DRVCTRL(pDev);
    if (pDrvCtrl == NULL)
        return ERROR;

    rc = sdhcCtrlInit(pDev);
    if (rc == ERROR)
        return ERROR;
    pDrvCtrl->sdHostCtrl.version = CSR_READ_4 (pDev, SDHC_HOSTVER) & 0xffff;
```

```
    /*
     * This 'water mark level register' is not defined by SD Host Controller
     * Simplified Specification, it should be Freescale specific thing.
     */

    if (pDrvCtrl->sdHostCtrl.dmaMode == SDHC_DMA_MODE_PIO)
        CSR_WRITE_4 (pDev, SDHC_WML, (WML_WR_MAX << WML_WR_SHIFT) |
                     WML_RD_MAX_PIO);
    else
        CSR_WRITE_4 (pDev, SDHC_WML, (WML_WR_MAX << WML_WR_SHIFT) |
                     WML_RD_MAX);

    /* enable cache snooping */

   CSR_WRITE_4 (pDev, SDHC_DCR, DCR_SNOOP);

   return OK;
   }
```

8. Write the hot plug monitor function.

   This routine monitors the SD/MMC/SDIO device hot plug status. If a device is inserted, set the host controller bus width to 1 bit and bus speed to 400K. Then call sdDeviceCreate( ) to find the device type and match the driver. If a device is removed, call vxbDevRemove( ) to delete the device from the VxBus.

9. Write the {vxbSdSpecInfoGet}( ) VxBus driver method.

   The SD, MMC, or SDIO bus needs the {vxbSdSpecInfoGet}( ) method to obtain host controller information such as the capability, the command issue function, and so forth. For more information, refer to the **SD_HOST_SPEC** structure and the fslSdhcSpecInfoGet( ) function in the following file:

   installDir **/vxworks-7/pkgs_v2/storage/bdm/sdmmc/host/sdhc/src/vxbFslImxShdcCtrl.c**

10. Implement the driver probe and attach methods.

    The probe and attach methods are standard for VxBus.

    Note that the attach method is responsible for resource allocation, retrieval of the flattened device tree (FDT) configuration parameters, and, finally, for calling the device initialization function.

Parent topic: SD/MMC Host Controller Devices on page 197

12.4. FLASH (NOR) Devices

When writing a VxBus device driver for a NOR flash device, you must follow certain guidelines.

NOR Flash Driver Overview

Note: The NOR flash device driver is used for both the non-volatile RAM (NVRAM) and the TrueFFS flash file system features.

The device driver for NOR flash devices is constructed from a library that provides code for the flash command set (the library supports both the DFI and JEDEC standards for obtaining device information). This library code resides under the following directory:

installDir **/vxworks-7/pkgs_v2/storage/bdm/flash/mtd/src/norflash**

The library includes the following files:

- **vxbNorFlashAmdCmdSetLib.c**
- **vxbNorFlashItlCmdSetLib.c**
- **vxbNorFlashLib.c**

Typically, you do not need to expand these files to support your specific flash device.

The library directory also contains the file **vxbNorFlash.c**. This is the main VxBus driver file which contains the VxBus probe and attach methods. In general, you do not need to modify this file aside from adding your flash device name (the compatibility string) to your flash device match table (vxbNorFlash_match).

Header Files

The NOR flash device driver requires the following header files (from the NOR library):

```
#include <norflash/vxbNorFlash.h>
#include <norflash/vxbNorFlashLib.h>
```

In addition, the driver requires the following standard VxBus and flattened device tree (FDT) headers:

```
#include <hwif/vxBus.h>
#include <hwif/buslib/vxbFdtLib.h>
```

Including the Driver in Your VxWorks Image

To include support for the NOR flash device driver in your VxWorks image, include the following component in your VxWorks image project (VIP):

- DRV_VXB_NORFLASH

Depending on your hardware, you may also need to include one of the following command set libraries:

- For AMD/Spansion NOR flash include the DRV_VXB_NORFLASH_CMDSET_AMD component.
- For Intel NOR flash include the DRV_VXB_NORFLASH_CMDSET_ITL component.

Device Tree Source (DTS) Requirements

Typically, the DTS bindings for a NOR flash controller include the fields similar to those shown in the following example (for an FSL p2020rdb):

```
norflash@5000
{
    compatible = "fsl,norflash";
    reg = <0xef000000 0x1000000>;
    maxSize = <0x1000000>;
};
```

The field maxSize specifies the size of the address space where the NOR flash chip is mapped into the host's address space.

Additional optional DTS bindings for a NOR flash driver are as follows:

WNDRVR

cfiPrior

Specifies whether the CFI interface is used to identify the flash chips first. If set, the default value is 1 which indicates that the CFI interface is used first. If this value is set to 0, the JEDEC interface is used first.

bigEndian

Specifies whether or not the flash chips are a big-endian device. If not set, big-endian mode is attempted first, followed by little-endian mode if big-endian is not successful.

portWidth

Specifies the width of the system port to the flash chips. If not set, all the possible widths are tried in order to identify the flash chips.

chipNum

Specifies the number of the Ffash chips that are paralleled to obtain the bigger port width. If not set, all possible numbers are tried in order to identify the flash chips.

chipWidth

Specifies the maximum bus width of the flash chips. If not set, all possible widths are tried in order to identify the flash chips.

wrEna

Specifies the address of the write-enable function supplied by the BSP. For some boards, write-protection must be disabled, or the VPP must be turned on, before any write operations are sent to the flash chips.

Note: VPP is hardware protection against program and erase operations of flash blocks.

legacyProbe

Specifies the address of the legacy identification function supplied by the BSP. If set, the CFI interface and JEDEC interface are skipped.

Initialization

Initialization is performed by the driver attach method which is called when VxBus discovers the flash device. In addition to the generic VxBus-related initialization, you must implement the following NOR-flash-specific initialization functionality:

- Set up the flash information structure so that the NOR flash library has the information it requires. This information includes the flash base access address and set up of function pointers to the provided enable, read, write, and compare functions.
- Initialize the flash subsystem by calling vxbNorFlashLibInit( ). The NOR flash library provides this function.
- Initialize the flash chip information to be used by the flash subsystem. You must implement this functionality as it is specific to your flash chip.
- Reset the flash chip. You must implement this functionality as it is specific to your flash chip.

Writing a NOR Flash Driver on page 206 When writing a NOR flash device driver, you must provide specific code and implement certain driver methods.

Related information

Overview on page 193
SATA Devices on page 193
SD/MMC Host Controller Devices on page 197
NVMe Devices on page 207

Migrating from a Previous Version of VxWorks on page 208

Writing a NOR Flash Driver

When writing a NOR flash device driver, you must provide specific code and implement certain driver methods.

About This Task

Your driver must implement both VxBus-specific functionality and hardware-specific (flash chip) functionality.

The following steps provide the basic requirements for writing a VxBus NOR flash driver:

Procedure

1. Create the driver files.

   The NOR flash controller source files must reside under the following directory:

   installDir **/vxworks-7/pkgs_v2/storage/bdm/flash/mtd/src/norflash**

   The NOR flash controller header files must reside under the following directory:

   installDir **/vxworks-7/pkgs_v2/storage/bdm/flash/mtd/h/norflash**.

2. Add the corresponding device tree source (DTS) description to the DTS file in the BSP folder.
3. In the source file, add a method table that defines, at a minimum, the driver probe and attach methods.

   For example:

```
LOCAL VXB_DRV_METHOD vxbMyNorFlashMethodList[] =
{
    { VXB_DEVMETHOD_CALL(vxbDevProbe), vxbMyNorFlashProbe },
    { VXB_DEVMETHOD_CALL(vxbDevAttach), vxbMyNorFlashAttach },
    VXB_DEVMETHOD_END
};
```

4. In the source file, add the driver declaration.

   For example:

```
VXB_DRV vxbFdtMyFlashCtrlDrv =
{
    { NULL } ,
    "norflash,                    /* Name */
    "VxBus NOR Flash driver",     /* Description */
    VXB_BUSID_FDT,                /* Class, may be VXB_BUSID_IADT as well */
    0,                            /* Flags */
    0,                            /* Reference count */
    &vxbMyNorFlashMethodList      /* Method table */
};
VXB_DRV_DEF(vxbFdtMyFlashCtrlDrv);
```

5. Add a match table to be used by the driver probe method.

For example:

```
LOCAL const VXB_FDT_DEV_MATCH_ENTRY   vxbNorFlash_match[] =
{
    {
     "fsl,norflash", /* compatible */
     (void *)NULL
    },
    {}
};
```

Note: In most cases, providing the **compatible** string is the only modification required for the existing driver (**vxbNorFlash.c**).

6. Create the driver attach method.

   In addition to the generic VxBus-related driver initialization, your attach method must implement the following NOR flash-specific initialization functionality:

   - Set up the flash information structure so that the NOR flash library has the information it requires, including the flash base access address and set up of function pointers to point to the provided enable, read, write, and compare functions.
   - Call vxbNorFlashLibInit( ) to initialize the flash library. (The NOR flash library provides this function.)
   - Initialize the flash chip information to be used by the flash subsystem. You must implement this functionality for your specific flash chips.
   - Reset the flash chip. This function is specific to your hardware (flash chip).

Parent topic: FLASH (NOR) Devices on page 203

12.5. NVMe Devices

VxWorks provides an NVM Express (NVMe) driver.

NVMe is a device interface specification for accessing non-volatile storage media by way of a PCI Express (PCIe) bus. The latest NVMe specification is NVM Express revision 1.3b. It is available from https://nvmexpress.org/

The VxWorks NVMe driver is ported from the FreeBSD NVMe driver. The FreeBSD source code is available from https://github.com/freebsd/freebsd.git.

Configuring VxWorks with NVMe Support on page 207 Configure your VSB project with the **BDM** VSB layer and, and your VIP project with the **INCLUDE_DRV_STORAGE_NVME** component.

Related information

Overview on page 193
SATA Devices on page 193
SD/MMC Host Controller Devices on page 197
FLASH (NOR) Devices on page 203
Migrating from a Previous Version of VxWorks on page 208

Configuring VxWorks with NVMe Support

Configure your VSB project with the **BDM** VSB layer and, and your VIP project with the **INCLUDE_DRV_STORAGE_NVME** component.

Procedure

1. Create a VSB project.
2. Add the **BDM** layer to your project.
3. Build your VSB project.
4. Create a VIP project based on your VSB project.
5. Add the **INCLUDE_DRV_STORAGE_NVME** component.
6. As required, add file system support.

   For example, add dosFs or HRFS (with **INCLUDE_DOSFS** or **INCLUDE_HRFS**).

7. Build the VIP project.

Next Steps

After you have booted VxWorks, use the devs command from the kernel shell to confirm that the NVME device has been instantiated. For example:

```
-> devs
drv name
0 /null
1 /ttyS0
1 /ttyS1
7 /nvme0:1
2 /nvme0:2
value = 25 = 0x19
```

Parent topic: NVMe Devices on page 207

12.6. Migrating from a Previous Version of VxWorks

Certain file systems and storage devices supported in earlier VxWorks releases are obsolete in VxWorks 7. In addition, there are new layers, components, and error codes that are not present in earlier versions of VxWorks.

File System Changes

The following file systems are not supported in VxWorks 7:

- Transaction-based reliable file system (TRFS) support for **dosFs**
- CD-ROM file system (**cdromFs**)
- MIPC file system (MFS)

Layer Changes

The layer **VXBUS_LEGACY** provided VxBus GEN1 backwards compatibility, but it is no longer supported in VxWorks 7.

Storage Device Changes

FLASH (NAND) devices are not supported in VxWorks 7.

Obsolete Components

The following storage components are obsolete in VxWorks 7 and cannot be added to your VxWorks image project (VIP):

- INCLUDE_ATA
- INCLUDE_ATA_SHOW
- INCLUDE_IDE
- INCLUDE_MFS
- INCLUDE_MFS_CLIENT
- INCLUDE_MFS_SERVER
- INCLUDE_MFS_SERVER_FD_CACHE
- INCLUDE_FD
- INCLUDE_CBIO
- INCLUDE_XBD_TRANS

New Error Codes

DOS FS error codes have been updated for compatibility with standard POSIX error codes. The new codes are defined in the following file:

installDir **/vxworks-7/pkgs_v2/storage/fs/dosfs/h/dosFsCommon.h**

Layer Structure of the Storage Group in VxWorks 7

The following information relates to the packaging and delivery of source code for VxWorks 7. Additional layers may be available. See the Wind River online support website for the latest information.

Group Name: **Storage**

Location: *installDir* **/vxworks-7/pkgs_v2/storage**

Description: This package provides the support infrastructure for the storage package class.

Note: The FS layer is a parent layer to a variety of child layers described below. However, the BDM layer does not contain child layers. Instead, the various BDM layer elements can be enabled or disabled from the VSB configuration tool, as supported by the BDM layer makefile.

Table 1. Top Layer of Storage Group

Layer Name	Description
TCPLAY	This layer contains the Truecrypt support libraries.
FS	Local file systems and file system core component.
BDM	Block device management (BDM) layers.

Table 2. File System Sub Layers

Layer Name	Description
FS_COMMON (**FS_CORE_COMMON**)	Core component for file system support, including the **fsMonitor** and **SoftXBD** components.
FS_UTIL	File system utilities for use from the VxWorks kernel shell.
FS_VRFS (**FS_CORE_VFS**)	Virtual file system support.
FS_DOSFS	MS-DOS-compatible file system support.
FS_HRFS	Highly reliable file system (HRFS) support.
FS_ROMFS	Read-only memory file system (ROMFS) support.
FS_NFS	Network file system (NFS) support.
FS_CDROMFS	CDROM compatible file system support

Table 3. Block Device Management (BDM) Layer Elements

BDM Layer Element Name	Description
sata	Provides various SATA controller drivers.
sdmmc	Provides MMC/SD/SDIO host controller driver support.
xbd	Extended block device support.
loopfs	Provides support for loop file system that mounts an image file as a virtual block device.
nvram	Provides facility to use flash memory as non-volitile RAM.
nvme	NVMe PCI controller driver.
flash/tffs	TrueFFS Flash File System
flash/tffs	TrueFFS Flash File System Driver
flash/sim	Flash simulator library.
flash/mtd	Memory technology device (MTD) support.

Related information

Overview on page 193
SATA Devices on page 193
SD/MMC Host Controller Devices on page 197
FLASH (NOR) Devices on page 203

NVMe Devices on page 207

13. Timer Drivers

13.1. Overview

Timer drivers are used to provide a functional interface between hardware timers and the various operating system services that make use of them.

In VxWorks, timer drivers are used to provide two distinct types of timing services:

- a periodic interrupt timer
- a timestamp timer

Timestamp drivers are used to allow middleware to quickly read a timestamp value from a device in order to place a timestamp value on an event that has occurred in the system. Timestamp drivers typically provide a high degree of precision with resolutions of a microsecond or less.

Periodic interrupt timer drivers are used to deliver periodic interrupts to an attached interrupt service routine (ISR). This type of driver is used as the basic heartbeat interrupt source for VxWorks. Because VxWorks requires periodic interrupts to perform its scheduling operations, each VxWorks system must include at least one timer driver that supports the generation of periodic interrupts.

This section presents the model for both timestamp and periodic interrupt timer drivers. Support for both timestamp and periodic interrupts can be provided within a single driver, or a driver can choose to provide just one of the two services.

A timer driver advertises the services that its hardware supports to the system. Operating system middleware chooses from among the available timers at runtime based on the advertised capabilities of the timer drivers in the system. Timing hardware often contains more than one timer within a single device. Your timer driver should normally be written so that a single instance supports all of the timers provided by the device hardware.

Related information

VxBus Driver Methods on page 211
Header Files on page 213
Configuring Timer Device Drivers on page 214
Available Utility Routines on page 214
Initialization on page 214
Data Structure Layout on page 215
Implementing Driver Service Routines on page 216
Integrating a Timer Driver on page 220
Debugging on page 221
SMP Considerations on page 222

13.2. VxBus Driver Methods

The timer device driver only requires the common VxBus methods.

The common methods required are:

- vxbDevProbe(VXB_DEV_ID pDev)
- vxbDevAttach(VXB_DEV_ID pDev)

All driver functionality is accessed by using the **vxbTimerFunctionality** structure. The driver fills in the contents of the **vxbTimerFunctionality** structure to describe the capabilities of the requested timer. The fields of the structure are listed in this

section, along with a description of each field's use. For the type definition for **vxbTimerFunctionality**, refer to the following header file: **vxworks-7/pkgs_v2/os/drv/vxbus/subsystem/h/subsys/timer/vxbTimerLib.h**.

SL_NODE *timerNode*

This field is used to maintain the timer device list in the timer subsystem.

BOOL *allocated*

This field holds a value that is maintained by the driver. The default value for "allocated" is **FALSE**. When a timer is allocated (using the **(*timerAllocate)( )** function pointer, see Implementing Driver Service Routines on page 216), the driver sets this field to **TRUE**. When a timer is released (using the **(*timerRelease)( )** function pointer, see Implementing Driver Service Routines on page 216), the driver sets this field to **FALSE** .

UINT32 *clkFrequency*

This field holds a value that describes the frequency (in Hz) that the timer's hardware counter increments (or decrements) when it is running.

UINT32 *minFrequency*

This fields holds a value that describes the minimum frequency (in interrupts per second) that a periodic interrupt timer can support. For timestamp drivers this field is not used.

UINT32 *maxFrequency*

This field holds a value that describes the maximum frequency (in interrupts per second) that a periodic interrupt timer can support. For time stamp drivers this field is not used.

UINT32 *features*

This field holds a bit-wise value that describes the capabilities of the timer. This value is constructed by performing a logical **OR** operation on the following values:

- **VXB_TIMER_CAN_INTERRUPT** - Set if the timer can generate interrupts.
- **VXB_TIMER_INTERMEDIATE_COUNT** - Set if the timer allows the hardware to read values while the counter is running without introducing any skew into the timing results.
- **VXB_TIMER_SIZE_16 , VXB_TIMER_SIZE_23 , VXB_TIMER_SIZE_32** - Set if the timer's counter register is 16, 23, or 32 bits (respectively).
- **VXB_TIMER_SIZE_64** - Set if the timer's counter register is 64 bits. When this value is set, the driver must also ensure that non-null values are provided for the **(*timerEnable64)( )**, **(*timerRolloverGet64)( )**, and **(*timerCountGet64)( )** function pointers (see Implementing Driver Service Routines on page 216).
- **VXB_TIMER_CANNOT_DISABLE** - Set if the underlying hardware timer cannot be disabled.
- **VXB_TIMER_STOP_WHILE_READ** - Set if the underlying hardware timer stops incrementing (or decrementing) while the timer is being read.
- **VXB_TIMER_AUTO_RELOAD** - Set if the underlying hardware timer automatically reloads itself when it reaches its terminal count, rather than requiring software intervention to restart the timer.
- **VXB_TIMER_CANNOT_MODIFY_ROLLOVER** - Set if the underlying hardware timer's rollover value is fixed at a single value. This is true for timers that (for example) always count from 0 to their maximum value, rather than to a software-controllable intermediate value.
- **VXB_TIMER_CANNOT_SUPPORT_ALL_FREQS** - Set if the underlying hardware timer cannot be configured to support interrupt frequencies continuously between **minFrequency** and **maxFrequency** .

UINT32 *ticksPerSecond*

This field holds a value that describes the current configuration of the timer hardware, in terms of interrupts per second that the hardware will generate. A timer driver sets this value to a reasonable default (typically between 60 and100), and maintains the value whenever the requested interrupt delivery rate changes. See the **(*timerEnable)( )** function pointer (see Implementing Driver Service Routines on page 216) for further information.

char *timerName* **[MAX_DRV_NAME_LEN + 1]**

This field holds the name of the timer driver. It is used for debug purposes. Timer drivers that support more than one timer within a single driver can choose to create a name that combines the name for the driver with the timer number. Timer drivers that only support a single timer should set this field to the name of the driver.

UINT32 *rolloverPeriod*

This field holds a parameter of type UINT32 that describes how long the timer takes to roll over, in seconds. Timers that count quickly have a shorter rollover period.

The following function pointers are described in Implementing Driver Service Routines on page 216:

- **STATUS (*timerAllocate)**
- **STATUS (*timerRelease)**
- **STATUS (*timerRolloverGet)**
- **STATUS (*timerCountGet)**
- **STATUS (*timerDisable)**
- **STATUS (*timerEnable)**
- **STATUS (*timerISRSet)**
- **STATUS (*timerEnable64)**
- **STATUS (*timerRolloverGet64)**
- **STATUS (*timerCountGet64)**

Once the driver's **{vxbTimerFuncGet}( )** routine is called, the various supported timer devices associated with the driver are available for allocation.

Related information

Overview on page 211
Header Files on page 213
Configuring Timer Device Drivers on page 214
Available Utility Routines on page 214
Initialization on page 214
Data Structure Layout on page 215
Implementing Driver Service Routines on page 216
Integrating a Timer Driver on page 220
Debugging on page 221
SMP Considerations on page 222

13.3. Header Files

Timer driver header files are contained in the **vxbTimerLib.h** file.

The **vxbTimerLib.h** file is located at: *installdir* **/vxworks-7/pkgs_v2/os/drv/vxbus/subsystem/h/subsys/timer/vxbTimerLib.h**.

Related information

Overview on page 211
VxBus Driver Methods on page 211
Configuring Timer Device Drivers on page 214
Available Utility Routines on page 214

Initialization on page 214
Data Structure Layout on page 215
Implementing Driver Service Routines on page 216
Integrating a Timer Driver on page 220
Debugging on page 221
SMP Considerations on page 222

13.4. Configuring Timer Device Drivers

Timer drivers do not typically require extra configuration information.

All the required configuration information is available in the FDT/ACPI or in some cases PCI query. Refer to Detecting and Configuring Devices with a Device Tree on page 21 for information on how to obtain device configuration values from FDT.

Related information

Overview on page 211
VxBus Driver Methods on page 211
Header Files on page 213
Available Utility Routines on page 214
Initialization on page 214
Data Structure Layout on page 215
Implementing Driver Service Routines on page 216
Integrating a Timer Driver on page 220
Debugging on page 221
SMP Considerations on page 222

13.5. Available Utility Routines

There are no class-specific utility routines required or available for timer drivers.

Related information

Overview on page 211
VxBus Driver Methods on page 211
Header Files on page 213
Configuring Timer Device Drivers on page 214
Initialization on page 214
Data Structure Layout on page 215
Implementing Driver Service Routines on page 216
Integrating a Timer Driver on page 220
Debugging on page 221
SMP Considerations on page 222

13.6. Initialization

This section describes the initialization procedure.

The timer device driver uses the generic initialization process for VxBus Gen2. The driver should also call vxbTimerRegister( ) to register to the timer subsystem so the system timer, auxiliary timer and timestamp timers can be associated with a proper timer device to supply timer functionality.

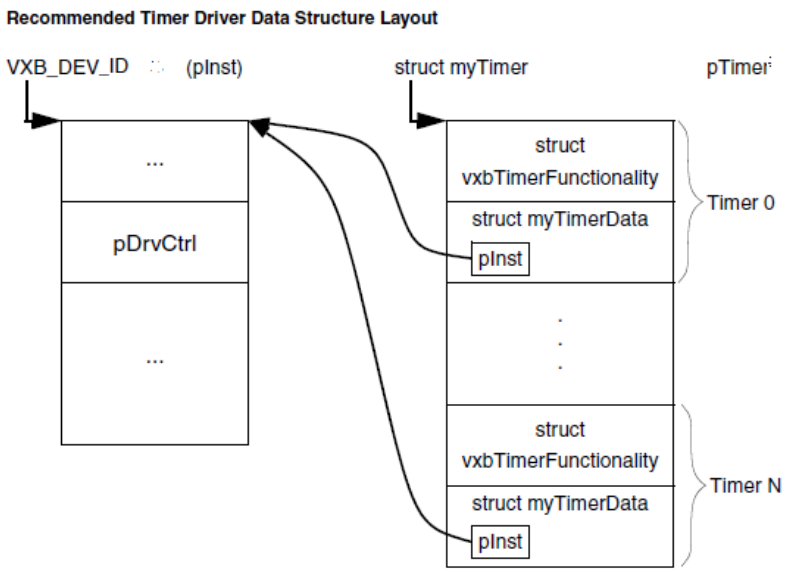
Related information

Overview on page 211
VxBus Driver Methods on page 211
Header Files on page 213
Configuring Timer Device Drivers on page 214
Available Utility Routines on page 214

Data Structure Layout on page 215
Implementing Driver Service Routines on page 216
Integrating a Timer Driver on page 220
Debugging on page 221
SMP Considerations on page 222

13.7. Data Structure Layout

This chapter documents recommended layout for the time driver data structure.



The two principal elements to this data structure are the **VXB_DEV_ID** instance data (by convention referred to as **pInst**) and the driver-specific data structure, which in this figure is labeled as **struct myTimerData**. Each of these data structures contains a pointer to the other. The **pInst->pDrvCtrl** field contains a pointer to the driver-specific data structure, and the driver-specific data structure contains a pointer back to the **pInst**.

When a timer driver initializes itself, it typically allocates its **struct myTimer** using **hwMemAlloc( )**, and then initializes the various data structures contained within it. This includes initializing the pointer(s) to **pInst**. Because the **VXB_DEV_ID pInst** pointer is not provided to the service routines, the stored pointer(s) to **pInst** is useful to the timer driver service routines.

This documentation assumes that the service routines are capable of accessing data within **VXB_DEV_ID**, even when **VXB_DEVI_ID** is not provided as a passed-in parameter to the service routine.

Related information

Overview on page 211
VxBus Driver Methods on page 211
Header Files on page 213
Configuring Timer Device Drivers on page 214
Available Utility Routines on page 214
Initialization on page 214
Implementing Driver Service Routines on page 216
Integrating a Timer Driver on page 220
Debugging on page 221
SMP Considerations on page 222

13.8. Implementing Driver Service Routines

Once a driver is registered with VxBus and a timer library (system/auxiliary/timestamp) wants to use a timer device, all subsequent interaction between the system and the driver occurs through the routines whose pointers are returned within the **vxbTimerFunctionality** data structure.

In the following sections, each of the service routines that can be supported by a timer driver are described. Not all of the service routines need to be implemented in a single driver. For each service routine, a note describing whether the service routine is required for a periodic interrupt driver, a timestamp driver, or for both types of drivers is provided..

- **(*timerAllocate)( )**

  The **(*timerAllocate)( )** routine is used to allocate a specific timer within a running VxWorks system. Both periodic interrupt and timestamp drivers are required to support this routine. The prototype for this routine is:

  ```
  STATUS (*timerAllocate)
  (
  void * pCookie, /* OUT */
  UINT32 flags, /* IN */
  );
  ```

  This routine tests its input parameters to ensure that it can comply with the requested allocation. If the requested timer is available, and if the requested timer supports the requested services (specified by the flags parameter), the driver:

  Marks the driver as allocated by setting the allocated field to **TRUE** within the **vxbTimerFunctionality** field associated with the timer hardware.
  Configures the timer hardware (if required, based on the **flags** parameter).
  Returns **OK** .

  If the requested timer does not exist, or if the timer cannot be configured according to the properties described in the flags parameter, the driver returns **ERROR** .

- **(*timerRelease)( )**

  The **(*timerRelease)( )** routine is used to release a specific timer that was previously allocated using **(*timerAllocate)( ).** Both periodic interrupt and timestamp drivers are required to support this routine. The prototype for this routine is:

  ```
  STATUS (*timerRelease)
  (
  void * pCookie, /* IN */
  );
  ```

  The void * parameter provided using **pCookie** points to the per-timer data area previously returned through a call to **vxbTimerEval( )** . The driver verifies that the requested timer is allocated. If the timer is allocated, the driver:

  Clears the allocation of the driver by setting the allocated field to **FALSE** within the **vxbTimerFunctionality** field associated with the timer hardware.
  Disables delivery of any interrupts from this timer source.
  Clears any associated ISR information from the per-timer data area.
  Returns **OK** .

  If the requested timer is not currently allocated, the driver returns **ERROR** .

- **(*timerRolloverGet)( )**

  The **(*timerRolloverGet)( )** routine is used to query the maximum value that the timer is capable of returning using its **(*timerCountGet)( )** routine. Timestamp drivers are required to support this routine. This routine is not used for periodic interrupt drivers. The prototype for this routine is:

  ```
  STATUS (*timerRolloverGet)
  (
  void * pCookie, /* IN */
  UINT32 pCount, /* OUT */
  );
  ```

The void * parameter provided through **pCookie** points to the per-timer data area previously returned through a call to **(*timerAllocate)( )**. The driver should test both **pCookie** and **pCount** to ensure that they are both non-null. If both pointers are valid, the driver:

Sets * **pCount** to the maximum value returnable from **(*timerCountGet)( )** .
Returns **OK** .

If either pointer is **NULL**, the driver returns **ERROR** .

- **(*timerCountGet)( )**

The **(*timerCountGet)( )** routine is used to query the current value of the timer. Timestamp drivers are required to support this routine. This routine is not used for periodic interrupt drivers.

> Note: In VxWorks, timers always count towards higher numeric values. If the underlying hardware on which the timer is based counts downward, the driver must perform the appropriate mathematics to ensure that the counter appears to count towards higher values from the caller's perspective.

The prototype for this routine is:

```
STATUS (*timerCountGet)
(
void * pCookie, /* IN */
UINT32 pCount, /* OUT */
);
```

The void * parameter provided using pCookie points to the per-timer data area previously returned through a call to **(*timerAllocate)( )**. Your driver should test both **pCookie** and **pCount** to ensure that they are both non-null. If both pointers are valid, the driver:

Sets * **pCount** to the current value of the hardware counter, with appropriate math operations to ensure that the counter appears to be counting towards higher values.
Returns **OK** .

If either **pCookie** or **pCount** are **NULL**, the driver returns **ERROR** .

> Note: Because this routine is used to create timestamps for events that can occur at a high frequency, it should be implemented as efficiently as possible in order to minimize its effect on overall system performance.

When a timer driver is used as the timestamp source for Wind River System Viewer, the kernel makes calls to **(*timerCountGet) ( )** at unpredictable times, such as when the kernel has interrupts locked, or while a spinlock is held. To allow **(*timerCountGet) ( )** to function correctly when used in this situation, the use of spinlocks is not allowed within **(*timerCountGet)( )**. In addition, the body of **(*timerCountGet)( )** must not perform any operations that result in a Wind River System Viewer event, because this causes an infinite recursion between System Viewer and the timer driver.

- **(*timerDisable)( )**

The **(*timerDisable)( )** routine is used to disable interrupts generated by the underlying timer hardware. Periodic interrupt drivers are required to support this routine. The prototype for this routine is:

The prototype for this routine is:

```
STATUS (*timerDisable)
(
void * pCookie, /* IN */

);
```

The **void** * parameter provided using **pCookie** points to the per-timer data area previously returned through a call to **(*timerAllocate)( )**. The driver should test **pCookie** to ensure that it is non-null. If **pCookie** is valid, the driver:

Disables interrupt generation for the requested hardware timer.
Returns **OK** .

If **pCookie** is **NULL**, the driver returns **ERROR** .

> **Note:** If the timer unit is always used as a free running counter—such as a timestamp timer or a delay timer—and the timer unit has the following characteristics:
>
> It starts at power on with the maximum count value supported by the hardware.
>
> The counter value is automatically rolled over when it reaches the maximum count.
>
> This routine can return **OK** with no other operations. (For an example, see the **vxbIntelTimestamp** driver.)

- **(*timerEnable)( )**

  The **(*timerEnable)( )** routine is used to enable generation of interrupts by the underlying timer hardware. Periodic interrupt drivers are required to support this routine. The prototype for this routine is:

```
STATUS (*timerEnable)
(
void * pCookie, /* IN */
UINT32 maxTimerCount /* IN */
);
```

  The **void** * parameter provided using **pCookie** points to the per-timer data area previously returned through a call to **(*timerAllocate)( )**. The driver should test **pCookie** to ensure that it is non-null. If **pCookie** is valid, the driver:

  Programs the underlying timer hardware so that it generates an interrupt each time **maxTimerCount** counts have occurred within the timer.
  Enables interrupt generation for the requested hardware timer.
  Returns **OK** .

  If **pCookie** is **NULL**, the driver returns **ERROR** .

> **Note:** If the timer unit is always used as a free running counter—such as a timestamp timer or a delay timer—and the timer unit has the following characteristics:
>
> It starts at power on with the maximum count value supported by the hardware.
>
> The counter value is automatically rolled over when it reaches the maximum count.
>
> This routine can return **OK** with no other operations. (For an example, see the **vxbIntelTimestamp** driver.)

- **(*timerISRSet)( )**

  The **(*timerISRSet)( )** routine is used to connect an ISR to the underlying timer hardware interrupt. Both periodic interrupt and timestamp drivers are required to support this routine (if your hardware supports interrupt generation). The prototype for this routine is:

```
STATUS (*timerISRSet)
(
void * pCookie, /* IN */
void (*pIsr)(_Vx_usr_arg_t),
_Vx_usr_arg_t arg
);
```

  The **void** * parameter provided using **pCookie** points to the per-timer data area previously returned through a call to **(*timerAllocate)( )**. The **pIsr** and **arg** parameters are caller-provided values that should be stored within the per-timer data area so that the values can be retrieved during interrupt handling for the timer hardware. After **(*timerISRSet)( )** is called to connect the requested ISR to the timer interrupt, the specified ISR is called each time a timer interrupt occurs, using the following code fragment:

```
(*pIsr)(arg);
```

- **(*timerRolloverGet64)( )**

The **(*timerRolloverGet64)( )** routine is used to query the maximum value that the timer is capable of returning using its **(*timerCountGet64)( )** routine. Support for 64-bit timers is optional. As such, neither timestamp drivers nor periodic interrupt drivers are required to support this routine. If you want to include support for 64-bit timers, this routine should be supported by the driver. In addition, the **VXB_TIMER_SIZE_64** property should be added to the features field of the **vxbTimerFunctionality** structure that is returned from **{vxbTimerFuncGet}( )** . The prototype for this routine is:

```
STATUS (*timeRolloverGet64)
(
void * pCookie, /* IN */
UINT64 * pCount /* OUT */
);
```

The **void** * parameter provided using **pCookie** points to the per-timer data area previously returned through a call to **(*timerAllocate)( )**. The driver should test both **pCookie** and **pCount** to ensure that they are both non-null. If both pointers are valid, the driver:

Sets * **pCount** to the maximum value returnable from **(*timerCountGet64)( )** .
Returns **OK** .

If either pointer is **NULL**, the driver returns **ERROR** .

- **(*timerEnable64)( )**

The **(*timerEnable64)( )** routine is used to enable generation of interrupts by the underlying timer hardware for timers that support 64-bit counters. Support for 64-bit timers is optional. As such, neither timestamp drivers nor periodic interrupt drivers are required to support this routine. If you want to include support for 64-bit timers, this routine should be supported by the driver. In addition, the **VXB_TIMER_SIZE_64** property should be added to the features field of the **vxbTimerFunctionality** structure that is returned from {vxbTimerFuncGet}( ). The prototype for this routine is:

```
STATUS (*timerEnable64)
(
void * pCookie, /* IN */
UINT64 maxTimerCount /* IN */
);
```

The **void** * parameter provided using **pCookie** points to the per-timer data area previously returned through a call to **(*timerAllocate)( )**. The driver should test **pCookie** to ensure that it is non-null. If **pCookie** is valid, the driver:

Programs the underlying timer hardware so that it generates an interrupt each time **maxTimerCount** counts have occurred within the timer.
Enables interrupt generation for the requested hardware timer.
Returns **OK** .

If **pCookie** is **NULL**, the driver returns **ERROR** .

- **(*timerCountGet64)( )**

The **(*timerCountGet64)( )** routine is used to query the current value of the timer for 64-bit timers. Support for 64-bit timers is optional. As such, neither timestamp drivers nor periodic interrupt drivers are required to support this routine.

> Note: In VxWorks, timers always count towards higher numeric values. If the underlying hardware on which the timer is based counts downward, the driver must perform the appropriate mathematics to ensure that the counter appears to count towards higher values from the caller's perspective.

```
STATUS (*timerCountGet64)
(
void * pCookie, /* IN */
UINT64 pcount /* OUT */
);
```

The **void** * parameter provided using **pCookie** points to the per-timer data area previously returned through a call to **(*timerAllocate)( )**. The driver should test both **pCookie** and **pCount** to ensure that they are both non-null. If both pointers are valid, the driver:

Sets * **pCount** to the current value of the hardware counter, with appropriate math operations to ensure that the counter appears to be counting towards higher values.
Returns **OK** .

If either **pCookie** or **pCount** is **NULL**, the driver returns **ERROR** .

> Note: Because this routine is used to create timestamps for events that can happen at a high frequency, it should be implemented as efficiently as possible in order to minimize its effect on overall system performance.

Related information

Overview on page 211
VxBus Driver Methods on page 211
Header Files on page 213
Configuring Timer Device Drivers on page 214
Available Utility Routines on page 214
Initialization on page 214
Data Structure Layout on page 215
Integrating a Timer Driver on page 220
Debugging on page 221
SMP Considerations on page 222

13.9. Integrating a Timer Driver

This section describes how to integrate a timer driver.

Traditionally, VxWorks uses between one and three different timers in a running system. All VxWorks operating systems use a standard periodic interrupt timer driver to support the kernel's heartbeat interrupt. Additionally, if a system is configured to support an auxiliary timer or timestamp driver, these services also make use of the timer drivers that are implemented according to this chapter. The following information discusses the integration of timer drivers to the system clock, auxiliary clock, and to the timestamp driver.

- **VxWorks Auxiliary Clock**

  VxBus supply following five APIs to compatible traditional legacy and VxBus GEN1 driver model.

  sysAuxClkConnect( )
  sysAuxClkEnable( )
  sysAuxClkDisable( )
  sysAuxClkRateSet( )
  sysAuxClkRateGet( )

  The work flow is:

  usrVxbTimerSys.c -> **vxbSysClkLib.c** -> VxBus timer driver

  The **usrClkInit** group is in charge of the system clock initialization and invoked by **usrRoot** :

```
void usrClkInit(void)
{
  usrVxbTimerSysInit();                /* vxBus Timer Support */
  vxbSysClkLibInit();                  /* vxBus Sys Clk Support */
   sysClkInit ();                       /* System clock component */
}
```

  usrVxbTimerSysInit() gets the system clock parameter from project, customer could use them to specify the hoped timer instance as system clock device:

  AUXCLK_TIMER_NAME

  AUXCLK_TIMER_NUM

  vxbSysClkLibInit( ) finds an available timer instance from timer subsystem; if the above two parameters are supplied, it uses them as the first match prerequisite.

You can then use the sysAux(*) API to connect/enable the ISR routine.

- **VxWorks Timestamp Driver**

  VxBus supplies the following APIs for compatibility with traditional legacy and VxBus GEN1 driver model:

  sysTimestampConnect( )
  sysTimestampEnable( )
  sysTimestampDisable( )
  sysTimestampPeriod( )
  sysTimestampFreq( )
  sysTimestamp( )
  sysTimestampLock( )

  The work flow is:

  usrVxbTimerSys.c -> **vxbSysClkLib.c** -> VxBus timer driver

  The **usrClkInit** group is in charge of the system clock initialization and invoked by **usrRoot** :

```
void usrClkInit(void)
{
  usrVxbTimerSysInit();                /* vxBus Timer Support */
  vxbSysClkLibInit();                  /* vxBus Sys Clk Support */
   sysClkInit ();                       /* System clock component */
}
```

  usrVxbTimerSysInit( ) gets the system clock parameter from the project. This can then be used to specify the timer instance as a system clock device:

  TIMESTAMP_TIMER_NAME

  TIMESTAMP_TIMER_NUM

  vxbTimestampLibInit( ) finds an available timer instance from the timer subsystem; if the above two parameters are supplied, it uses them as the first match prerequisite.

  The **sysTimestamp(*)( )** API can then be used to operate the timestamp function.

Related information

Overview on page 211
VxBus Driver Methods on page 211
Header Files on page 213
Configuring Timer Device Drivers on page 214
Available Utility Routines on page 214
Initialization on page 214
Data Structure Layout on page 215
Implementing Driver Service Routines on page 216
Debugging on page 221
SMP Considerations on page 222

13.10. Debugging

When debugging a timer driver, as with all driver development, it is convenient to have a fully functional system to test the driver on. A fully functioning system provides you with full access to the debug capabilities of VxWorks as well as the VxBus show routines.

Because timer drivers can be allocated to the VxWorks system clock during system boot, use a functional timer driver as the VxWorks system clock so that VxWorks can boot into a fully functional system that you can then use to debug your timer driver. The simplest way to prevent your driver from being selected as the system clock is to delay the registration of your timer driver until after the system has booted. When you delay registration, your driver is unavailable during the period when the system clock is evaluated; therefore, it cannot be selected as the system clock.

Related information

Overview on page 211
VxBus Driver Methods on page 211

Header Files on page 213
Configuring Timer Device Drivers on page 214
Available Utility Routines on page 214
Initialization on page 214
Data Structure Layout on page 215
Implementing Driver Service Routines on page 216
Integrating a Timer Driver on page 220
SMP Considerations on page 222

13.11. SMP Considerations

In VxWorks SMP, any CPU in the system can utilize the services of a timestamp driver.

This can present a unique problem if CPU-specific registers are used to implement a timestamp service. If these registers are not synchronized in the SMP system, the timestamp driver using these registers returns inconsistent results unless it is only used on a single CPU within the system.

If CPU-specific registers are used as a time base for a timer driver, the registers must be synchronized across all CPUs in the SMP system. The steps required to synchronize these registers is, by definition, architecture and CPU-specific. Unless your driver handles this situation, you should not advertise the **VXB_TIMER_INTERMEDIATE_COUNT** capability when compiled for SMP.

Related information

Overview on page 211
VxBus Driver Methods on page 211
Header Files on page 213
Configuring Timer Device Drivers on page 214
Available Utility Routines on page 214
Initialization on page 214
Data Structure Layout on page 215
Implementing Driver Service Routines on page 216
Integrating a Timer Driver on page 220
Debugging on page 221

14. Mailbox Controller Drivers

14.1. Overview - Mailbox Controller Drivers

Mailbox controller drivers are used to provide a functional interface between mailbox hardware and the various VxBus device drivers that uses them.

Related tasks

Implementing a Mailbox Controller Driver on page 231
Implementing the Mailbox Client on page 231

Related information

VxBus Driver Methods for Mailbox Controller Drivers on page 223
Header Files for Mailbox Controller Drivers on page 223
Device Tree Binding for Mailbox Controller Drivers on page 224
Available Utility Routines for Mailbox Controller Drivers on page 225
Initialization Process for Mailbox Controller Drivers on page 225
Mailbox Device Structure on page 226
Mailbox Channel Structure on page 227
Mailbox Node Structure on page 228
Mailbox Message Structure on page 228
Mailbox Controller Driver Functions on page 229

Mailbox Client Functions on page 230

14.2. VxBus Driver Methods for Mailbox Controller Drivers

Mailbox controller drivers only require the common VxBus methods.

The common methods required are:

- vxbDevProbe(VXB_DEV_ID pDev)
- vxbDevAttach(VXB_DEV_ID pDev)

All driver functionality is accessed by using the **vxbMailboxDev** structure.

Related tasks

Implementing a Mailbox Controller Driver on page 231
Implementing the Mailbox Client on page 231

Related information

Overview - Mailbox Controller Drivers on page 222
Header Files for Mailbox Controller Drivers on page 223
Device Tree Binding for Mailbox Controller Drivers on page 224
Available Utility Routines for Mailbox Controller Drivers on page 225
Initialization Process for Mailbox Controller Drivers on page 225
Mailbox Device Structure on page 226
Mailbox Channel Structure on page 227
Mailbox Node Structure on page 228
Mailbox Message Structure on page 228
Mailbox Controller Driver Functions on page 229
Mailbox Client Functions on page 230

14.3. Header Files for Mailbox Controller Drivers

Mailbox controller driver header files are contained in the **vxbMailboxLib.h** file.

The **vxbMailboxLib.h** file is located at: *installdir* **/vxworks-7/pkgs_v2/os/drv/vxbus/subsystem/h/subsys/mailbox/vxbMailboxLib.h**.

Related tasks

Implementing a Mailbox Controller Driver on page 231
Implementing the Mailbox Client on page 231

Related information

Overview - Mailbox Controller Drivers on page 222
VxBus Driver Methods for Mailbox Controller Drivers on page 223
Device Tree Binding for Mailbox Controller Drivers on page 224
Available Utility Routines for Mailbox Controller Drivers on page 225
Initialization Process for Mailbox Controller Drivers on page 225
Mailbox Device Structure on page 226
Mailbox Channel Structure on page 227
Mailbox Node Structure on page 228
Mailbox Message Structure on page 228
Mailbox Controller Driver Functions on page 229
Mailbox Client Functions on page 230

14.4. Device Tree Binding for Mailbox Controller Drivers

The properties of a mailbox controller can be configured using a device tree for systems that support the flattened device tree (FDT) model.

The properties of a mailbox controller can be managed using a device tree source (DTS) file. When a DTS file is used, and depending on the device type, certain device tree properties are supported.

Mailbox Controllers

For mailbox controllers, the following property is supported:

#mbox-cells

This property indicates how many parameters the mailbox controller requires to choose a mailbox channel. The parameters are defined by the mailbox controller driver. They can be a mailbox channel **phandle** in the mailbox controller or any other property the driver supports.

When the mailbox client node is referring to the mailbox channel, it must read the mailbox controller driver's device tree source file to obtain the parameters.

In this example, the mailbox controller requires one parameter to identify the mailbox channel.

```
mailbox0: mailbox@31f81000
    {
    compatible = "someone,mailbox";
    #mbox-cells = <1>;
    ...
    };
```

Mailbox Client

A mailbox client is any device node that uses a mailbox channel. A mailbox client can be any device node. For mailbox client, the following properties are supported:

mboxes

This property contains the list of mailbox controller phandle and mailbox parameter pairs; each pair represents a single mailbox channel that can be used.

mbox-names

This property is optional. It contains the list of mailbox channel names. Each name represents a single mailbox channel that specified in **mboxes** property.

```
mailboxclient0: mailboxclient
    {
    compatible = "mailbox-client";
    mboxes = <&mailbox0 0>,
             <&mailbox0 1>;
    mbox-names = "rx", "tx";
    };
```

Related tasks

Implementing a Mailbox Controller Driver on page 231
Implementing the Mailbox Client on page 231

Related information

Overview - Mailbox Controller Drivers on page 222
VxBus Driver Methods for Mailbox Controller Drivers on page 223
Header Files for Mailbox Controller Drivers on page 223
Available Utility Routines for Mailbox Controller Drivers on page 225
Initialization Process for Mailbox Controller Drivers on page 225
Mailbox Device Structure on page 226
Mailbox Channel Structure on page 227
Mailbox Node Structure on page 228
Mailbox Message Structure on page 228
Mailbox Controller Driver Functions on page 229
Mailbox Client Functions on page 230

14.5. Available Utility Routines for Mailbox Controller Drivers

There are no class-specific utility routines required or available for mailbox controller drivers.

Related tasks

Implementing a Mailbox Controller Driver on page 231
Implementing the Mailbox Client on page 231

Related information

Overview - Mailbox Controller Drivers on page 222
VxBus Driver Methods for Mailbox Controller Drivers on page 223
Header Files for Mailbox Controller Drivers on page 223
Device Tree Binding for Mailbox Controller Drivers on page 224
Initialization Process for Mailbox Controller Drivers on page 225
Mailbox Device Structure on page 226
Mailbox Channel Structure on page 227
Mailbox Node Structure on page 228
Mailbox Message Structure on page 228
Mailbox Controller Driver Functions on page 229
Mailbox Client Functions on page 230

14.6. Initialization Process for Mailbox Controller Drivers

This section describes the initialization procedure.

Mailbox controller drivers use the generic **VxBus GEN2** initialization process.

The vxbMailboxRegister( ) function must be called to register the mailbox controller to the mailbox subsystem during the **VxBus** initialization attach phase - vxbDevAttach( ). This step guarantees that the **VxBus** mailbox library (**vxbMailboxLib**) is available to other device drivers in the system.

Related tasks

Implementing a Mailbox Controller Driver on page 231
Implementing the Mailbox Client on page 231

Related information

Overview - Mailbox Controller Drivers on page 222
VxBus Driver Methods for Mailbox Controller Drivers on page 223
Header Files for Mailbox Controller Drivers on page 223
Device Tree Binding for Mailbox Controller Drivers on page 224
Available Utility Routines for Mailbox Controller Drivers on page 225
Mailbox Device Structure on page 226
Mailbox Channel Structure on page 227
Mailbox Node Structure on page 228

Mailbox Message Structure on page 228
Mailbox Controller Driver Functions on page 229
Mailbox Client Functions on page 230

14.7. Mailbox Device Structure

The VxBus mailbox subsystem uses a mailbox device structure to manage mailbox controllers.

The mailbox device structure contains variables and function pointers to manage the mailbox controller. When implementing a mailbox controller driver, you must fill this structure according to your implementation.

The mailbox device structure is defined as follows:

```
struct vxbMailboxDev {
    SL_NODE     mailboxDevNode;                 /* mailbox node list */
    VXB_DEV_ID  pDev;                           /* mailbox device */
    UINT32      chanNum;                        /* channel number */

    /* allocate a channel */

    STATUS (*mailboxChanAlloc)
        (
        void *                  pCookie,        /* IN */
        VXB_MBOX_NODE *         pMailboxNode,   /* IN */
        VXB_MAILBOX_CHAN *      pChan           /* OUT */
        );

    /* free a channel */

    STATUS (*mailboxChanFree)
        (
        void *                  pCookie,        /* IN */
        VXB_MAILBOX_CHAN *      pChan           /* IN */
        );

    /* start the reception on a mailbox channel */

    STATUS (*mailboxChanRxStart)
        (
        void *                  pCookie,        /* IN */
        VXB_MAILBOX_CHAN *      pChan           /* IN */
        );

    /* send message from channel */

    STATUS (*mailboxChanSend)
        (
        void *                  pCookie,        /* IN */
        VXB_MAILBOX_CHAN *      pChan,          /* IN */
        void *                  pMsg            /* IN */
        );

    /* shutdown a channel */

    STATUS (*mailboxChanShutdown)
        (
        void *                  pCookie,        /* IN */
        VXB_MAILBOX_CHAN *      pChan           /* IN */
```

```
        );

    /* status of a channel */

    STATUS (*mailboxChanStatusGet)
        (
        void *                  pCookie,        /* IN */
        VXB_MAILBOX_CHAN *      pChan,          /* IN */
        void *                  pStatus         /* OUT */
        );
}
```

Note: Depending on your specific target hardware, not all functions provided in the above example are implemented. If your hardware does not support a particular function, define it as NULL.

Related tasks

Implementing a Mailbox Controller Driver on page 231
Implementing the Mailbox Client on page 231

Related information

Overview - Mailbox Controller Drivers on page 222
VxBus Driver Methods for Mailbox Controller Drivers on page 223
Header Files for Mailbox Controller Drivers on page 223
Device Tree Binding for Mailbox Controller Drivers on page 224
Available Utility Routines for Mailbox Controller Drivers on page 225
Initialization Process for Mailbox Controller Drivers on page 225
Mailbox Channel Structure on page 227
Mailbox Node Structure on page 228
Mailbox Message Structure on page 228
Mailbox Controller Driver Functions on page 229
Mailbox Client Functions on page 230

14.8. Mailbox Channel Structure

The VxBus mailbox channel structure is used by mailbox client to send or receive mailbox message.

The mailbox channel structure is defined as follows:

```
struct vxbMailboxChan {
    VXB_DEV_ID                  pDev;
    VXB_MBOX_NODE *             pMailboxNode;   /* mailbox node information   */
    VXB_MAILBOX_DEV *           pMailboxDev;    /* controller device          */
    UINT32                      flag;

    /* channel receive callback */

    STATUS (*chanReceive)
        (
        VXB_MAILBOX_CHAN *      pChan,          /* IN */
        VXB_MAILBOX_MSG *       pMsg            /* IN */
        );
};
```

Related tasks

Implementing a Mailbox Controller Driver on page 231
Implementing the Mailbox Client on page 231

Related information

Overview - Mailbox Controller Drivers on page 222
VxBus Driver Methods for Mailbox Controller Drivers on page 223
Header Files for Mailbox Controller Drivers on page 223
Device Tree Binding for Mailbox Controller Drivers on page 224
Available Utility Routines for Mailbox Controller Drivers on page 225
Initialization Process for Mailbox Controller Drivers on page 225
Mailbox Device Structure on page 226
Mailbox Node Structure on page 228
Mailbox Message Structure on page 228
Mailbox Controller Driver Functions on page 229
Mailbox Client Functions on page 230

14.9. Mailbox Node Structure

The mailbox node structure is used to describe a specific mailbox node in the device tree source (DTS) file.

The mailbox node structure is defined as follows:

```
typedef struct vxbMboxNode {
    UINT32              phandle;        /* mbox controller's phandle */
    int                 index;          /* mbox channel specifier */
    const void *        mboxesProp;     /* mbox property */
    int                 mboxCell;       /* mbox-cell */
    VXB_MAILBOX_CHAN *  pChan;          /* active mbox channel */
    UINT32              flag;           /* this flag can be used for more */
                                        /* information about the mbox node */

} VXB_MBOX_NODE;
```

Related tasks

Implementing a Mailbox Controller Driver on page 231
Implementing the Mailbox Client on page 231

Related information

Overview - Mailbox Controller Drivers on page 222
VxBus Driver Methods for Mailbox Controller Drivers on page 223
Header Files for Mailbox Controller Drivers on page 223
Device Tree Binding for Mailbox Controller Drivers on page 224
Available Utility Routines for Mailbox Controller Drivers on page 225
Initialization Process for Mailbox Controller Drivers on page 225
Mailbox Device Structure on page 226
Mailbox Channel Structure on page 227
Mailbox Message Structure on page 228
Mailbox Controller Driver Functions on page 229
Mailbox Client Functions on page 230

14.10. Mailbox Message Structure

The VxBus mailbox subsystem uses mailbox message structure to manage mailbox message.

The mailbox message structure is defined as follows:

```
typedef struct vxbMailboxMsg {
    size_t      msgSize;                /* mbox message size   */
    void *      msgBuf;                 /* mbox message buffer */
} VXB_MAILBOX_MSG;
```

Related tasks
Implementing a Mailbox Controller Driver on page 231
Implementing the Mailbox Client on page 231

Related information
Overview - Mailbox Controller Drivers on page 222
VxBus Driver Methods for Mailbox Controller Drivers on page 223
Header Files for Mailbox Controller Drivers on page 223
Device Tree Binding for Mailbox Controller Drivers on page 224
Available Utility Routines for Mailbox Controller Drivers on page 225
Initialization Process for Mailbox Controller Drivers on page 225
Mailbox Device Structure on page 226
Mailbox Channel Structure on page 227
Mailbox Node Structure on page 228
Mailbox Controller Driver Functions on page 229
Mailbox Client Functions on page 230

14.11. Mailbox Controller Driver Functions

The following functions are available for your mailbox controller driver.

vxbMailboxRegister()

This function registers a mailbox controller to VxBus mailbox subsystem.

Note: The memory designated in the **VXB_MAILBOX_DEV** structure must be allocated by the controller driver.

vxbMailboxUnregister()

This function unregisters a mailbox controller from VxBus mailbox subsystem.

Note: The memory designated in the **VXB_MAILBOX_DEV** structure must be freed by the controller driver after it is unregistered.

vxbMailboxChanReceive()

This function notifies mailbox client to receive message from the mailbox channel.

Note: Your implementation defines whether or not vxbMailboxChanReceive( ) is invoked from the *ISR* context or the *task* context.

Related tasks
Implementing a Mailbox Controller Driver on page 231
Implementing the Mailbox Client on page 231

Related information
Overview - Mailbox Controller Drivers on page 222
VxBus Driver Methods for Mailbox Controller Drivers on page 223
Header Files for Mailbox Controller Drivers on page 223
Device Tree Binding for Mailbox Controller Drivers on page 224
Available Utility Routines for Mailbox Controller Drivers on page 225
Initialization Process for Mailbox Controller Drivers on page 225
Mailbox Device Structure on page 226
Mailbox Channel Structure on page 227

Mailbox Node Structure on page 228
Mailbox Message Structure on page 228
Mailbox Client Functions on page 230

14.12. Mailbox Client Functions

The following functions are available to mailbox client.

vxbMailboxChanAlloc()

Allocates a mailbox channel by index. The accepted parameters are the mailbox client VxBus device ID, mailbox channel index and allocate flag. This function must be called before any other mailbox functions that require the mailbox channel parameter.

vxbMailboxChanAllocByName()

Allocates a mailbox channel by name. The accepted parameters are the mailbox client VxBus device ID, mailbox channel name and allocate flag. This function must be called before any other mailbox functions that require the mailbox channel parameter.

vxbMailboxChanFree()

Free a mailbox channel. The accepted parameter is the mailbox channel.

vxbMailboxChanSetRxHandle()

Set mailbox channel receive handle. The accepted parameters are the mailbox channel and mailbox message receive callback function.

Note: It is implementation defined whether mailbox message receive callback function will be invoked from *ISR* context or *task* context.

vxbMailboxChanRxStart()

Start the reception on a mailbox channel. The accepted parameter is the mailbox channel. vxbMailboxChanSetRxHandle ( ) must be called to set receive handle on specified mailbox channel before this function is called.

vxbMailboxChanSend()

Send mailbox message. The accepted parameters are the mailbox channel and mailbox message. This function sends mailbox message via specific mailbox channel and it is implemented as synchronous operation which is blocked until mailbox message is sent.

vxbMailboxChanStatusGet()

Get the mailbox channel status. The accepted parameters are the mailbox channel and pointer to save mailbox status.

Related tasks

Implementing a Mailbox Controller Driver on page 231
Implementing the Mailbox Client on page 231

Related information

Overview - Mailbox Controller Drivers on page 222
VxBus Driver Methods for Mailbox Controller Drivers on page 223

Header Files for Mailbox Controller Drivers on page 223
Device Tree Binding for Mailbox Controller Drivers on page 224
Available Utility Routines for Mailbox Controller Drivers on page 225
Initialization Process for Mailbox Controller Drivers on page 225
Mailbox Device Structure on page 226
Mailbox Channel Structure on page 227
Mailbox Node Structure on page 228
Mailbox Message Structure on page 228
Mailbox Controller Driver Functions on page 229

14.13. Implementing a Mailbox Controller Driver

To implement a mailbox controller driver, you must implement functions and configure your device.

About This Task

Complete this workflow to implement a mailbox controller driver:

Procedure

1. Decide what functions must be implemented for your target hardware system.
2. Set the mailbox controller parameters for your target system in the device tree source (DTS) for flattened device tree (FDT)-based systems (PowerPC and ARM).
   For more information, see Mailbox Device Structure on page 226 and Mailbox Controller Driver Functions on page 229.
3. Call the vxbMailboxRegister( ) function to add the mailbox device to the mailbox subsystem.

Related tasks

Implementing the Mailbox Client on page 231

Related information

Overview - Mailbox Controller Drivers on page 222
VxBus Driver Methods for Mailbox Controller Drivers on page 223
Header Files for Mailbox Controller Drivers on page 223
Device Tree Binding for Mailbox Controller Drivers on page 224
Available Utility Routines for Mailbox Controller Drivers on page 225
Initialization Process for Mailbox Controller Drivers on page 225
Mailbox Device Structure on page 226
Mailbox Channel Structure on page 227
Mailbox Node Structure on page 228
Mailbox Message Structure on page 228
Mailbox Controller Driver Functions on page 229
Mailbox Client Functions on page 230

14.14. Implementing the Mailbox Client

For mailbox client, you must get the mailbox channel, allocate it, and set its allocation flag.

About This Task

For mailbox client in a flattened device tree (FDT)-based environment (ARM and PowerPC targets), the mailbox subsystem API usage is similar to the following:

```
VXB_MAILBOX_CHAN *          pChan;

/* create mailbox channel by index */

pChan = vxbMailboxChanAlloc (pDev, 0, VXB_MBOX_CHAN_DIR_TX | VXB_MBOX_CHAN_DIR_RX);

/* create mailbox channel by name */

pChan = vxbMailboxChanAllocByName (pDev, "name", VXB_MBOX_CHAN_DIR_TX | VXB_MBOX_CHAN_DIR_RX);

/* construct mailbox message */

VXB_MAILBOX_MSG             mailboxMsg;
mailboxMsg.msgSize = "message to send";
mailboxMsg.msgBuf  = sizeof ("message to send");

/* send mailbox message */

vxbMailboxChanSend (pChan, &mailboxMsg);

/* set mailbox message receive handle */

vxbMailboxChanSetRxHandle (pChan, chanReceive);

/* start reception */

vxbMailboxChanRxStart (pChan);

/* free mailbox channel */

vxbMailboxChanFree (pChan);
```

Related tasks

Implementing a Mailbox Controller Driver on page 231

Related information

Overview - Mailbox Controller Drivers on page 222
VxBus Driver Methods for Mailbox Controller Drivers on page 223
Header Files for Mailbox Controller Drivers on page 223
Device Tree Binding for Mailbox Controller Drivers on page 224
Available Utility Routines for Mailbox Controller Drivers on page 225
Initialization Process for Mailbox Controller Drivers on page 225
Mailbox Device Structure on page 226
Mailbox Channel Structure on page 227
Mailbox Node Structure on page 228
Mailbox Message Structure on page 228
Mailbox Controller Driver Functions on page 229
Mailbox Client Functions on page 230

10. PORTING A VXBUS GEN1 DRIVER TO VXBUS GEN2

1. VxBus GEN1 Driver Migration

While the basic rationale behind VxBus GEN2 is the same for GEN1, differences in design and implementation between the two mean that GEN1 drivers cannot be used unmodified with VxBus GEN2.

Devices Versus Buses

In VxBus GEN1, a bus is considered to be conceptually different from a device. One consequence of this is that there are data structures unique to each type. Another is that it is the responsibility of a bus driver to provide a **devMatch** method to be used as part of the driver probe phase. While GEN1 drivers can include their own probe functions, doing so is optional, and many drivers do not include one. Instead, the parent bus driver is expected to provide a match function that is used against all new devices announced on the bus. For example, the PCI code includes a **devMatch** method that checks the device's vendor and device ID against a list provided by the driver. This implies a built-in assumption that PCI-class devices must always be matched using their vendor and device IDs.

This assumption is not entirely valid. Depending on the circumstances, you might also need to test the revision register or other criteria in order to perform a match. Enforcing a single match policy for a given bus type is overly restrictive, and while a custom probe function can be used to override the policy, the semantics are not always clear.

VxBus GEN2 chooses instead to leave it up to you to make these policy choices on a case by case basis rather than having them included as part of the parent bus class. Each driver you write must provide a probe function which allows you to choose the match criteria. You can use the helper functions provided for some common cases, if desired.

Old Driver Methods Versus New

In VxBus GEN1, a driver is required to provide the following three functions:

- instInit( )
- instInit2( )
- instConnect( )

These three functions are part of a table that is directly defined in the **VXB_DEVICE_ID** handle structure. An optional instUnlink( ) function can be provided as a method.

The instInit( )/instInit2( ) scheme is intended to mirror the VxWorks bootstrap process, which required BSPs to provide sysHwInit( ) and sysHwInit2( ) functions. The instConnect( ) function is intended to provide a way to execute some setup code in a separate task context (**tDevConn**, rather than **tRootTask**). (This turned out to be more of a hindrance than a help due to the difficulty in predicting just when **tDevConn** will execute.) According to the design, the instInit( ) function runs before the kernel is initialized and instInit2( ) runs after. The instInit( ) function is unable to allocate memory from the heap.

There are several problems with this strategy. While it was born out of a desire to preserve existing design practices, it made BSP-independent driver implementation difficult. Furthermore, the implementation is inconsistent. The instInit( ), instInit2( ), and instConnect( ) functions are part of a static function table while the instUnlink( ) function is a method. This is to avoid changing the static function table (and hence the **VXB_DEVICE_ID** structure).

In VxBus GEN2, the driver initialization process is completely divorced from any BSP or OS design constraints and now more closely resembles the driver frameworks of other OS platforms. Some features:

- The instInit( )/instInit2( )/instConnect( )/instUnlink( ) construction is replaced by the probe/attach/shutdown/detach strategy.
- The probe/attach/shutdown/detach functions are all implemented as methods (that is, there are no more baked-in function pointers).

- Given the new VxWorks 7 memory model design, all VxBus operations now take place post-kernel. This means that it is safe to use malloc( )/free( ) in all driver methods, including probe and attach.

FDT Versus hwconf.c

VxBus GEN2 now uses flattened device tree (FDT) for static device configuration instead of an **hwconf.c** file. The FDT specification is an industry standard with a more flexible format than **hwconf.c**. VxBus GEN2 uses the standard FDT library for parsing FDT structures, and BSPs can include device tree source (**.dts**) files that can be customized for specific boards. (The **hwconf.c** file in VxBus GEN1 served the same general purpose as **.dts** files do in VxWorks 7.)

The use of FDT is also mandated as part of the VxWorks 7 BSP format.

Device Handles Are Opaque

In VxBus GEN1, drivers directly refer to structure members in the **VXB_DEVICE_ID** handle. In VxBus GEN2, this is no longer possible. The definition of the **VXB_DEV_ID** structure is kept internal to VxBus, and the handle is manipulated entirely using accessor functions or other APIs.

Introduction of VXB_RESOURCE Handles

One consequence of the change to opaque device handles is that the **pRegBase[]** array from the **VXB_DEVICE_ID** handle, which lists the base addresses of a device's register banks, is no longer present. Instead, BARs are referenced using **VXB_RESOURCE** structures. One benefit to this design is that where the **pRegBase[]** array was limited to 10 slots, there is no limit to the number of **VXB_RESOURCE** entries that can be associated with a given device handle. (Devices with 10 distinct register banks are probably rare, but VxBus GEN1 would have difficulty dealing with one regardless.)

In addition, the **VXB_RESOURCE** abstraction no longer requires that a function driver call vxbRegMap( )/vxbRegUnmap( ) directly. This is instead done by the parent resource allocation method provided by the parent bus driver.

Changes to the Interrupt Handling Model

The interrupt handling API in VxBus GEN1 provided the vxbIntConnect( ), vxbIntDisconnect( ), vxbIntEnable( ), and vxbIntDisable( ) functions. Each of these functions required the caller to provide the following:

- The device handle.
- The ISR function pointer.
- The ISR function connect handle.
- The index value that represented the desired interrupt (most devices only have one interrupt, therefore the index is often 0).

VxBus GEN2 has these functions as well. However, they are simplified and adapted to the **VXB_RESOURCE** model. First, the ISR function pointer and context handle are only needed when calling the vxbIntConnect( ) function. They are not required for the other functions (they are redundant for everything except the connect phase). Also, rather than using an index number, the vxbIntConnect( ), vxbIntDisconnect( ), vxbIntEnable( ), and vxbIntDisable( ) functions now accept a **VXB_RESOURCE** handle that describes the desired interrupt.

Removal of hwMemLib.c

Some of the functions in VxBus GEN1 drivers, particularly the instInit( ) method, can be executed before the kernel is fully initialized, and before the heap is available. It is not possible to use malloc( ) in these functions. Instead, a separate allocation library is provided. With VxWorks 7 and VxBus GEN2, all driver initialization takes place after the basic kernel facilities are initialized. Therefore, the special allocation library (**hwMemLib**) is no longer needed.

Removal of Explicit Registration Functions

Using the VxBus GEN2 linker set feature means it is no longer necessary to have explicit C code to register a driver with VxBus. Instead, as long as the **VXB_DRV_DEF()** macro is used to include the driver's **VXB_DRV** structure in the driver linker set, the driver is registered automatically.

When changing from instInit( )/instIinit2( )/instConnect( ) to the probe/attach model, it might be possible for you to simply convert instInit( ) to the probe function and instInit2( ) to the attach function. However, this depends on your driver. The instInit( ) method is intended for code that runs before the kernel is initialized. That is, it is analagous to the sysHwInit( ) function in a BSP. There is no heap available yet, kernel facilities such as semaphores are not used, and the MMU is not yet enabled. With VxBus GEN2, the probe and attach functions all execute after the kernel is initialized, so that no special distinction is drawn between pre-kernel and post-kernel initialization.

vxbDevAcquireByName() Versus vxbInstByNameFind()

The vxbInstByNameFind( ) function in VxBus GEN1 has a serious shortcoming in that it returns pointers to device nodes that can later become stale. The vxbDevAcquireByName( ) function can also be used to look up a node in the VxBus tree. However, it increments the node's reference count before returning a pointer to it. The caller must use vxbDevRelease( ) to decrement the reference count again once it is finished using the node.

2. Porting Guidelines for VxBus GEN1 to VxBus GEN2

There are several guidelines you can follow to help as you port your VxBus GEN1 device drivers to VxBus GEN2.

The best place to start when converting a driver from VxBus GEN1 to GEN2 is to create a set of probe, attach, shutdown, and detach methods and a **VXB_DRV** structure, to replace the existing instInit( ), instInit2( ), instConnect( ), and (if present) drvUnlink( ) methods and the old style driver structure. If there is a driver registration function in your driver, remove it. Any instances of **VXB_DEVICE_ID** must also be changed to **VXB_DEV_ID**.

> Note: It is now always necessary to provide a probe function. You cannot rely on the parent bus driver to provide a class-specific devMatch( ) method. The probe logic is left up to you as the driver developer, though some class libraries provide helper functions to avoid code duplication.

Code that references **pDev->regBase[]** must be removed and replaced with code that instead acquires **VXB_RESOURCE** handles for any necessary register banks. Also, you do not need to call vxbRegMap( ) directly as you did in VxBus GEN1. Now, this is done internally by the parent bus controller driver in its VXB_RESOURCE_ALLOC( ) method.

> Note: In VxBus GEN1 on 32-bit platforms, vxbRegMap( ) only served to acquire the correct vxbReadXX( )/vxbWriteXX( ) handle. In VxWorks 6.9. 32-bit platforms used a 1:1 virtual to physical mapping strategy, where device register bank mappings were often preset by the BSP. In contrast, 64-bit VxWorks on the Intel platform required vxbRegMap( ) in order to create virtual to physical mappings as well. In VxWorks 7, both the 32-bit and 64-bit platforms use the same memory model, and neither one uses the 1:1 identity mapping scheme. This means that vxbRegMap( ) is always needed to both create the necessary virtual mapping for a BAR and for returning the correct register access handle. (Fortunately, this is now handled automatically.)

You may also need to re-write any register access macros. Ideally, your driver should use the **CSR_READ()**/**CSR_WRITE()** macros when accessing registers, as this allows the actual access mechanism to be more easily abstracted.

An example of accessing registers using resources is as follows:

```
    typedef struct myDrvCtrl
        {
```


WNDRVR

```
        void * pBase;
        void * pHandle;
        } MY_DRV_CTRL;

    #define CSR_READ_4(dev, offset) \
        vxbRead32 (dev->pHandle, (UINT32 *)(dev->pBase + offset))
    #define CSR_WRITE_4(dev, offset, val) \
        vxbWrite32 (dev->pHandle, (UINT32 *)(dev->pBase + offset), val)

    STATUS myAttach
        (
        VXB_DEV_ID pDev
        )
        {
        MY_DRV_CTRL * pDrvCtrl;
        VXB_RESOURCE * pRes;
        VXB_RESOURCE_ADR * pAdr;
        UINT32 someVal;

        pDrvCtrl = malloc (sizeof (MY_DRV_CTRL));
        vxbDevSoftcSet (pDev, pDrvCtrl);

        /* Get a handle to the 0th memory-mapped BAR */

        pRes = vxbResourceAlloc (pDev, VXB_RES_MEMORY, 0);
        pAdr = pRes->pRes;

        /* Save the base address and handle */

        pDrvCtrl->pBase = pAdr->virtual;
        pDrvCtrl->pHandle = pAdr->pHandle;

        /* Do a read/modify/write of a register */

        someVal = CSR_READ_4(pDrvCtrl, 0x10);
        someVal |= 0x1;
        CSR_WRITE_4(pDrvCtrl, 0x10, someVal);

        return (OK);
        }
```

Any uses of hwMemAlloc( ) and hwMemFree( ) must be replaced with malloc( ) and free( ). A vxbMemAlloc( )/vxbMemFree( ) API exists. However, it is currently only a wrapper around malloc( ) and free( ).

Use of vxbIntConnect( ), vxbIntDisconnect( ), vxbIntEnable( ), and vxbIntDisable( ) can remain largely the same as in GEN1. The only major differences are that these functions now operate on resource handles rather than interrupt index numbers, and you do not need to always specify the ISR function pointer and connect pointer for anything except vxbIntConnect( ).

For example, where a VxBus GEN1 driver does the following:

```
     vxbIntConnect (pDev, myIsrFunc, myIsrCtx, 0);
     vxbIntEnable (pDev, myIsrFunc, myIsrCtx, 0);
```

In VxBus GEN2 it now does this:

```
    VXB_RESOURCE pIrq;

    /* Acquire handle to the 0th interrupt resource */

    pIrq = vxbResourceAlloc (pDev, VXB_RES_IRQ, 0);

    vxbIntConnect (pDev, pIrq, myIsrFunc, myIsrCtx);
    vxbIntEnable (pDev, pIrq);
```

Drivers that previously used **vxbDmaBufLib** for handling bus-master DMA can continue to do so. This library is provided in VxBus GEN2 and works exactly as it did in VxBus GEN1.

For PCI drivers, the **VXB_PCI_BUS_CFG_READ()** and **VXB_PCI_BUS_CFG_WRITE()** macros are still provided, and work as they did in VxBus GEN1.

Your remaining driver code can probably be used as-is. The new VxBus driver model mostly only affects the front-end parts of the driver. The overall driver structure typically remains unchanged.